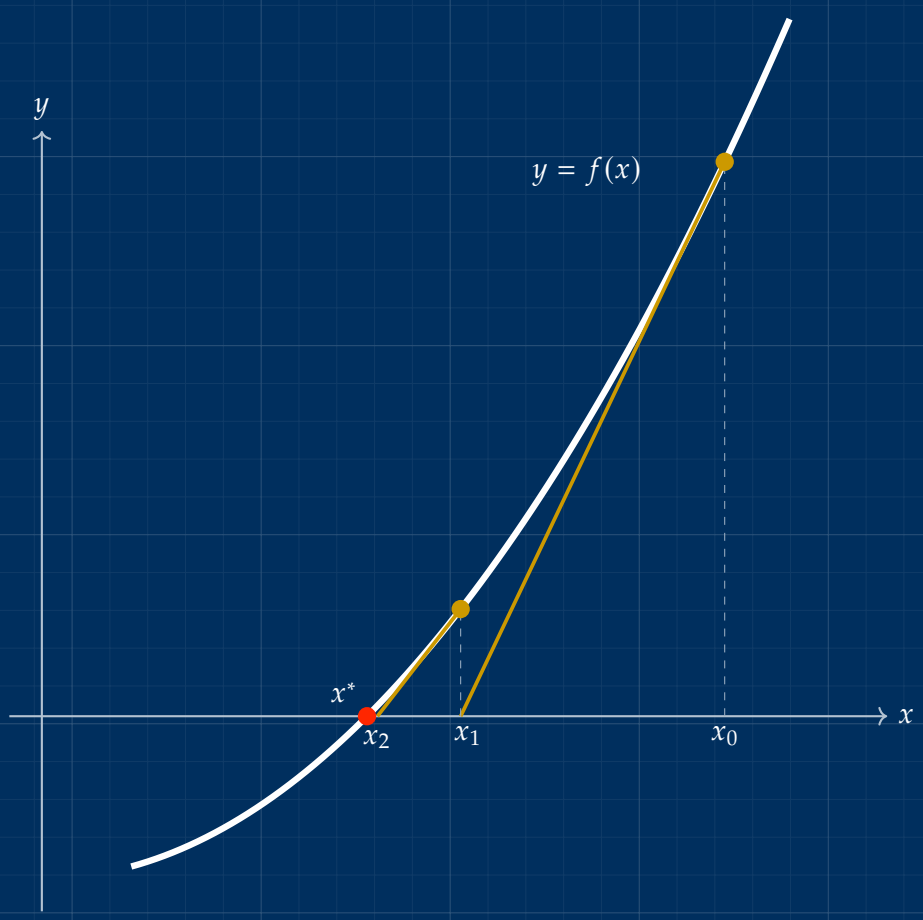
Lecture Notes for

Third Edition

Numerical Analysis

with Python



$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Stephen E. Moore

Department of Mathematics · University of Cape Coast, Ghana

Lecture Notes for Numerical Analysis with Python
MAT 407: Numerical Analysis I
Third edition

Stephen E. Moore
Department of Mathematics, University of Cape Coast, Cape Coast, Ghana
https://moorestephen.info/

© 2022, 2024, 2026 Stephen E. Moore.

First edition 2022
Second edition 2024
Third edition 2026

Licence. These notes and their Python programs are licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International Licence (CC BY-NC-ND 4.0, https://creativecommons.org/licenses/by-nc-nd/4.0/): you may copy and share them in unchanged form for non-commercial purposes, provided you give appropriate credit to the author; you may not use them commercially or distribute modified versions. Both the book and the programs are available at https://github.com/moorekwesi/numerical-analysis-with-python.

Earlier editions of these notes were typed by Joshua Badu and edited by Aaron Antwi.

All Python programs referred to in the text are distributed with these notes in the folder `code/`, organised by chapter. The output printed beneath each listing is the actual output of the program, produced with Python 3, NumPy, SciPy and Matplotlib.

Cover illustration: two steps of Newton's method for $f(x) = \frac{1}{4}x^2 - 1$.

Typeset by the author in LaTeX.

For my Hilda, Elisa and Yael.

Contents

List of Python Listings ix

Preface xi

Acknowledgements xiii

Course Map xv

1 Introduction to Python Programming 1
- 1.1 Getting started . . . 1
  - 1.1.1 Numbers, variables and strings . . . 2
- 1.2 Vectors with NumPy . . . 3
- 1.3 Matrices . . . 5
- 1.4 Control flow: loops and decisions . . . 7
- 1.5 Functions . . . 9
- 1.6 Plotting with Matplotlib . . . 10
- 1.7 Case study: the falling bungee jumper . . . 11

2 Number Systems 15
- 2.1 Positional notation in base β . . . 15
  - 2.1.1 From base β to decimal: Horner's rule . . . 16
  - 2.1.2 From decimal to base β: repeated division . . . 16
  - 2.1.3 Fractions: repeated multiplication . . . 17
  - 2.1.4 Binary $\leftrightarrow$ octal $\leftrightarrow$ hexadecimal . . . 17
  - 2.1.5 Python . . . 17
- 2.2 Binary arithmetic . . . 19
- 2.3 Integers in a computer . . . 19

3 Floating-Point Arithmetic and Round-off Errors 23
- 3.1 Floating-point number systems . . . 23
  - 3.1.1 The IEEE 754 standard . . . 25
- 3.2 Rounding and chopping . . . 28
- 3.3 Propagation of errors and catastrophic cancellation . . . 30
  - 3.3.1 Accumulation of errors in sums . . . 32
- 3.4 Conditioning and stability . . . 34
- 3.5 Truncation error, Taylor's theorem and the O notation . . . 36

4 Solutions of Nonlinear Equations 39
- 4.1 Locating roots and bracketing . . . 40
- 4.2 The bisection method . . . 40
- 4.3 Order of convergence . . . 44

4.4 Fixed-point iteration . . . 45
4.5 Newton's method . . . 49
4.5.1 When Newton's method fails . . . 52
4.6 The secant method . . . 55
4.7 The method of false position (regula falsi) . . . 57
4.8 Comparison, acceleration and software . . . 60
4.8.1 Aitken's Δ^2 process and Steffensen's method . . . 62
4.8.2 Root finding in SciPy . . . 63

5 Systems of Nonlinear Equations 67
5.1 Functions of several variables . . . 67
5.2 Newton's method for systems . . . 68
5.2.1 Finite-difference Jacobians . . . 71
5.3 Quasi-Newton methods and software . . . 73

6 Direct Methods for Linear Systems: Matrix Factorisations 77
6.1 Triangular systems . . . 78
6.2 Gaussian elimination and pivoting . . . 78
6.2.1 Why pivoting is necessary . . . 79
6.3 LU factorisation . . . 82
6.3.1 Tridiagonal systems . . . 85
6.4 Cholesky factorisation . . . 86
6.5 QR factorisation . . . 89

7 Iterative Methods for Linear Systems 95
7.1 Stationary iterative methods . . . 96
7.2 The Jacobi method . . . 97
7.3 The Gauss–Seidel method . . . 97
7.3.1 Sufficient conditions for convergence . . . 99
7.4 Successive over-relaxation (SOR) . . . 100
7.5 Gradient methods: steepest descent and conjugate gradients . . . 102
7.6 A large sparse example: the Poisson equation . . . 105

8 Norms, Condition Numbers and Error Estimates 109
8.1 Vector norms . . . 109
8.2 Matrix norms and the spectral radius . . . 112
8.3 The condition number of a matrix . . . 113
8.3.1 Iterative refinement . . . 115
8.4 Stopping criteria for iterative methods . . . 117

9 Polynomial Interpolation 119
9.1 Polynomial approximation . . . 119
9.2 The interpolation problem . . . 120
9.2.1 Method of undetermined coefficients . . . 120
9.3 Lagrange form . . . 121
9.4 Newton's divided-difference form . . . 124
9.5 The error of polynomial interpolation . . . 127
9.6 Runge's phenomenon and Chebyshev nodes . . . 129
9.7 Piecewise interpolation and splines . . . 130

10 Least-Squares Approximation and Orthogonal Polynomials **133**
10.1 Discrete least squares . . . 134
10.1.1 The normal equations . . . 134
10.1.2 Normal equations versus QR . . . 136
10.2 Nonlinear models and linearisation . . . 137
10.3 Continuous least squares . . . 140
10.4 Orthogonal polynomials . . . 142
10.4.1 Legendre polynomials . . . 143
10.4.2 Chebyshev polynomials . . . 143

11 Approximation by Rational Functions **147**
11.1 Padé approximation . . . 147
11.2 Chebyshev rational approximation . . . 152
11.2.1 Best approximation . . . 153

12 Numerical Differentiation **155**
12.1 Finite-difference formulas . . . 155
12.2 Rounding error and the choice of h . . . 158
12.3 Richardson extrapolation . . . 161

13 Numerical Integration and Adaptive Quadrature **165**
13.1 Newton–Cotes formulas . . . 166
13.2 Composite rules . . . 168
13.3 Romberg integration . . . 171
13.4 Gaussian quadrature . . . 172
13.5 Adaptive quadrature . . . 175

A Python Quick Reference **179**

B Answers and Hints to Selected Exercises **183**

References **187**

List of Python Listings

1.1 Numbers, the `math` module and strings . . . 2
1.2 Creating and manipulating vectors . . . 4
1.3 Matrix operations, errors and linear systems . . . 5
1.4 Loops, conditions and list comprehensions . . . 7
1.5 User-defined functions and functions as arguments . . . 9
1.6 Plotting functions . . . 10
1.7 Bungee jumper: Euler's method against the exact solution . . . 12
2.1 Base conversion by repeated division, Horner's rule and repeated multiplication 17
2.2 Fixed-width integers, two's complement and overflow . . . 20
3.1 Enumerating a toy floating-point system . . . 24
3.2 Inspecting the bits of a double . . . 25
3.3 Machine epsilon, overflow, underflow, infinity and NaN . . . 26
3.4 k-digit chopping and rounding arithmetic . . . 29
3.5 Catastrophic cancellation and its cures . . . 31
3.6 Naive and compensated summation . . . 32
3.7 Reproducing the Patriot clock drift . . . 33
3.8 Condition numbers of some elementary functions . . . 35
4.1 The bisection method . . . 42
4.2 Three fixed-point iterations for $x^2 - 2x - 3 = 0$ and cobweb diagrams . . . 46
4.3 Newton's method and its quadratic convergence . . . 51
4.4 Poor starting values, cycles and multiple roots . . . 52
4.5 The secant method . . . 56
4.6 False position and the Illinois modification . . . 58
4.7 All methods on $x^3 - x^2 - x - 1 = 0$ . . . 60
4.8 Aitken acceleration and Steffensen's method for $x = \cos x$ . . . 62
4.9 SciPy and NumPy root finders . . . 63
5.1 Newton's method for a 2×2 system . . . 70
5.2 Lorenz equilibria by Newton's method with exact and finite-difference Jacobians 72
5.3 Broyden's method against Newton's method . . . 74
5.4 Nonlinear systems in SciPy . . . 75
6.1 The effect of a tiny pivot . . . 79
6.2 Gaussian elimination with partial pivoting . . . 81
6.3 LU factorisation with partial pivoting . . . 84
6.4 The Thomas algorithm for a boundary-value problem . . . 85
6.5 Cholesky factorisation and tests for positive definiteness . . . 88
6.6 Gram–Schmidt QR, solving by QR, and loss of orthogonality . . . 91
7.1 Jacobi and Gauss–Seidel . . . 98
7.2 SOR for several values of ω . . . 101
7.3 Steepest descent and conjugate gradients . . . 103
7.4 Iterative methods for the 2D Poisson equation ($n = 400$) . . . 106

8.1 Vector norms, matrix norms and the spectral radius . . . 110
8.2 Condition numbers and lost digits . . . 115
8.3 Mixed-precision iterative refinement . . . 116
9.1 Undetermined coefficients and the conditioning of Vandermonde matrices . . . 121
9.2 Lagrange interpolation . . . 123
9.3 Divided differences, Newton form and error estimates . . . 126
9.4 Runge's phenomenon . . . 129
9.5 Piecewise linear and cubic spline interpolation of Runge's function . . . 131
10.1 The least-squares line . . . 135
10.2 Normal equations versus QR for polynomial fitting . . . 136
10.3 Exponential fits: linearisation and nonlinear least squares . . . 139
10.4 Continuous least squares in the monomial and Legendre bases . . . 141
10.5 Legendre and Chebyshev polynomials: recurrences and orthogonality . . . 143
11.1 Padé approximation of e^{-x} in exact rational arithmetic and with SciPy . . . 149
11.2 Padé approximation reproduces the pole of $\tan x$ . . . 150
11.3 Chebyshev rational approximation of e^{-x} on $[-1, 1]$ . . . 152
12.1 Orders of accuracy of difference formulas . . . 157
12.2 Total error of difference quotients as a function of h . . . 158
12.3 Differentiating noisy velocity measurements . . . 160
12.4 The Richardson extrapolation table . . . 162
12.5 The complex-step derivative . . . 163
13.1 Newton–Cotes rules and their degree of precision . . . 166
13.2 Composite trapezoidal and Simpson rules . . . 169
13.3 Romberg integration . . . 171
13.4 Gauss–Legendre quadrature . . . 173
13.5 Adaptive Simpson quadrature . . . 176

Preface

Numerical analysis is the study of algorithms for the problems of continuous mathematics. Whenever a scientist or an engineer computes a root, solves a linear system, fits a curve to data, or evaluates an integral on a computer, the answer is produced by an algorithm that works with finitely many digits and finitely many steps. Numerical analysis asks three questions of every such algorithm:

1. **Does it converge?** Will the algorithm eventually produce the answer we want, and under what conditions?

2. **How fast?** How many steps (and how much computing work) are needed to reach a required accuracy?

3. **How reliable?** How sensitive is the answer to rounding errors and to errors in the data?

These notes accompany MAT 407 (Numerical Analysis I) at the University of Cape Coast. They are written for final-year undergraduate students, with a second aim of preparing students for graduate study in mathematics, statistics, physics, engineering, data science and related fields. Every algorithm is derived, analysed and then *implemented in Python*. Every Python listing in these notes has been executed, and the output shown beneath it is the actual output of the program. The source files are provided in the folder `code/`, organised by chapter, so that you can run, modify and experiment with them.

How to use these notes

The text uses a small number of highlighted boxes so that important ideas stand out.

⋆ Key Concept: what it looks like

A *Key Concept* box contains an idea, formula or result that you must understand and remember. Examination questions are very often built around these.

Historical Note: the people behind the mathematics

A *Historical Note* tells the story of how an idea arose, who discovered it, and why. Mathematics is a human activity, and knowing its history helps you see why an idea matters.

△ Caution: common mistakes

A *Caution* box warns about a common mistake, a trap in computer arithmetic, or a situation in which a method fails.

Towards Graduate Studies: looking ahead

A *Towards Graduate Studies* box connects the topic to more advanced mathematics (functional analysis, numerical linear algebra, numerical PDEs, optimisation, scientific machine learning). These are not examined in MAT 407, but they are the questions you will meet in a Master's or PhD programme.

Exercises. Exercises are placed at the end of each major section and are tagged: Theory for pencil-and-paper analysis and proofs, Python for programming exercises, and ★ Graduate for harder problems aimed at students planning graduate study. Brief answers and hints to selected exercises are given in Appendix B.

Software. We use Python 3 with the libraries NumPy, SciPy and Matplotlib. The easiest way to obtain all of them is to install the free Anaconda distribution, or to use Google Colab in a web browser. Appendix A lists the most useful commands and their MATLAB/Octave equivalents.

Course outline. The chapters follow the MAT 407 course outline: Python programming (Chapter 1); number systems (Chapter 2); round-off errors and floating-point arithmetic (Chapter 3); nonlinear equations (Chapters 4 and 5) [*Quiz 1*]; direct and iterative methods for linear systems, followed by norms and error estimates (Chapters 6 to 8) [*Quiz 2*]; polynomial interpolation, orthogonal polynomials, least squares and rational approximation (Chapters 9 to 11); and numerical differentiation, integration and adaptive quadrature (Chapters 12 and 13). The Course Map on page xv shows how the chapters fit together.

This edition. The first edition of these notes appeared in 2022 and the second in 2024. This third edition (2026) adds Python implementations with verified output throughout, exercises after every section, historical notes and graduate-level pointers, and new chapters on rational approximation, numerical differentiation and adaptive quadrature.

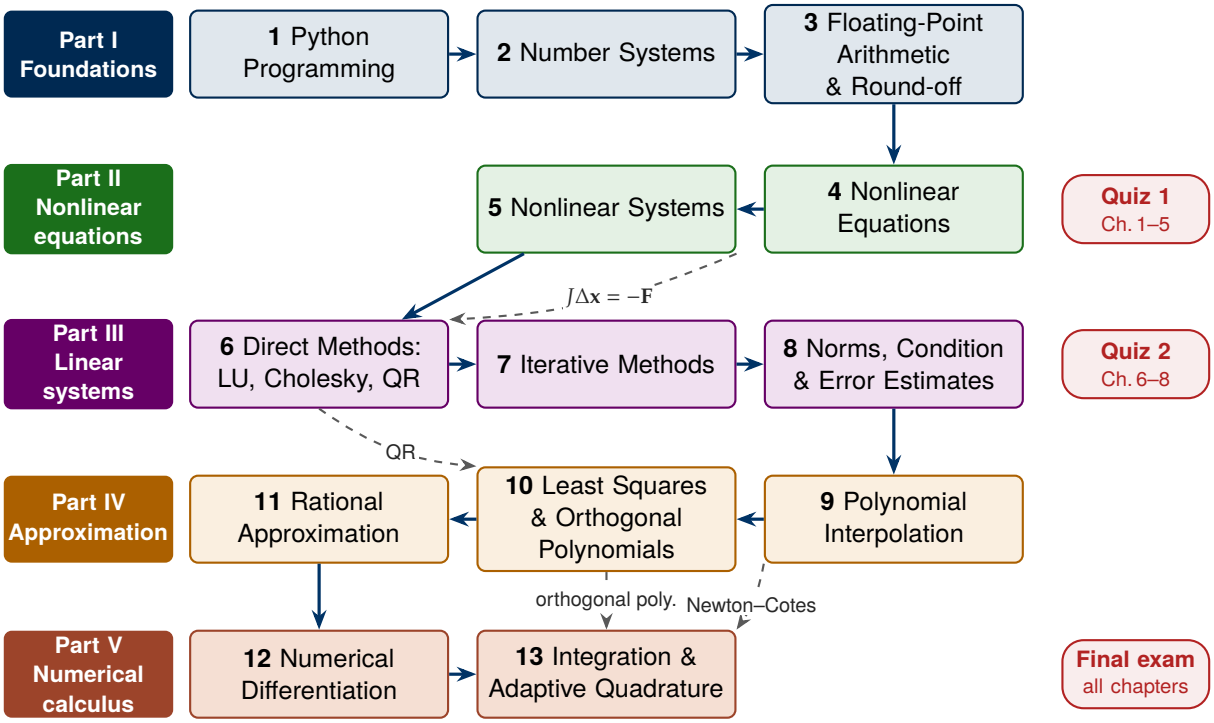
Acknowledgements

This is a work in progress. Firstly, thank you to all the students who painfully sat through the lectures. Also to Joshua Badu, who took time to type out the first version. Then my teaching assistant Aaron Antwi, who did a lot of proof-reading and editing towards the previous version, and my current teaching assistant Solomon Aikins.

Stephen E. Moore
Cape Coast, 2026

Course Map

The thirteen chapters are grouped into five parts that follow the MAT 407 course outline. Solid arrows show the order of study; dashed arrows show an important idea that is used again later. Python (Chapter 1) is used in every chapter.



Part	Chapters	Central question
I Foundations	1–3	How does a computer represent numbers, and which errors does this cause?
II Nonlinear equations	4–5	How do we solve $f(x) = 0$ and $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ quickly and reliably?
III Linear systems	6–8	How do we solve $A\mathbf{x} = \mathbf{b}$ for small dense and large sparse A, and how accurate is the answer?
IV Approximation	9–11	How can a complicated function, or noisy data, be replaced by a simple one?
V Numerical calculus	12–13	How do we differentiate and integrate using only function values?

Suggested use. Read each chapter with the Python listings open and run them; a chapter is not finished until you can reproduce its tables. Work at least the Theory and Python exercises of every section. Students aiming at graduate study should also read the *Towards Graduate Studies* boxes and attempt the ★ Graduate exercises.

Chapter 1

Introduction to Python Programming

Learning Objectives

After studying this chapter you should be able to:

- run Python programs in a script, in the interactive shell and in a Jupyter notebook;
- use Python numbers, strings, lists and `f`-strings for formatted output;
- create and manipulate vectors and matrices with NumPy;
- write `for`/`while` loops, `if` statements and your own functions;
- produce publication-quality plots with Matplotlib;
- translate a simple mathematical model (the falling bungee jumper) into a numerical algorithm, and observe how its error depends on the step size.

Numerical analysis is a subject about *algorithms*, and an algorithm is only really understood when you have implemented it and watched it work (or fail!). Throughout these notes we use the programming language Python together with three libraries: **NumPy** (arrays and linear algebra), **SciPy** (scientific algorithms) and **Matplotlib** (plotting).

Historical Note: From Fortran to Python

The first high-level programming language, **Fortran** ("FORmula TRANslation"), was created by John Backus and his team at IBM in 1957 precisely to do numerical computation. In the 1970s the numerical analyst **Cleve Moler** wrote **MATLAB** ("MATrix LABoratory") so that his students could use the Fortran linear-algebra libraries LINPACK and EISPACK without writing Fortran; GNU **Octave**, used in earlier versions of this course, is a free MATLAB clone. **Python** was created by the Dutch programmer **Guido van Rossum** and first released in 1991. It became a scientific language with the array package Numeric (1995), which Travis Oliphant merged with Numarray to create **NumPy** in 2006. SciPy appeared in 2001, and John D. Hunter wrote **Matplotlib** in 2003 to imitate MATLAB's plotting. Today the same Python stack is used to analyse data from the Large Hadron Collider, to process the first image of a black hole (2019), and to train machine-learning models. Under the surface, `numpy.linalg` still calls the Fortran libraries BLAS and LAPACK: the work of numerical analysts from the 1970s–1990s is inside every modern computer.

1.1 Getting started

There are three common ways of working with Python.

1. **Interactive shell.** Type `python` in a terminal and enter commands one at a time; useful as a calculator.
2. **Scripts.** Save commands in a text file such as `bisection.py` and run it with `python`

`bisection.py`, or with the Run button of an editor such as Spyder, VS Code or Thonny. Every listing in these notes is a script.

3. **Notebooks.** Jupyter notebooks (and Google Colab, which runs in a web browser without any installation) mix code, output, plots and text in one document. They are excellent for experiments and assignments.

To use a library we must *import* it. The standard abbreviations, used everywhere in these notes, are

```
import numpy as np
import matplotlib.pyplot as plt
import scipy
```

1.1.1 Numbers, variables and strings

Python has integers (`int`), floating-point numbers (`float`) and complex numbers (`complex`). A variable is created simply by assigning to it.

Listing 1.1: Numbers, the `math` module and strings `ch01/basics.py`

```
# Basic Python: numbers, variables, strings and printing
import math

a = 7            # an int
b = 2.5          # a float
c = 3 + 4j       # a complex number
print(type(a), type(b), type(c))

print("a / 2  =", a / 2)      # true division
print("a // 2 =", a // 2)     # integer (floor) division
print("a % 2  =", a % 2)      # remainder
print("a ** 2 =", a ** 2)     # power
print("|c|    =", abs(c))

# The math module
print("pi      =", math.pi)
print("sqrt(2) =", math.sqrt(2))
print("e^1     =", math.exp(1))

# Strings
name = "Numerical Analysis"
print(name.upper())
print(name.lower())
print(len(name), "characters")
print(name[0:9])          # slicing: characters 0,...,8
print(name.split())       # split into words
print(name.replace("Analysis", "Methods"))

# f-strings: formatted output (very useful for tables)
x = math.pi
print(f"pi to 4 decimal places: {x:.4f}")
print(f"pi in scientific form : {x:.6e}")
print(f"pi in a field of 12   : |{x:12.5f}|")
```

Output

```
<class 'int'> <class 'float'> <class 'complex'>
a / 2  = 3.5
a // 2 = 3
a % 2  = 1
a ** 2 = 49
|c|    = 5.0
pi      = 3.141592653589793
sqrt(2) = 1.4142135623730951
e^1     = 2.718281828459045
NUMERICAL ANALYSIS
numerical analysis
18 characters
Numerical
['Numerical', 'Analysis']
Numerical Methods
pi to 4 decimal places: 3.1416
pi in scientific form : 3.141593e+00
pi in a field of 12   : |     3.14159|
```

△ Caution: integer and true division

In Python 3, `7/2` is `3.5` but `7//2` is `3`. The operator for powers is `**`, *not* `^` (in Python `^` is the "exclusive or" of the bits of two integers, so `2^3` gives `1`!).

★ Key Concept: formatted output

Numerical results should be printed with a controlled number of digits. In an `f`-string, `{x:10.6f}` prints `x` in a field of width 10 with 6 digits after the decimal point, and `{x:.3e}` prints it in scientific notation with 3 digits after the point. We use this to print iteration tables throughout the course.

Exercises

1.1.1 Python Use Python as a calculator to evaluate (a) $\sqrt{17} + \ln 5$, (b) $e^{\pi} - \pi^{e}$, (c) $(1+i)^{10}$, (d) $\sin(10^{6})$. Print each answer to 8 significant digits.

1.1.2 Python Given `s = "MAT 407 Numerical Analysis"`, write commands to (a) extract the course code `MAT 407`; (b) count the letters `a` (both cases) using `s.lower().count("a")`; (c) print the string backwards using the slice `s[::-1]`.

1.1.3 Python Print a table of n, n^2, n^3 and $\sqrt{n}$ for $n = 1, \dots, 10$ with neatly aligned columns using `f`-strings.

1.1.4 Theory What is printed by `print(0.1 + 0.2 == 0.3)`? Try it, and write down a guess for the reason. (We explain it completely in Chapter 3.)

1.2 Vectors with NumPy

A NumPy *array* is the basic object of numerical computing. Unlike a Python list, all its elements have the same type and arithmetic acts *element by element*.

Listing 1.2: Creating and manipulating vectors ch01/vectors.py

```python
# Vectors with NumPy
import numpy as np

L = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("L          =", L)
print("L.shape    =", L.shape, " len(L) =", len(L))

K = L.reshape(-1, 1)          # a column vector (10 x 1)
print("K.shape    =", K.shape)

s = np.linspace(1, 10, 5)     # 5 equally spaced points from 1 to 10
print("linspace   =", s)
V = np.arange(1, 21, 1)       # 1,2,...,20   (the end point is excluded!)
B = np.arange(0, 21, 2)       # 0,2,...,20
print("arange     =", V)
print("even       =", B)

A = np.array([1, 2, 4, 8])
Bv = np.array([-1, 2, 1, 0])
print("A + Bv     =", A + Bv)
print("A - Bv     =", A - Bv)
print("A * Bv     =", A * Bv, "  (element-wise)")
print("A @ Bv     =", A @ Bv, "  (dot product)")
print("2*A        =", 2 * A)
print("A**2       =", A ** 2)
print("sqrt(A)    =", np.sqrt(A))

# Indexing starts at 0 in Python (it starts at 1 in MATLAB/Octave)
print("A[0], A[-1] =", A[0], A[-1])
print("A[1:3]      =", A[1:3])

# Concatenation
F = np.concatenate([A, Bv])
print("F          =", F)
# Deleting an element
O = np.array([1, -1, 0, 2, 0])
O = np.delete(O, 2)           # remove the third element (index 2)
print("O          =", O)
print("sum, max, min, mean of A:", A.sum(), A.max(), A.min(), A.mean())
```

Output

```
L          = [ 1  2  3  4  5  6  7  8  9 10]
L.shape    = (10,)  len(L) = 10
K.shape    = (10, 1)
linspace   = [ 1.    3.25  5.5   7.75 10.  ]
arange     = [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
even       = [ 0  2  4  6  8 10 12 14 16 18 20]
A + Bv     = [0 4 5 8]
A - Bv     = [2 0 3 8]
A * Bv     = [-1  4  4  0]   (element-wise)
A @ Bv     = 7   (dot product)
2*A        = [ 2  4  8 16]
A**2       = [ 1  4 16 64]
```

```
sqrt(A)    = [1.         1.41421356 2.         2.82842712]
A[0], A[-1] = 1 8
A[1:3]      = [2 4]
F          = [ 1  2  4  8 -1  2  1  0]
O          = [ 1 -1  2  0]
sum, max, min, mean of A: 15 8 1 3.75
```

Note the difference between `np.linspace(a, b, n)`, which returns n equally spaced points *including* both end points, and `np.arange(a, b, h)`, which uses the step h and *excludes* the end point b.

⋆ Key Concept: vectorisation

Operations such as `A + B`, `2*A`, `np.sqrt(A)` and `A @ B` act on whole arrays at once. Such *vectorised* code is shorter, is closer to mathematical notation, and is typically 10–100 times faster than an equivalent Python `for` loop, because the loop is carried out in compiled C code inside NumPy.

△ Caution: indexing starts at zero

The first element of a Python array is `A[0]` and the last is `A[-1]`. The slice `A[i:j]` contains `A[i]`, …, `A[j-1]`, and not `A[j]`. Mathematical formulas are usually written with indices $1, \dots, n$, so be careful when you translate an algorithm into Python: x_i becomes `x[i-1]`.

Exercises

1.2.1 Python Create the vector $\mathbf{x} = (0, 0.1, 0.2, \dots, 2)$ in two different ways and verify that they agree using `np.allclose`.

1.2.2 Python Let $\mathbf{u} = (1, -2, 3)$ and $\mathbf{v} = (4, 0, -1)$. Compute $\mathbf{u} + \mathbf{v}$, $\mathbf{u} \cdot \mathbf{v}$, the element-wise product, $\|\mathbf{u}\|_2 = \sqrt{\mathbf{u} \cdot \mathbf{u}}$ and the cross product (`np.cross`).

1.2.3 Python Without a loop, compute $\sum_{k=1}^{1000} 1/k^2$ and compare it with $\pi^2/6$. How many correct digits do you obtain? (This is Euler's famous solution of the *Basel problem*, 1734.)

1.3 Matrices

A matrix is a two-dimensional array. The operator `@` is matrix multiplication, while `*` multiplies element by element.

Listing 1.3: Matrix operations, errors and linear systems ch01/matrices.py

```python
# Matrices with NumPy
import numpy as np

A = np.array([1, 2, 4, 8])
O = np.array([1, -1, 2, 0])
B = np.array([-1, 2, 1, 0])
H = np.vstack([A, O, B])          # stack rows
print("H =\n", H)
print("H.shape =", H.shape)

try:
```

```
    np.vstack([A, np.array([1, -1, 0, 2, 0]), B])
except ValueError as err:
    print("Error:", err)

Z = np.array([[1, -1], [0, 2]])
I2 = np.array([[1, 2], [-3, 4]])
print("Z^T =\n", Z.T)
print("2Z =\n", 2 * Z)
print("Z + I2 =\n", Z + I2)
print("Z @ I2 (matrix product) =\n", Z @ I2)
print("Z * I2 (element-wise)   =\n", Z * I2)

Mq = np.array([[2, 1], [-3, 6], [1, -1], [0, 2]])
print("Mq.shape =", Mq.shape)
print("Mq[3, 0] =", Mq[3, 0], "   row 2:", Mq[2, :], "   column 1:", Mq[:, 1])
try:
    np.linalg.inv(Mq)
except np.linalg.LinAlgError as err:
    print("Error:", err)

print("inv(I2) =\n", np.linalg.inv(I2))
print("det(I2) =", np.linalg.det(I2))

# Special matrices
print("eye(3) =\n", np.eye(3))
print("zeros((2,3)) =\n", np.zeros((2, 3)))
print("diag([1,2,3]) =\n", np.diag([1, 2, 3]))

# Solving a linear system  I2 x = b  (better than computing the inverse!)
b = np.array([5.0, 5.0])
x = np.linalg.solve(I2, b)
print("solution of I2 x = b:", x)
```

Output

```
H =
 [[ 1  2  4  8]
 [ 1 -1  2  0]
 [-1  2  1  0]]
H.shape = (3, 4)
Error: all the input array dimensions for the concatenation axis must match exactly, but
    ↪ along dimension 1, the array at index 0 has size 4 and the array at index 1 has size
    ↪  5
Z^T =
 [[ 1  0]
 [-1  2]]
2Z =
 [[ 2 -2]
 [ 0  4]]
Z + I2 =
 [[ 2  1]
 [-3  6]]
Z @ I2 (matrix product) =
 [[ 4 -2]
 [-6  8]]
Z * I2 (element-wise)   =
```

```
 [[ 1 -2]
 [ 0  8]]
Mq.shape = (4, 2)
Mq[3, 0] = 0    row 2: [ 1 -1]    column 1: [ 1  6 -1  2]
Error: Last 2 dimensions of the array must be square
inv(I2) =
 [[ 0.4 -0.2]
 [ 0.3  0.1]]
det(I2) = 10.000000000000002
eye(3) =
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
zeros((2,3)) =
 [[0. 0. 0.]
 [0. 0. 0.]]
diag([1,2,3]) =
 [[1 0 0]
 [0 2 0]
 [0 0 3]]
solution of I2 x = b: [1. 2.]
```

Look carefully at `det(I2)`: the exact determinant of $\left(\begin{smallmatrix}1&2\\-3&4\end{smallmatrix}\right)$ is 10, but the computer prints `10.000000000000002`. This tiny discrepancy is a *rounding error*, the subject of Chapter 3.

⋆ Key Concept: never invert a matrix to solve a system

To solve $A\mathbf{x} = \mathbf{b}$ use `np.linalg.solve(A, b)`, *not* `np.linalg.inv(A) @ b`. Solving by elimination is about three times cheaper and is more accurate. Chapter 6 explains why.

Exercises

1.3.1 Python Enter $A = \left(\begin{smallmatrix}4&-1&0\\-1&4&-1\\0&-1&4\end{smallmatrix}\right)$ and $\mathbf{b} = (2, 4, 10)^T$. Compute A^T, A^2, $\det A$, the eigenvalues (`np.linalg.eigvals`) and the solution of $A\mathbf{x} = \mathbf{b}$. Check the solution by computing the residual $\mathbf{b} - A\mathbf{x}$.

1.3.2 Python Build the 5×5 matrix with 2 on the diagonal and -1 on the sub- and super-diagonals using `np.diag` three times. (This matrix appears again in Chapter 7.)

1.3.3 Theory Explain the difference between `A*B`, `A@B` and `A.dot(B)` for two 2×2 arrays.

1.4 Control flow: loops and decisions

Iterative algorithms, which make up most of this course, repeat a computation until a stopping criterion is satisfied. In Python this is done with **`for`** and **`while`** loops. Blocks of code are defined by *indentation* (four spaces), not by `end` as in MATLAB.

Listing 1.4: Loops, conditions and list comprehensions `ch01/loops.py`

```
# Control flow: for loops, while loops and if statements
import numpy as np

v = np.array([1, 2, 3, 4, 5, 6])
```

```
5
6  # Sum of neighbouring entries  S[i] = v[i] + v[i+1]
7  S = np.zeros(len(v) - 1)
8  for i in range(len(v) - 1):
9      S[i] = v[i] + v[i + 1]
10 print("S (loop)       =", S)
11 print("S (vectorised) =", v[:-1] + v[1:])
12
13 # Accumulating a sum
14 total = 0
15 for value in v:
16     total = total + value
17 print("sum with loop =", total, "  np.sum =", np.sum(v))
18
19 # while loop: how many times can we halve 1 before it becomes < 1e-3?
20 x, n = 1.0, 0
21 while x >= 1e-3:
22     x = x / 2
23     n += 1
24 print("halvings needed:", n, " final x =", x)
25
26 # if / elif / else
27 for t in [-2, 0, 3]:
28     if t < 0:
29         print(t, "is negative")
30     elif t == 0:
31         print(t, "is zero")
32     else:
33         print(t, "is positive")
34
35 # list comprehension: squares of odd numbers below 10
36 print([k**2 for k in range(10) if k % 2 == 1])
37
38 # Partial sums of the harmonic series  1 + 1/2 + ... + 1/n
39 for n in [10, 100, 1000, 10000]:
40     print(f"n = {n:6d}   H_n = {sum(1 / k for k in range(1, n + 1)):.6f}")
```

Output

```
S (loop)       = [ 3.  5.  7.  9. 11.]
S (vectorised) = [ 3  5  7  9 11]
sum with loop = 21   np.sum = 21
halvings needed: 10  final x = 0.0009765625
-2 is negative
0 is zero
3 is positive
[1, 9, 25, 49, 81]
n =     10   H_n = 2.928968
n =    100   H_n = 5.187378
n =   1000   H_n = 7.485471
n =  10000   H_n = 9.787606
```

The last lines show the harmonic numbers $H_n = \sum_{k=1}^{n} 1/k$. They grow without bound, but so slowly (like $\ln n + 0.5772\ldots$) that a computer can never "see" the divergence by summing terms: a warning that numerical experiments must be combined with analysis.

★ Key Concept: the shape of an iterative algorithm

Almost every algorithm in this course has the form

```
x = initial_guess
for k in range(max_iter):
    x_new = update(x)
    if stopping_criterion(x_new, x):
        break
    x = x_new
```

Always include a maximum number of iterations, so that a method which does not converge cannot run forever.

Exercises

1.4.1 Python Use a `while` loop to find the smallest n such that $n! > 10^{10}$.

1.4.2 Python The Fibonacci numbers satisfy $F_1 = F_2 = 1$, $F_{n+1} = F_n + F_{n-1}$. Print the first 30 of them and the ratios F_{n+1}/F_n. To what number do the ratios converge? (We meet it again as the order of convergence of the secant method in Chapter 4.)

1.4.3 Python Babylonian square root: $x_{k+1} = \tfrac{1}{2}(x_k + 2/x_k)$, $x_0 = 1$. Iterate until $|x_{k+1} - x_k| < 10^{-12}$ and print each iterate with 15 digits. Count how many correct digits you gain per step. (This ancient method is Newton's method in disguise.)

1.5 Functions

A function packages an algorithm so that it can be reused. In numerical analysis we often pass *functions as arguments*: a root finder receives the function whose root is sought; an integrator receives the integrand.

Listing 1.5: User-defined functions and functions as arguments ch01/functions.py

```
# Functions, default arguments, lambda and passing functions as arguments
import math


def quadratic_roots(a, b, c):
    """Return the two roots of a x^2 + b x + c = 0 (real or complex)."""
    d = b * b - 4 * a * c
    sq = math.sqrt(d) if d >= 0 else complex(0, math.sqrt(-d))
    return (-b + sq) / (2 * a), (-b - sq) / (2 * a)


print("roots of x^2 - 3x + 2:", quadratic_roots(1, -3, 2))
print("roots of x^2 + 2x + 5:", quadratic_roots(1, 2, 5))


def forward_difference(f, x, h=1e-5):
    """Approximate f'(x) by (f(x+h) - f(x))/h.  f is itself a function!"""
    return (f(x + h) - f(x)) / h


square = lambda t: t**2                 # a small anonymous function
print("d/dx x^2 at x=3   ~", forward_difference(square, 3.0))
print("d/dx sin at x=0   ~", forward_difference(math.sin, 0.0))
```

```
print("with h = 0.1      ~", forward_difference(math.sin, 0.0, h=0.1))
```

Output

```
roots of x^2 - 3x + 2: (2.0, 1.0)
roots of x^2 + 2x + 5: ((-1+2j), (-1-2j))
d/dx x^2 at x=3    ~ 6.000009999951316
d/dx sin at x=0    ~ 0.9999999999833332
with h = 0.1       ~ 0.9983341664682815
```

Notice the last two lines: $\frac{d}{dx}\sin x$ at $x = 0$ is exactly 1, the forward difference with $h = 10^{-5}$ is correct to about 10 digits, but with $h = 0.1$ only to about 3 digits. How to choose h is studied in Chapter 12.

△ Caution: write a docstring

Every function you write should begin with a *docstring* (the text in triple quotes) saying what it computes, what the arguments are, and what is returned. In a few weeks' time you will not remember, and neither will anyone else who reads your code.

Exercises

1.5.1 Python Write a function `taylor_exp(x, n)` that returns $\sum_{k=0}^{n} x^k/k!$. Compare with `math.exp(x)` for $x = 1, 5, -5, -20$ and $n = 10, 20, 50, 100$. Why is the result for $x = -20$ so poor? (Hint: look at the size of the terms; see Chapter 3.)

1.5.2 Python Write a function `horner(a, x)` that evaluates the polynomial $a_0 + a_1 x + \cdots + a_n x^n$ by nested multiplication $a_0 + x(a_1 + x(a_2 + \cdots + x a_n))$. Count the multiplications used compared with the obvious method.

1.5.3 Theory The function `quadratic_roots` in Listing 1.5 uses the school formula. For $a = 1$, $b = 10^8$, $c = 1$ one root is computed very inaccurately. Find out which one and propose a better formula (answer in Chapter 3).

1.6 Plotting with Matplotlib

A picture of a function is the first step in every root-finding or approximation problem.

Listing 1.6: Plotting functions ch01/plotting.py

```
# A first plot with Matplotlib
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2 * np.pi, 2 * np.pi, 400)
plt.figure(figsize=(6, 3.4))
plt.plot(x, np.sin(x), label=r"$\sin x$")
plt.plot(x, np.cos(x), "--", label=r"$\cos x$")
plt.plot(x, x - x**3 / 6, ":", label=r"$x - x^3/6$ (Taylor)")
plt.ylim(-2, 2)
plt.xlabel("x")
plt.ylabel("y")
plt.title("sin, cos and a Taylor polynomial")
```

```
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch01_plot.pdf")     # save to a file ...
plt.show()                       # ... and/or display on the screen
```

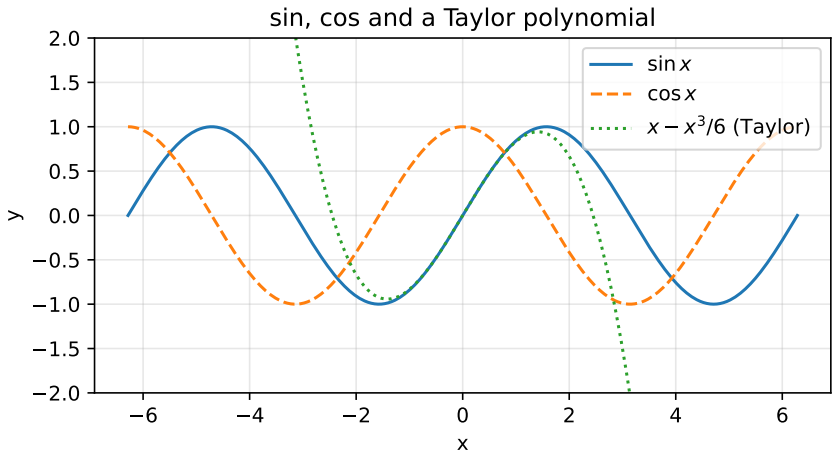


Figure 1.1: Output of Listing 1.6. The Taylor polynomial $x - x^3/6$ is an excellent approximation of $\sin x$ near 0 but useless far away from 0.

Exercises

1.6.1 Python Plot $f(x) = x^3 - x^2 - x - 1$ on $[-1, 3]$ together with the line $y = 0$. Read off an interval containing the real root. (This function is used in Chapter 4.)

1.6.2 Python Plot $y = e^{-x}$ and $y = x$ on $[0, 1]$. Where do they intersect? What equation does the intersection point satisfy?

1.6.3 Python Use `plt.semilogy` to plot 2^{-n}, n^{-2} and 2^{-2^n} for $n = 1, \ldots, 10$. These three sequences represent linear, sublinear and quadratic convergence (Section 4.3).

1.7 Case study: the falling bungee jumper

This example, taken from Chapra [3], shows the whole process of scientific computing: a physical law, a mathematical model, an exact solution, a numerical algorithm, and an analysis of the error.

The model. A jumper of mass m falls under gravity g and experiences air resistance proportional to the square of the velocity, with drag coefficient c_d. Newton's second law gives the initial-value problem

$$\frac{dv}{dt} = g - \frac{c_d}{m} v^2, \qquad v(0) = 0. \tag{1.1}$$

Separating variables one can show that the exact solution is

$$v(t) = \sqrt{\frac{gm}{c_d}} \tanh\left(\sqrt{\frac{gc_d}{m}}\, t \right), \tag{1.2}$$

which tends to the *terminal velocity* $\sqrt{gm/c_d}$ as $t \to \infty$.

A numerical method. Suppose we could not solve (1.1) exactly (which is the usual situation for realistic models). Replace the derivative by a difference quotient on a grid $t_i = ih$:

$$\frac{v_{i+1} - v_i}{h} \approx g - \frac{c_d}{m} v_i^2 \quad \Longrightarrow \quad v_{i+1} = v_i + h\left(g - \frac{c_d}{m} v_i^2\right).$$

This is **Euler's method** (Leonhard Euler, 1768).

Listing 1.7: Bungee jumper: Euler's method against the exact solution ch01/bungee.py

```
# Bungee jumper: analytical velocity and Euler's method
import numpy as np
import matplotlib.pyplot as plt

g, m, cd = 9.81, 68.1, 0.25          # gravity (m/s^2), mass (kg), drag (kg/m)


def v_exact(t):
    """Analytical velocity v(t) = sqrt(g m/cd) tanh( sqrt(g cd/m) t )."""
    return np.sqrt(g * m / cd) * np.tanh(np.sqrt(g * cd / m) * t)


def v_euler(t_end, h):
    """Euler's method for dv/dt = g - (cd/m) v^2, v(0) = 0."""
    n = int(round(t_end / h))
    t = np.linspace(0, t_end, n + 1)
    v = np.zeros(n + 1)
    for i in range(n):
        v[i + 1] = v[i] + h * (g - cd / m * v[i] ** 2)
    return t, v


t, v = v_euler(12, 2.0)
print(" t (s)   Euler v   exact v   error")
for ti, vi in zip(t, v):
    print(f"{ti:5.1f} {vi:9.4f} {v_exact(ti):9.4f} {abs(vi - v_exact(ti)):8.4f}")
print("terminal velocity sqrt(g m/cd) =", np.sqrt(g * m / cd))

# The error at t = 12 s decreases in proportion to the step size h
for h in [2.0, 1.0, 0.5, 0.25, 0.125]:
    t, v = v_euler(12, h)
    print(f"h = {h:6.3f}   error at t = 12: {abs(v[-1] - v_exact(12)):.6f}")

tt = np.linspace(0, 12, 200)
t, v = v_euler(12, 2.0)
plt.figure(figsize=(6, 3.6))
plt.plot(tt, v_exact(tt), label="exact")
plt.plot(t, v, "o--", label="Euler, h = 2")
plt.xlabel("t (s)")
plt.ylabel("v (m/s)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch01_bungee.pdf")
```

Output

```
 t (s)   Euler v   exact v   error
  0.0    0.0000    0.0000   0.0000
  2.0   19.6200   18.7292   0.8908
  4.0   36.4137   33.1118   3.3019
  6.0   46.2983   42.0762   4.2221
  8.0   50.1802   46.9575   3.2227
 10.0   51.3123   49.4214   1.8909
 12.0   51.6008   50.6175   0.9833
terminal velocity sqrt(g m/cd) = 51.69375204026111
h =  2.000   error at t = 12: 0.983305
h =  1.000   error at t = 12: 0.583336
h =  0.500   error at t = 12: 0.308428
h =  0.250   error at t = 12: 0.157841
h =  0.125   error at t = 12: 0.079766
```

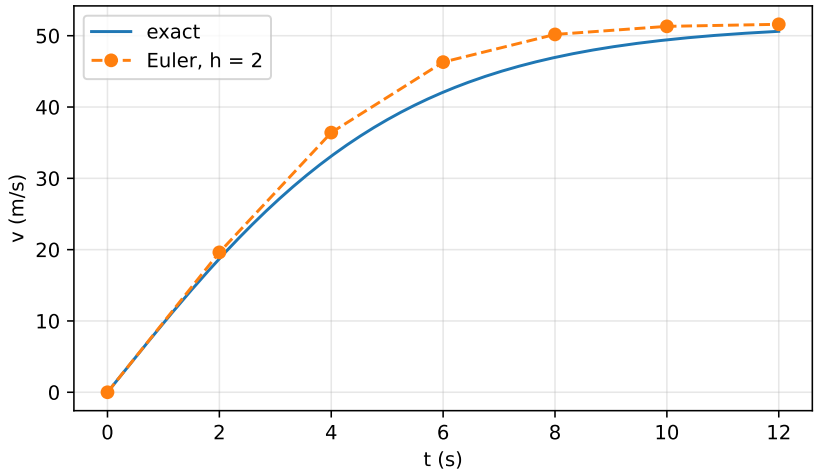


Figure 1.2: Velocity of the bungee jumper ($m = 68.1\,\text{kg}$, $c_d = 0.25\,\text{kg/m}$).

The second table is the important one. Each time h is halved, the error at $t = 12\,\text{s}$ is (approximately) halved: the error behaves like Ch. We say Euler's method is *first-order accurate*. This is our first encounter with the main themes of the course: *approximation*, *error* and *rate of convergence*.

⋆ Key Concept: sources of error

A computed answer differs from the true answer for several reasons:

1. **modelling error**: the equation (1.1) is only a model of reality;
2. **data error**: m, c_d and g are measured quantities;
3. **truncation (discretisation) error**: replacing a derivative by a difference quotient, an infinite series by a finite sum, etc. (here $\approx Ch$);
4. **rounding error**: the computer stores only about 16 significant digits.

Numerical analysis is mainly concerned with (3) and (4), and with how (2) is amplified.

Towards Graduate Studies: numerical ODEs

Euler's method is the simplest member of large families of methods (Runge–Kutta, linear multistep, implicit methods) for ordinary differential equations. At graduate level one studies their *consistency*, *stability* and *convergence*, and the celebrated *Dahlquist equivalence theorem* (1956): a consistent linear multistep method converges if and only if it is zero-stable. The same "consistency + stability $\Rightarrow$ convergence" principle, known for PDEs as the *Lax equivalence theorem*, is one of the deepest ideas in numerical analysis. In Python, `scipy.integrate.solve_ivp` provides professional adaptive ODE solvers. See Hairer, Nørsett & Wanner [10].

Exercises

1.7.1 Python Modify Listing 1.7 to find how small h must be for the error at $t = 12$ to be below 10^{-3}. Estimate the answer first from the table above.

1.7.2 Python The jumper's position satisfies $dx/dt = v$. Extend the Euler scheme to compute $x(t)$ and determine (approximately) the distance fallen after 10 s.

1.7.3 Python Solve (1.1) with `scipy.integrate.solve_ivp` and compare with the exact solution at $t = 12$.

1.7.4 Theory Verify by differentiation that (1.2) satisfies (1.1).

1.7.5 ★ Graduate Using Taylor's theorem, show that the *local* error of one Euler step is $O(h^2)$, and argue informally why the *global* error after T/h steps is $O(h)$.

Chapter Summary

- Python with NumPy/SciPy/Matplotlib is a complete environment for numerical computing.
- Arrays support vectorised arithmetic; `@` is matrix multiplication; indices start at 0.
- Iterative algorithms are loops with a stopping test *and* an iteration limit.
- Solve linear systems with `np.linalg.solve`, not with the inverse.
- Numerical answers contain modelling, data, truncation and rounding errors. Halving h halves the error of a first-order method.

Chapter 2

Number Systems

Learning Objectives

After studying this chapter you should be able to:

- explain positional notation in an arbitrary base β;
- convert integers and fractions between bases 2, 8, 10 and 16, by hand and in Python;
- justify the repeated-division and repeated-multiplication algorithms;
- explain why numbers such as 0.1 cannot be stored exactly in binary;
- describe how a computer stores integers (two's complement) and what overflow is.

Humans count in *decimal* (base 10), probably because we have ten fingers, and still use *duodecimal* (base 12: a dozen, 12 months, 12 inches) and *sexagesimal* (base 60: minutes, seconds, degrees) for time and angles. Computers are built from transistors, electronic switches with two states, ON and OFF, and so they use the *binary* system (base 2). Because long strings of bits are hard for people to read, programmers group them into *octal* (base 8) or *hexadecimal* (base 16) digits.

Historical Note: zero, place value and the binary system

The Babylonians (c. 2000 BC) had a positional base-60 system, which is why an hour has 60 minutes; the Maya of Central America used base 20. Our decimal place-value system with a symbol for zero comes from India: the rules of arithmetic with zero were written down by **Brahmagupta** in 628 AD. It reached the Islamic world through the Baghdad scholar **Muhammad ibn Mūsā al-Khwārizmī** (c. 780–850), whose name gives us the word *algorithm* and whose book title (*al-jabr*) gives us *algebra*. **Leonardo of Pisa (Fibonacci)** brought the "Hindu–Arabic" numerals to Europe in his *Liber Abaci* (1202). **Gottfried Wilhelm Leibniz** published a systematic account of binary arithmetic in 1703, and was delighted to find the same patterns in the ancient Chinese *I Ching*. In 1937 **Claude Shannon** showed in his Master's thesis that Boolean algebra (George Boole, 1854) could describe electrical switching circuits, which is the theoretical foundation of every digital computer. Hexadecimal notation became standard with the IBM System/360 computers of 1964.

2.1 Positional notation in base β

Definition 2.1 (base-β representation)**.** Let $\beta \geq 2$ be an integer. The digit string

$(d_n d_{n-1} \cdots d_1 d_0 . d_{-1} d_{-2} \cdots)_\beta$, with digits $d_k \in \{0, 1, \ldots, \beta - 1\}$, represents the real number

$$x = \sum_{k=-\infty}^{n} d_k \, \beta^k = d_n \beta^n + \cdots + d_1 \beta + d_0 + d_{-1} \beta^{-1} + d_{-2} \beta^{-2} + \cdots . \tag{2.1}$$

The part before the point is the *integer part*; the part after it is the *fractional part*. The digit d_n is the *most significant* digit.

Example 2.2 (decimal, binary and hexadecimal)**.**

$$\begin{aligned}
(735)_{10} &= 7 \times 10^2 + 3 \times 10^1 + 5 \times 10^0, \\
(10110)_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 16 + 4 + 2 = 22, \\
(\mathrm{A3E})_{16} &= 10 \times 16^2 + 3 \times 16^1 + 14 \times 16^0 = 2560 + 48 + 14 = 2622, \\
(0.101)_2 &= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0.5 + 0.125 = 0.625.
\end{aligned}$$

In hexadecimal the digits $10, \ldots, 15$ are written A, B, C, D, E, F. In Python (and C, MATLAB, ...) hexadecimal constants carry the prefix `0x`, binary ones `0b` and octal ones `0o`: `0xA3E == 2622` is `True`.

A binary digit is called a **bit**; eight bits form a **byte**. One hexadecimal digit corresponds to exactly four bits (a "nibble"), because $16 = 2^4$; one octal digit corresponds to three bits.

Dec	Bin	Oct	Hex	Dec	Bin	Oct	Hex
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

2.1.1 From base β to decimal: Horner's rule

To evaluate (2.1) for an integer, it is wasteful to compute every power β^k. Instead write

$$(d_n d_{n-1} \cdots d_0)_\beta = \big(\cdots \big((d_n \beta + d_{n-1})\beta + d_{n-2}\big)\beta + \cdots\big)\beta + d_0 .$$

Starting with $v = 0$, for each digit from left to right set $v \leftarrow v\beta + d$. This *nested multiplication* (Horner's rule) uses only n multiplications. For example $(\mathrm{A3E})_{16}$: $v = 10$, then $v = 10{\cdot}16{+}3 = 163$, then $v = 163 \cdot 16 + 14 = 2622$.

2.1.2 From decimal to base β: repeated division

Theorem 2.3 (repeated division)**.** *Let N be a positive integer. Divide N by β to get $N = \beta q_0 + r_0$ with $0 \le r_0 < \beta$, then divide q_0 by β, and so on, until a quotient is zero. Then the remainders are the digits of N in base β, read from the last to the first: $N = (r_m r_{m-1} \cdots r_1 r_0)_\beta$.*

Proof. If $N = (d_m \cdots d_1 d_0)_\beta$ then $N = \beta\,(d_m \beta^{m-1} + \cdots + d_1) + d_0$ with $0 \le d_0 < \beta$. By uniqueness of quotient and remainder in the division algorithm, $r_0 = d_0$ and $q_0 = (d_m \cdots d_1)_\beta$. Repeating the argument on q_0 gives the other digits. The process stops because $q_0 < N$, $q_1 < q_0, \ldots$ is a strictly decreasing sequence of non-negative integers. □

Example 2.4. To convert 9 to binary: $9 = 2\cdot 4 + \mathbf{1}$, $4 = 2\cdot 2 + \mathbf{0}$, $2 = 2\cdot 1 + \mathbf{0}$, $1 = 2\cdot 0 + \mathbf{1}$. Reading the remainders upwards, $9 = (1001)_2$.

2.1.3 Fractions: repeated multiplication

If $0 \le x < 1$ and $x = (0.d_{-1}d_{-2}d_{-3}\cdots)_\beta$, then $\beta x = d_{-1} + (0.d_{-2}d_{-3}\cdots)_\beta$, so the integer part of βx is the first digit. Take the fractional part and repeat.

Example 2.5 (0.1 has no finite binary expansion)**.**

$$\begin{aligned} 0.1\times 2 &= 0.2 \;\to 0, & 0.6\times 2 &= 1.2 \;\to 1,\\ 0.2\times 2 &= 0.4 \;\to 0, & 0.2\times 2 &= 0.4 \;\to 0, \quad \text{(repeats)}\\ 0.4\times 2 &= 0.8 \;\to 0, & 0.4\times 2 &= 0.8 \;\to 0,\\ 0.8\times 2 &= 1.6 \;\to 1, & \dots & \end{aligned}$$

Hence $(0.1)_{10} = (0.\overline{00011})_2 = 0.000110011001100\ldots_2$, an *infinite* repeating expansion. A computer that stores a fixed number of bits cannot represent 0.1 exactly.

⋆ Key Concept: terminating expansions

A rational number p/q in lowest terms has a terminating expansion in base β if and only if every prime factor of q divides β. Since $10 = 2\cdot 5$ has the prime factor 5 that 2 lacks, numbers like $0.1 = 1/(2\cdot 5)$, 0.2 and 0.3 terminate in decimal but *not* in binary. This single fact explains why `0.1 + 0.2 == 0.3` is `False` in Python.

2.1.4 Binary ↔ octal ↔ hexadecimal

Because $16 = 2^4$, group binary digits in fours starting from the binary point and replace each group by one hex digit (use groups of three for octal):

$$(\mathrm{A3C5})_{16} = (\underbrace{1010}_{\mathrm{A}}\ \underbrace{0011}_{3}\ \underbrace{1100}_{\mathrm{C}}\ \underbrace{0101}_{5})_2, \qquad (1\,011\,011\,111)_2 = (1337)_8.$$

2.1.5 Python

The following listing implements the three algorithms and checks them against Python's built-in functions `bin`, `oct`, `hex` and `int(s, base)`.

Listing 2.1: Base conversion by repeated division, Horner's rule and repeated multiplication
ch02/base_conversion.py

```
# Conversion between number bases
DIGITS = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"


def to_base(n, beta):
    """Digits of the non-negative integer n in base beta (repeated division)."""
    if n == 0:
        return "0"
    digits = []
    while n > 0:
        n, r = divmod(n, beta)       # n = beta*q + r
        digits.append(DIGITS[r])     # remainders give the digits right-to-left
    return "".join(reversed(digits))


```

```
def from_base(s, beta):
    """Value of the digit string s in base beta, by Horner's rule."""
    value = 0
    for ch in s.upper():
        value = value * beta + DIGITS.index(ch)
    return value


def fraction_to_base(x, beta, ndigits=20):
    """First ndigits digits of 0 <= x < 1 in base beta (repeated multiplication)."""
    digits = []
    for _ in range(ndigits):
        x = x * beta
        d = int(x)
        digits.append(DIGITS[d])
        x = x - d
        if x == 0:
            break
    return "0." + "".join(digits)


print("Repeated division of 9 by 2:")
n = 9
while n > 0:
    print(f"   {n:3d} = 2 x {n // 2:3d} + {n % 2}")
    n //= 2
print("9 in binary       :", to_base(9, 2))
print("735 in binary     :", to_base(735, 2), "  built-in:", bin(735))
print("2622 in hex       :", to_base(2622, 16), "   built-in:", hex(2622))
print("2622 in octal     :", to_base(2622, 8), "  built-in:", oct(2622))
print("10110 (base 2)    =", from_base("10110", 2), "  built-in:", int("10110", 2))
print("A3E   (base 16)   =", from_base("A3E", 16), "  built-in:", int("A3E", 16))
print("A3C5  (base 16)   =", from_base("A3C5", 16))
print("A3C5 in binary    :", to_base(from_base("A3C5", 16), 2))
print("0.625 in binary   :", fraction_to_base(0.625, 2))
print("0.1   in binary   :", fraction_to_base(0.1, 2, 24), "... (repeats)")
print("1/3   in base 3   :", fraction_to_base(1 / 3, 3, 5))
```

Output

```
Repeated division of 9 by 2:
     9 = 2 x   4 + 1
     4 = 2 x   2 + 0
     2 = 2 x   1 + 0
     1 = 2 x   0 + 1
9 in binary       : 1001
735 in binary     : 1011011111   built-in: 0b1011011111
2622 in hex       : A3E    built-in: 0xa3e
2622 in octal     : 5076   built-in: 0o5076
10110 (base 2)    = 22   built-in: 22
A3E   (base 16)   = 2622   built-in: 2622
A3C5  (base 16)   = 41925
A3C5 in binary    : 1010001111000101
0.625 in binary   : 0.101
0.1   in binary   : 0.000110011001100110011001 ... (repeats)
1/3   in base 3   : 0.1
```

Exercises

2.1.1 Theory Convert to decimal: (a) $(110101)_2$; (b) $(777)_8$; (c) $(\mathrm{BEEF})_{16}$; (d) $(0.0111)_2$; (e) $(12.21)_3$.

2.1.2 Theory Convert $(2026)_{10}$ to bases 2, 8 and 16, by hand, and check with Python.

2.1.3 Theory Find the binary expansions of 0.3, 0.75 and $1/3$. Which are finite?

2.1.4 Theory Prove the Key Concept on terminating expansions for $\beta = 2$ and $\beta = 10$.

2.1.5 Python Extend `fraction_to_base` to detect a repeating block (store the remainders you have seen in a dictionary) and print the answer in the form $0.00\overline{011}$.

2.1.6 Theory Explain why multiplying a binary integer by 2 amounts to shifting all its bits one place to the left. What does shifting right do?

2.1.7 Theory Show that $(0.\overline{1})_2 = 1$ in the same way that $0.\overline{9} = 1$ in decimal.

2.2 Binary arithmetic

Arithmetic in base 2 follows the familiar school algorithms with the tables

$$0+0=0,\quad 0+1=1,\quad 1+1=10_2 \text{ (write 0, carry 1)},\qquad 0\times 1=0,\quad 1\times 1=1.$$

For example

$$\begin{array}{r} 1011_2 \\ +\ 0111_2 \\ \hline 10010_2 \end{array} \quad (11+7=18), \qquad \begin{array}{r} 101_2 \\ \times\ 11_2 \\ \hline 101 \\ 1010 \\ \hline 1111_2 \end{array} \quad (5\times 3=15).$$

Multiplication in binary is just "shift and add", which is why it is easy to build in hardware.

Exercises

2.2.1 Theory Compute in binary and check in decimal: (a) $11011_2 + 1101_2$; (b) $11011_2 - 1101_2$; (c) $1101_2 \times 101_2$; (d) $110110_2 \div 110_2$.

2.2.2 Theory Add $(3\mathrm{F})_{16} + (\mathrm{A7})_{16}$ directly in hexadecimal.

2.3 Integers in a computer

Hardware stores integers in a fixed number of bits: 8, 16, 32 or 64. With n bits there are 2^n patterns. Unsigned integers represent $0, \dots, 2^n - 1$. Signed integers are almost always stored in **two's complement**: the pattern $(b_{n-1}b_{n-2}\cdots b_0)_2$ represents

$$-b_{n-1}2^{n-1} + \sum_{k=0}^{n-2} b_k 2^k,$$

so the range is $-2^{n-1}, \dots, 2^{n-1} - 1$. To negate a number, invert all its bits and add one. The beauty of two's complement is that addition and subtraction use exactly the same circuit as for unsigned numbers.

Listing 2.2: Fixed-width integers, two's complement and overflow ch02/integers.py

```python
# Integers in a computer: fixed width, two's complement and overflow
import numpy as np

for bits, t in [(8, np.int8), (16, np.int16), (32, np.int32), (64, np.int64)]:
    info = np.iinfo(t)
    print(f"{bits:2d}-bit signed integers: {info.min} ... {info.max}")

# two's complement bit patterns of some 8-bit integers
for k in [5, 1, 0, -1, -5, 127, -128]:
    print(f"{k:5d} -> {np.binary_repr(k, width=8)}")

# overflow: 127 + 1 "wraps around" in 8-bit arithmetic
a = np.array([127], dtype=np.int8)
print("int8: 127 + 1 =", (a + np.int8(1))[0])
b = np.array([32767], dtype=np.int16)
print("int16: 32767 + 1 =", (b + np.int16(1))[0])
# Python's own integers have unlimited size
print("Python int: 2**100 =", 2**100)
```

Output

```
 8-bit signed integers: -128 ... 127
16-bit signed integers: -32768 ... 32767
32-bit signed integers: -2147483648 ... 2147483647
64-bit signed integers: -9223372036854775808 ... 9223372036854775807
    5 -> 00000101
    1 -> 00000001
    0 -> 00000000
   -1 -> 11111111
   -5 -> 11111011
  127 -> 01111111
 -128 -> 10000000
int8: 127 + 1 = -128
int16: 32767 + 1 = -32768
Python int: 2**100 = 1267650600228229401496703205376
```

△ Caution: integer overflow

Fixed-width integer arithmetic silently "wraps around": in 8 bits, $127 + 1 = -128$. Python's built-in `int` has unlimited precision and never overflows, but NumPy arrays of type `int8`, `int16`, `int32` (and C, Fortran, Java programs) do. On 4 June 1996 the European rocket **Ariane 5** self-destructed 37 seconds after launch because a 64-bit floating-point value (the horizontal velocity) was converted to a 16-bit signed integer, whose largest value is 32 767. The overflow shut down both inertial reference systems. The loss was estimated at about US$370 million.

Towards Graduate Studies: exact and multiple-precision arithmetic

Computer algebra systems (SymPy, Mathematica, Maple) use arbitrary-length integers and exact rationals, and libraries such as `mpmath` provide floating-point numbers with hundreds of digits. Fast multiplication of huge integers (Karatsuba 1962, Schönhage–Strassen 1971, and Harvey–van der Hoeven 2019 with $O(n \log n)$ operations) is an active research area connecting number theory, the Fast Fourier Transform and complexity theory. Modern

machine learning goes the other way: it uses *low*-precision formats (16-bit `bfloat16`, even 8- and 4-bit numbers), and the analysis of algorithms in mixed precision is a current topic of research in numerical linear algebra.

Exercises

2.3.1 Theory Write -37 as an 8-bit two's complement number. Check with `np.binary_repr(-37, width=8)`.

2.3.2 Theory Show that in n-bit two's complement the negation rule "invert the bits and add one" is correct, i.e. that $x + \bar{x} + 1 \equiv 0 \pmod{2^n}$, where $\bar{x}$ has all bits of x inverted.

2.3.3 Python The Ariane 5 velocity value was of the order of 40 000 (in internal units). Use `np.array([40000.0]).astype(np.int16)` to see what value a 16-bit conversion produces on your computer.

2.3.4 ★ Graduate How many bits are needed to store $n!$ exactly? Use Stirling's formula to show that the answer grows like $n \log_2 n$, and verify with `math.factorial(n).bit_length()`.

Chapter Summary

- In base β, $(d_n \cdots d_0.d_{-1} \cdots)_\beta = \sum d_k \beta^k$.
- Base $\beta \to 10$: Horner's rule. $10 \to \beta$: repeated division for integers, repeated multiplication for fractions.
- One hex digit = 4 bits, one octal digit = 3 bits.
- 0.1 has an infinite binary expansion, so it cannot be stored exactly in a computer.
- Integers are stored in two's complement with a fixed number of bits and can overflow.

Chapter 3

Floating-Point Arithmetic and Round-off Errors

Learning Objectives

After studying this chapter you should be able to:

- describe a floating-point number system $\mathbb{F}(\beta, p, L, U)$ and the IEEE 754 standard;
- define and compute absolute and relative errors and significant digits;
- state and prove the bound $\mathrm{fl}(x) = x(1+\delta)$, $|\delta| \le u$, for chopping and rounding;
- recognise and avoid catastrophic cancellation;
- distinguish the *conditioning* of a problem from the *stability* of an algorithm;
- use Taylor's theorem and the $O(\cdot)$ notation to describe truncation errors.

Try the following in Python: `0.1 + 0.2 == 0.3`. The answer is `False`. This is not a bug in Python: it happens in every programming language on every modern computer, and it is a consequence of the way real numbers are stored. In this chapter we study the finite set of numbers that a computer can represent, the errors made when a real number is replaced by its nearest machine number, and how these errors propagate through a computation.

Historical Note: floating point, from Zuse to Kahan

The German engineer **Konrad Zuse** built the Z3 in 1941, the first working programmable computer, and it already used binary floating-point numbers. For the next forty years every manufacturer used its own floating-point format with its own rounding rules, so a program could give different answers on different machines. The Canadian mathematician **William Kahan** (University of California, Berkeley) led the committee that produced the **IEEE 754 standard for binary floating-point arithmetic** in 1985 (revised in 2008 and 2019). It prescribes the formats, the rounding ("correctly rounded" $+, -, \times, \div, \sqrt{}$), and the special values $\pm\infty$ and NaN. Kahan received the Turing Award, the "Nobel Prize of computing", in 1989 for this work. Today essentially every processor implements IEEE 754, which is why the outputs in these notes will be identical on your computer.
The systematic study of rounding errors was begun by **John von Neumann** and **Herman Goldstine** (1947) and by **Alan Turing** (1948), and brought to maturity by **James H. Wilkinson**, whose book *Rounding Errors in Algebraic Processes* (1963) introduced *backward error analysis*.

3.1 Floating-point number systems

In scientific notation we write, for example, $30\,000\,000 \times 0.000\,000\,15 = 3.0 \times 10^{7} \times 1.5 \times 10^{-7} = 4.5 \times 10^{0}$. Computers store real numbers in a similar way.

Definition 3.1 (floating-point number system)**.** A *floating-point number system* $\mathbb{F} = \mathbb{F}(\beta, p, L, U)$ consists of 0 and all numbers of the form

$$x = \pm (0.d_1 d_2 \cdots d_p)_\beta \times \beta^e = \pm \left(\frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_p}{\beta^p} \right) \beta^e, \tag{3.1}$$

where

(i) $\beta \geq 2$ is the *base* (usually 2, sometimes 10 or 16),

(ii) p is the *precision* (number of significant digits),

(iii) e is the *exponent*, an integer with $L \leq e \leq U$,

(iv) the digits satisfy $0 \leq d_i \leq \beta - 1$, and the number is *normalised*: $d_1 \neq 0$.

The fraction $m = (0.d_1 d_2 \cdots d_p)_\beta$ is called the *mantissa* (or significand). Normalisation means $\beta^{-1} \leq m < 1$.

Example 3.2. With $\beta = 10$, $p = 4$: $81.34 = 0.8134 \times 10^2$, $3294 = 0.3294 \times 10^4$, $0.0098 = 0.9800 \times 10^{-2}$, $967.314 \approx 0.9673 \times 10^3$.

Example 3.3 (a toy system)**.** Take $\beta = 2$, $p = 3$, $L = -1$, $U = 2$. The normalised mantissas are $(0.100)_2 = 0.5$, $(0.101)_2 = 0.625$, $(0.110)_2 = 0.75$, $(0.111)_2 = 0.875$, and multiplying by $2^{-1}, 2^0, 2^1, 2^2$ gives 16 positive numbers. Together with their negatives and zero, $\mathbb{F}$ has 33 elements.

Listing 3.1: Enumerating a toy floating-point system ch03/toy_system.py

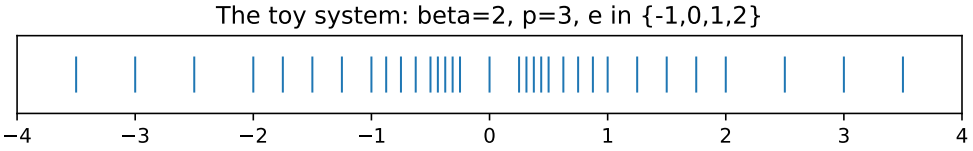
```
# All numbers of a toy floating-point system  x = +/- (0.d1 d2 d3)_2 * 2^e,
# d1 = 1 (normalised), e = -1, 0, 1, 2
import numpy as np
import matplotlib.pyplot as plt

beta, p, emin, emax = 2, 3, -1, 2
mantissas = [sum(d * beta ** -(k + 1) for k, d in enumerate(digits))
             for digits in [(1, a, b) for a in (0, 1) for b in (0, 1)]]
positive = sorted(m * beta ** e for e in range(emin, emax + 1) for m in mantissas)
print("mantissas :", mantissas)
print("positive numbers:", positive)
print("count (with negatives and zero):", 2 * len(positive) + 1)
print("gaps:", np.diff(positive))

allnums = [-v for v in positive] + [0.0] + positive
plt.figure(figsize=(7, 1.3))
plt.plot(allnums, np.zeros(len(allnums)), "|", markersize=18)
plt.yticks([])
plt.xticks(range(-4, 5))
plt.title("The toy system: beta=2, p=3, e in {-1,0,1,2}")
plt.tight_layout()
plt.savefig("ch03_toy.pdf")
```

Output

```
mantissas : [0.5, 0.625, 0.75, 0.875]
positive numbers: [0.25, 0.3125, 0.375, 0.4375, 0.5, 0.625, 0.75, 0.875, 1.0, 1.25, 1.5,
    ↪ 1.75, 2.0, 2.5, 3.0, 3.5]
count (with negatives and zero): 33
gaps: [0.0625 0.0625 0.0625 0.0625 0.125  0.125  0.125  0.125  0.25   0.25
 0.25   0.25   0.5    0.5    0.5   ]
```



The picture reveals three fundamental facts.

★ Key Concept: the machine numbers

1. $\mathbb{F}$ is a **finite** set. There is a largest number ($(0.111)_2 \times 2^2 = 3.5$ above) and a smallest positive normalised number ($(0.100)_2 \times 2^{-1} = 0.25$).
2. The numbers are **not equally spaced**: the gap doubles at every power of β.
3. The **relative** spacing is roughly constant: between β^{e-1} and β^e the gap is β^{e-p}, so gap$/|x|$ lies between β^{-p} and β^{1-p}.

Hence floating-point numbers are designed to make *relative* errors small, not absolute errors.

3.1.1 The IEEE 754 standard

In IEEE double precision (Python's `float`, NumPy's `float64`) a number is stored in 64 bits: 1 sign bit s, 11 exponent bits E and 52 fraction bits f. A normal number has the value

$$x = (-1)^s \times (1.f)_2 \times 2^{E-1023}, \qquad 1 \le E \le 2046.$$

The leading bit 1 is not stored (the "hidden bit"), so the precision is $p = 53$ bits, about 16 decimal digits. The exponent $E = 0$ is used for zero and for *subnormal* numbers $(0.f)_2 \times 2^{-1022}$ that fill the gap near zero, and $E = 2047$ encodes $\pm\infty$ and NaN ("Not a Number").

Format	bits	p (bits)	unit roundoff u	range (approx.)
half (`float16`)	16	11	4.9×10^{-4}	$6 \times 10^{-5} - 6.6 \times 10^{4}$
single (`float32`)	32	24	6.0×10^{-8}	$1.2 \times 10^{-38} - 3.4 \times 10^{38}$
double (`float64`)	64	53	1.1×10^{-16}	$2.2 \times 10^{-308} - 1.8 \times 10^{308}$

Listing 3.2: Inspecting the bits of a double ch03/ieee_bits.py

```
# The bit pattern of an IEEE 754 double:  1 sign bit, 11 exponent bits, 52 fraction bits
import struct


def double_bits(x):
    """Return (sign, exponent bits, fraction bits) of the double x."""
    (n,) = struct.unpack(">Q", struct.pack(">d", x))
```

```
    b = f"{n:064b}"
    return b[0], b[1:12], b[12:]


for x in [1.0, -2.0, 0.1, 13.25, 1e-310]:
    s, e, f = double_bits(x)
    E = int(e, 2)
    kind = "subnormal" if E == 0 else "normal"
    print(f"x = {x!r}")
    print(f"   sign={s}  exponent={e} (={E}, unbiased {E - 1023})  [{kind}]")
    print(f"   fraction={f}")
```

Output

```
x = 1.0
   sign=0  exponent=01111111111 (=1023, unbiased 0)  [normal]
   fraction=0000000000000000000000000000000000000000000000000000
x = -2.0
   sign=1  exponent=10000000000 (=1024, unbiased 1)  [normal]
   fraction=0000000000000000000000000000000000000000000000000000
x = 0.1
   sign=0  exponent=01111111011 (=1019, unbiased -4)  [normal]
   fraction=1001100110011001100110011001100110011001100110011010
x = 13.25
   sign=0  exponent=10000000010 (=1026, unbiased 3)  [normal]
   fraction=1010100000000000000000000000000000000000000000000000
x = 1e-310
   sign=0  exponent=00000000000 (=0, unbiased -1023)  [subnormal]
   fraction=0000000100100110100010001011011100001110011000101011
```

For $13.25 = (1101.01)_2 = (1.10101)_2 \times 2^3$ we see the exponent 3 and the fraction bits 10101000 . . .. For 0.1 the fraction is the repeating pattern 1001 1001 . . . of Example 2.5, rounded up in the last place.

Listing 3.3: Machine epsilon, overflow, underflow, infinity and NaN ch03/float_basics.py

```
# Surprises and facts about IEEE double precision
import sys
from decimal import Decimal
import numpy as np

print("0.1 + 0.2        =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3 :", 0.1 + 0.2 == 0.3)
print("exact value stored for 0.1:")
print("   ", Decimal(0.1))

# machine epsilon: the gap between 1 and the next larger double
eps = 1.0
while 1.0 + eps / 2 > 1.0:
    eps = eps / 2
print("machine epsilon (loop)    =", eps)
print("np.finfo(float).eps       =", np.finfo(float).eps)
print("1 + eps/2 == 1 ?          ", 1.0 + eps / 2 == 1.0)
print("largest double            =", sys.float_info.max)
print("smallest normalised double=", sys.float_info.min)
```

```python
print("smallest subnormal double =", 5e-324, "=", 2.0**-1074)
print("significant decimal digits:", sys.float_info.dig)
print("overflow : 1e308 * 10     =", 1e308 * 10)
print("underflow: 1e-320 / 1e10  =", 1e-320 / 1e10)
print("inf - inf                 =", float("inf") - float("inf"))
nan = float("nan")
print("nan == nan                :", nan == nan)
print("hex form of 0.1           :", (0.1).hex())
print("hex form of 1.0           :", (1.0).hex())
```

Output

```
0.1 + 0.2        = 0.30000000000000004
0.1 + 0.2 == 0.3 : False
exact value stored for 0.1:
    0.1000000000000000055511151231257827021181583404541015625
machine epsilon (loop)    = 2.220446049250313e-16
np.finfo(float).eps       = 2.220446049250313e-16
1 + eps/2 == 1 ?           True
largest double            = 1.7976931348623157e+308
smallest normalised double= 2.2250738585072014e-308
smallest subnormal double = 5e-324 = 5e-324
significant decimal digits: 15
overflow : 1e308 * 10     = inf
underflow: 1e-320 / 1e10  = 0.0
inf - inf                 = nan
nan == nan                : False
hex form of 0.1           : 0x1.999999999999ap-4
hex form of 1.0           : 0x1.0000000000000p+0
```

Definition 3.4 (machine epsilon and unit roundoff)**.** The *machine epsilon* $\varepsilon_{\text{mach}}$ is the distance from 1 to the next larger floating-point number; for IEEE double $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22\times10^{-16}$. The *unit roundoff* is $u = \frac{1}{2}\varepsilon_{\text{mach}} = 2^{-53} \approx 1.11\times10^{-16}$ for rounding to nearest. (Some books call u the machine epsilon, so check the definition when you read.)

Exercises

3.1.1 Theory List all positive numbers of $\mathbb{F}(10, 2, -1, 1)$. How many are there? What are the largest and smallest?

3.1.2 Theory Show that $\mathbb{F}(\beta, p, L, U)$ contains exactly $2(\beta-1)\beta^{p-1}(U-L+1)+1$ numbers. Check with Example 3.3.

3.1.3 Theory Write -6.75 and 0.2 in IEEE double format (sign, unbiased exponent, fraction). Check with Listing 3.2.

3.1.4 Python Repeat the machine-epsilon loop of Listing 3.3 with `np.float32` and `np.float16` numbers and compare with `np.finfo`.

3.1.5 Python Find the smallest positive integer n such that `float(n) + 1.0 == float(n)`. Explain your answer in terms of $p = 53$.

3.2 Rounding and chopping

Let x be a real number within the range of $\mathbb{F}$, with the exact (possibly infinite) normalised expansion

$$x = \pm(0.d_1 d_2 \cdots d_p d_{p+1} d_{p+2} \cdots)_\beta \times \beta^e .$$

Its floating-point representation $\mathrm{fl}(x)$ is obtained by one of two rules:

- **Chopping**: discard the digits $d_{p+1}, d_{p+2}, \ldots$: $\mathrm{fl}(x) = \pm(0.d_1 \cdots d_p)_\beta \times \beta^e$.
- **Rounding**: chop, but add one unit in the last place, β^{e-p}, if $d_{p+1} \geq \beta/2$. (IEEE uses "round to nearest, ties to even".)

The error $x - \mathrm{fl}(x)$ is called the **round-off error**.

Example 3.5. With $\beta = 10$, $p = 7$, $\pi = 0.314159265\ldots \times 10^1$ has $\mathrm{fl}_{\mathrm{chop}}(\pi) = 0.3141592 \times 10^1$ and $\mathrm{fl}_{\mathrm{round}}(\pi) = 0.3141593 \times 10^1$.

Definition 3.6 (absolute and relative error)**.** If $\hat{x}$ is an approximation of x, the *absolute error* is $|x - \hat{x}|$ and, for $x \neq 0$, the *relative error* is $\dfrac{|x - \hat{x}|}{|x|}$. We say that $\hat{x}$ approximates x to t *significant digits* if t is the largest non-negative integer with

$$\frac{|x - \hat{x}|}{|x|} \leq 5 \times 10^{-t}.$$

Example 3.7. Let $x = 2.51$ and $\hat{x} = 2.50$. The absolute error is 0.01 and the relative error is $0.01/2.51 \approx 3.98 \times 10^{-3} \leq 5 \times 10^{-3}$, so $\hat{x}$ has 3 significant digits. The same absolute error 0.01 in approximating $x = 0.02$ by $\hat{x} = 0.01$ gives a relative error of 50% and no correct digits at all.

Theorem 3.8 (relative error of fl)**.** *Let $x \neq 0$ lie in the range of $\mathbb{F}(\beta, p, L, U)$. Then*

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \leq u, \qquad u = \begin{cases} \tfrac{1}{2}\beta^{1-p}, & \text{rounding}, \\ \beta^{1-p}, & \text{chopping}. \end{cases}$$

Equivalently, $\mathrm{fl}(x) = x(1 + \delta)$ for some δ with $|\delta| \leq u$.

Proof. Write $x = m\beta^e$ with $\beta^{-1} \leq |m| < 1$. The two floating-point numbers adjacent to x differ by one unit in the p-th digit, that is by β^{e-p}. Chopping moves x to one of them, so $|x - \mathrm{fl}(x)| < \beta^{e-p}$; rounding moves x to the nearest one, so $|x - \mathrm{fl}(x)| \leq \tfrac{1}{2}\beta^{e-p}$. Dividing by $|x| = |m|\beta^e \geq \beta^{e-1}$ gives

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \leq \frac{\beta^{e-p}}{\beta^{e-1}} = \beta^{1-p} \quad \text{(chopping)}, \qquad \frac{|x - \mathrm{fl}(x)|}{|x|} \leq \tfrac{1}{2}\beta^{1-p} \quad \text{(rounding)}.$$

Finally, define $\delta = (\mathrm{fl}(x) - x)/x$; then $|\delta| \leq u$ and $\mathrm{fl}(x) = x(1 + \delta)$. □

For IEEE double precision (which uses the form $(1.f)_2 \times 2^E$ with $p = 53$) the same argument gives $u = 2^{-53}$.

★ Key Concept: the fundamental model of floating-point arithmetic

IEEE 754 guarantees that the result of each basic operation is the *correctly rounded* exact result. Therefore, for $\circ \in \{+, -, \times, \div\}$,

$$\mathrm{fl}(x \circ y) = (x \circ y)(1 + \delta), \qquad |\delta| \leq u. \tag{3.2}$$

Every single operation has a tiny *relative* error. Large errors can nevertheless accumulate, or be magnified, in a long computation. This model is the starting point of all rounding-error

analysis.

We can simulate k-digit decimal arithmetic with Python's `decimal` module, which lets us repeat classic textbook experiments exactly.

Listing 3.4: k-digit chopping and rounding arithmetic ch03/kdigit.py

```python
# Simulating k-digit decimal arithmetic with chopping or rounding
from decimal import Decimal, getcontext, ROUND_DOWN, ROUND_HALF_UP
import math


def set_arithmetic(k, mode):
    """Make every Decimal operation keep k significant digits."""
    getcontext().prec = k
    getcontext().rounding = ROUND_HALF_UP if mode == "round" else ROUND_DOWN


def fl(x, k, mode="round"):
    """k significant decimal digits of x, by chopping or rounding."""
    set_arithmetic(k, mode)
    return +Decimal(repr(x))          # unary + applies the context


for k in [3, 5, 7]:
    for mode in ["chop", "round"]:
        f = fl(math.pi, k, mode)
        rel = abs(float(f) - math.pi) / math.pi
        bound = 10.0 ** (1 - k) * (0.5 if mode == "round" else 1.0)
        print(f"k={k} {mode:5s}: fl(pi) = {str(f):9s} rel.err = {rel:.2e} <= {bound:.1e}")


def poly(x, k, mode):
    """f(x) = x^3 - 6.1 x^2 + 3.2 x + 1.5 in k-digit arithmetic, two ways."""
    set_arithmetic(k, mode)
    x = +Decimal(str(x))
    a2, a1, a0 = Decimal("-6.1"), Decimal("3.2"), Decimal("1.5")
    direct = x * x * x + a2 * (x * x) + a1 * x + a0     # every operation rounded
    nested = ((x + a2) * x + a1) * x + a0               # Horner's rule
    return direct, nested


x = 4.71
exact = x**3 - 6.1 * x**2 + 3.2 * x + 1.5
print(f"\nexact f(4.71) = {exact:.6f}")
for mode in ["chop", "round"]:
    d, n = poly(x, 3, mode)
    print(f"3-digit {mode:5s}: direct = {d},  nested (Horner) = {n}")
```

Output

```
k=3 chop : fl(pi) = 3.14      rel.err = 5.07e-04 <= 1.0e-02
k=3 round: fl(pi) = 3.14      rel.err = 5.07e-04 <= 5.0e-03
k=5 chop : fl(pi) = 3.1415    rel.err = 2.95e-05 <= 1.0e-04
k=5 round: fl(pi) = 3.1416    rel.err = 2.34e-06 <= 5.0e-05
```

```
k=7 chop : fl(pi) = 3.141592  rel.err = 2.08e-07 <= 1.0e-06
k=7 round: fl(pi) = 3.141593  rel.err = 1.10e-07 <= 5.0e-07

exact f(4.71) = -14.263899
3-digit chop : direct = -13.5,  nested (Horner) = -14.2
3-digit round: direct = -13.4,  nested (Horner) = -14.3
```

The second experiment evaluates $f(x) = x^3 - 6.1x^2 + 3.2x + 1.5$ at $x = 4.71$ (an example from Burden & Faires [2]) in 3-digit arithmetic. The exact value is -14.263899. The direct evaluation has a relative error of about 5–6%, while *Horner's rule* (nested multiplication), $f(x) = ((x - 6.1)x + 3.2)x + 1.5$, uses fewer operations and is far more accurate.

△ Caution: floating-point arithmetic breaks the laws of algebra

Floating-point addition is commutative but **not associative**: $(10^{16} + (-10^{16})) + 1 = 1$, but $10^{16} + (-10^{16} + 1) = 0$, since $-10^{16} + 1$ rounds to -10^{16}. Likewise the distributive law fails. Never test floating-point numbers for exact equality; test `abs(a - b) <= tol * abs(b)` or use `math.isclose` / `np.isclose`.

Exercises

3.2.1 Theory Compute the absolute and relative errors, and the number of significant digits, when (a) π is approximated by $22/7$; (b) π by $355/113$; (c) e by 2.718; (d) $\sqrt{2}$ by 1.414.

3.2.2 Theory Use 4-digit rounding arithmetic to compute $\frac{13}{14} - \frac{6}{7}$, then $(2e - 5.4)$ and finally their quotient. Find the relative error.

3.2.3 Theory Prove that $\mathrm{fl}(x \cdot y \cdot z)$, evaluated as $\mathrm{fl}(\mathrm{fl}(xy)z)$, equals $xyz(1 + \theta)$ with $|\theta| \le 2u + u^2$.

3.2.4 Python Use Listing 3.4 to evaluate $e^{0.5}$ by the Taylor polynomial of degree 6 in 4-digit chopping arithmetic, summing (a) from the largest term to the smallest, (b) from the smallest to the largest. Which is more accurate, and why?

3.2.5 ★ Graduate Show that with rounding to nearest, $\mathrm{fl}(x) = x/(1 + \eta)$ with $|\eta| \le u$ as well. Why is this alternative form sometimes more convenient?

3.3 Propagation of errors and catastrophic cancellation

Suppose $\hat{x} = x(1 + \epsilon_x)$ and $\hat{y} = y(1 + \epsilon_y)$ are inexact data.

- **Multiplication and division are safe:** $\hat{x}\hat{y} = xy(1 + \epsilon_x + \epsilon_y + \epsilon_x\epsilon_y)$, so relative errors (approximately) *add*.
- **Subtraction can be dangerous:** if $x \neq y$,

$$\frac{|(\hat{x} - \hat{y}) - (x - y)|}{|x - y|} = \frac{|x\epsilon_x - y\epsilon_y|}{|x - y|} \le \frac{|x| + |y|}{|x - y|} \max(|\epsilon_x|, |\epsilon_y|).$$

  When $x \approx y$ the factor $(|x| + |y|)/|x - y|$ is huge.

★ Key Concept: catastrophic cancellation

Subtracting two nearly equal numbers that are already inexact destroys significant digits: the leading digits cancel and the result is dominated by the earlier errors. For example,

$0.1234567 - 0.1234561 = 0.0000006$ has at most one correct digit even though each operand had seven. The subtraction itself is exact; it merely *reveals* the errors already present. The cure is to **rewrite the formula** so that the subtraction is avoided.

Example 3.9 (three classical cases).

1. **Quadratic equations.** For $x^2 + 10^8 x + 1 = 0$ the roots are $x_1 \approx -10^{-8}$ and $x_2 \approx -10^8$. The formula $x_1 = (-b + \sqrt{b^2 - 4ac})/(2a)$ subtracts two numbers that agree in all their digits. Instead compute $x_2 = (-b - \operatorname{sign}(b)\sqrt{b^2 - 4ac})/(2a)$ (no cancellation) and then $x_1 = c/(a x_2)$ from Vieta's formula $x_1 x_2 = c/a$.
2. $1 - \cos x$ **for small** x**.** Use the identity $1 - \cos x = 2\sin^2(x/2)$.
3. **Alternating series.** $e^{-20} = \sum_{n \geq 0} (-20)^n / n!$ has terms as large as 4×10^7 that must cancel to give 2×10^{-9}; each term carries a rounding error of about $10^7 \times 10^{-16} = 10^{-9}$. Instead compute $1/e^{20}$, where all terms are positive.

Listing 3.5: Catastrophic cancellation and its cures ch03/cancellation.py

```python
# Catastrophic cancellation and how to avoid it
import math

# 1. The quadratic formula for x^2 + 1e8 x + 1 = 0
a, b, c = 1.0, 1e8, 1.0
d = math.sqrt(b * b - 4 * a * c)
x1_naive = (-b + d) / (2 * a)            # subtracts two nearly equal numbers
x2 = (-b - d) / (2 * a)
x1_stable = c / (a * x2)                 # Vieta: x1 * x2 = c / a
print("small root, school formula :", x1_naive)
print("small root, stable formula :", x1_stable)
print("(true value is -1.0000000000000000e-08 to 16 digits)")

# 2. f(x) = (1 - cos x)/x^2  -> 1/2 as x -> 0
print("\n   x        (1-cos x)/x^2       2 sin^2(x/2)/x^2")
for k in range(1, 9):
    x = 10.0 ** (-k)
    f1 = (1 - math.cos(x)) / x**2
    f2 = 2 * math.sin(x / 2) ** 2 / x**2
    print(f"1e-{k:d}   {f1:.15f}   {f2:.15f}")

# 3. exp(-20) from its Taylor series
s, term = 0.0, 1.0
largest = 0.0
for n in range(1, 120):
    s += term
    largest = max(largest, abs(term))
    term *= -20.0 / n
print("\nTaylor series for e^-20   :", s)
print("largest term in the series:", largest)
print("math.exp(-20)             :", math.exp(-20))
s2, term = 0.0, 1.0
for n in range(1, 120):
    s2 += term
    term *= 20.0 / n
print("1 / (series for e^20)     :", 1 / s2)
```

Output

```
small root, school formula : -7.450580596923828e-09
small root, stable formula : -1e-08
(true value is -1.0000000000000000e-08 to 16 digits)

   x        (1-cos x)/x^2       2 sin^2(x/2)/x^2
1e-1   0.499583472197418   0.499583472197423
1e-2   0.499995833347366   0.499995833347222
1e-3   0.499999958325503   0.499999958333335
1e-4   0.499999996961265   0.499999999583333
1e-5   0.500000041370185   0.499999999995833
1e-6   0.500044450291171   0.499999999999958
1e-7   0.499600361081320   0.500000000000000
1e-8   0.000000000000000   0.500000000000000

Taylor series for e^-20   : 6.147561828914626e-09
largest term in the series: 43099804.12182178
math.exp(-20)             : 2.061153622438558e-09
1 / (series for e^20)     : 2.061153622438557e-09
```

The school formula gives the small root with *no* correct digits (-7.45×10^{-9} instead of -1.00×10^{-8}), and at $x = 10^{-8}$ the expression $(1 - \cos x)/x^2$ returns 0 instead of 0.5: in double precision $\cos(10^{-8})$ rounds to exactly 1. The Taylor series for e^{-20} is wrong in every digit.

3.3.1 Accumulation of errors in sums

Adding n numbers one after another can accumulate an error that grows like nu in the worst case. The following listing adds 0.1 a million times. **Kahan's compensated summation** (1965) keeps track of the low-order part lost in each addition and feeds it back.

Listing 3.6: Naive and compensated summation ch03/summation.py

```python
# Summation errors: naive summation, Kahan's compensated summation and math.fsum
import math
import numpy as np

n = 10**6
naive = 0.0
for _ in range(n):
    naive += 0.1

kahan, comp = 0.0, 0.0
for _ in range(n):
    y = 0.1 - comp          # correct the next term by the lost low-order part
    t = kahan + y
    comp = (t - kahan) - y  # what was lost when adding y
    kahan = t

print(f"naive sum of 0.1 (10^6 times) = {naive:.15f}")
print(f"Kahan compensated sum         = {kahan:.15f}")
print(f"math.fsum (correctly rounded) = {math.fsum([0.1] * n):.15f}")

# Floating-point addition is not associative
a, b, c = 1e16, -1e16, 1.0
```

```
print("(a + b) + c =", (a + b) + c, "   a + (b + c) =", a + (b + c))

# Order matters: summing 1/k^2 forwards and backwards in single precision
N = 10**6
terms = (1.0 / np.arange(1, N + 1, dtype=np.float64) ** 2).astype(np.float32)
fwd = np.float32(0)
for t in terms:
    fwd += t
bwd = np.float32(0)
for t in terms[::-1]:
    bwd += t
exact = math.fsum(1.0 / k**2 for k in range(1, N + 1))
print(f"float32 forward  sum = {fwd:.8f}")
print(f"float32 backward sum = {bwd:.8f}")
print(f"true value           = {exact:.8f}")
```

Output

```
naive sum of 0.1 (10^6 times) = 100000.000001332882675
Kahan compensated sum         = 100000.000000000000000
math.fsum (correctly rounded) = 100000.000000000000000
(a + b) + c = 1.0    a + (b + c) = 0.0
float32 forward  sum = 1.64472532
float32 backward sum = 1.64493299
true value           = 1.64493307
```

In single precision, adding $\sum 1/k^2$ from the largest term to the smallest loses about 4 digits: once the partial sum is about 1.64, the terms $1/k^2 < 10^{-8}$ are smaller than half a unit in the last place and are simply lost. Adding from small to large keeps them.

Historical Note: disasters caused by rounding

Patriot missile, Dhahran, 25 February 1991. The system clock counted time in tenths of a second, and multiplied the count by 1/10 stored in a 24-bit fixed-point register. Since 0.1 has no finite binary expansion, the stored value was too small by about 9.5×10^{-8}. After the battery had run for 100 hours the time error was 0.34 s, during which an incoming Scud missile travels more than half a kilometre. The Patriot failed to intercept the Scud, which struck a barracks and killed 28 American soldiers (US General Accounting Office report GAO/IMTEC-92-26).

Vancouver Stock Exchange, 1982–83. A new index was started at 1000.000 and recomputed after every trade, with the result *chopped* to three decimal places. After 22 months the index stood at about 525 although its correctly computed value was about 1099: thousands of tiny chopping errors per day, all of the same sign, had accumulated.

Listing 3.7: Reproducing the Patriot clock drift ch03/patriot.py

```
# The Patriot missile failure (Dhahran, 25 February 1991)
# Time was counted in tenths of a second and multiplied by 0.1 stored in a
# 24-bit fixed-point register, i.e. 0.1 chopped after 23 binary digits.
import math

bits = 23
tenth = math.floor(0.1 * 2**bits) / 2**bits
err_per_tick = 0.1 - tenth
print(f"stored value of 0.1   : {tenth:.15f}")
```

```
print(f"error per 0.1 s tick  : {err_per_tick:.3e}")
for hours in [1, 8, 20, 48, 72, 100]:
    ticks = hours * 3600 * 10
    drift = ticks * err_per_tick
    print(f"after {hours:3d} h: clock error = {drift:.4f} s,"
          f" Scud travels {1676 * drift:6.0f} m in that time")
```

Output

```
stored value of 0.1   : 0.099999904632568
error per 0.1 s tick  : 9.537e-08
after   1 h: clock error = 0.0034 s, Scud travels      6 m in that time
after   8 h: clock error = 0.0275 s, Scud travels     46 m in that time
after  20 h: clock error = 0.0687 s, Scud travels    115 m in that time
after  48 h: clock error = 0.1648 s, Scud travels    276 m in that time
after  72 h: clock error = 0.2472 s, Scud travels    414 m in that time
after 100 h: clock error = 0.3433 s, Scud travels    575 m in that time
```

Exercises

3.3.1 Theory Rewrite each expression to avoid cancellation for the stated values of x: (a) $\sqrt{x+1}-\sqrt{x}$, x large; (b) $\ln(x+1)-\ln x$, x large; (c) $\dfrac{1-\cos x}{\sin x}$, $x \approx 0$; (d) $e^x - 1$, $x \approx 0$ (compare with `np.expm1`); (e) $\sin(a+x)-\sin a$, $x \approx 0$.

3.3.2 Python Verify each of your formulas in the previous exercise numerically for $x = 10^{k}$ or $x = 10^{-k}$, $k = 1, \dots, 10$.

3.3.3 Python The sample variance can be computed as $s^2 = \frac{1}{n-1}\left(\sum x_i^2 - n\bar{x}^2\right)$ (one pass) or $s^2 = \frac{1}{n-1}\sum (x_i-\bar{x})^2$ (two passes). Compare both for the data $x_i = 10^8 + i$, $i = 1, \dots, 10$. Explain the result.

3.3.4 Theory Show that for $a = 1$, $b = 10^8$, $c = 1$, the relative error of the school formula for the small root is at least of the order $b^2 u \approx 10^0$, i.e. no digits are correct. (Hint: $\sqrt{b^2-4ac}$ carries an absolute error of about bu.)

3.3.5 ★ Graduate Prove that Kahan's summation algorithm computes $\sum_{i=1}^{n} x_i(1+\eta_i)$ with $|\eta_i| \le 2u + O(nu^2)$, i.e. the error bound is essentially independent of n (see Higham [11], §4.3).

3.4 Conditioning and stability

Two different questions must be separated.

1. **Conditioning** is a property of the *problem*: how much can the exact answer change when the data change slightly?
2. **Stability** is a property of the *algorithm*: does the algorithm produce an answer that is as good as the conditioning of the problem allows?

> **Definition 3.10** (condition number of a function)**.** Let f be differentiable. The (relative)

condition number of evaluating f at x is

$$\kappa_f(x) = \lim_{\epsilon \to 0} \sup_{|\Delta x| \le \epsilon |x|} \frac{|f(x+\Delta x) - f(x)|/|f(x)|}{|\Delta x|/|x|} = \left| \frac{x\, f'(x)}{f(x)} \right|.$$

It is the factor by which a small relative error in x is amplified into a relative error in $f(x)$. A problem with small κ is *well conditioned*; with large κ, *ill conditioned*.

Example 3.11. $f(x) = \sqrt{x}$: $\kappa = \frac{1}{2}$ (very well conditioned). $f(x) = x - 1$ near $x = 1$: $\kappa = |x/(x-1)|$, which is huge; this is cancellation seen as ill-conditioning. $f(x) = \tan x$ near $\pi/2$: $\kappa = |x/(\sin x \cos x)|$, huge.

Listing 3.8: Condition numbers of some elementary functions ch03/conditioning.py

```python
# Condition number of evaluating a function: kappa(x) = |x f'(x) / f(x)|
import numpy as np

cases = [
    ("sqrt(x)", np.sqrt, lambda x: 0.5 / np.sqrt(x), 2.0),
    ("exp(x)", np.exp, np.exp, 10.0),
    ("x - 1", lambda x: x - 1, lambda x: 1.0, 1.000001),
    ("tan(x)", np.tan, lambda x: 1 / np.cos(x) ** 2, 1.5707),
    ("log(x)", np.log, lambda x: 1 / x, 1.0001),
]
print(f"{'f':8s} {'x':>10s} {'kappa':>12s}  perturb x by 1e-10 relatively")
for name, f, df, x in cases:
    kappa = abs(x * df(x) / f(x))
    xp = x * (1 + 1e-10)
    rel_out = abs(f(xp) - f(x)) / abs(f(x))
    print(f"{name:8s} {x:10.6f} {kappa:12.4e}  rel. change in f = {rel_out:.2e}")
```

Output

```
f                x        kappa  perturb x by 1e-10 relatively
sqrt(x)      2.000000   5.0000e-01  rel. change in f = 5.00e-11
exp(x)      10.000000   1.0000e+01  rel. change in f = 1.00e-09
x - 1        1.000001   1.0000e+06  rel. change in f = 1.00e-04
tan(x)       1.570700   1.6306e+04  rel. change in f = 1.63e-06
log(x)       1.000100   1.0000e+04  rel. change in f = 1.00e-06
```

In every row the measured relative change in f equals $\kappa \times 10^{-10}$, just as the definition predicts.

Definition 3.12 (backward stability)**.** An algorithm $\hat{f}$ for computing f is *backward stable* if for every x the computed result is the exact result for slightly perturbed data: $\hat{f}(x) = f(x + \Delta x)$ with $|\Delta x|/|x| = O(u)$.

★ Key Concept: the golden rule of error analysis

$$\text{relative forward error} \lesssim \text{condition number} \times \text{relative backward error}.$$

A backward stable algorithm applied to a well-conditioned problem gives an accurate answer. A backward stable algorithm applied to an ill-conditioned problem may give an inaccurate

answer, but this is the fault of the problem, not of the algorithm. An *unstable* algorithm (like the school quadratic formula for the small root) can give a poor answer even for a well-conditioned problem.

Towards Graduate Studies: backward error analysis

Wilkinson's idea of *backward error analysis* (1960s) transformed numerical analysis. Instead of tracking how errors grow forward through thousands of operations, one proves that the computed answer is the exact answer of a nearby problem. For example, Gaussian elimination with partial pivoting computes the exact solution of $(A + \Delta A)\hat{\mathbf{x}} = \mathbf{b}$ with $\|\Delta A\| \le c_n \rho u \, \|A\|$ (Chapter 8). The standard graduate reference is Higham, *Accuracy and Stability of Numerical Algorithms* [11]. Wilkinson's famous example $w(x) = (x-1)(x-2)\cdots(x-20)$ shows that the roots of a polynomial can be extremely ill-conditioned functions of its coefficients: changing the coefficient of x^{19} by 2^{-23} turns ten of the roots into complex numbers.

Exercises

3.4.1 Theory Compute the condition numbers of $f(x) = e^x$, $\ln x$, $\sin x$, x^n and $1/(1-x)$. Where is each ill conditioned?

3.4.2 Theory Show that the problem of computing the small root of $x^2 + bx + 1 = 0$, as a function of b, is well conditioned for large b (compute κ). Conclude that the poor result of the school formula is due to instability, not ill-conditioning.

3.4.3 ★ Graduate Show that the computed sum $\hat{s}$ of $x_1 + x_2 + \cdots + x_n$ (added left to right) satisfies $\hat{s} = \sum_i x_i(1+\theta_i)$ with $|\theta_i| \le (n-1)u + O(u^2)$. Hence recursive summation is backward stable. What is its forward error bound?

3.5 Truncation error, Taylor's theorem and the O notation

Rounding errors are caused by finite *precision*. **Truncation errors** are caused by replacing an infinite process by a finite one: a series by a partial sum, a derivative by a difference quotient, an integral by a sum, an iteration by finitely many steps. The main tool for analysing them is Taylor's theorem.

Theorem 3.13 (Taylor's theorem with Lagrange remainder)**.** *Let $f \in C^{n+1}[a,b]$ and $x_0 \in [a,b]$. For every $x \in [a,b]$ there is a ξ between x_0 and x such that*

$$f(x) = \underbrace{\sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x-x_0)^k}_{P_n(x)} + \underbrace{\frac{f^{(n+1)}(\xi)}{(n+1)!}(x-x_0)^{n+1}}_{R_n(x)}.$$

P_n is the Taylor polynomial of degree n and R_n is the remainder (truncation) error.

Example 3.14. For $f(x) = \sin x$, $x_0 = 0$, $n = 3$: $P_3(x) = x - x^3/6$ and $|R_3(x)| = |\sin^{(4)}(\xi)x^4/24| \le x^4/24$ (Fig. 1.1). At $x = 0.1$ the bound is 4.2×10^{-6}; the actual error is 8.3×10^{-8} (since $R_3 = R_4$ for odd functions, the better bound $x^5/120 = 8.3 \times 10^{-8}$ also holds).

Definition 3.15 (big-O notation)**.** We write $g(h) = O(h^p)$ as $h \to 0$ if there are constants C and $h_0 > 0$ with $|g(h)| \le Ch^p$ for $0 < h < h_0$. For sequences, $a_n = O(b_n)$ as $n \to \infty$ if $|a_n| \le C|b_n|$ for all large n.

Thus $\sin h = h + O(h^3)$, $\cos h = 1 - \frac{1}{2}h^2 + O(h^4)$, and the error of Euler's method in Section 1.7 is $O(h)$. If an error is $O(h^p)$, halving h reduces it by a factor of about 2^p: this is how we recognise the order of a method experimentally.

⋆ Key Concept: the total error

Most numerical methods have a truncation error that decreases as $h \to 0$ and a rounding error that *increases* as $h \to 0$ (because we subtract nearly equal quantities or perform more operations). The best achievable accuracy is obtained where the two are balanced. We will see this clearly for numerical differentiation in Chapter 12.

Exercises

3.5.1 Theory Find the Taylor polynomial P_4 of $f(x) = e^x$ about 0 and use the remainder to bound the error in approximating $e^{0.5}$. Verify numerically.

3.5.2 Theory How many terms of the Maclaurin series of e^x are needed to compute e with an error less than 10^{-10}?

3.5.3 Theory Show that $\dfrac{1}{1-h} = 1 + h + O(h^2)$ and $\sqrt{1+h} = 1 + \frac{1}{2}h - \frac{1}{8}h^2 + O(h^3)$.

3.5.4 Python Plot $\ln|R_n(x)|$ for the Taylor polynomials of $\ln(1+x)$ about 0 at $x = 0.5$ and $x = 1$ as a function of n. What do you observe at $x = 1$?

3.5.5 ★ Graduate The function $f(x) = e^{-1/x^2}$ ($f(0) = 0$) has all derivatives equal to zero at 0. What is its Taylor series? What does this say about the remainder term?

Chapter Summary

- Machine numbers form a finite set $\mathbb{F}(\beta, p, L, U)$ with constant *relative* spacing. IEEE double: $p = 53$, $u = 2^{-53} \approx 1.1 \times 10^{-16}$.
- $\mathrm{fl}(x) = x(1+\delta)$ and $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$ with $|\delta| \le u$.
- Subtracting nearly equal numbers causes catastrophic cancellation; rewrite formulas to avoid it.
- Conditioning (problem) $\neq$ stability (algorithm). Forward error $\lesssim$ condition number $\times$ backward error.
- Truncation errors are analysed with Taylor's theorem and described with $O(h^p)$.

Chapter 4

Solutions of Nonlinear Equations

Learning Objectives

After studying this chapter you should be able to:

- locate roots graphically and by the Intermediate Value Theorem;
- apply the bisection, false position, secant, Newton and fixed-point methods, by hand and in Python;
- define the order of convergence and estimate it from computed iterates;
- prove convergence of bisection, of fixed-point iteration (contraction mapping) and of Newton's method;
- recognise when a method fails (bad starting value, multiple root, stagnation) and choose a suitable remedy;
- accelerate a linearly convergent sequence with Aitken's Δ^2 process, and use SciPy's root finders.

We seek numerical solutions of equations of the form

$$f(x) = 0, \tag{4.1}$$

where f is a continuous (usually differentiable) function of one real variable. A value r with $f(r) = 0$ is a *root* of the equation, or a *zero* of f. For example, $x^2 + 4x + 4 = 0$ or $x \sin x + 1 = 0$. Only in special cases (linear and quadratic equations, some cubics and quartics, factorable polynomials) can the roots be written down by a formula; Abel and Galois proved in the 1820s–30s that no formula using radicals exists for general polynomial equations of degree five or more. Equations such as $x = \cos x$ or $e^{-x} = x$ have no closed-form solution at all. We therefore use *iterative methods*: starting from one or more initial guesses, we compute a sequence $x_0, x_1, x_2, \ldots$ that (we hope) converges to a root.

Historical Note: four thousand years of root finding

The Babylonian clay tablet YBC 7289 (c. 1800–1600 BC) shows $\sqrt{2}$ as $1;24,51,10$ in base 60, i.e. $1.41421296\ldots$, correct to six decimal places. It was probably computed with the iteration $x_{n+1} = \tfrac{1}{2}(x_n + 2/x_n)$, which is Newton's method for $x^2 - 2 = 0$. The Egyptian Rhind papyrus (c. 1650 BC) and the Chinese *Nine Chapters on the Mathematical Art* (c. 200 BC) use the method of *false position*. In 1225 **Leonardo of Pisa (Fibonacci)** solved $x^3 + 2x^2 + 10x = 20$, a problem posed at the court of Emperor Frederick II, and gave the root in sexagesimal as $1;22,7,42,33,4,40$, which is correct to about ten decimal places. Nobody knows how he did it.

4.1 Locating roots and bracketing

The first step is always to find a rough location of the roots, for instance from a graph (`plt.plot`) or a table of values. The theoretical tool is the following theorem.

Theorem 4.1 (Intermediate Value Theorem)**.** *If f is continuous on $[a, b]$ and $f(a)f(b) < 0$, then there exists at least one $r \in (a, b)$ with $f(r) = 0$.*

An interval $[a, b]$ with $f(a)f(b) < 0$ is called a *bracket*. Methods that keep the root inside a bracket (bisection, false position) are called *bracketing methods*; they always converge but can be slow. Methods that do not (secant, Newton, fixed point) are called *open methods*; they are usually much faster but may fail to converge.

Historical Note: Bolzano

The Intermediate Value Theorem looks obvious from a picture, and for a long time nobody felt it needed proof. The Bohemian priest and mathematician **Bernard Bolzano** gave the first rigorous proof in 1817, using the idea of repeatedly halving an interval, which is exactly the bisection method! A complete proof requires the completeness of the real numbers, which was only made precise by Dedekind and Cantor in the 1870s.

△ Caution: a sign change is sufficient, not necessary

$f(x) = (x - 1)^2$ has a root at $x = 1$ but no sign change, so no bracket exists and bracketing methods cannot find it. Conversely, $f(x) = 1/x$ changes sign on $[-1, 1]$ but has no root: the IVT requires continuity.

4.2 The bisection method

Suppose f is continuous on $[a, b]$ with $f(a)f(b) < 0$. Set $a_1 = a$, $b_1 = b$ and $p_1 = \frac{1}{2}(a_1 + b_1)$. Exactly one of the following holds:

1. $f(p_1) = 0$: then p_1 is a root and we stop;
2. $f(a_1)f(p_1) < 0$: a root lies in (a_1, p_1); set $a_2 = a_1$, $b_2 = p_1$;
3. $f(p_1)f(b_1) < 0$: a root lies in (p_1, b_1); set $a_2 = p_1$, $b_2 = b_1$.

Repeat with $[a_2, b_2]$, whose length is half that of $[a_1, b_1]$.

Algorithm 1 Bisection

Require: f continuous, $a < b$ with $f(a)f(b) < 0$, tolerance ε, maximum iterations N

for $n = 1, 2, \ldots, N$ **do**
  $p \leftarrow a + (b - a)/2$
  if $f(p) = 0$ **or** $(b - a)/2 < \varepsilon$ **then**
    return p
  end if
  if sign $f(a) =$ sign $f(p)$ **then**
    $a \leftarrow p$
  else
    $b \leftarrow p$
  end if
end for
error: maximum number of iterations exceeded

Theorem 4.2 (convergence of bisection)**.** *Let $f \in C[a,b]$ with $f(a)f(b) < 0$. The bisection method generates a sequence $\{p_n\}$ converging to a root r of f, with*

$$|p_n - r| \le \frac{b-a}{2^n}, \qquad n \ge 1.$$

Proof. By construction $b_n - a_n = (b-a)/2^{n-1}$ and $r \in (a_n, b_n)$ for every n (the IVT applied on each subinterval). Since p_n is the midpoint of $[a_n, b_n]$,

$$|p_n - r| \le \tfrac{1}{2}(b_n - a_n) = \frac{b-a}{2^n} \longrightarrow 0. \qquad \square$$

Corollary 4.3 (number of iterations)**.** *To guarantee $|p_n - r| \le \varepsilon$ it suffices to take*

$$n \;\ge\; \frac{\log\big((b-a)/\varepsilon\big)}{\log 2} = \log_2 \frac{b-a}{\varepsilon}.$$

Proof. We need $(b-a)/2^n \le \varepsilon$, i.e. $2^n \ge (b-a)/\varepsilon$; take logarithms. $\square$

Example 4.4. For $[a,b] = [1,2]$ and $\varepsilon = 10^{-4}$: $n \ge \log_2 10^4 = 4/\log_{10} 2 \approx 13.29$, so 14 iterations suffice. For $\varepsilon = 10^{-6}$ we need $n \ge 19.93$, i.e. 20 iterations. Note that the number of iterations depends only on a, b and ε, not on f.

Example 4.5. The equation $f(x) = x^3 - x^2 - x - 1 = 0$ has a root in $[1,2]$ since $f(1) = -2 < 0 < 1 = f(2)$. (The root $r = 1.839286755\ldots$ is the *tribonacci constant*, the limit of ratios of consecutive terms of $1, 1, 1, 3, 5, 9, 17, 31, \ldots$, where each term is the sum of the previous three.)

Listing 4.1: The bisection method ch04/bisection.py

```python
# The bisection method
import math


def bisection(f, a, b, tol=1e-6, max_iter=100, verbose=False):
    """Find a root of f in [a, b], where f(a) and f(b) have opposite signs.

    Returns the approximate root p and the number of iterations used.
    Stops when the half-length of the bracketing interval is below tol.
    """
    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("f(a) and f(b) must have opposite signs")
    if verbose:
        print(f"{'n':>3} {'a_n':>12} {'b_n':>12} {'p_n':>12} {'f(p_n)':>12}")
    for n in range(1, max_iter + 1):
        p = a + (b - a) / 2          # midpoint (this form avoids overflow)
        fp = f(p)
        if verbose:
            print(f"{n:3d} {a:12.8f} {b:12.8f} {p:12.8f} {fp:12.4e}")
        if fp == 0 or (b - a) / 2 < tol:
            return p, n
        if math.copysign(1, fa) == math.copysign(1, fp):   # same sign: root in [p, b]
            a, fa = p, fp
        else:                                              # root in [a, p]
            b, fb = p, fp
    raise RuntimeError("bisection did not converge")


def f(x):
    return x**3 - x**2 - x - 1


root, n = bisection(f, 1, 2, tol=1e-6, verbose=True)
print(f"\nroot ~ {root:.10f} after {n} iterations")
print("predicted number of iterations:",
      math.ceil(math.log2((2 - 1) / 1e-6)))
print("exact root (tribonacci constant) 1.839286755214161")
```

Output

```
 n          a_n          b_n          p_n       f(p_n)
 1   1.00000000   2.00000000   1.50000000  -1.3750e+00
 2   1.50000000   2.00000000   1.75000000  -4.5312e-01
 3   1.75000000   2.00000000   1.87500000   2.0117e-01
 4   1.75000000   1.87500000   1.81250000  -1.4331e-01
 5   1.81250000   1.87500000   1.84375000   2.4506e-02
 6   1.81250000   1.84375000   1.82812500  -6.0497e-02
 7   1.82812500   1.84375000   1.83593750  -1.8271e-02
 8   1.83593750   1.84375000   1.83984375   3.0484e-03
 9   1.83593750   1.83984375   1.83789062  -7.6285e-03
10   1.83789062   1.83984375   1.83886719  -2.2944e-03
11   1.83886719   1.83984375   1.83935547   3.7591e-04
12   1.83886719   1.83935547   1.83911133  -9.5951e-04
13   1.83911133   1.83935547   1.83923340  -2.9187e-04
14   1.83923340   1.83935547   1.83929443   4.2004e-05
15   1.83923340   1.83929443   1.83926392  -1.2494e-04
16   1.83926392   1.83929443   1.83927917  -4.1467e-05
17   1.83927917   1.83929443   1.83928680   2.6797e-07
18   1.83927917   1.83928680   1.83928299  -2.0600e-05
19   1.83928299   1.83928680   1.83928490  -1.0166e-05
20   1.83928490   1.83928680   1.83928585  -4.9490e-06

root ~ 1.8392858505 after 20 iterations
predicted number of iterations: 20
exact root (tribonacci constant) 1.839286755214161
```

Exactly the predicted 20 iterations were needed. Observe that $|f(p_n)|$ does not decrease monotonically (compare $n = 17$ and $n = 18$): bisection ignores the *size* of f and uses only its *sign*.

★ Key Concept: bisection in one sentence

Bisection gains exactly **one binary digit per iteration** (about 3.3 iterations per decimal digit), regardless of f. It is slow but **guaranteed** to converge whenever a bracket is known, and it needs only function values and their signs.

△ Caution: implementation details

Compute the midpoint as `a + (b - a)/2` rather than `(a + b)/2`, which can overflow for huge a, b, and compare *signs* rather than testing `f(a)*f(p) < 0`, since the product can underflow to 0 when both values are tiny.

Stopping criteria

For any iterative method generating $p_1, p_2, \ldots$ we choose a tolerance $\varepsilon > 0$ and a maximum number of iterations N_0, and stop when one of the following holds:

(i) $|p_n - p_{n-1}| < \varepsilon$ (absolute change),

(ii) $\dfrac{|p_n - p_{n-1}|}{|p_n|} < \varepsilon,\ p_n \neq 0$ (relative change; $\varepsilon = 5 \times 10^{-k}$ asks for about k significant digits),

(iii) $|f(p_n)| < \varepsilon$ (small residual),

(iv) $n \geq N_0$ (safeguard: report failure).

None of (i)–(iii) is perfect: a sequence with $|p_n - p_{n-1}| \to 0$ may diverge ($p_n = \sum_{k \le n} 1/k$), and $|f(p_n)|$ can be tiny far from a root if f is very flat. Criterion (ii) is usually preferred because it is independent of the scale of the problem.

Exercises

4.2.1 Theory Show that $f(x) = x^3 + 4x^2 - 10$ has exactly one root in $[1, 2]$, and perform three bisection steps by hand.

4.2.2 Theory How many bisection steps are needed to find $\sqrt{3}$ to within 10^{-8} starting from $[1, 2]$? Verify with Python.

4.2.3 Python Use bisection to find all roots of $f(x) = x \sin x + 1$ in $[0, 10]$ to 8 decimal places. (Plot f first to find brackets.)

4.2.4 Python Modify Listing 4.1 to use the relative stopping criterion (ii) and to count function evaluations. Use it to find the positive root of $e^x = 3x^2$.

4.2.5 Theory Let $f(x) = (x + 2)(x + 1)x(x - 1)^3(x - 2)$. To which root does bisection converge on (a) $[-3, 2.5]$, (b) $[-1.5, 1.5]$, (c) $[-0.5, 2.4]$?

4.2.6 Theory The bisection method is applied to $f(x) = 1/x$ on $[-1, 2]$. What happens? Which hypothesis of Theorem 4.2 is violated?

4.2.7 ★ Graduate Prove that the sequence of midpoints of the bisection method is a Cauchy sequence, and deduce the IVT from the completeness of $\mathbb{R}$ (this is Bolzano's proof).

4.3 Order of convergence

To compare methods we need a precise measure of speed.

Definition 4.6 (order of convergence)**.** Let $x_n \to r$ with $x_n \neq r$, and let $e_n = x_n - r$. If there are constants $\alpha \ge 1$ and $\lambda > 0$ with

$$\lim_{n \to \infty} \frac{|e_{n+1}|}{|e_n|^\alpha} = \lambda,$$

the sequence converges with *order* α and *asymptotic error constant* λ.

- $\alpha = 1$ (with $0 < \lambda < 1$): **linear** convergence;
- $\alpha = 2$: **quadratic** convergence;
- $\alpha = 1$, $\lambda = 0$: **superlinear** convergence;
- $\alpha = 1$, $\lambda = 1$: **sublinear** convergence.

Example 4.7. $e_n = 2^{-n}$: linear with $\lambda = \frac{1}{2}$. $e_n = 1/n$: sublinear. $e_n = 2^{-2^n}$: $e_{n+1} = e_n^2$, quadratic with $\lambda = 1$. $e_n = 1/n!$: superlinear.

★ Key Concept: what the order means in practice

With linear convergence the number of correct digits increases by a *constant amount* per step (about $-\log_{10} \lambda$ digits). With quadratic convergence the number of correct digits roughly *doubles* at every step: $10^{-2} \to 10^{-4} \to 10^{-8} \to 10^{-16}$. Once a quadratically convergent method is close to the root, only a handful of iterations are needed to reach machine precision.

Since $|e_{n+1}| \approx \lambda |e_n|^\alpha$ and $|e_n| \approx \lambda |e_{n-1}|^\alpha$, dividing and taking logarithms gives a practical estimate of the order from three consecutive errors:

$$\alpha \approx \frac{\ln\bigl(|e_{n+1}|/|e_n|\bigr)}{\ln\bigl(|e_n|/|e_{n-1}|\bigr)}. \tag{4.2}$$

If the exact root is unknown, one may replace e_n by $x_{n+1} - x_n$.

Exercises

4.3.1 Theory Determine the order and asymptotic error constant of (a) $x_n = 1/3^n$, (b) $x_n = 10^{-3^n}$, (c) $x_n = 1/n^2$, (d) $x_n = 1/n^n$, all converging to 0.

4.3.2 Theory Show that a sequence of order $\alpha > 1$ is superlinearly convergent.

4.3.3 Theory Show that if $|e_{n+1}| \le \lambda |e_n|$ with $\lambda < 1$, then $|e_n| \le \lambda^n |e_0|$, and find how many iterations are needed to reduce the error by a factor 10^{-6} when $\lambda = 0.9$ and when $\lambda = 0.1$.

4.4 Fixed-point iteration

Definition 4.8. A number p is a *fixed point* of a function g if $g(p) = p$.

For example, $x = 0$ is a fixed point of $g(x) = x(1-x)$ and $x = 1$ is a fixed point of $g(x) = x^2$. Root finding and fixed-point problems are equivalent: $f(x) = 0$ can be rewritten in many ways as $x = g(x)$, e.g. $g(x) = x - f(x)$ or $g(x) = x - c\,f(x)$ for any $c \neq 0$. Conversely, if $g(p) = p$ then p is a root of $f(x) = x - g(x)$. The *fixed-point iteration* is

$$x_{n+1} = g(x_n), \qquad n = 0, 1, 2, \ldots$$

If g is continuous and $x_n \to p$, then $p = \lim x_{n+1} = \lim g(x_n) = g(\lim x_n) = g(p)$: the limit is a fixed point.

Example 4.9. Consider $f(x) = x^2 - 2x - 3 = (x-3)(x+1) = 0$. Three rearrangements are

$$g_1(x) = \sqrt{2x+3}, \qquad g_2(x) = \frac{3}{x-2}, \qquad g_3(x) = \frac{x^2-3}{2}.$$

Starting from $x_0 = 4$ with g_1: $x_1 = \sqrt{11} = 3.316625$, $x_2 = \sqrt{9.633250} = 3.103748$, $x_3 = 3.034385$, $x_4 = 3.011440$, $x_5 = 3.003811$, $\ldots \to 3$.

Listing 4.2: Three fixed-point iterations for $x^2 - 2x - 3 = 0$ and cobweb diagrams
ch04/fixed_point.py

```python
# Fixed-point iteration x_{n+1} = g(x_n) for x^2 - 2x - 3 = 0 (roots 3 and -1)
import math
import numpy as np
import matplotlib.pyplot as plt


def fixed_point(g, x0, tol=1e-8, max_iter=50):
    """Iterate x_{n+1} = g(x_n) until |x_{n+1} - x_n| < tol.

    Returns the list of all iterates (so that we can study convergence)."""
    xs = [x0]
    for _ in range(max_iter):
        try:
            x_new = g(xs[-1])
        except (ValueError, ZeroDivisionError, OverflowError):
            break
        xs.append(x_new)
        if abs(xs[-1] - xs[-2]) < tol or abs(x_new) > 1e8:
            break
    return xs


g1 = lambda x: math.sqrt(2 * x + 3)      # g1'(3) = 1/3
g2 = lambda x: 3 / (x - 2)               # g2'(3) = -3
g3 = lambda x: (x**2 - 3) / 2            # g3'(3) = 3

for name, g in [("g1 = sqrt(2x+3)", g1), ("g2 = 3/(x-2)", g2), ("g3 = (x^2-3)/2", g3)]:
    xs = fixed_point(g, 4.0, max_iter=60)
    shown = ", ".join(f"{x:.6f}" for x in xs[:7])
    print(f"{name:16s}: {shown}, ...  ({len(xs) - 1} its, last = {xs[-1]:.6g})")

# Error ratios for g1: |x_{n+1}-3| / |x_n-3|  ->  |g1'(3)| = 1/3
xs = fixed_point(g1, 4.0, tol=1e-14)
print("\n n      x_n              |e_n|        |e_n|/|e_{n-1}|")
for n, x in enumerate(xs[:12]):
    e = abs(x - 3)
    ratio = e / abs(xs[n - 1] - 3) if n > 0 else float("nan")
    print(f"{n:2d}  {x:.12f}  {e:.3e}   {ratio:.6f}")

# Cobweb diagrams
fig, axes = plt.subplots(1, 2, figsize=(8, 3.6))
for ax, g, x0, title, lim in [(axes[0], g1, 4.0, r"$g(x)=\sqrt{2x+3}$ (converges)", (0, 5)),
                              (axes[1], g3, 3.1, r"$g(x)=(x^2-3)/2$ (diverges)", (2.5, 5))]:
    xx = np.linspace(*lim, 200)
    ax.plot(xx, [g(t) for t in xx], label="y = g(x)")
    ax.plot(xx, xx, "k--", lw=0.8, label="y = x")
    xs = fixed_point(g, x0, max_iter=6)
    for k in range(len(xs) - 1):
        ax.plot([xs[k], xs[k]], [xs[k], xs[k + 1]], "r", lw=0.8)
        ax.plot([xs[k], xs[k + 1]], [xs[k + 1], xs[k + 1]], "r", lw=0.8)
    ax.set_xlim(*lim)
    ax.set_ylim(*lim)
    ax.set_title(title, fontsize=10)
    ax.legend(fontsize=8, loc="upper left")
plt.tight_layout()
plt.savefig("ch04_cobweb.pdf")
```

Output

```
g1 = sqrt(2x+3) : 4.000000, 3.316625, 3.103748, 3.034385, 3.011440, 3.003811, 3.001270, ...
    ↪   (18 its, last = 3)
g2 = 3/(x-2)    : 4.000000, 1.500000, -6.000000, -0.375000, -1.263158, -0.919355,
    ↪ -1.027624, ...  (21 its, last = -1)
g3 = (x^2-3)/2  : 4.000000, 6.500000, 19.625000, 191.070312, 18252.432159,
    ↪ 166575638.367185, ...  (5 its, last = 1.66576e+08)

 n     x_n              |e_n|        |e_n|/|e_{n-1}|
 0  4.000000000000  1.000e+00   nan
 1  3.316624790355  3.166e-01   0.316625
 2  3.103747667049  1.037e-01   0.327668
 3  3.034385495302  3.439e-02   0.331434
 4  3.011440019426  1.144e-02   0.332699
 5  3.003810919291  3.811e-03   0.333122
 6  3.001270037598  1.270e-03   0.333263
 7  3.000423316000  4.233e-04   0.333310
 8  3.000141102015  1.411e-04   0.333325
 9  3.000047033636  4.703e-05   0.333331
10  3.000015677838  1.568e-05   0.333332
11  3.000005225941  5.226e-06   0.333333
```

The same equation gives three completely different behaviours: g_1 converges to 3 linearly with ratio $1/3$; g_2 is repelled by 3 and converges to the other root -1; g_3 diverges. The theory below explains why: $|g_1'(3)| = \frac{1}{3} < 1$, $|g_2'(3)| = 3 > 1$, $|g_3'(3)| = 3 > 1$.

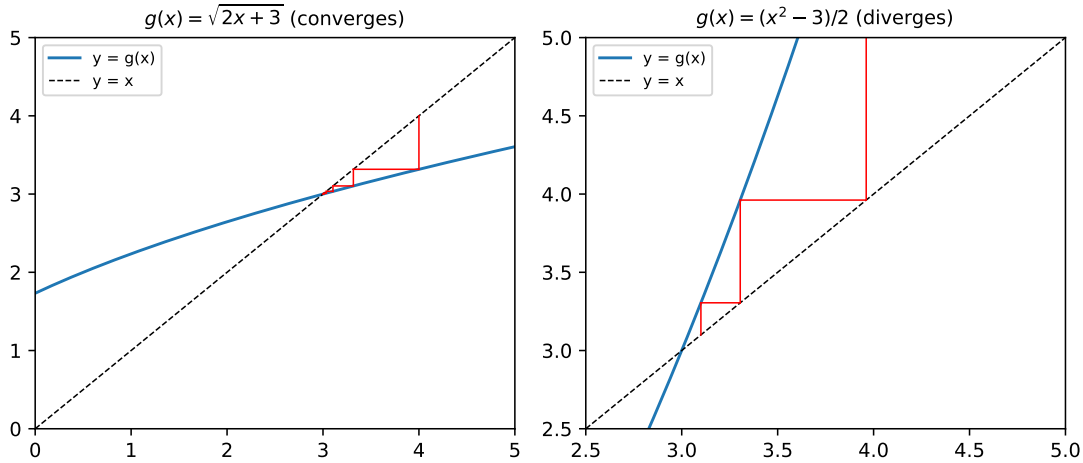


Figure 4.1: Cobweb diagrams: move vertically to the curve $y = g(x)$, then horizontally to the line $y = x$. Left: $|g'| < 1$ near the fixed point (convergence). Right: $|g'| > 1$ (divergence).

Theorem 4.10 (fixed-point theorem). *1. **(Existence)** If $g \in C[a,b]$ and $g(x) \in [a,b]$ for all $x \in [a,b]$, then g has at least one fixed point in $[a,b]$.*

*2. **(Uniqueness)** If, in addition, g' exists on (a,b) and there is a constant $0 < k < 1$ with $|g'(x)| \le k$ for all $x \in (a,b)$, then the fixed point is unique.*

*3. **(Convergence)** Under the hypotheses of (1) and (2), for* any *$x_0 \in [a,b]$ the iteration $x_{n+1} = g(x_n)$*

converges to the fixed point p, and

$$|x_n - p| \le k^n \max\{x_0 - a,\ b - x_0\}, \qquad |x_n - p| \le \frac{k^n}{1-k}|x_1 - x_0|.$$

Proof. (1) If $g(a) = a$ or $g(b) = b$ we are done. Otherwise $g(a) > a$ and $g(b) < b$ because g maps into $[a, b]$. The function $h(x) = g(x) - x$ is continuous with $h(a) > 0 > h(b)$, so by the IVT there is $p \in (a, b)$ with $h(p) = 0$, i.e. $g(p) = p$.

(2) Suppose $p \neq q$ are both fixed points. By the Mean Value Theorem there is ξ between p and q with

$$|p - q| = |g(p) - g(q)| = |g'(\xi)|\,|p - q| \le k|p - q| < |p - q|,$$

a contradiction. Hence $p = q$.

(3) All iterates lie in $[a, b]$. By the MVT, $|x_n - p| = |g(x_{n-1}) - g(p)| \le k|x_{n-1} - p|$, so $|x_n - p| \le k^n|x_0 - p| \le k^n \max\{x_0 - a, b - x_0\} \to 0$. For the second bound, $|x_{n+1} - x_n| \le k^n|x_1 - x_0|$, and for $m > n$,

$$|x_m - x_n| \le \sum_{j=n}^{m-1} |x_{j+1} - x_j| \le |x_1 - x_0| \sum_{j=n}^{\infty} k^j = \frac{k^n}{1-k}|x_1 - x_0|.$$

Letting $m \to \infty$ gives the result. □

★ Key Concept: the derivative decides

Near a fixed point p, $e_{n+1} = g(x_n) - g(p) = g'(\xi_n)e_n \approx g'(p)e_n$. Hence

- $|g'(p)| < 1$: p is **attracting**; convergence is linear with rate $|g'(p)|$ for x_0 close to p;
- $|g'(p)| > 1$: p is **repelling**; the iteration moves away from p;
- $g'(p) = 0$: convergence is faster than linear (Theorem 4.11).

The smaller $|g'(p)|$, the faster the convergence. Designing a good iteration means choosing g with $|g'(p)|$ as small as possible.

Theorem 4.11 (higher-order fixed-point iteration)**.** *Let $p = g(p)$, $g'(p) = 0$ and let g'' be continuous with $|g''(x)| \le M$ near p. Then for x_0 sufficiently close to p the iteration converges at least quadratically:*

$$|x_{n+1} - p| \le \frac{M}{2}|x_n - p|^2, \qquad \lim_{n\to\infty} \frac{x_{n+1} - p}{(x_n - p)^2} = \frac{g''(p)}{2}.$$

Proof. By Taylor's theorem, $g(x_n) = g(p) + g'(p)(x_n - p) + \frac{1}{2}g''(\xi_n)(x_n - p)^2$ with ξ_n between x_n and p. Since $g(p) = p$ and $g'(p) = 0$, $x_{n+1} - p = \frac{1}{2}g''(\xi_n)(x_n - p)^2$. Convergence for x_0 close to p follows from Theorem 4.10 applied on a small interval around p on which $|g'| \le k < 1$, and $\xi_n \to p$ gives the limit. □

Historical Note: Brouwer, Banach and Picard

Theorem 4.10 is a special case of two of the most important theorems of twentieth century analysis. **L. E. J. Brouwer** proved in 1911 that every continuous map of a closed ball in $\mathbb{R}^n$ into itself has a fixed point (part (1) is the case $n = 1$). In his 1920 doctoral thesis (published 1922) the Polish mathematician **Stefan Banach** proved the *contraction mapping principle*: if T maps a complete metric space into itself and $d(Tx, Ty) \le k\,d(x, y)$ with $k < 1$, then T has a unique fixed point, and the iteration $x_{n+1} = Tx_n$ converges to it from any starting point. **Émile Picard** had used such iterations in 1890 to prove existence of solutions of differential equations.

Towards Graduate Studies: fixed points everywhere

Banach's theorem is used to prove the Picard–Lindelöf existence theorem for ODEs, the inverse and implicit function theorems, and existence of solutions of integral equations; iterative methods for linear systems (Chapter 7) are fixed-point iterations in $\mathbb{R}^n$; value iteration in dynamic programming and reinforcement learning is a contraction mapping. Graduate courses in functional analysis and in the numerical analysis of PDEs rely on this theorem constantly.

Exercises

4.4.1 Theory Show that $g(x) = (x^2-1)/3$ has a unique fixed point in $[-1,1]$. Estimate the number of iterations needed to reach accuracy 10^{-5} from $x_0 = 0$ using the a priori bound.

4.4.2 Theory For $f(x) = x^3+4x^2-10$ on $[1,2]$ consider $g_a(x) = x-x^3-4x^2+10$, $g_b(x) = \tfrac{1}{2}(10-x^3)^{1/2}$, $g_c(x) = \big(10/(4+x)\big)^{1/2}$ and $g_d(x) = x - \dfrac{x^3+4x^2-10}{3x^2+8x}$. Predict from $|g'(p)|$ (with $p \approx 1.365$) which iterations converge and how fast. Verify in Python. (This is a classic example from Burden & Faires [2].)

4.4.3 Python Solve $x = e^{-x}$ by fixed-point iteration. Estimate the rate of convergence from the output and compare with $|g'(p)|$.

4.4.4 Theory Show that $g(x) = \cos x$ is a contraction on $[0,1]$, and hence that $x = \cos x$ has a unique solution there (the "Dottie number" $0.739085\ldots$).

4.4.5 Theory For which values of c does $x_{n+1} = x_n - c(x_n^2 - 2)$ converge to $\sqrt{2}$ for x_0 near $\sqrt{2}$? Which value of c gives quadratic convergence?

4.4.6 ★ Graduate (Logistic map) Study $x_{n+1} = \mu x_n(1-x_n)$ for $\mu = 2.8, 3.2, 3.5, 3.9$ and $x_0 = 0.2$. Find the fixed points and determine their stability. Plot the iterates and describe what happens (period doubling and chaos; see May, *Nature* 1976).

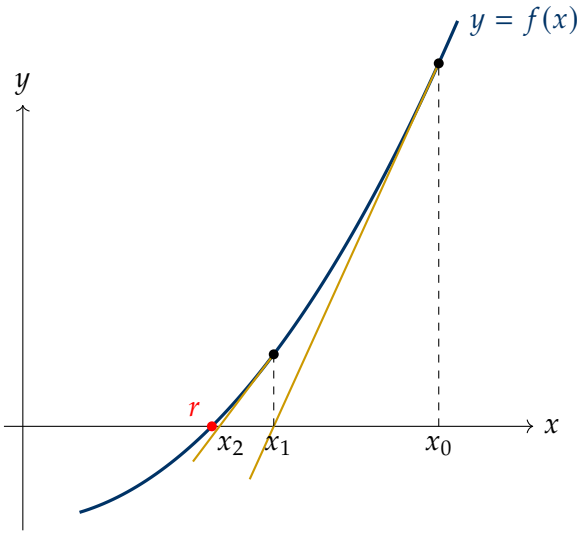
4.5 Newton's method

Newton's method (also called the Newton–Raphson method) is the most important method for solving nonlinear equations.

Derivation (geometric). At the current approximation x_n, replace the curve $y = f(x)$ by its tangent line $y = f(x_n) + f'(x_n)(x - x_n)$ and take the point where the tangent crosses the x-axis as the next approximation:

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \qquad f'(x_n) \neq 0.} \tag{4.3}$$

Derivation (Taylor). If $r = x_n + h$ is the root, then $0 = f(x_n + h) = f(x_n) + h f'(x_n) + O(h^2)$. Neglecting the $O(h^2)$ term and solving for h gives $h \approx -f(x_n)/f'(x_n)$. This derivation generalises to systems (Chapter 5).



Algorithm 2 Newton's method

Require: f, f', initial guess x_0, tolerance ε, maximum iterations N

1: **for** $n = 0, 1, \ldots, N-1$ **do**
2: **if** $f'(x_n) = 0$ **then**
3: **error:** zero derivative
4: **end if**
5: $x_{n+1} \leftarrow x_n - f(x_n)/f'(x_n)$
6: **if** $|x_{n+1} - x_n| < \varepsilon |x_{n+1}|$ **then**
7: **return** x_{n+1}
8: **end if**
9: **end for**
10: **error:** no convergence

Example 4.12. For $f(x) = x^3 - x^2 - x - 1$, $f'(x) = 3x^2 - 2x - 1$ and $x_0 = 1.5$:

$$x_1 = 1.5 - \frac{f(1.5)}{f'(1.5)} = 1.5 - \frac{-1.375}{2.75} = 2, \qquad x_2 = 2 - \frac{f(2)}{f'(2)} = 2 - \frac{1}{7} = 1.857142857, \ldots$$

Listing 4.3: Newton's method and its quadratic convergence ch04/newton.py

```python
# Newton's method, with a table showing quadratic convergence
def newton(f, df, x0, tol=1e-12, max_iter=50):
    """Newton's method x_{n+1} = x_n - f(x_n)/f'(x_n).

    Returns the list of iterates. Stops when |x_{n+1} - x_n| < tol."""
    xs = [x0]
    for _ in range(max_iter):
        d = df(xs[-1])
        if d == 0:
            raise ZeroDivisionError("f'(x_n) = 0: Newton's method breaks down")
        xs.append(xs[-1] - f(xs[-1]) / d)
        if abs(xs[-1] - xs[-2]) < tol:
            return xs
    raise RuntimeError("Newton's method did not converge")


f = lambda x: x**3 - x**2 - x - 1
df = lambda x: 3 * x**2 - 2 * x - 1
r = 1.839286755214161                 # the exact root, for measuring errors

xs = newton(f, df, 1.5)
print(" n        x_n                 |e_n|       |e_n|/|e_{n-1}|^2")
for n, x in enumerate(xs):
    e = abs(x - r)
    # once e_n reaches rounding level (~1e-16) the ratio is meaningless
    ratio = f"{e / abs(xs[n - 1] - r) ** 2:.4f}" if 0 < n and e > 1e-15 else "-"
    print(f"{n:2d}  {x:.16f}  {e:.3e}   {ratio}")
print("theory: |f''(r) / (2 f'(r))| =", abs((6 * r - 2) / (2 * df(r))))

# Leonardo of Pisa's equation (1225):  x^3 + 2x^2 + 10x - 20 = 0
g = lambda x: x**3 + 2 * x**2 + 10 * x - 20
dg = lambda x: 3 * x**2 + 4 * x + 10
xs = newton(g, dg, 1.0)
print("\nFibonacci's equation:", ", ".join(f"{x:.10f}" for x in xs))
```

Output

```
 n        x_n                 |e_n|       |e_n|/|e_{n-1}|^2
 0  1.5000000000000000  3.393e-01   -
 1  2.0000000000000000  1.607e-01   1.3961
 2  1.8571428571428572  1.786e-02   0.6913
 3  1.8395445134575570  2.578e-04   0.8084
 4  1.8392868100680193  5.485e-08   0.8256
 5  1.8392867552141636  2.665e-15   0.8855
 6  1.8392867552141612  2.220e-16   -
theory: |f''(r) / (2 f'(r))| = 0.8258808180165279

Fibonacci's equation: 1.0000000000, 1.4117647059, 1.3693364706, 1.3688081886, 1.3688081078,
    ↪ 1.3688081078
```

The number of correct digits goes $1, 2, 4, 8, 15$: it doubles at every step, and the ratio $|e_{n+1}|/|e_n|^2$ approaches $|f''(r)/(2f'(r))| = 0.826$ exactly as Theorem 4.13 below predicts. Newton's method also reproduces Fibonacci's root 1.3688081078 in four steps.

Theorem 4.13 (local quadratic convergence of Newton's method). *Let $f \in C^2[a,b]$, and let $r \in (a,b)$ satisfy $f(r) = 0$ and $f'(r) \neq 0$. Then there is a $\delta > 0$ such that Newton's method converges to r for every $x_0 \in [r-\delta, r+\delta]$, and*

$$\lim_{n\to\infty} \frac{x_{n+1} - r}{(x_n - r)^2} = \frac{f''(r)}{2f'(r)}.$$

Proof. Let $e_n = x_n - r$. Expand f about x_n and evaluate at r:

$$0 = f(r) = f(x_n) + f'(x_n)(r - x_n) + \tfrac{1}{2} f''(\xi_n)(r - x_n)^2,$$

with ξ_n between x_n and r. Divide by $f'(x_n)$ (non-zero near r by continuity) and use $x_{n+1} = x_n - f(x_n)/f'(x_n)$:

$$x_{n+1} - r = \frac{f''(\xi_n)}{2f'(x_n)}(x_n - r)^2, \qquad \text{i.e.} \qquad e_{n+1} = \frac{f''(\xi_n)}{2f'(x_n)} e_n^2.$$

Choose $\delta > 0$ so that $f'(x) \neq 0$ on $I = [r-\delta, r+\delta]$ and set $M = \max_I |f''|/(2\min_I |f'|)$. Shrinking δ if necessary, $M\delta < 1$. If $x_n \in I$ then $|e_{n+1}| \leq M|e_n|^2 \leq (M\delta)|e_n| < |e_n|$, so $x_{n+1} \in I$; by induction $|e_n| \leq (M\delta)^n |e_0| \to 0$. Finally $\xi_n \to r$ and $x_n \to r$ give the limit. □

⋆ Key Concept: Newton's method as a fixed-point iteration

Newton's method is the fixed-point iteration for $g(x) = x - f(x)/f'(x)$, and

$$g'(x) = \frac{f(x)f''(x)}{f'(x)^2} \quad \Longrightarrow \quad g'(r) = 0 \ \text{ when } f'(r) \neq 0.$$

By Theorem 4.11, this is exactly why the convergence is quadratic.

4.5.1 When Newton's method fails

Theorem 4.13 is a *local* result: it guarantees convergence only for x_0 sufficiently close to r. Things can go wrong.

Listing 4.4: Poor starting values, cycles and multiple roots ch04/newton_fail.py

```
# When Newton's method struggles: bad starting values, cycles and multiple roots
def newton_iterates(f, df, x0, n):
    xs = [x0]
    for _ in range(n):
        xs.append(xs[-1] - f(xs[-1]) / df(xs[-1]))
    return xs


# 1. A poor starting value: x0 = 0.5 for x^3 - x^2 - x - 1
f = lambda x: x**3 - x**2 - x - 1
df = lambda x: 3 * x**2 - 2 * x - 1
xs = newton_iterates(f, df, 0.5, 30)
print("x0 = 0.5 :", ", ".join(f"{x:.4f}" for x in xs[:12]), "...")
print("   iterate 30 =", xs[-1])

# 2. A 2-cycle: x^3 - 2x + 2 with x0 = 0
g = lambda x: x**3 - 2 * x + 2
dg = lambda x: 3 * x**2 - 2
print("cycle    :", newton_iterates(g, dg, 0.0, 6))

```

```python
# 3. A double root: f(x) = (x - 1)^2 (x + 2), root r = 1 of multiplicity 2
h = lambda x: (x - 1) ** 2 * (x + 2)
dh = lambda x: 2 * (x - 1) * (x + 2) + (x - 1) ** 2
print("\n n   Newton e_n     ratio    | modified Newton (m=2) e_n")
x, y = 2.0, 2.0
e_old = abs(x - 1)
for n in range(1, 9):
    x = x - h(x) / dh(x)
    y = y - 2 * h(y) / dh(y) if abs(y - 1) > 1e-15 else y
    e = abs(x - 1)
    print(f"{n:2d}   {e:.3e}    {e / e_old:.4f}   | {abs(y - 1):.3e}")
    e_old = e
```

Output

```
x0 = 0.5 : 0.5000, -0.8000, -0.2635, -3.3769, -2.1875, -1.3942, -0.8352, -0.3122, -10.1326,
    ↪ -6.6681, -4.3673, -2.8431 ...
   iterate 30 = 1.8392867552141612
cycle    : [0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0]

 n   Newton e_n     ratio    | modified Newton (m=2) e_n
 1   5.556e-01    0.5556   | 1.111e-01
 2   2.979e-01    0.5362   | 1.949e-03
 3   1.554e-01    0.5216   | 6.327e-07
 4   7.956e-02    0.5120   | 6.661e-14
 5   4.029e-02    0.5064   | 0.000e+00
 6   2.028e-02    0.5033   | 0.000e+00
 7   1.017e-02    0.5017   | 0.000e+00
 8   5.095e-03    0.5008   | 0.000e+00
```

1. **Poor starting value.** For $f(x) = x^3 - x^2 - x - 1$ with $x_0 = 0.5$, we have $f'(0.5) = -1.25 < 0$ and the tangent sends $x_1 = -0.8$ far away. The iterates wander for many steps before (by luck) landing near the root.
2. **Cycling.** For $f(x) = x^3 - 2x + 2$ and $x_0 = 0$ the iterates alternate $0, 1, 0, 1, \ldots$ forever.
3. **Zero derivative.** If $f'(x_n) = 0$ the tangent is horizontal and the method breaks down.
4. **Multiple roots.** If $f(r) = f'(r) = 0$ convergence is only *linear*.

Theorem 4.14 (multiple roots). *If $f(x) = (x-r)^m q(x)$ with $q(r) \neq 0$ and $m \geq 2$ (a root of multiplicity m), Newton's method converges linearly with $\lambda = (m-1)/m$. The* modified Newton method

$$x_{n+1} = x_n - m\,\frac{f(x_n)}{f'(x_n)}$$

restores quadratic convergence.

Sketch. Here $g(x) = x - \frac{(x-r)q(x)}{mq(x)+(x-r)q'(x)}$, and a direct computation gives $g'(r) = 1 - \frac{1}{m} \neq 0$. For the modified method $g(x) = x - mf/f'$ gives $g'(r) = 0$. □

The output confirms the ratio $(2-1)/2 = 0.5$ for the double root of $(x-1)^2(x+2)$, and the quadratic convergence of the modified method. In practice m is usually unknown; an alternative is to apply Newton's method to $\mu(x) = f(x)/f'(x)$, which has a *simple* root at r.

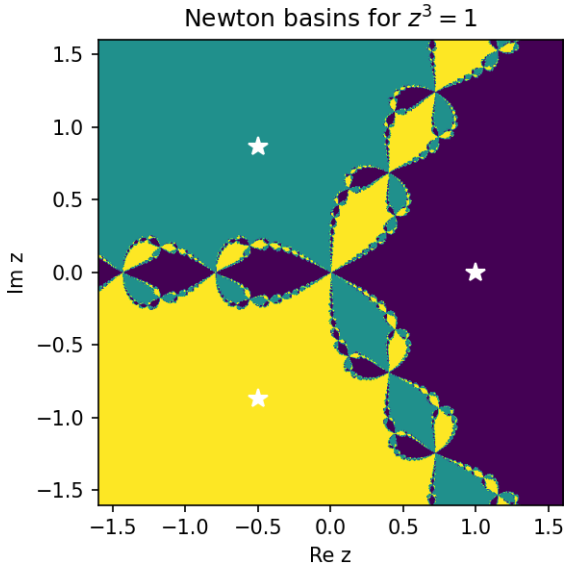


Figure 4.2: Basins of attraction of Newton's method for $z^3 = 1$. Each colour shows the starting points that converge to one of the three cube roots of unity (stars). The boundaries are fractals. Produced by `code/ch04/newton_fractal.py`.

Historical Note: Newton, Raphson and Simpson

Isaac Newton described his method in *De analysi* (written 1669, published 1711), applying it to $x^3 - 2x - 5 = 0$. His version was quite different from ours: he successively substituted $x = 2 + p$, then $p = 0.1 + q$, and so on, working with a new polynomial each time. **Joseph Raphson** published a simpler, iterative formulation in 1690, still only for polynomials. The general formula with derivatives, $x_{n+1} = x_n - f(x_n)/f'(x_n)$, is due to **Thomas Simpson** (1740), who also extended it to systems of two equations. Convergence was first analysed by Fourier (1818) and Cauchy (1829). In 1879 Arthur **Cayley** asked which starting points in the complex plane converge to which root of $z^3 = 1$; the answer, revealed by computer graphics a century later, is the fractal of Fig. 4.2.

Towards Graduate Studies: Kantorovich and Smale

Theorem 4.13 assumes a root exists. **Kantorovich's theorem** (1948) makes no such assumption: from computable quantities at x_0 (namely $f(x_0)$, $f'(x_0)$ and a Lipschitz constant of f') it proves that a root exists and that Newton's method converges to it, in Banach spaces of any dimension. Smale's α-theory (1986) gives a similar criterion using only derivatives at x_0. Newton's method in function spaces is the basis of modern methods for nonlinear PDEs and of optimisation algorithms (Newton, quasi-Newton, Gauss–Newton, interior point methods). See Deuflhard, *Newton Methods for Nonlinear Problems*.

Exercises

4.5.1 Theory Perform three Newton steps for $f(x) = x^3 + 2x^2 + 10x - 20$ with $x_0 = 1$, by hand, and compare with the output above.

4.5.2 Theory Show that Newton's method for $f(x) = x^2 - a$ ($a > 0$) is the Babylonian iteration $x_{n+1} = \tfrac{1}{2}(x_n + a/x_n)$. Prove that it converges for every $x_0 > 0$ (show $x_n \geq \sqrt{a}$ for $n \geq 1$ and that the sequence decreases).

4.5.3 Theory Derive a Newton iteration for computing $1/a$ that uses no divisions. (Early computers, and some modern processors, divide in exactly this way.)

4.5.4 Theory Show that Newton's method for $f(x) = x^{1/3}$ diverges for every $x_0 \neq 0$. Which hypothesis of Theorem 4.13 fails?

4.5.5 Python Use Newton's method to find the smallest positive root of $\tan x = x$ to 12 digits. Choose x_0 carefully (plot first).

4.5.6 Python A GH¢20,000 loan is repaid with 36 monthly payments of GH¢800. The monthly interest rate i satisfies $20000 = 800\,\frac{1-(1+i)^{-36}}{i}$. Find i with Newton's method, and the corresponding annual rate.

4.5.7 Python For $f(x) = (x-2)^3(x+1)$, compare Newton, modified Newton with $m = 3$, and Newton applied to $\mu = f/f'$, starting from $x_0 = 3$. Estimate the order of each.

4.5.8 ★ Graduate Suppose $f \in C^3$, $f(r) = 0$, $f'(r) \neq 0$. Show that *Halley's method* $x_{n+1} = x_n - \dfrac{2f(x_n)f'(x_n)}{2f'(x_n)^2 - f(x_n)f''(x_n)}$ converges cubically.

4.5.9 ★ Graduate Modify `newton_fractal.py` to draw the basins of attraction for $z^4 = 1$ and colour each point by the number of iterations needed.

4.6 The secant method

Newton's method needs $f'(x)$, which may be expensive or unavailable. Replacing the derivative by the slope of the *secant* through the last two iterates, $f'(x_n) \approx \dfrac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$, gives the **secant method**

$$x_{n+1} = x_n - f(x_n)\,\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}, \qquad n = 1, 2, \dots \tag{4.4}$$

Geometrically, x_{n+1} is where the straight line through $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$ crosses the x-axis. Two starting values are needed, but only *one new function evaluation* per step.

> **Example 4.15.** For $f(x) = x^3 - x^2 - x - 1$, $x_0 = 1$, $x_1 = 2$: $f(1) = -2$, $f(2) = 1$, so $x_2 = 2 - 1 \cdot \frac{2-1}{1-(-2)} = 1.6667$, $f(x_2) = -0.8148$; $x_3 = 1.6667 + 0.8148 \cdot \frac{1.6667-2}{-0.8148-1} = 1.8163$, and so on.

Listing 4.5: The secant method ch04/secant.py

```python
# The secant method and its order of convergence (the golden ratio!)
import math


def secant(f, x0, x1, tol=1e-12, max_iter=50):
    """Secant method. Returns the list of iterates x0, x1, x2, ..."""
    xs = [x0, x1]
    for _ in range(max_iter):
        f0, f1 = f(xs[-2]), f(xs[-1])
        if f1 == f0:
            break
        xs.append(xs[-1] - f1 * (xs[-1] - xs[-2]) / (f1 - f0))
        if abs(xs[-1] - xs[-2]) < tol:
            break
    return xs


f = lambda x: x**3 - x**2 - x - 1
r = 1.839286755214161
xs = secant(f, 1.0, 2.0)
e = [abs(x - r) for x in xs]
print(" n        x_n                 |e_n|      order estimate")
for n, x in enumerate(xs):
    q = ""
    if n >= 4 and e[n] > 1e-15:     # skip the first steps and rounding level
        q = f"{math.log(e[n] / e[n - 1]) / math.log(e[n - 1] / e[n - 2]):.4f}"
    print(f"{n:2d}  {x:.16f}  {e[n]:.3e}   {q}")
print("golden ratio (1 + sqrt 5)/2 =", (1 + math.sqrt(5)) / 2)
```

Output

```
 n        x_n                 |e_n|      order estimate
 0  1.0000000000000000  8.393e-01
 1  2.0000000000000000  1.607e-01
 2  1.6666666666666667  1.726e-01
 3  1.8163265306122449  2.296e-02
 4  1.8429939112321743  3.707e-03   0.9039
 5  1.8392156332109642  7.112e-05   2.1682
 6  1.8392865379398711  2.173e-07   1.4647
 7  1.8392867552269239  1.276e-11   1.6823
 8  1.8392867552141612  2.220e-16
 9  1.8392867552141612  2.220e-16
golden ratio (1 + sqrt 5)/2 = 1.618033988749895
```

Theorem 4.16 (order of the secant method). *If $f \in C^2$, $f(r) = 0$, $f'(r) \neq 0$ and x_0, x_1 are sufficiently close to r, the secant method converges with order*

$$\alpha = \frac{1+\sqrt{5}}{2} \approx 1.618 \quad \textit{(the golden ratio)}.$$

Sketch. A Taylor expansion (see [2, 1]) gives $e_{n+1} \approx C e_n e_{n-1}$ with $C = f''(r)/(2f'(r))$. Try $|e_{n+1}| \approx \lambda |e_n|^\alpha$: then $\lambda |e_n|^\alpha \approx |C||e_n|\,(|e_n|/\lambda)^{1/\alpha}$, and comparing powers of $|e_n|$ gives $\alpha = 1 + 1/\alpha$, i.e. $\alpha^2 - \alpha - 1 = 0$, whose positive root is the golden ratio. (Taking logarithms, $\ln|e_{n+1}| \approx \ln|e_n| + \ln|e_{n-1}|$ plus a constant: the exponents grow like Fibonacci numbers!) □

⋆ Key Concept: Newton or secant?

Newton has order 2 but needs f and f' per step; secant has order 1.618 but needs only one new f. Two secant steps (cost: 2 evaluations) have order $1.618^2 = 2.618 > 2$, so if evaluating f' costs about as much as f, the secant method is actually *more efficient* than Newton's method.

Exercises

4.6.1 Theory Perform three secant steps for $f(x) = \cos x - x$ with $x_0 = 0.5$, $x_1 = \pi/4$.

4.6.2 Theory Show that formula (4.4) can be written as $x_{n+1} = \dfrac{x_{n-1}f(x_n) - x_n f(x_{n-1})}{f(x_n) - f(x_{n-1})}$. Why is this form less suitable for computation? (Think about cancellation.)

4.6.3 Python Apply the secant method to $f(x) = x^2 - 2$ with $x_0 = 1$, $x_1 = 2$ and verify numerically that $\ln|e_n|$ behaves like a Fibonacci sequence.

4.6.4 ★ Graduate Complete the proof that the secant method has order $(1+\sqrt{5})/2$, including the derivation of $e_{n+1} \approx C e_n e_{n-1}$ using divided differences (Chapter 9).

4.7 The method of false position (regula falsi)

The secant method is fast but may diverge. **Regula falsi** combines the secant idea with bracketing: start with a bracket $[a, b]$, $f(a)f(b) < 0$, take the x-intercept of the chord

$$c = b - f(b)\,\frac{b-a}{f(b)-f(a)},$$

and keep whichever of $[a, c]$ and $[c, b]$ still contains a sign change. The only difference from bisection is the choice of c; the only difference from the secant method is that the bracket is retained.

> **Example 4.17.** For $f(x) = x^3 + 2x^2 + 10x - 20$ on $[1, 2]$ (Fibonacci's equation) we have $f(1) = -7$, $f(2) = 16$ and $c_1 = 2 - 16 \cdot \frac{1}{23} = 1.30435$, $f(c_1) = -1.33475 < 0$, so the new bracket is $[1.30435, 2]$.

Listing 4.6: False position and the Illinois modification ch04/regula_falsi.py

```python
# Regula falsi (false position) and the Illinois modification
def regula_falsi(f, a, b, tol=1e-10, max_iter=200, illinois=False, verbose=False):
    """False position on [a, b] with f(a) f(b) < 0.

    If illinois=True, the function value at an endpoint that is retained twice in
    a row is halved (Dowell & Jarratt, 1971), which prevents stagnation.
    Returns (root, number of iterations)."""
    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("f(a) and f(b) must have opposite signs")
    side = 0
    c_old = a
    for n in range(1, max_iter + 1):
        c = b - fb * (b - a) / (fb - fa)       # x-intercept of the chord
        fc = f(c)
        if verbose:
            print(f"{n:3d} {a:12.8f} {b:12.8f} {c:12.8f} {fc:12.4e}")
        if fc == 0 or abs(c - c_old) < tol:
            return c, n
        c_old = c
        if fa * fc < 0:          # root in [a, c]: replace b
            b, fb = c, fc
            if illinois and side == -1:
                fa /= 2
            side = -1
        else:                    # root in [c, b]: replace a
            a, fa = c, fc
            if illinois and side == +1:
                fb /= 2
            side = +1
    raise RuntimeError("no convergence")


f = lambda x: x**3 + 2 * x**2 + 10 * x - 20
print(f"{'n':>3} {'a_n':>12} {'b_n':>12} {'c_n':>12} {'f(c_n)':>12}")
root, n = regula_falsi(f, 1, 2, verbose=True)
print(f"regula falsi: root = {root:.10f} in {n} iterations")
root, n = regula_falsi(f, 1, 2, illinois=True)
print(f"Illinois    : root = {root:.10f} in {n} iterations")

# A convex function on which plain regula falsi is very slow
g = lambda x: x**10 - 1
for ill in [False, True]:
    root, n = regula_falsi(g, 0, 1.3, illinois=ill, max_iter=1000)
    print(f"x^10 - 1 on [0, 1.3], illinois={ill!s:5}: root = {root:.10f}, {n} iterations"
    ↪ )
```

Output

```
  n          a_n          b_n          c_n       f(c_n)
  1   1.00000000   2.00000000   1.30434783  -1.3348e+00
  2   1.30434783   2.00000000   1.35791230  -2.2914e-01
  3   1.35791230   2.00000000   1.36697780  -3.8592e-02
  4   1.36697780   2.00000000   1.36850098  -6.4787e-03
  5   1.36850098   2.00000000   1.36875658  -1.0870e-03
  6   1.36875658   2.00000000   1.36879946  -1.8237e-04
  7   1.36879946   2.00000000   1.36880666  -3.0597e-05
  8   1.36880666   2.00000000   1.36880786  -5.1331e-06
  9   1.36880786   2.00000000   1.36880807  -8.6118e-07
 10   1.36880807   2.00000000   1.36880810  -1.4448e-07
 11   1.36880810   2.00000000   1.36880811  -2.4239e-08
 12   1.36880811   2.00000000   1.36880811  -4.0665e-09
 13   1.36880811   2.00000000   1.36880811  -6.8223e-10
 14   1.36880811   2.00000000   1.36880811  -1.1445e-10
regula falsi: root = 1.3688081078 in 14 iterations
Illinois    : root = 1.3688081078 in 8 iterations
x^10 - 1 on [0, 1.3], illinois=False: root = 0.9999999997, 91 iterations
x^10 - 1 on [0, 1.3], illinois=True : root = 1.0000000000, 15 iterations
```

△ Caution: one-sided convergence

Look at the column b_n: it is stuck at 2 for the whole computation. For a function that is convex (or concave) on the bracket, one endpoint never moves; the bracket length does not tend to zero and the convergence is only *linear*, sometimes extremely slowly (91 iterations for $x^{10} - 1$). The **Illinois modification** (Dowell & Jarratt, 1971) halves the stored function value at an endpoint that has been retained twice in succession. This simple trick restores superlinear convergence (8 and 15 iterations in the output).

Historical Note: the rule of false position

"False position" means: guess an answer (a false position), see how wrong it is, and correct the guess proportionally. For linear problems a single correction gives the exact answer, and the method appears in the Rhind papyrus. The Chinese *Nine Chapters* used two guesses ("excess and deficit"), which is exactly the chord formula. Via Arabic mathematicians, who called it *hisab al-khata'ayn* ("reckoning by two errors"), it reached Fibonacci's *Liber Abaci* (1202) and European schoolbooks, where it was taught until the 19th century.

Exercises

4.7.1 Theory Perform three steps of regula falsi for $f(x) = x^3 - x^2 - x - 1$ on $[1, 2]$ and compare with the bisection and secant iterates.

4.7.2 Theory Show that if $f'' > 0$ and $f' > 0$ on $[a, b]$ then the right endpoint b is never replaced in regula falsi.

4.7.3 Python Compare bisection, regula falsi, Illinois, secant and Newton on $f(x) = e^x - x - 2$ on $[0, 2]$ with tolerance 10^{-10}. Tabulate iterations and function evaluations.

4.8 Comparison, acceleration and software

Listing 4.7: All methods on $x^3 - x^2 - x - 1 = 0$ ch04/compare_methods.py

```
# Comparing bisection, regula falsi, fixed point, secant and Newton on one problem
import math
import numpy as np
import matplotlib.pyplot as plt

f = lambda x: x**3 - x**2 - x - 1
df = lambda x: 3 * x**2 - 2 * x - 1
r = 1.839286755214161
N = 40


def errors_bisection(a, b):
    e = []
    for _ in range(N):
        p = (a + b) / 2
        e.append(abs(p - r))
        if f(a) * f(p) < 0:
            b = p
        else:
            a = p
    return e


def errors_regula(a, b):
    e = []
    for _ in range(N):
        c = b - f(b) * (b - a) / (f(b) - f(a))
        e.append(abs(c - r))
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return e


def errors_fixed(x):
    # x = g(x) = (x^2 + x + 1)^(1/3); g'(r) ~ 0.44
    e = []
    for _ in range(N):
        x = (x * x + x + 1) ** (1 / 3)
        e.append(abs(x - r))
    return e


def errors_secant(x0, x1):
    e = []
    for _ in range(N):
        if f(x1) == f(x0):
            break
        x0, x1 = x1, x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
        e.append(abs(x1 - r))
    return e

```

```python

def errors_newton(x):
    e = []
    for _ in range(N):
        x = x - f(x) / df(x)
        e.append(abs(x - r))
    return e


runs = {"bisection": errors_bisection(1, 2),
        "regula falsi": errors_regula(1, 2),
        "fixed point": errors_fixed(1.5),
        "secant": errors_secant(1, 2),
        "Newton": errors_newton(1.5)}

print(f"{'method':14s} {'its to 1e-10':>13s} {'order estimate':>15s}")
plt.figure(figsize=(6.5, 3.8))
for name, e in runs.items():
    e = [max(v, 1e-17) for v in e]
    its = next(k + 1 for k, v in enumerate(e) if v < 1e-10)
    # order estimate log(e_{n+1}/e_n) / log(e_n/e_{n-1}) just before the error hits 1e-10
    k = its - 1
    q = math.log(e[k] / e[k - 1]) / math.log(e[k - 1] / e[k - 2])
    q = "n/a (not monotone)" if name == "bisection" else f"{q:.3f}"
    print(f"{name:14s} {its:13d} {q:>18s}")
    plt.semilogy(range(1, len(e) + 1), e, "o-", ms=3, label=name)
plt.ylim(1e-16, 1)
plt.xlim(0, 36)
plt.xlabel("iteration n")
plt.ylabel("|x_n - r|")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch04_compare.pdf")
```

Output

```
method          its to 1e-10  order estimate
bisection                 31 n/a (not monotone)
regula falsi              12              1.000
fixed point               29              1.000
secant                     6              1.682
Newton                     5              1.992
```

Method	needs	bracket?	order	evaluations/step	guaranteed?
Bisection	sign of f	yes	1 ($\lambda = \frac{1}{2}$)	1	yes
Regula falsi	f	yes	1	1	yes
Fixed point	g	no	1 ($\lambda = \lvert g'(p)\rvert$)	1	if contraction
Secant	f	no	1.618	1	locally
Newton	f, f'	no	2	2	locally

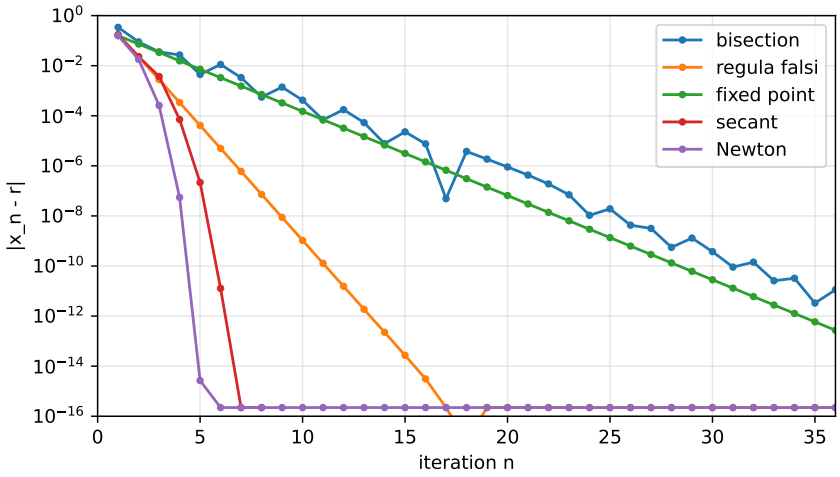


Figure 4.3: Error against iteration number on a logarithmic scale. Linear convergence appears as a straight line; superlinear convergence curves downwards.

4.8.1 Aitken's Δ^2 process and Steffensen's method

If $x_n \to p$ linearly, then $x_{n+1} - p \approx \lambda(x_n - p)$ and $x_{n+2} - p \approx \lambda(x_{n+1} - p)$. Eliminating λ gives $(x_{n+1} - p)^2 \approx (x_{n+2} - p)(x_n - p)$, which we solve for p:

$$\hat{x}_n = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n} = x_n - \frac{(\Delta x_n)^2}{\Delta^2 x_n}. \tag{4.5}$$

Theorem 4.18 (Aitken). *If $x_n \to p$ linearly with $\lim(x_{n+1} - p)/(x_n - p) = \lambda$, $|\lambda| < 1$, and $x_n \neq p$, then $\hat{x}_n$ converges to p faster: $\lim_{n\to\infty}(\hat{x}_n - p)/(x_n - p) = 0$.*

Steffensen's method applies Aitken's formula after every two fixed-point steps and restarts from $\hat{x}$; for $g'(p) \neq 1$ it converges *quadratically* without any derivatives.

Listing 4.8: Aitken acceleration and Steffensen's method for $x = \cos x$ ch04/aitken.py

```
# Aitken's Delta^2 acceleration and Steffensen's method for x = cos(x)
import math

g = math.cos
r = 0.7390851332151607        # the Dottie number: the solution of cos x = x

# plain fixed-point iteration
x = [1.0]
for _ in range(12):
    x.append(g(x[-1]))

# Aitken's formula applied to the sequence x_n
print(" n    x_n (fixed point)   error      Aitken x^_n        error")
for n in range(len(x) - 2):
    d1 = x[n + 1] - x[n]
    d2 = x[n + 2] - 2 * x[n + 1] + x[n]
    xa = x[n] - d1 * d1 / d2
    print(f"{n:2d}  {x[n]:.12f}  {abs(x[n] - r):.2e}  {xa:.12f}  {abs(xa - r):.2e}")

# Steffensen: restart the fixed-point iteration from each Aitken value
p = 1.0
print("\nSteffensen's method:")
```

```
for k in range(5):
    p1 = g(p)
    p2 = g(p1)
    denom = p2 - 2 * p1 + p
    if denom == 0:
        break
    p = p - (p1 - p) ** 2 / denom
    print(f"  k = {k + 1}: p = {p:.15f}, error = {abs(p - r):.2e}")
```

Output

```
 n    x_n (fixed point)   error      Aitken x^_n        error
 0  1.000000000000  2.61e-01  0.728010361468  1.11e-02
 1  0.540302305868  1.99e-01  0.733665164585  5.42e-03
 2  0.857553215846  1.18e-01  0.736906294340  2.18e-03
 3  0.654289790498  8.48e-02  0.738050421372  1.03e-03
 4  0.793480358743  5.44e-02  0.738636096882  4.49e-04
 5  0.701368773623  3.77e-02  0.738876582817  2.09e-04
 6  0.763959682901  2.49e-02  0.738992243027  9.29e-05
 7  0.722102425027  1.70e-02  0.739042511328  4.26e-05
 8  0.750417761764  1.13e-02  0.739065949600  1.92e-05
 9  0.731404042423  7.68e-03  0.739076383319  8.75e-06
10  0.744237354901  5.15e-03  0.739081177260  3.96e-06

Steffensen's method:
  k = 1: p = 0.728010361467617, error = 1.11e-02
  k = 2: p = 0.739066966908674, error = 1.82e-05
  k = 3: p = 0.739085133166076, error = 4.91e-11
  k = 4: p = 0.739085133215161, error = 0.00e+00
```

4.8.2 Root finding in SciPy

In practice one uses library routines. The workhorse is **Brent's method** (`scipy.optimize.brentq`), which combines bisection, the secant method and inverse quadratic interpolation: it is guaranteed to converge like bisection, but is usually as fast as the secant method.

Listing 4.9: SciPy and NumPy root finders ch04/scipy_roots.py

```
# Professional root finders in SciPy and NumPy
import numpy as np
from scipy import optimize

f = lambda x: x**3 - x**2 - x - 1
df = lambda x: 3 * x**2 - 2 * x - 1

r1 = optimize.bisect(f, 1, 2, xtol=1e-12)
r2 = optimize.brentq(f, 1, 2, xtol=1e-14)        # Brent's method: safe AND fast
r3 = optimize.newton(f, 1.5, fprime=df)            # Newton
r4 = optimize.newton(f, 1.5)                       # no derivative given -> secant
sol = optimize.root_scalar(f, bracket=[1, 2], method="brentq")
print("bisect :", r1)
print("brentq :", r2)
print("newton :", r3)
print("secant :", r4)
print("root_scalar:", sol.root, " function calls:", sol.function_calls)
```

```python

# All roots of a polynomial: eigenvalues of the companion matrix
coeffs = [1, -1, -1, -1]                          # x^3 - x^2 - x - 1
print("np.roots:", np.roots(coeffs))
```

Output

```
bisect : 1.8392867552147436
brentq : 1.8392867552141612
newton : 1.8392867552141612
secant : 1.8392867552141616
root_scalar: 1.8392867552141612  function calls: 9
np.roots: [ 1.83928676+0.j         -0.41964338+0.60629073j -0.41964338-0.60629073j]
```

`np.roots` finds all roots of a polynomial, including complex ones, by computing the eigenvalues of the *companion matrix* of the polynomial, a beautiful link between root finding and linear algebra.

Historical Note: Aitken, Steffensen, Dekker and Brent

Alexander **Aitken**, a New Zealand mathematician famous for his extraordinary mental arithmetic, published the Δ^2 process in 1926 (the idea goes back to Seki Takakazu in 17th-century Japan). The Danish actuary J. F. **Steffensen** gave his method in 1933. T. J. **Dekker** (Amsterdam, 1969) and Richard **Brent** (Stanford/Canberra, 1973) developed the safeguarded algorithm that is today in MATLAB's `fzero`, SciPy's `brentq` and most other libraries.

Exercises

4.8.1 Python Apply Aitken's process to the partial sums of $\sum_{k\ge 0}(-1)^k/(2k+1) = \pi/4$. How many digits of π do you get from the first 10 terms, with and without acceleration?

4.8.2 Theory Prove Aitken's theorem. (Write $x_{n+1} - p = (\lambda + \delta_n)(x_n - p)$ with $\delta_n \to 0$.)

4.8.3 Python Use `scipy.optimize.brentq` to find all roots of the Legendre polynomial $P_5(x) = \frac{1}{8}(63x^5 - 70x^3 + 15x)$ in $[-1, 1]$, and compare with `np.roots`. (These roots are the nodes of Gauss quadrature, Chapter 13.)

4.8.4 Python **Project: the Colebrook equation.** The friction factor f of turbulent pipe flow satisfies $\dfrac{1}{\sqrt{f}} = -2\log_{10}\!\left(\dfrac{\varepsilon/D}{3.7} + \dfrac{2.51}{\mathrm{Re}\sqrt{f}}\right)$. For $\mathrm{Re} = 10^5$ and $\varepsilon/D = 10^{-4}$, solve for f by fixed-point iteration (in the variable $1/\sqrt{f}$), by Newton's method, and by `brentq`. Compare the costs.

4.8.5 ★ Graduate **Project: Kepler's equation.** The eccentric anomaly E of a planet satisfies $E - e\sin E = M$. Show that fixed-point iteration $E_{n+1} = M + e\sin E_n$ converges for $0 \le e < 1$. Compare it with Newton's method for $e = 0.1, 0.5, 0.9, 0.99$ and $M \in [0, 2\pi]$. (Kepler used fixed-point iteration in 1621; this equation has motivated root-finding research for four centuries.)

Chapter Summary

- Bracketing methods (bisection, regula falsi) always converge; open methods (fixed point, secant, Newton) converge fast but only locally.

- Bisection: $|p_n - r| \leq (b-a)/2^n$; $n \geq \log_2((b-a)/\varepsilon)$.
- Fixed point: converges if g maps $[a, b]$ into itself and $|g'| \leq k < 1$; linear rate $|g'(p)|$.
- Newton: $x_{n+1} = x_n - f(x_n)/f'(x_n)$; quadratic for simple roots, linear for multiple roots.
- Secant: order 1.618 with one function evaluation per step.
- Order estimate: $\alpha \approx \ln(e_{n+1}/e_n)/\ln(e_n/e_{n-1})$.
- In practice: bracket the root, then use a safeguarded fast method (Brent: `brentq`).

Chapter 5

Systems of Nonlinear Equations

Learning Objectives

After studying this chapter you should be able to:

- write a nonlinear system in the form $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ and compute its Jacobian matrix;
- derive Newton's method for systems from the multivariable Taylor expansion;
- carry out Newton iterations by hand for 2×2 systems and in Python for $n \times n$ systems;
- approximate a Jacobian by finite differences, and describe quasi-Newton (Broyden) methods;
- use `scipy.optimize.fsolve` and `scipy.optimize.root`.

Most mathematical models of real systems are nonlinear and involve several unknowns: steady states of chemical reactions and epidemic models, equilibria of mechanical structures, intersections of curves and surfaces, and the discretised equations of nonlinear differential equations. We extend the ideas of Chapter 4 from one equation to n equations in n unknowns,

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &= 0, \\ f_2(x_1, x_2, \dots, x_n) &= 0, \\ &\vdots \\ f_n(x_1, x_2, \dots, x_n) &= 0, \end{aligned} \qquad \text{or briefly} \qquad \mathbf{F}(\mathbf{x}) = \mathbf{0}, \tag{5.1}$$

where $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ and $\mathbf{F} = (f_1, \dots, f_n)^T : \mathbb{R}^n \to \mathbb{R}^n$.

△ Caution: no bisection in higher dimensions

In one dimension a sign change brackets a root. In two or more dimensions there is no such simple test, and hence no analogue of bisection. The solutions of $f_1 = 0$ and $f_2 = 0$ are the intersections of two *curves*, which may have any number of intersection points, or none. Good starting values, obtained from a graph, from physical insight, or from a simpler model, are therefore essential.

5.1 Functions of several variables

Definition 5.1 (limit and continuity)**.** Let $f : D \subseteq \mathbb{R}^n \to \mathbb{R}$. We say $\lim_{\mathbf{x}\to\mathbf{x}_0} f(\mathbf{x}) = L$ if for every $\varepsilon > 0$ there is $\delta > 0$ such that $|f(\mathbf{x}) - L| < \varepsilon$ whenever $\mathbf{x} \in D$ and $0 < \|\mathbf{x} - \mathbf{x}_0\| < \delta$. f is continuous at $\mathbf{x}_0$ if $\lim_{\mathbf{x}\to\mathbf{x}_0} f(\mathbf{x}) = f(\mathbf{x}_0)$. A vector function $\mathbf{F}$ is continuous if each

component f_i is.

(Any norm on $\mathbb{R}^n$ may be used here, e.g. $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$; norms are studied in Chapter 8.)

Definition 5.2 (Jacobian matrix)**.** If all partial derivatives of $\mathbf{F}$ exist, the *Jacobian matrix* of $\mathbf{F}$ at $\mathbf{x}$ is the $n \times n$ matrix

$$J(\mathbf{x}) = \mathbf{F}'(\mathbf{x}) = \begin{pmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_n}{\partial x_1} & \cdots & \dfrac{\partial f_n}{\partial x_n} \end{pmatrix}, \qquad J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

Row i is the gradient of f_i; column j contains the derivatives with respect to x_j.

Theorem 5.3 (first-order Taylor expansion)**.** *If $\mathbf{F}$ has continuous second partial derivatives near $\mathbf{x}$, then*

$$\mathbf{F}(\mathbf{x}+\mathbf{h}) = \mathbf{F}(\mathbf{x}) + J(\mathbf{x})\mathbf{h} + O(\|\mathbf{h}\|^2).$$

For $n = 2$ with $\mathbf{F} = (f, g)$ and $\mathbf{h} = (\Delta x, \Delta y)$ this reads

$$\begin{aligned} f(x+\Delta x, y+\Delta y) &\approx f(x,y) + f_x(x,y)\Delta x + f_y(x,y)\Delta y, \\ g(x+\Delta x, y+\Delta y) &\approx g(x,y) + g_x(x,y)\Delta x + g_y(x,y)\Delta y. \end{aligned}$$

Fixed-point iteration for systems

As in one dimension, write $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ as $\mathbf{x} = \mathbf{G}(\mathbf{x})$ and iterate $\mathbf{x}^{(k+1)} = \mathbf{G}(\mathbf{x}^{(k)})$. The analogue of Theorem 4.10 holds: if $\mathbf{G}$ maps a closed set D into itself and is a *contraction*, $\|\mathbf{G}(\mathbf{x}) - \mathbf{G}(\mathbf{y})\| \le k\,\|\mathbf{x}-\mathbf{y}\|$ with $k < 1$, then $\mathbf{G}$ has a unique fixed point in D and the iteration converges to it from any $\mathbf{x}^{(0)} \in D$ (Banach's theorem). A sufficient condition is that $\|\mathbf{G}'(\mathbf{x})\| \le k < 1$ on a convex set D. Convergence is linear, however, and finding a suitable $\mathbf{G}$ is harder than in one dimension, so we move directly to Newton's method.

Exercises

5.1.1 Theory Compute the Jacobian of (a) $\mathbf{F}(x,y) = (x^2 + y^2 - 4,\ xy - 1)$; (b) $\mathbf{F}(x,y,z) = (3x - \cos(yz) - \tfrac{1}{2},\ x^2 - 81(y+0.1)^2 + \sin z + 1.06,\ e^{-xy} + 20z + \tfrac{10\pi - 3}{3})$.

5.1.2 Theory Show that $\mathbf{G}(x,y) = \left(\tfrac{1}{4}\cos y,\ \tfrac{1}{3}\sin x\right)$ is a contraction on $\mathbb{R}^2$ in the norm $\|\cdot\|_\infty$, and find its fixed point numerically.

5.2 Newton's method for systems

Let $\mathbf{x}^{(k)}$ be an approximation of a solution $\mathbf{x}^*$ and write $\mathbf{x}^* = \mathbf{x}^{(k)} + \mathbf{h}$. By Taylor's theorem,

$$\mathbf{0} = \mathbf{F}(\mathbf{x}^*) = \mathbf{F}(\mathbf{x}^{(k)} + \mathbf{h}) \approx \mathbf{F}(\mathbf{x}^{(k)}) + J(\mathbf{x}^{(k)})\mathbf{h}.$$

Solving the *linear* system $J(\mathbf{x}^{(k)})\mathbf{h} = -\mathbf{F}(\mathbf{x}^{(k)})$ for $\mathbf{h}$ gives the next approximation. Formally, $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J(\mathbf{x}^{(k)})^{-1}\mathbf{F}(\mathbf{x}^{(k)})$, which generalises $x_{n+1} = x_n - f(x_n)/f'(x_n)$.

Algorithm 3 Newton's method for systems

Require: $\mathbf{F}$, J, initial guess $\mathbf{x}^{(0)}$, tolerance ε, maximum iterations N

for $k = 0, 1, \ldots, N-1$ **do**
  Evaluate $\mathbf{F}(\mathbf{x}^{(k)})$ and $J(\mathbf{x}^{(k)})$
  Solve the linear system $J(\mathbf{x}^{(k)})\,\Delta\mathbf{x} = -\mathbf{F}(\mathbf{x}^{(k)})$ ▷ e.g. by LU, Chapter 6
  $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \Delta\mathbf{x}$
  if $\|\Delta\mathbf{x}\| < \varepsilon\,(1 + \|\mathbf{x}^{(k+1)}\|)$ **then**
    return $\mathbf{x}^{(k+1)}$
  end if
end for

★ Key Concept: Newton = a sequence of linear systems

Each Newton step *linearises* the nonlinear problem and solves a linear system. This is why linear algebra (Chapters 6 and 7) is at the heart of nonlinear problems. Never compute J^{-1}: solve $J\Delta\mathbf{x} = -\mathbf{F}$ with `np.linalg.solve`.

△ Caution: a common programming error

In MATLAB/Octave, `y - F./J` is *not* Newton's method: `./` divides element by element, which is meaningless for a matrix J and a vector $\mathbf{F}$. The correct Newton step is `y - J\F` in MATLAB, or `y - np.linalg.solve(J, F)` in Python.

Theorem 5.4 (quadratic convergence). *Let $\mathbf{F}(\mathbf{x}^*) = \mathbf{0}$ with $J(\mathbf{x}^*)$ nonsingular, and let J be Lipschitz continuous near $\mathbf{x}^*$. Then there is $\delta > 0$ such that for $\|\mathbf{x}^{(0)} - \mathbf{x}^*\| < \delta$ Newton's method converges and $\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \le C\,\|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2$.*

Example 5.5. Perform Newton iterations for

$$f(x,y) = -x(x+1) + 2y - 18 = 0, \qquad g(x,y) = (x-1)^2 + (y-6)^2 - 25 = 0,$$

starting from $(x_0, y_0) = (2, 11)$.

Solution. The Jacobian is $J(x,y) = \begin{pmatrix} -2x-1 & 2 \\ 2(x-1) & 2(y-6) \end{pmatrix}$. At $(2,11)$: $f = -6 + 22 - 18 = -2$, $g = 1 + 25 - 25 = 1$, and $J = \begin{pmatrix} -5 & 2 \\ 2 & 10 \end{pmatrix}$. Solve

$$\begin{pmatrix} -5 & 2 \\ 2 & 10 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}:$$

$$\Delta x = \frac{2\cdot 10 - 2\cdot(-1)}{-54} = -0.407407, \qquad \Delta y = \frac{-5\cdot(-1) - 2\cdot 2}{-54} = -0.018519,$$

(Cramer's rule, $\det J = -54$), so $(x_1, y_1) = (1.592593,\ 10.981481)$. A second step gives $(1.547518,\ 10.970149)$. The program below confirms these values and shows the quadratic convergence: $\|\mathbf{F}\|$ goes $10^{-1}, 10^{-3}, 10^{-7}, 10^{-14}$. A different starting point converges to the second real solution $(-2, 10)$.

Listing 5.1: Newton's method for a 2×2 system ch05/newton_system.py

```python
# Newton's method for a system of nonlinear equations F(x) = 0
import numpy as np
import matplotlib.pyplot as plt


def newton_system(F, J, x0, tol=1e-12, max_iter=50, verbose=False):
    """Solve F(x) = 0, where F: R^n -> R^n has Jacobian matrix J(x).

    Each step solves the LINEAR system J(x_k) dx = -F(x_k); the inverse of J
    is never formed.  Returns the solution and the number of iterations."""
    x = np.array(x0, dtype=float)
    for k in range(1, max_iter + 1):
        dx = np.linalg.solve(J(x), -F(x))
        x = x + dx
        if verbose:
            print(f"k={k}: x = {np.array2string(x, precision=10)}, "
                  f"||dx|| = {np.linalg.norm(dx):.2e}, ||F(x)|| = {np.linalg.norm(F(x))
    ↪ :.2e}")
        if np.linalg.norm(dx) < tol * (1 + np.linalg.norm(x)):
            return x, k
    raise RuntimeError("Newton's method did not converge")


# Example 5.1:  -x(x+1) + 2y = 18,  (x-1)^2 + (y-6)^2 = 25
def F(v):
    x, y = v
    return np.array([-x * (x + 1) + 2 * y - 18,
                     (x - 1) ** 2 + (y - 6) ** 2 - 25])


def J(v):
    x, y = v
    return np.array([[-2 * x - 1, 2.0],
                     [2 * (x - 1), 2 * (y - 6)]])


print("Starting from (2, 11):")
sol, k = newton_system(F, J, [2.0, 11.0], verbose=True)
print("Starting from (-1.5, 10.5):")
sol, k = newton_system(F, J, [-1.5, 10.5], verbose=True)

# Plot the two curves: the solutions are their intersection points
x = np.linspace(-4, 5, 400)
y = np.linspace(4, 14, 400)
X, Y = np.meshgrid(x, y)
plt.figure(figsize=(5, 4))
plt.contour(X, Y, -X * (X + 1) + 2 * Y - 18, levels=[0], colors="C0")
plt.contour(X, Y, (X - 1) ** 2 + (Y - 6) ** 2 - 25, levels=[0], colors="C1")
plt.plot([-2, 1.54694647], [10, 10.96999493], "ko")
plt.xlabel("x")
plt.ylabel("y")
plt.title("f = 0 (parabola) and g = 0 (circle)")
plt.gca().set_aspect("equal")
plt.tight_layout()
plt.savefig("ch05_curves.pdf")
```

Output

```
Starting from (2, 11):
k=1: x = [ 1.5925925926 10.9814814815], ||dx|| = 4.08e-01, ||F(x)|| = 2.35e-01
k=2: x = [ 1.5475180862 10.9701493011], ||dx|| = 4.65e-02, ||F(x)|| = 2.97e-03
k=3: x = [ 1.546946562  10.9699949506], ||dx|| = 5.92e-04, ||F(x)|| = 4.79e-07
k=4: x = [ 1.54694647   10.9699949254], ||dx|| = 9.54e-08, ||F(x)|| = 1.59e-14
k=5: x = [ 1.54694647   10.9699949254], ||dx|| = 2.59e-15, ||F(x)|| = 5.02e-15
Starting from (-1.5, 10.5):
k=1: x = [-2.1160714286  9.9910714286], ||dx|| = 7.99e-01, ||F(x)|| = 7.43e-01
k=2: x = [-2.0035188594  9.9989504399], ||dx|| = 1.13e-01, ||F(x)|| = 1.80e-02
k=3: x = [-2.0000034938  9.9999990618], ||dx|| = 3.67e-03, ||F(x)|| = 1.83e-05
k=4: x = [-2. 10.], ||dx|| = 3.62e-06, ||F(x)|| = 1.79e-11
k=5: x = [-2. 10.], ||dx|| = 3.57e-12, ||F(x)|| = 0.00e+00
```

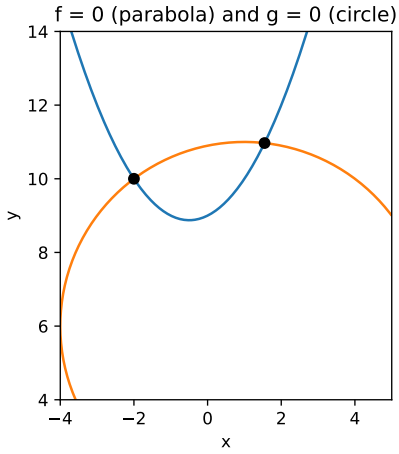


Figure 5.1: The solutions of Example 5.5 are the intersections of a parabola and a circle.

5.2.1 Finite-difference Jacobians

If the partial derivatives are difficult to derive, approximate column j of the Jacobian by

$$J(\mathbf{x})\mathbf{e}_j \approx \frac{\mathbf{F}(\mathbf{x} + h\mathbf{e}_j) - \mathbf{F}(\mathbf{x})}{h}, \qquad h \approx \sqrt{u}\,\max(1, |x_j|) \approx 10^{-8}.$$

This costs n extra evaluations of $\mathbf{F}$ per iteration. The choice $h \approx \sqrt{u}$ balances truncation and rounding errors (Chapter 12).

Example 5.6 (equilibria of the Lorenz system)**.** The Lorenz equations (Edward Lorenz, 1963), the famous model of atmospheric convection that launched chaos theory, are

$$\dot{x} = \sigma(y - x), \qquad \dot{y} = \rho x - y - xz, \qquad \dot{z} = xy - \beta z.$$

Equilibria (steady states) satisfy $\mathbf{F}(x, y, z) = \mathbf{0}$ where $\mathbf{F}$ is the right-hand side. For $\sigma = 10$, $\rho = 28$, $\beta = 8/3$ they are $(0, 0, 0)$ and $(\pm\sqrt{\beta(\rho-1)}, \pm\sqrt{\beta(\rho-1)}, \rho - 1)$.

Listing 5.2: Lorenz equilibria by Newton's method with exact and finite-difference Jacobians
ch05/lorenz_equilibria.py

```
# Equilibria of the Lorenz system with Newton's method and a finite-difference Jacobian
import numpy as np

sigma, rho, beta = 10.0, 28.0, 8.0 / 3.0


def lorenz(v):
    x, y, z = v
    return np.array([sigma * (y - x),
                     rho * x - y - x * z,
                     x * y - beta * z])


def jacobian_exact(v):
    x, y, z = v
    return np.array([[-sigma, sigma, 0.0],
                     [rho - z, -1.0, -x],
                     [y, x, -beta]])


def jacobian_fd(F, x, h=1e-7):
    """Forward-difference approximation of the Jacobian: column j is
    (F(x + h e_j) - F(x)) / h."""
    n = len(x)
    Fx = F(x)
    Jm = np.zeros((n, n))
    for j in range(n):
        xh = x.copy()
        step = h * max(1.0, abs(x[j]))
        xh[j] += step
        Jm[:, j] = (F(xh) - Fx) / step
    return Jm


def newton(F, x0, jac=None, tol=1e-12, max_iter=50):
    x = np.array(x0, dtype=float)
    for k in range(1, max_iter + 1):
        Jm = jac(x) if jac is not None else jacobian_fd(F, x)
        dx = np.linalg.solve(Jm, -F(x))
        x += dx
        if np.linalg.norm(dx) < tol * (1 + np.linalg.norm(x)):
            return x, k
    raise RuntimeError("no convergence")


x0 = np.array([-1.0, 0.0, 1.05])
print("difference between exact and FD Jacobian at x0:",
      np.max(np.abs(jacobian_exact(x0) - jacobian_fd(lorenz, x0))))
for start in ([-1.0, 0.0, 1.05], [5.0, 5.0, 20.0], [-5.0, -5.0, 20.0]):
    xe, k = newton(lorenz, start, jacobian_exact)
    xf, kf = newton(lorenz, start)          # finite-difference Jacobian
    print(f"start {start}: exact J -> {np.round(xe, 10)} ({k} its), "
          f"FD J -> {kf} its")
c = np.sqrt(beta * (rho - 1))
print(f"theory: (0,0,0) and (+-{c:.10f}, +-{c:.10f}, {rho - 1})")
```

Output

```
difference between exact and FD Jacobian at x0: 4.388375529629229e-08
start [-1.0, 0.0, 1.05]: exact J -> [0. 0. 0.] (5 its), FD J -> 5 its
start [5.0, 5.0, 20.0]: exact J -> [ 8.48528137  8.48528137 27.        ] (7 its), FD J ->
    ↪ 7 its
start [-5.0, -5.0, 20.0]: exact J -> [-8.48528137 -8.48528137 27.        ] (7 its), FD J
    ↪ -> 7 its
theory: (0,0,0) and (+-8.4852813742, +-8.4852813742, 27.0)
```

The finite-difference Jacobian agrees with the exact one to about 10^{-8}, and Newton's method needs the same number of iterations with either.

Historical Note: from Simpson to Lorenz

Thomas Simpson already applied Newton's method to two equations in two unknowns in 1740. The general n-dimensional method, and its analysis in Banach spaces, is due to **L. V. Kantorovich** (1948), who received the Nobel Prize in Economics in 1975 for his work on optimal allocation of resources (linear programming). **Edward Lorenz**, a meteorologist at MIT, discovered in 1961 that rounding his initial data from six to three digits completely changed his computed weather forecast; this "butterfly effect" shows the sensitive dependence on initial conditions of chaotic systems, a dramatic example of *ill-conditioning*.

Exercises

5.2.1 Theory Do two Newton iterations by hand for $x^2 + y^2 = 4$, $xy = 1$ from $(2, 1)$.

5.2.2 Theory Do one Newton iteration by hand for $x^2 - 10x + y^2 + 8 = 0$, $xy^2 + x - 10y + 8 = 0$ from $(0, 0)$. (The exact solution nearby is $(1, 1)$.)

5.2.3 Python Solve the 3×3 system of Exercise 5.1.1(b) with Newton's method from $\mathbf{x}^{(0)} = (0.1, 0.1, -0.1)$ and print $\|\mathbf{F}(\mathbf{x}^{(k)})\|_\infty$ at each step.

5.2.4 Python **SIR endemic equilibrium.** For the SIR model with births and deaths, $\dot S = \mu N - \beta SI/N - \mu S$, $\dot I = \beta SI/N - (\gamma + \mu)I$, $\dot R = \gamma I - \mu R$, with $N = 1$, $\beta = 0.5$, $\gamma = 0.1$, $\mu = 0.01$, find the equilibrium with $I > 0$ by Newton's method and compare with the exact formula $S^* = (\gamma + \mu)/\beta$.

5.2.5 Python Find all real solutions of $x^3 - 3xy^2 - 1 = 0$, $3x^2y - y^3 = 0$ (these are the equations $\mathrm{Re}(z^3 - 1) = 0$, $\mathrm{Im}(z^3 - 1) = 0$ with $z = x + iy$). Compare with Fig. 4.2.

5.2.6 ★ Graduate Show that Newton's method for $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ is *affine invariant*: applying it to $A\mathbf{F}(\mathbf{x}) = \mathbf{0}$ with a nonsingular matrix A produces exactly the same iterates.

5.3 Quasi-Newton methods and software

For large n the cost of Newton's method is dominated by forming J (n^2 partial derivatives) and solving the linear system ($\approx \tfrac{2}{3}n^3$ operations, Chapter 6). **Quasi-Newton methods** replace $J(\mathbf{x}^{(k)})$ by an approximation B_k that is cheaply updated from step to step. **Broyden's method** (1965), the multidimensional secant method, requires the *secant condition*

$$B_{k+1}(\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}) = \mathbf{F}(\mathbf{x}^{(k+1)}) - \mathbf{F}(\mathbf{x}^{(k)})$$

and changes B_k as little as possible, by a rank-one matrix:

$$B_{k+1} = B_k + \frac{(\mathbf{y}_k - B_k\mathbf{s}_k)\mathbf{s}_k^T}{\mathbf{s}_k^T\mathbf{s}_k}, \qquad \mathbf{s}_k = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}, \quad \mathbf{y}_k = \mathbf{F}(\mathbf{x}^{(k+1)}) - \mathbf{F}(\mathbf{x}^{(k)}).$$

Broyden's method converges superlinearly and needs no derivatives after the first step.

Listing 5.3: Broyden's method against Newton's method ch05/broyden.py

```python
# Broyden's quasi-Newton method compared with Newton's method
import numpy as np


def F(v):
    x, y = v
    return np.array([x**2 + y**2 - 4, x * y - 1])


def J(v):
    x, y = v
    return np.array([[2 * x, 2 * y], [y, x]])


def newton(x, tol=1e-12):
    errs = []
    for k in range(50):
        dx = np.linalg.solve(J(x), -F(x))
        x = x + dx
        errs.append(np.linalg.norm(dx))
        if errs[-1] < tol:
            return x, errs
    return x, errs


def broyden(x, B, tol=1e-12):
    """Broyden's 'good' method: B approximates the Jacobian and is updated by a
    rank-one correction so that B_new (x_new - x) = F(x_new) - F(x)."""
    errs = []
    Fx = F(x)
    for k in range(100):
        s = np.linalg.solve(B, -Fx)
        x_new = x + s
        F_new = F(x_new)
        yv = F_new - Fx
        B = B + np.outer(yv - B @ s, s) / (s @ s)
        x, Fx = x_new, F_new
        errs.append(np.linalg.norm(s))
        if errs[-1] < tol:
            return x, errs
    return x, errs


x0 = np.array([2.0, 1.0])
xn, en = newton(x0)
xb, eb = broyden(x0, J(x0))       # start from the exact Jacobian at x0 only
print("Newton :", xn, f"{len(en)} iterations (each needs the Jacobian)")
print("Broyden:", xb, f"{len(eb)} iterations (Jacobian evaluated once)")
print("step sizes Newton :", " ".join(f"{e:.1e}" for e in en))
```

```
print("step sizes Broyden:", " ".join(f"{e:.1e}" for e in eb))
print("exact: x = sqrt(2+sqrt3) =", np.sqrt(2 + np.sqrt(3)), " y = 1/x =", 1 / np.sqrt(2 +
    ↪ np.sqrt(3)))
```

Output

```
Newton : [1.93185165 0.51763809] 6 iterations (each needs the Jacobian)
Broyden: [1.93185165 0.51763809] 9 iterations (Jacobian evaluated once)
step sizes Newton : 5.0e-01 6.9e-02 1.8e-03 1.5e-06 1.1e-12 1.3e-16
step sizes Broyden: 5.0e-01 8.6e-02 2.1e-02 1.8e-03 1.5e-04 6.4e-06 7.9e-08 3.8e-11 1.3e-16
exact: x = sqrt(2+sqrt3) = 1.9318516525781366  y = 1/x = 0.5176380902050415
```

Listing 5.4: Nonlinear systems in SciPy ch05/scipy_systems.py

```
# Solving nonlinear systems with SciPy
import numpy as np
from scipy import optimize


def F(v):
    x, y = v
    return [x**2 + y**2 - 4, x * y - 1]


# fsolve: a classic MINPACK routine (Powell's hybrid method)
print("fsolve:", optimize.fsolve(F, [2, 1]))

# root: a common interface to several methods
for method in ["hybr", "lm", "broyden1"]:
    sol = optimize.root(F, [2, 1], method=method)
    print(f"root(method={method!r:11}): x = {sol.x}, success = {sol.success}")

# the four solutions, found from four starting points
for start in ([2, 1], [1, 2], [-2, -1], [-1, -2]):
    print(start, "->", np.round(optimize.fsolve(F, start), 8))
```

Output

```
fsolve: [1.93185165 0.51763809]
root(method='hybr'     ): x = [1.93185165 0.51763809], success = True
root(method='lm'       ): x = [1.93185165 0.51763809], success = True
root(method='broyden1' ): x = [0.51763809 1.93185165], success = True
[2, 1] -> [1.93185165 0.51763809]
[1, 2] -> [0.51763809 1.93185165]
[-2, -1] -> [-1.93185165 -0.51763809]
[-1, -2] -> [-0.51763809 -1.93185165]
```

Note that `method="broyden1"` converged to a *different* solution from the same starting point. A nonlinear system may have several solutions, and which one an iterative method finds depends on the method and on the starting point.

Towards Graduate Studies: large nonlinear systems

Discretising a nonlinear PDE such as the Bratu problem $-\Delta u = \lambda e^u$ or the steady Navier–Stokes equations produces systems with millions of unknowns. There one uses *inexact Newton* or *Newton–Krylov* methods (the linear systems are solved approximately by iterative methods such as GMRES, Chapter 7), *globalisation* by line searches or trust regions so that convergence is obtained from poor starting values, and *continuation* in a parameter such as λ to follow solution branches through turning points and bifurcations. Newton's method applied to $\nabla\phi(\mathbf{x}) = \mathbf{0}$ is Newton's method of optimisation, and BFGS (the optimisation cousin of Broyden's method) is one of the most widely used algorithms in science and machine learning. See Kelley, *Iterative Methods for Linear and Nonlinear Equations* (SIAM, 1995), and Nocedal & Wright, *Numerical Optimization*.

Exercises

5.3.1 Python Apply Broyden's method with $B_0 = I$ (the identity) instead of $B_0 = J(\mathbf{x}^{(0)})$ to the example of Listing 5.3. Does it still converge?

5.3.2 Python **A nonlinear boundary-value problem.** Discretise $y'' = \tfrac{1}{8}(32 + 2x^3 - yy')$, $y(1) = 17$, $y(3) = \tfrac{43}{3}$, on $N = 19$ interior points with central differences ($y'' \approx (y_{i+1} - 2y_i + y_{i-1})/h^2$, $y' \approx (y_{i+1} - y_{i-1})/(2h)$), and solve the resulting nonlinear system by Newton's method. Compare with the exact solution $y = x^2 + 16/x$.

5.3.3 ★ Graduate Show that Broyden's update is the solution of $\min\{\|B - B_k\|_F : B\mathbf{s}_k = \mathbf{y}_k\}$, where $\|\cdot\|_F$ is the Frobenius norm.

Chapter Summary

- A nonlinear system $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ generalises $f(x) = 0$; the Jacobian J generalises f'.
- Newton: solve $J(\mathbf{x}^{(k)})\Delta\mathbf{x} = -\mathbf{F}(\mathbf{x}^{(k)})$, set $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}$. Quadratic convergence near a solution with nonsingular Jacobian.
- Good starting values are essential; there is no bisection in several dimensions.
- Jacobians can be approximated by finite differences; Broyden updates avoid them entirely.
- Library routines: `scipy.optimize.fsolve`, `scipy.optimize.root`.

Chapter 6

Direct Methods for Linear Systems: Matrix Factorisations

Learning Objectives

After studying this chapter you should be able to:

- solve triangular systems by forward and back substitution and count the operations;
- perform Gaussian elimination with partial pivoting, and explain why pivoting is needed;
- compute the factorisations $PA = LU$, $A = LL^T$ (Cholesky) and $A = QR$ by hand for small matrices;
- use a factorisation to solve $A\mathbf{x} = \mathbf{b}$ for several right-hand sides and to compute determinants;
- test a symmetric matrix for positive definiteness;
- use `numpy.linalg` and `scipy.linalg` for these tasks.

Linear systems of equations

$$A\mathbf{x} = \mathbf{b}, \qquad A \in \mathbb{R}^{n\times n},\ \mathbf{b} \in \mathbb{R}^n, \tag{6.1}$$

are the most common problem in scientific computing. They arise directly (electrical circuits, structures, economics, statistics) and, even more often, as a sub-problem of other methods: every Newton step in Chapter 5, every least-squares fit in Chapter 10, every implicit time step when solving a differential equation. In this chapter we study *direct methods*, which compute the exact solution in a finite number of operations (apart from rounding errors) by *factorising* the matrix into simpler ones.

⋆ Key Concept: the factorisation idea

A *factorisation* (or *decomposition*) writes A as a product of matrices whose systems are easy to solve (triangular, orthogonal, diagonal, permutation):

Factorisation	applies to	cost (flops)
$PA = LU$	any nonsingular A	$\frac{2}{3}n^3$
$A = LL^T$ (Cholesky)	symmetric positive definite A	$\frac{1}{3}n^3$
$A = QR$	any A (also rectangular)	$\frac{4}{3}n^3$ (Householder)

The expensive factorisation is done once; each additional right-hand side then costs only

$O(n^2)$ operations.

Historical Note: from the Nine Chapters to Turing

Elimination for linear systems appears in Chapter 8 (*Fangcheng*, "rectangular arrays") of the Chinese *Nine Chapters on the Mathematical Art* (compiled c. 200 BC–100 AD, with a commentary by **Liu Hui** in 263 AD), where coefficients were arranged as counting rods on a board and eliminated column by column, exactly as we do today. In Europe the method was described by **Newton** (lecture notes of the 1670s, published 1707) and used systematically by **Carl Friedrich Gauss** (1809–1810) to solve the normal equations for the orbit of the asteroid Pallas; it has carried his name ever since. The American geodesist M. H. **Doolittle** (1878) organised the computation in the form now called LU factorisation for the US Coast Survey. The matrix formulation $A = LU$, and the first analysis of rounding errors in elimination, are due to **Alan Turing** (1948), who also introduced the term "condition number".

6.1 Triangular systems

A system $U\mathbf{x} = \mathbf{c}$ with U upper triangular ($u_{ij} = 0$ for $i > j$) and $u_{ii} \neq 0$ is solved from the last equation upwards, by **back substitution**:

$$x_n = \frac{c_n}{u_{nn}}, \qquad x_i = \frac{1}{u_{ii}}\Big(c_i - \sum_{j=i+1}^{n} u_{ij}x_j\Big), \quad i = n-1, \dots, 1.$$

Similarly a lower triangular system $L\mathbf{z} = \mathbf{c}$ is solved by **forward substitution**: $z_i = \big(c_i - \sum_{j<i} l_{ij}z_j\big)/l_{ii}$, $i = 1, \dots, n$. Computing x_i needs $n - i$ multiplications, $n - i$ subtractions and one division, so the total cost is $\sum_{i=1}^{n}(2(n-i)+1) = n^2$ floating-point operations (*flops*).

⋆ Key Concept: operation counts

Counting operations tells us how the cost grows with n. Back substitution costs n^2 flops; Gaussian elimination costs $\approx \frac{2}{3}n^3$. Doubling n multiplies the cost of elimination by 8. On a laptop performing about 10^{11} flops per second, elimination for $n = 10^4$ takes about 7 seconds, but for $n = 10^6$ about 11 weeks (and 8 terabytes of memory to store A): large problems need the methods of Chapter 7.

6.2 Gaussian elimination and pivoting

Gaussian elimination transforms (6.1) into an equivalent upper triangular system by subtracting multiples of one row from the rows below it. At step k, for $i = k+1, \dots, n$:

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \qquad \text{row}_i \leftarrow \text{row}_i - m_{ik}\,\text{row}_k, \qquad b_i \leftarrow b_i - m_{ik}b_k.$$

The element $a_{kk}^{(k)}$ is the *pivot* and the numbers m_{ik} are the *multipliers*.

Operation count. Step k updates an $(n-k)\times(n-k)$ block: $(n-k)$ divisions and $(n-k)^2$ multiplications and subtractions. Summing,

$$\sum_{k=1}^{n-1} 2(n-k)^2 \approx 2\cdot\frac{n^3}{3} = \frac{2}{3}n^3 \text{ flops}.$$

More precisely, solving $A\mathbf{x} = \mathbf{b}$ by elimination and back substitution requires $\frac{n^3}{3} + n^2 - \frac{n}{3}$ multiplications/divisions and $\frac{n^3}{3} + \frac{n^2}{2} - \frac{5n}{6}$ additions/subtractions [2].

6.2.1 Why pivoting is necessary

If a pivot $a_{kk}^{(k)}$ is zero, elimination breaks down; the remedy is to interchange rows. But a pivot that is merely *small* is just as dangerous.

Example 6.1 (a tiny pivot). Solve $\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, whose solution is $x_1 \approx x_2 \approx 1$ for small ϵ. Without pivoting, $m_{21} = 1/\epsilon$ and

$$\begin{pmatrix} \epsilon & 1 \\ 0 & 1 - 1/\epsilon \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1 \\ 2 - 1/\epsilon \end{pmatrix}.$$

For $\epsilon < u$ the computer rounds $1 - 1/\epsilon$ to $-1/\epsilon$ and $2 - 1/\epsilon$ to $-1/\epsilon$, giving $x_2 = 1$ (fine) but $x_1 = (1 - x_2)/\epsilon = 0$ (completely wrong). Interchanging the rows first gives multiplier ϵ and an accurate answer.

Listing 6.1: The effect of a tiny pivot ch06/pivoting.py

```python
# Why pivoting is necessary: a tiny pivot destroys the solution
import numpy as np


def solve_2x2_no_pivot(A, b):
    m = A[1, 0] / A[0, 0]
    u22 = A[1, 1] - m * A[0, 1]
    c2 = b[1] - m * b[0]
    x2 = c2 / u22
    x1 = (b[0] - A[0, 1] * x2) / A[0, 0]
    return np.array([x1, x2])


for eps in [1e-4, 1e-8, 1e-12, 1e-16, 1e-20]:
    A = np.array([[eps, 1.0], [1.0, 1.0]])
    b = np.array([1.0, 2.0])
    x_np = solve_2x2_no_pivot(A, b)
    x_pp = solve_2x2_no_pivot(A[::-1], b[::-1])   # swap the rows = partial pivoting
    print(f"eps = {eps:.0e}:  no pivoting x = {x_np},   with pivoting x = {x_pp}")
print("exact solution is x1 = 1/(1-eps) ~ 1, x2 = (1-2 eps)/(1-eps) ~ 1")
```

Output

```
eps = 1e-04:  no pivoting x = [1.00010001 0.99989999],   with pivoting x = [1.00010001
    ↪ 0.99989999]
eps = 1e-08:  no pivoting x = [1.00000001 0.99999999],   with pivoting x = [1.00000001
    ↪ 0.99999999]
eps = 1e-12:  no pivoting x = [0.99997788 1.        ],   with pivoting x = [1. 1.]
eps = 1e-16:  no pivoting x = [2.22044605 1.        ],   with pivoting x = [1. 1.]
eps = 1e-20:  no pivoting x = [0. 1.],   with pivoting x = [1. 1.]
exact solution is x1 = 1/(1-eps) ~ 1, x2 = (1-2 eps)/(1-eps) ~ 1
```

Definition 6.2 (partial pivoting). At step k, choose the row $p \geq k$ with the largest $|a_{pk}^{(k)}|$ and interchange rows k and p before eliminating. Then all multipliers satisfy $|m_{ik}| \leq 1$.

△ Caution: always pivot

Gaussian elimination *without* pivoting is unstable in general, even when no pivot is exactly zero. Every professional code (LAPACK, `np.linalg.solve`) uses partial pivoting. Pivoting can be omitted only for special classes of matrices: symmetric positive definite matrices (Section 6.4) and strictly diagonally dominant matrices.

Example 6.3. Solve

$$x_1 + 2x_2 + 6x_3 = 3, \qquad 4x_1 + 8x_2 - x_3 = -13, \qquad -2x_1 + 3x_2 + 5x_3 = -3.$$

By hand, with the minimum of row interchanges. Eliminate x_1 with $m_{21} = 4$, $m_{31} = -2$:

$$\left(\begin{array}{ccc|c} 1 & 2 & 6 & 3 \\ 4 & 8 & -1 & -13 \\ -2 & 3 & 5 & -3 \end{array}\right) \to \left(\begin{array}{ccc|c} 1 & 2 & 6 & 3 \\ 0 & 0 & -25 & -25 \\ 0 & 7 & 17 & 3 \end{array}\right).$$

The $(2,2)$ pivot is zero, so interchange rows 2 and 3; the system is now triangular: $-25x_3 = -25$, $7x_2 + 17x_3 = 3$, $x_1 + 2x_2 + 6x_3 = 3$, giving $x_3 = 1$, $x_2 = (3 - 17)/7 = -2$, $x_1 = 3 - 2(-2) - 6 = 1$. *With partial pivoting* (as a computer does it), row 2 (pivot 4) is used first:

Listing 6.2: Gaussian elimination with partial pivoting ch06/gauss_elim.py

```python
# Triangular solves and Gaussian elimination with partial pivoting
import numpy as np


def back_substitution(U, c):
    """Solve U x = c for upper triangular U (about n^2 flops)."""
    n = len(c)
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (c[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x


def forward_substitution(L, c):
    """Solve L z = c for lower triangular L."""
    n = len(c)
    z = np.zeros(n)
    for i in range(n):
        z[i] = (c[i] - L[i, :i] @ z[:i]) / L[i, i]
    return z


def gauss_solve(A, b, verbose=False):
    """Solve A x = b by Gaussian elimination with partial pivoting."""
    A = np.array(A, dtype=float)          # work on copies
    b = np.array(b, dtype=float)
    n = len(b)
    for k in range(n - 1):
        p = k + np.argmax(np.abs(A[k:, k]))      # row of the largest pivot candidate
        if A[p, k] == 0:
            raise ValueError("matrix is singular")
        if p != k:                               # swap rows k and p
            A[[k, p]] = A[[p, k]]
            b[[k, p]] = b[[p, k]]
        for i in range(k + 1, n):
            m = A[i, k] / A[k, k]                # multiplier
            A[i, k:] -= m * A[k, k:]
            b[i] -= m * b[k]
        if verbose:
            print(f"after step {k + 1} (pivot row {p + 1}):")
            print(np.column_stack([A, b]))
    return back_substitution(A, b)


A = np.array([[1, 2, 6], [4, 8, -1], [-2, 3, 5]], dtype=float)
b = np.array([3, -13, -3], dtype=float)
x = gauss_solve(A, b, verbose=True)
print("x =", x)
print("residual b - A x =", b - A @ x)
print("numpy.linalg.solve:", np.linalg.solve(A, b))
```

Output

```
after step 1 (pivot row 2):
[[  4.     8.    -1.   -13.  ]
 [  0.     0.     6.25   6.25]
 [  0.     7.     4.5   -9.5 ]]
after step 2 (pivot row 3):
[[  4.     8.    -1.   -13.  ]
 [  0.     7.     4.5   -9.5 ]
 [  0.     0.     6.25   6.25]]
x = [ 1. -2.  1.]
residual b - A x = [0. 0. 0.]
numpy.linalg.solve: [ 1. -2.  1.]
```

Exercises

6.2.1 Theory Solve by Gaussian elimination with partial pivoting, showing each augmented matrix:

$$2x_1 - x_2 + x_3 = -1, \quad 3x_1 + 3x_2 + 9x_3 = 0, \quad 3x_1 + 3x_2 + 5x_3 = 4.$$

6.2.2 Theory Show that the number of multiplications and divisions in Gaussian elimination (reduction to triangular form, including the right-hand side) is $\sum_{k=1}^{n-1}(n-k)(n-k+2) = \frac{2n^3+3n^2-5n}{6}$.

6.2.3 Python Use Listing 6.2 to solve the $n \times n$ system with $a_{ij} = 1/(i+j-1)$ (the Hilbert matrix) and $\mathbf{b} = A\,(1,\dots,1)^T$ for $n = 4, 8, 12$. How many correct digits does the solution have? (We explain this in Chapter 8.)

6.2.4 Theory *Scaled partial pivoting* chooses the pivot row maximising $|a_{ik}|/s_i$, where $s_i = \max_j |a_{ij}|$. Apply it in 4-digit arithmetic to

$$30x_1 + 591400x_2 = 591700, \qquad 5.291x_1 - 6.130x_2 = 46.78,$$

and compare with partial pivoting. (Exact solution $(10, 1)$.)

6.2.5 Python Measure the time taken by `np.linalg.solve` for random matrices with $n = 500, 1000, 2000$ and 4000 (use `time.perf_counter`). Does the time grow like n^3?

6.3 LU factorisation

Gaussian elimination secretly computes a factorisation of A. Eliminating below the pivot in column k is multiplication by the *elementary matrix*

$$E_k = I - \mathbf{m}_k \mathbf{e}_k^T, \qquad \mathbf{m}_k = (0,\dots,0, m_{k+1,k}, \dots, m_{nk})^T,$$

(the identity with the negated multipliers inserted below the diagonal in column k).

Definition 6.4. An *elementary matrix* is obtained from the identity I_n by a single row operation. A *permutation matrix* P has exactly one entry 1 in each row and each column and zeros elsewhere; PA permutes the rows of A, and $P^{-1} = P^T$.

Without row interchanges, $E_{n-1}\cdots E_2E_1A = U$ is upper triangular. Each $E_k^{-1} = I + \mathbf{m}_k\mathbf{e}_k^T$ is obtained simply by changing the signs of the multipliers, and their product is

$$L = E_1^{-1}E_2^{-1}\cdots E_{n-1}^{-1} = \begin{pmatrix} 1 & & & & \\ m_{21} & 1 & & & \\ m_{31} & m_{32} & 1 & & \\ \vdots & & & \ddots & \\ m_{n1} & m_{n2} & \cdots & m_{n,n-1} & 1 \end{pmatrix}.$$

Hence $A = LU$: *L is unit lower triangular and contains the multipliers; U is the final upper triangular matrix.*

Theorem 6.5 (existence of LU)**.** *1. If all leading principal submatrices $A_k = A(1{:}k, 1{:}k)$, $k = 1, \ldots, n-1$, of A are nonsingular, then $A = LU$ with L unit lower triangular and U upper triangular, and this factorisation is unique.*

2. For every nonsingular A there is a permutation matrix P with $PA = LU$ (partial pivoting).

Solving with the LU factorisation. If $PA = LU$ then $A\mathbf{x} = \mathbf{b} \iff LU\mathbf{x} = P\mathbf{b}$. Set $\mathbf{z} = U\mathbf{x}$:

1. solve $L\mathbf{z} = P\mathbf{b}$ by forward substitution;

2. solve $U\mathbf{x} = \mathbf{z}$ by back substitution.

Also $\det A = \det(P^T)\det L \det U = \pm\prod_i u_{ii}$ (the sign is $(-1)^{\text{number of row swaps}}$).

Example 6.6 (Example 6.3 continued)**.** In the hand computation we interchanged rows 2 and 3 after the first elimination step. Collecting the multipliers (and swapping them with the rows),

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}}_{P}\underbrace{\begin{pmatrix} 1 & 2 & 6 \\ 4 & 8 & -1 \\ -2 & 3 & 5 \end{pmatrix}}_{A} = \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 4 & 0 & 1 \end{pmatrix}}_{L}\underbrace{\begin{pmatrix} 1 & 2 & 6 \\ 0 & 7 & 17 \\ 0 & 0 & -25 \end{pmatrix}}_{U}.$$

Forward substitution $L\mathbf{z} = P\mathbf{b} = (3, -3, -13)^T$ gives $z_1 = 3$, $z_2 = -3 + 2\cdot 3 = 3$, $z_3 = -13 - 4\cdot 3 = -25$; back substitution $U\mathbf{x} = \mathbf{z}$ gives $x_3 = 1$, $x_2 = (3 - 17)/7 = -2$, $x_1 = 1$. Moreover $\det A = -(1\cdot 7\cdot(-25)) = 175$ (one row swap).
With partial pivoting a different (but equally valid) factorisation is obtained:

Listing 6.3: LU factorisation with partial pivoting ch06/lu.py

```
# LU factorisation with partial pivoting:  P A = L U
import numpy as np
import scipy.linalg as sla


def lu_pivot(A):
    """Return P, L, U with P A = L U (Doolittle form: L has unit diagonal)."""
    A = np.array(A, dtype=float)
    n = A.shape[0]
    perm = np.arange(n)
    for k in range(n - 1):
        p = k + np.argmax(np.abs(A[k:, k]))
        if p != k:
            A[[k, p]] = A[[p, k]]            # swap whole rows (including stored multipliers)
            perm[[k, p]] = perm[[p, k]]
        A[k + 1:, k] /= A[k, k]              # multipliers l_ik, stored below the diagonal
        A[k + 1:, k + 1:] -= np.outer(A[k + 1:, k], A[k, k + 1:])
    L = np.tril(A, -1) + np.eye(n)
    U = np.triu(A)
    P = np.eye(n)[perm]
    return P, L, U


def lu_solve(P, L, U, b):
    """Solve A x = b using P A = L U:  L z = P b, then U x = z."""
    z = sla.solve_triangular(L, P @ b, lower=True, unit_diagonal=True)
    return sla.solve_triangular(U, z)


A = np.array([[1, 2, 6], [4, 8, -1], [-2, 3, 5]], dtype=float)
b = np.array([3, -13, -3], dtype=float)
P, L, U = lu_pivot(A)
print("P =\n", P, "\nL =\n", L, "\nU =\n", U)
print("check ||PA - LU|| =", np.linalg.norm(P @ A - L @ U))
print("x =", lu_solve(P, L, U, b))
print("det A = det(P^T) * prod(diag U) =", np.linalg.det(P.T) * np.prod(np.diag(U)))

# The same with SciPy.  Note: scipy returns A = P L U, i.e. its P is our P^T.
P2, L2, U2 = sla.lu(A)
print("scipy U =\n", U2)
lu, piv = sla.lu_factor(A)        # factorise once ...
for rhs in ([3, -13, -3], [1, 0, 0]):
    print("solve with factors, b =", rhs, "->", sla.lu_solve((lu, piv), rhs))  # ... reuse
```

Output

```
P =
 [[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]]
L =
 [[ 1.    0.    0.  ]
 [-0.5   1.    0.  ]
 [ 0.25  0.    1.  ]]
U =
 [[ 4.    8.   -1.  ]
 [ 0.    7.    4.5 ]
 [ 0.    0.    6.25]]
check ||PA - LU|| = 0.0
x = [ 1. -2.  1.]
det A = det(P^T) * prod(diag U) = 175.0
scipy U =
 [[ 4.    8.   -1.  ]
 [ 0.    7.    4.5 ]
 [ 0.    0.    6.25]]
solve with factors, b = [3, -13, -3] -> [ 1. -2.  1.]
solve with factors, b = [1, 0, 0] -> [ 0.24571429 -0.10285714  0.16      ]
```

△ Caution: conventions differ

Our P satisfies $PA = LU$. SciPy's `scipy.linalg.lu` returns P with $A = PLU$, i.e. the transpose of ours. Always check the documentation, and check your factors by computing $\|PA - LU\|$.

6.3.1 Tridiagonal systems

If A is tridiagonal, L and U are bidiagonal and elimination (without pivoting, when A is diagonally dominant) costs only $O(n)$ operations: the **Thomas algorithm**. Such systems arise from finite-difference discretisations of boundary-value problems, e.g. $-u'' = f$ discretised as $(-u_{i-1} + 2u_i - u_{i+1})/h^2 = f_i$.

Listing 6.4: The Thomas algorithm for a boundary-value problem ch06/tridiagonal.py

```python
# The Thomas algorithm for tridiagonal systems (O(n) operations)
import numpy as np


def thomas(a, d, c, b):
    """Solve a tridiagonal system.

    a: sub-diagonal (length n-1), d: diagonal (n), c: super-diagonal (n-1), b: rhs (n).
    This is LU factorisation without pivoting specialised to tridiagonal matrices."""
    n = len(d)
    d = np.array(d, dtype=float)
    b = np.array(b, dtype=float)
    for i in range(1, n):                 # elimination
        m = a[i - 1] / d[i - 1]
        d[i] -= m * c[i - 1]
        b[i] -= m * b[i - 1]
    x = np.zeros(n)
```

```
    x[-1] = b[-1] / d[-1]
    for i in range(n - 2, -1, -1):        # back substitution
        x[i] = (b[i] - c[i] * x[i + 1]) / d[i]
    return x


# -u'' = pi^2 sin(pi x) on (0,1), u(0) = u(1) = 0; exact solution u = sin(pi x)
for n in [10, 100, 1000, 10000]:
    h = 1.0 / (n + 1)
    xg = np.linspace(h, 1 - h, n)
    rhs = h**2 * np.pi**2 * np.sin(np.pi * xg)
    u = thomas(-np.ones(n - 1), 2 * np.ones(n), -np.ones(n - 1), rhs)
    print(f"n = {n:5d}: max error = {np.max(np.abs(u - np.sin(np.pi * xg))):.3e}")
```

Output

```
n =    10: max error = 6.756e-03
n =   100: max error = 8.062e-05
n =  1000: max error = 8.208e-07
n = 10000: max error = 8.232e-09
```

The error decreases by a factor of 100 each time n is multiplied by 10: the finite-difference method is $O(h^2)$ accurate. Solving with $n = 10^4$ unknowns takes a fraction of a second.

Exercises

6.3.1 Theory Find the LU factorisation (without pivoting) of $A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}$ and use it to solve $A\mathbf{x} = (1, 2, 3)^T$ and $A\mathbf{x} = (0, 1, 0)^T$. Compute $\det A$.

6.3.2 Theory Show that $A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ has no LU factorisation, but PA has one for a suitable P.

6.3.3 Theory Prove that the product of two unit lower triangular matrices is unit lower triangular, and that the inverse of a unit lower triangular matrix is unit lower triangular. Deduce the uniqueness part of Theorem 6.5.

6.3.4 Python Compute A^{-1} for the matrix of Example 6.3 by solving $A\mathbf{x} = \mathbf{e}_j$, $j = 1, 2, 3$, with one LU factorisation. How many flops does computing A^{-1} cost, compared with solving one system?

6.3.5 Python The *Crout* factorisation has U unit upper triangular instead of L. Derive the formulas and implement them.

6.3.6 ★ Graduate Show that Gaussian elimination with partial pivoting applied to the matrix with $a_{ii} = 1$, $a_{in} = 1$, $a_{ij} = -1$ for $i > j$ and zeros elsewhere produces $u_{nn} = 2^{n-1}$. This *growth factor* is the worst possible. Why is it almost never observed in practice? (Trefethen & Bau [18], Lecture 22, call this one of the great unsolved puzzles of numerical analysis.)

6.4 Cholesky factorisation

Many matrices in applications (covariance matrices in statistics, stiffness matrices in engineering, normal equations in least squares, discretised elliptic PDEs) are symmetric and positive definite.

Definition 6.7 (positive definite)**.** A symmetric matrix $A \in \mathbb{R}^{n\times n}$ is *positive definite* (SPD) if $\mathbf{x}^T A\mathbf{x} > 0$ for every $\mathbf{x} \neq \mathbf{0}$, and *positive semi-definite* if $\mathbf{x}^T A\mathbf{x} \geq 0$ for all $\mathbf{x}$. The function $\mathbf{x} \mapsto \mathbf{x}^T A\mathbf{x} = \sum_{i,j} a_{ij}x_i x_j$ is called a *quadratic form*.

Example 6.8. For $A = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix}$,

$$\mathbf{x}^T A\mathbf{x} = 4x_1^2 + 2x_1x_2 + 2x_2^2 = 4\Big(x_1 + \tfrac{1}{4}x_2\Big)^2 + \tfrac{7}{4}x_2^2 > 0 \quad (\mathbf{x} \neq \mathbf{0}),$$

so A is positive definite (complete the square!).

Theorem 6.9 (characterisations of SPD matrices)**.** *For a symmetric matrix A the following are equivalent:*

1. *A is positive definite;*
2. *all eigenvalues of A are positive;*
3. *all leading principal minors* $\det A_k$, $k = 1, \ldots, n$, *are positive (Sylvester's criterion);*
4. *$A = LL^T$ with L lower triangular with positive diagonal entries (Cholesky);*
5. *$A = LDL^T$ with L unit lower triangular and D diagonal with positive entries.*

Moreover, if A is SPD then $a_{ii} > 0$ for all i and the largest entry in absolute value lies on the diagonal. A symmetric strictly diagonally dominant *matrix with positive diagonal entries is SPD.*

Example 6.10. $B = \begin{pmatrix} 10 & -2 & -1 \\ -2 & 8 & -3 \\ -1 & -3 & 6 \end{pmatrix}$ has leading principal minors 10, $\begin{vmatrix} 10 & -2 \\ -2 & 8 \end{vmatrix} = 76$ and $\det B = 346$, all positive, so B is SPD. (It is also strictly diagonally dominant.)

Deriving the algorithm. Equate entries in $A = LL^T$ for $n = 3$:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{31}l_{11} & l_{31}l_{21} + l_{32}l_{22} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{pmatrix}.$$

Column by column: $l_{11} = \sqrt{a_{11}}$, $l_{21} = a_{21}/l_{11}$, $l_{31} = a_{31}/l_{11}$; $l_{22} = \sqrt{a_{22} - l_{21}^2}$, $l_{32} = (a_{32} - l_{31}l_{21})/l_{22}$; $l_{33} = \sqrt{a_{33} - l_{31}^2 - l_{32}^2}$. In general, for $k = 1, \ldots, n$:

$$l_{kk} = \Big(a_{kk} - \sum_{m=1}^{k-1} l_{km}^2\Big)^{1/2}, \qquad l_{ik} = \frac{1}{l_{kk}}\Big(a_{ik} - \sum_{m=1}^{k-1} l_{im}l_{km}\Big), \quad i = k+1, \ldots, n. \tag{6.2}$$

The quantity under the square root is positive exactly when A is SPD, so the algorithm is also the cheapest practical *test* for positive definiteness. It needs no pivoting, and costs $\frac{1}{3}n^3$ flops, half the cost of LU.

Example 6.11. Factorise $A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{pmatrix}$.

$$l_{11} = \sqrt{4} = 2, \quad l_{21} = -\tfrac{1}{2}, \quad l_{31} = \tfrac{1}{2}; \qquad l_{22} = \sqrt{4.25 - \tfrac{1}{4}} = 2,$$

$$l_{32} = \frac{2.75 - (\frac{1}{2})(-\frac{1}{2})}{2} = 1.5; \qquad l_{33} = \sqrt{3.5 - \frac{1}{4} - 2.25} = 1.$$

To solve $A\mathbf{x} = \mathbf{b}$: solve $L\mathbf{z} = \mathbf{b}$ (forward), then $L^T\mathbf{x} = \mathbf{z}$ (backward).

Listing 6.5: Cholesky factorisation and tests for positive definiteness ch06/cholesky.py

```python
# Cholesky factorisation A = L L^T of a symmetric positive definite matrix
import numpy as np


def cholesky(A):
    """Return lower triangular L with A = L L^T (A symmetric positive definite)."""
    A = np.array(A, dtype=float)
    n = A.shape[0]
    L = np.zeros((n, n))
    for k in range(n):
        s = A[k, k] - L[k, :k] @ L[k, :k]
        if s <= 0:
            raise ValueError("matrix is not positive definite")
        L[k, k] = np.sqrt(s)
        for i in range(k + 1, n):
            L[i, k] = (A[i, k] - L[i, :k] @ L[k, :k]) / L[k, k]
    return L


A = np.array([[4, -1, 1], [-1, 4.25, 2.75], [1, 2.75, 3.5]])
L = cholesky(A)
print("L =\n", L)
print("||A - L L^T|| =", np.linalg.norm(A - L @ L.T))
print("numpy cholesky agrees:", np.allclose(L, np.linalg.cholesky(A)))

# solve A x = b:  L z = b (forward),  L^T x = z (backward)
b = np.array([4.0, 6.0, 7.25])
z = np.linalg.solve(L, b)
x = np.linalg.solve(L.T, z)
print("solution x =", x)

# Tests for positive definiteness of B
B = np.array([[10, -2, -1], [-2, 8, -3], [-1, -3, 6]], dtype=float)
minors = [np.linalg.det(B[:k, :k]) for k in range(1, 4)]
print("leading principal minors of B:", np.round(minors, 6))
print("eigenvalues of B:", np.round(np.linalg.eigvalsh(B), 6))
C = np.array([[1, 2], [2, 1]], dtype=float)
try:
    cholesky(C)
except ValueError as err:
    print("C =", C.tolist(), "->", err)
```

Output

```
L =
 [[ 2.   0.   0. ]
 [-0.5  2.   0. ]
 [ 0.5  1.5  1. ]]
||A - L L^T|| = 0.0
numpy cholesky agrees: True
solution x = [1. 1. 1.]
leading principal minors of B: [ 10.  76. 346.]
eigenvalues of B: [ 3.243409  9.36285  11.393741]
C = [[1.0, 2.0], [2.0, 1.0]] -> matrix is not positive definite
```

Historical Note: André-Louis Cholesky

André-Louis Cholesky (1875–1918) was a French artillery officer and geodesist who developed his method around 1910 to solve the least-squares problems of map-making in Crete and North Africa. He was killed in action near the end of the First World War. His method was published in 1924 by a fellow officer, Commandant Benoît; Cholesky's own manuscript was only rediscovered in the family archives in 2003 and published in 2005.

Exercises

6.4.1 Theory Determine which of the following matrices are positive definite, using two different tests: $\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}$, $\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$, $\begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{pmatrix}$.

6.4.2 Theory Compute the Cholesky factor of $\begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{pmatrix}$ by hand and use it to solve $A\mathbf{x} = (2, 1, 4)^T$. Then write $A = LDL^T$.

6.4.3 Theory Prove that if A is SPD then $a_{ii} > 0$ and $|a_{ij}| < \sqrt{a_{ii}a_{jj}}$ for $i \neq j$. (Use $\mathbf{x} = \mathbf{e}_i$ and $\mathbf{x} = \mathbf{e}_i \pm t\mathbf{e}_j$.)

6.4.4 Theory Prove that A^TA is SPD whenever A has linearly independent columns.

6.4.5 Python Generate a random SPD matrix as `A = M.T @ M + n*np.eye(n)` and compare the results and running times of your Cholesky code, `np.linalg.cholesky` and `np.linalg.solve`.

6.4.6 ★ Graduate The Cholesky factorisation of a covariance matrix $\Sigma = LL^T$ lets us sample from the multivariate normal distribution: if $\mathbf{z} \sim N(\mathbf{0}, I)$ then $L\mathbf{z} \sim N(\mathbf{0}, \Sigma)$. Prove this and test it in Python by computing the sample covariance of 10^5 samples.

6.5 QR factorisation

Definition 6.12. A matrix $Q \in \mathbb{R}^{m \times n}$ has *orthonormal columns* if $Q^TQ = I_n$. A square matrix with this property is *orthogonal*; then $Q^{-1} = Q^T$.

The great virtue of orthogonal matrices is that they are trivially inverted and that they do not amplify errors:

Theorem 6.13. *If Q has orthonormal columns then $\|Q\mathbf{x}\|_2 = \|\mathbf{x}\|_2$ for all $\mathbf{x}$.*

Proof. $\|Q\mathbf{x}\|_2^2 = (Q\mathbf{x})^T(Q\mathbf{x}) = \mathbf{x}^T Q^T Q\mathbf{x} = \mathbf{x}^T\mathbf{x} = \|\mathbf{x}\|_2^2$. □

Theorem 6.14 (QR factorisation)**.** *Let $A \in \mathbb{R}^{m\times n}$, $m \geq n$, have linearly independent columns $\mathbf{a}_1, \ldots, \mathbf{a}_n$. Then $A = QR$, where $Q \in \mathbb{R}^{m\times n}$ has orthonormal columns forming a basis of the column space of A, and $R \in \mathbb{R}^{n\times n}$ is upper triangular with positive diagonal entries.*

Gram–Schmidt. The columns of Q are obtained by orthonormalising the columns of A one at a time:

$$\mathbf{v}_k = \mathbf{a}_k - \sum_{j=1}^{k-1} r_{jk}\mathbf{q}_j, \quad r_{jk} = \mathbf{q}_j^T\mathbf{a}_k, \qquad r_{kk} = \|\mathbf{v}_k\|_2, \quad \mathbf{q}_k = \mathbf{v}_k/r_{kk}.$$

Rearranging, $\mathbf{a}_k = \sum_{j=1}^{k} r_{jk}\mathbf{q}_j$, which is exactly $A = QR$.

Example 6.15. Find the QR factorisation of $A = \begin{pmatrix} 1 & -1 & 4 \\ 1 & 4 & -2 \\ 1 & 4 & 2 \\ 1 & -1 & 0 \end{pmatrix}$.

Solution. $r_{11} = \|\mathbf{a}_1\| = \sqrt{4} = 2$, $\mathbf{q}_1 = \frac{1}{2}(1,1,1,1)^T$. Next $r_{12} = \mathbf{q}_1^T\mathbf{a}_2 = \frac{1}{2}(-1+4+4-1) = 3$ and $\mathbf{v}_2 = \mathbf{a}_2 - 3\mathbf{q}_1 = (-\frac{5}{2}, \frac{5}{2}, \frac{5}{2}, -\frac{5}{2})^T$, so $r_{22} = 5$ and $\mathbf{q}_2 = \frac{1}{2}(-1,1,1,-1)^T$. Finally $r_{13} = \mathbf{q}_1^T\mathbf{a}_3 = 2$, $r_{23} = \mathbf{q}_2^T\mathbf{a}_3 = -2$, $\mathbf{v}_3 = \mathbf{a}_3 - 2\mathbf{q}_1 + 2\mathbf{q}_2 = (2,-2,2,-2)^T$, $r_{33} = 4$, $\mathbf{q}_3 = \frac{1}{2}(1,-1,1,-1)^T$. Therefore

$$Q = \frac{1}{2}\begin{pmatrix} 1 & -1 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 1 \\ 1 & -1 & -1 \end{pmatrix}, \qquad R = \begin{pmatrix} 2 & 3 & 2 \\ 0 & 5 & -2 \\ 0 & 0 & 4 \end{pmatrix}.$$

Listing 6.6: Gram–Schmidt QR, solving by QR, and loss of orthogonality ch06/qr.py

```python
# QR factorisation: classical and modified Gram-Schmidt, and Householder (numpy)
import numpy as np


def qr_cgs(A):
    """Classical Gram-Schmidt: A = Q R, Q with orthonormal columns."""
    A = np.array(A, dtype=float)
    m, n = A.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for k in range(n):
        v = A[:, k].copy()
        for j in range(k):
            R[j, k] = Q[:, j] @ A[:, k]       # project the ORIGINAL column
            v -= R[j, k] * Q[:, j]
        R[k, k] = np.linalg.norm(v)
        Q[:, k] = v / R[k, k]
    return Q, R


def qr_mgs(A):
    """Modified Gram-Schmidt: numerically much better than CGS."""
    V = np.array(A, dtype=float)
    m, n = V.shape
    Q = np.zeros((m, n))
    R = np.zeros((n, n))
    for k in range(n):
        R[k, k] = np.linalg.norm(V[:, k])
        Q[:, k] = V[:, k] / R[k, k]
        for j in range(k + 1, n):
            R[k, j] = Q[:, k] @ V[:, j]       # project the UPDATED column
            V[:, j] -= R[k, j] * Q[:, k]
    return Q, R


A = np.array([[1, -1, 4], [1, 4, -2], [1, 4, 2], [1, -1, 0]], dtype=float)
Q, R = qr_cgs(A)
print("Q =\n", Q, "\nR =\n", R)
print("Q^T Q =\n", np.round(Q.T @ Q, 12))
print("||A - QR|| =", np.linalg.norm(A - Q @ R))

# Solving a square system by QR:  R x = Q^T b
M = np.array([[1, 2, 6], [4, 8, -1], [-2, 3, 5]], dtype=float)
b = np.array([3, -13, -3], dtype=float)
Qm, Rm = np.linalg.qr(M)                      # Householder reflections (LAPACK)
print("QR solution:", np.linalg.solve(Rm, Qm.T @ b))

# Loss of orthogonality on an ill-conditioned matrix
print("\nloss of orthogonality ||I - Q^T Q|| for Hilbert matrices:")
for n in [4, 6, 8, 10, 12]:
    H = 1.0 / (np.arange(1, n + 1)[:, None] + np.arange(n)[None, :])
    errs = []
    for f in (qr_cgs, qr_mgs, np.linalg.qr):
        Qh = f(H)[0]
        errs.append(np.linalg.norm(np.eye(n) - Qh.T @ Qh))
    print(f"n = {n:2d}: CGS {errs[0]:.1e}   MGS {errs[1]:.1e}   Householder {errs[2]:.1e}
    ↪ ")
```

Output

```
Q =
 [[ 0.5 -0.5  0.5]
 [ 0.5  0.5 -0.5]
 [ 0.5  0.5  0.5]
 [ 0.5 -0.5 -0.5]]
R =
 [[ 2.  3.  2.]
 [ 0.  5. -2.]
 [ 0.  0.  4.]]
Q^T Q =
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
||A - QR|| = 0.0
QR solution: [ 1. -2.  1.]

loss of orthogonality ||I - Q^T Q|| for Hilbert matrices:
n =  4: CGS 4.9e-11   MGS 4.2e-13   Householder 9.6e-16
n =  6: CGS 2.8e-04   MGS 1.0e-09   Householder 5.2e-16
n =  8: CGS 1.4e+00   MGS 3.6e-07   Householder 1.5e-15
n = 10: CGS 3.5e+00   MGS 2.5e-04   Householder 1.3e-15
n = 12: CGS 5.5e+00   MGS 1.6e-01   Householder 1.5e-15
```

Solving with QR. For square nonsingular A: $A\mathbf{x} = \mathbf{b} \iff QR\mathbf{x} = \mathbf{b} \iff R\mathbf{x} = Q^T\mathbf{b}$, a triangular system. For rectangular A ($m > n$) the same formula gives the *least-squares* solution (Chapter 10), which is the main application of QR.

△ Caution: classical Gram–Schmidt is unstable

The output shows that for the (ill-conditioned) Hilbert matrices the vectors computed by classical Gram–Schmidt (CGS) are far from orthogonal: for $n = 8$, $\|I - Q^TQ\| \approx 1$! The *modified* Gram–Schmidt (MGS) algorithm, which subtracts each projection from the *updated* vector, is much better. Library routines (`np.linalg.qr`) use **Householder reflections** $H = I - 2\mathbf{v}\mathbf{v}^T/(\mathbf{v}^T\mathbf{v})$, which produce Q orthogonal to machine precision regardless of the conditioning of A.

Historical Note: Gram, Schmidt, Householder

The orthogonalisation process is named after the Danish actuary **Jørgen Pedersen Gram** (1883) and the German analyst **Erhard Schmidt** (1907), who used it in infinite-dimensional function spaces; it already appears in work of Laplace (1812). **Alston Householder** (1958) and **Wallace Givens** (1958) introduced orthogonal reflections and rotations, which made QR a stable algorithm. In 1961 **John Francis** (UK) and **Vera Kublanovskaya** (USSR) independently invented the *QR algorithm* for computing eigenvalues by repeated QR factorisations, named one of the ten most important algorithms of the 20th century.

Towards Graduate Studies: numerical linear algebra

At graduate level one studies these factorisations through the lens of *backward stability*: Householder QR is backward stable; Gaussian elimination with partial pivoting is backward stable "in practice" (the growth factor puzzle); Cholesky is unconditionally stable. Other key topics are the *singular value decomposition* $A = U\Sigma V^T$ (the most important factorisation of

all, used in data compression, principal component analysis and recommender systems), eigenvalue algorithms, and *sparse* direct solvers, which exploit the zeros of large matrices through graph-theoretic reordering. Standard texts: Trefethen & Bau [18], Golub & Van Loan [8], Demmel, *Applied Numerical Linear Algebra*.

Exercises

6.5.1 Theory Find the QR factorisation of $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$ by Gram–Schmidt, and of $\begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$.

6.5.2 Theory Use the QR factorisation of Example 6.15 to find the least-squares solution of $A\mathbf{x} = (1, 2, 3, 4)^T$ by solving $R\mathbf{x} = Q^T\mathbf{b}$.

6.5.3 Theory Show that the Householder matrix $H = I - 2\mathbf{v}\mathbf{v}^T/(\mathbf{v}^T\mathbf{v})$ is symmetric and orthogonal, and that $H\mathbf{v} = -\mathbf{v}$ while $H\mathbf{w} = \mathbf{w}$ for $\mathbf{w} \perp \mathbf{v}$. Interpret geometrically.

6.5.4 Theory Show that if $\mathbf{x}$ and $\mathbf{y}$ have the same 2-norm, the Householder matrix with $\mathbf{v} = \mathbf{x} - \mathbf{y}$ maps $\mathbf{x}$ to $\mathbf{y}$. How is this used to create zeros below the diagonal?

6.5.5 Python Implement Householder QR and compare with Listing 6.6 on the Hilbert matrices.

6.5.6 ★ Graduate Prove Theorem 6.14, including the uniqueness of Q and R when the diagonal of R is required to be positive.

Chapter Summary

- Triangular systems: n^2 flops. Gaussian elimination: $\tfrac{2}{3}n^3$ flops.
- Always use partial pivoting: $PA = LU$ exists for every nonsingular A.
- Solve $A\mathbf{x} = \mathbf{b}$ via $L\mathbf{z} = P\mathbf{b}$, $U\mathbf{x} = \mathbf{z}$; $\det A = \pm \prod u_{ii}$.
- SPD matrices: $A = LL^T$ (Cholesky), $\tfrac{1}{3}n^3$ flops, no pivoting; it also tests definiteness.
- $A = QR$ by Gram–Schmidt (use MGS) or Householder (stable); $R\mathbf{x} = Q^T\mathbf{b}$.
- Python: `np.linalg.solve`, `scipy.linalg.lu_factor/lu_solve`, `np.linalg.cholesky`, `np.linalg.qr`.

Chapter 7

Iterative Methods for Linear Systems

Learning Objectives

After studying this chapter you should be able to:

- explain when iterative methods are preferable to direct methods;
- write the Jacobi, Gauss–Seidel and SOR methods in component and matrix form and apply them by hand;
- prove that a stationary iteration converges if and only if $\rho(T) < 1$, and use diagonal dominance to guarantee convergence;
- choose a good relaxation parameter ω for SOR;
- derive the steepest descent and conjugate gradient methods from the minimisation of a quadratic function;
- use sparse matrices and `scipy.sparse.linalg` in Python.

Direct methods cost $O(n^3)$ operations and $O(n^2)$ memory. For the huge linear systems of modern applications, which come from discretised partial differential equations, networks, image processing and machine learning, n may be 10^6 to 10^9, and direct methods are out of the question. Fortunately such matrices are usually **sparse**: almost all entries are zero. An **iterative method** starts from an initial guess $\mathbf{x}^{(0)}$ and generates a sequence $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots \to \mathbf{x}$, using the matrix only through products $A\mathbf{v}$, which cost $O(\text{number of nonzeros})$ operations.

⋆ Key Concept: direct versus iterative

Direct (LU, Cholesky, QR)	**Iterative** (Jacobi, GS, SOR, CG)
exact answer after a fixed number of steps	approximate answer, improved at each step
$O(n^3)$ work, $O(n^2)$ memory (dense)	$O(\text{nnz})$ work per iteration
robust: works for any nonsingular A	convergence depends on properties of A
best for small/medium dense matrices	best for large sparse matrices

⋆ Key Concept: measuring vectors and matrices: a preview of Chapter 8

To say that $\mathbf{x}^{(k)} \to \mathbf{x}$ we measure the error with a *norm*. In this chapter we mainly use

$$\|\mathbf{x}\|_\infty = \max_{1\le i\le n} |x_i|, \qquad \|\mathbf{x}\|_2 = \Big(\sum_{i=1}^{n} x_i^2\Big)^{1/2},$$

$$\|A\|_\infty = \max_{\mathbf{x}\neq\mathbf{0}} \frac{\|A\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \max_i \sum_{j=1}^{n} |a_{ij}| \ \text{(maximum row sum)},$$

which satisfy $\|A\mathbf{x}\| \le \|A\|\,\|\mathbf{x}\|$. The *spectral radius* of a square matrix is $\rho(A) = \max\{|\lambda| : \lambda \text{ an eigenvalue of } A\}$. Two facts, proved in Chapter 8 (Theorem 8.7), are all we need here:

$$\rho(A) \le \|A\| \quad \text{for every induced norm}, \qquad A^k \to 0 \ \text{as } k\to\infty \iff \rho(A) < 1.$$

For symmetric positive definite A we also use the *condition number* $\kappa_2(A) = \lambda_{\max}/\lambda_{\min}$, studied in Section 8.3.

7.1 Stationary iterative methods

Write $A = M - N$ with M nonsingular and "easy to invert". Then $A\mathbf{x} = \mathbf{b} \iff M\mathbf{x} = N\mathbf{x} + \mathbf{b}$, which suggests the iteration

$$M\mathbf{x}^{(k+1)} = N\mathbf{x}^{(k)} + \mathbf{b}, \qquad \text{i.e.} \qquad \mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}, \quad T = M^{-1}N,\ \mathbf{c} = M^{-1}\mathbf{b}. \tag{7.1}$$

This is a fixed-point iteration for $\mathbf{x} = T\mathbf{x} + \mathbf{c}$; T is the *iteration matrix*. For the standard methods we split A into its diagonal, strictly lower and strictly upper parts,

$$A = D - L - U, \qquad D = \operatorname{diag}(a_{11}, \dots, a_{nn}),$$
$$-L = \text{strictly lower part of } A, \qquad -U = \text{strictly upper part of } A.$$

(Note the minus signs, which follow Burden & Faires [2].)

Theorem 7.1 (convergence of stationary iterations). *For any $\mathbf{x}^{(0)}$, the iteration $\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}$ converges to the unique solution of $\mathbf{x} = T\mathbf{x} + \mathbf{c}$ if and only if $\rho(T) < 1$. Moreover, for any induced norm with $\|T\| < 1$,*

$$\left\|\mathbf{x}^{(k)} - \mathbf{x}\right\| \le \|T\|^k \left\|\mathbf{x}^{(0)} - \mathbf{x}\right\|, \qquad \left\|\mathbf{x}^{(k)} - \mathbf{x}\right\| \le \frac{\|T\|^k}{1 - \|T\|} \left\|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\right\|.$$

Proof. Subtracting $\mathbf{x} = T\mathbf{x} + \mathbf{c}$ from the iteration gives $\mathbf{e}^{(k+1)} = T\mathbf{e}^{(k)}$ for the error $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}$, hence $\mathbf{e}^{(k)} = T^k\mathbf{e}^{(0)}$. By Theorem 8.7, $T^k\mathbf{e}^{(0)} \to \mathbf{0}$ for every $\mathbf{e}^{(0)}$ if and only if $\rho(T) < 1$. The bounds follow as in Theorem 4.10. □

⋆ Key Concept: rate of convergence

Asymptotically the error is reduced by a factor $\rho(T)$ per iteration. To gain one decimal digit requires about $-1/\log_{10}\rho(T)$ iterations: 2.3 for $\rho = 0.37$, 23 for $\rho = 0.9$, 230 for $\rho = 0.99$. The quantity $R(T) = -\log_{10}\rho(T)$ is called the *rate of convergence*.

7.2 The Jacobi method

Solve the i-th equation for x_i, using the old values of all other unknowns:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j\neq i} a_{ij}x_j^{(k)}\Big), \qquad i = 1,\dots,n. \tag{7.2}$$

In matrix form ($M = D$, $N = L + U$):

$$\mathbf{x}^{(k+1)} = T_J\mathbf{x}^{(k)} + \mathbf{c}_J, \qquad T_J = D^{-1}(L+U), \quad \mathbf{c}_J = D^{-1}\mathbf{b}.$$

All components can be updated simultaneously, which makes Jacobi ideal for parallel computers.

Example 7.2. Solve

$$4x_1 + 3x_2 = 24, \qquad 3x_1 + 4x_2 - x_3 = 30, \qquad -x_2 + 4x_3 = -24,$$

whose solution is $\mathbf{x} = (3, 4, -5)^T$, starting from $\mathbf{x}^{(0)} = (1, 1, 1)^T$. The Jacobi iteration is

$$x_1^{(k+1)} = \frac{24 - 3x_2^{(k)}}{4}, \quad x_2^{(k+1)} = \frac{30 - 3x_1^{(k)} + x_3^{(k)}}{4}, \quad x_3^{(k+1)} = \frac{-24 + x_2^{(k)}}{4}.$$

First iteration: $x_1^{(1)} = (24-3)/4 = 5.25$, $x_2^{(1)} = (30-3+1)/4 = 7$, $x_3^{(1)} = (-24+1)/4 = -5.75$. Second iteration: $x_1^{(2)} = (24-21)/4 = 0.75$, $x_2^{(2)} = (30-15.75-5.75)/4 = 2.125$, $x_3^{(2)} = (-24+7)/4 = -4.25$.

7.3 The Gauss–Seidel method

In (7.2), when $x_i^{(k+1)}$ is computed the new values $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ are already available. Using them immediately gives the **Gauss–Seidel** method:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\Big), \qquad i = 1,\dots,n. \tag{7.3}$$

In matrix form $(D - L)\mathbf{x}^{(k+1)} = U\mathbf{x}^{(k)} + \mathbf{b}$ ($M = D - L$, $N = U$):

$$\mathbf{x}^{(k+1)} = T_G\mathbf{x}^{(k)} + \mathbf{c}_G, \qquad T_G = (D-L)^{-1}U, \quad \mathbf{c}_G = (D-L)^{-1}\mathbf{b}.$$

In practice one never forms $(D - L)^{-1}$; the component formula (7.3) *is* the forward substitution with the lower triangular matrix $D - L$.

Example 7.3 (Example 7.2 continued). $x_1^{(1)} = (24 - 3\cdot 1)/4 = 5.25$, then $x_2^{(1)} = (30 - 3\cdot 5.25 + 1)/4 = 3.8125$ (using the new x_1), then $x_3^{(1)} = (-24 + 3.8125)/4 = -5.046875$. So $\mathbf{x}^{(1)} = (5.25,\ 3.8125,\ -5.0469)^T$, already much closer to $(3, 4, -5)^T$ than the Jacobi iterate. (Equivalently, in matrix form, $\mathbf{x}^{(1)} = (D-L)^{-1}U\mathbf{x}^{(0)} + (D-L)^{-1}\mathbf{b} = (-\tfrac{3}{4}, \tfrac{13}{16}, \tfrac{13}{64})^T + (6, 3, -\tfrac{21}{4})^T$.)

Listing 7.1: Jacobi and Gauss–Seidel ch07/jacobi_gs.py

```python
# Jacobi and Gauss-Seidel iterations
import numpy as np


def jacobi(A, b, x0, tol=1e-10, max_iter=500):
    """Jacobi: every component of x^(k+1) uses only the OLD vector x^(k)."""
    n = len(b)
    x = np.array(x0, dtype=float)
    history = [x.copy()]
    for k in range(1, max_iter + 1):
        x_new = np.empty(n)
        for i in range(n):
            s = A[i, :i] @ x[:i] + A[i, i + 1:] @ x[i + 1:]
            x_new[i] = (b[i] - s) / A[i, i]
        history.append(x_new.copy())
        if np.linalg.norm(x_new - x, np.inf) < tol * np.linalg.norm(x_new, np.inf):
            return x_new, k, history
        x = x_new
    raise RuntimeError("Jacobi did not converge")


def gauss_seidel(A, b, x0, tol=1e-10, max_iter=500):
    """Gauss-Seidel: new components are used as soon as they are computed."""
    n = len(b)
    x = np.array(x0, dtype=float)
    history = [x.copy()]
    for k in range(1, max_iter + 1):
        x_old = x.copy()
        for i in range(n):
            s = A[i, :i] @ x[:i] + A[i, i + 1:] @ x_old[i + 1:]   # x[:i] already updated
            x[i] = (b[i] - s) / A[i, i]
        history.append(x.copy())
        if np.linalg.norm(x - x_old, np.inf) < tol * np.linalg.norm(x, np.inf):
            return x, k, history
    raise RuntimeError("Gauss-Seidel did not converge")


A = np.array([[4, 3, 0], [3, 4, -1], [0, -1, 4]], dtype=float)
b = np.array([24, 30, -24], dtype=float)
x0 = np.ones(3)
xs = np.array([3.0, 4.0, -5.0])

xj, kj, hj = jacobi(A, b, x0)
xg, kg, hg = gauss_seidel(A, b, x0)
print(" k        Jacobi x^(k)                    Gauss-Seidel x^(k)")
for k in range(6):
    print(f"{k:2d}  {np.array2string(hj[k], precision=4, floatmode='fixed')}"
          f"   {np.array2string(hg[k], precision=4, floatmode='fixed')}")
print(f"Jacobi      : {kj} iterations, x = {xj}")
print(f"Gauss-Seidel: {kg} iterations, x = {xg}")

# Iteration matrices from the splitting A = D - L - U
D = np.diag(np.diag(A))
L = -np.tril(A, -1)
U = -np.triu(A, 1)
TJ = np.linalg.solve(D, L + U)
TG = np.linalg.solve(D - L, U)
rho = lambda T: np.max(np.abs(np.linalg.eigvals(T)))
print(f"rho(T_J) = {rho(TJ):.6f},  rho(T_GS) = {rho(TG):.6f},  rho(T_J)^2 = {rho(TJ)
    ↪ **2:.6f}")
```

Output

```
k        Jacobi x^(k)                    Gauss-Seidel x^(k)
0  [1.0000 1.0000 1.0000]   [1.0000 1.0000 1.0000]
1  [ 5.2500  7.0000 -5.7500]   [ 5.2500  3.8125 -5.0469]
2  [ 0.7500  2.1250 -4.2500]   [ 3.1406  3.8828 -5.0293]
3  [ 4.4062  5.8750 -5.4688]   [ 3.0879  3.9268 -5.0183]
4  [ 1.5938  2.8281 -4.5312]   [ 3.0549  3.9542 -5.0114]
5  [ 3.8789  5.1719 -5.2930]   [ 3.0343  3.9714 -5.0072]
Jacobi      : 100 iterations, x = [ 3.  4. -5.]
Gauss-Seidel: 43 iterations, x = [ 3.  4. -5.]
rho(T_J) = 0.790569,  rho(T_GS) = 0.625000,  rho(T_J)^2 = 0.625000
```

Gauss–Seidel needs fewer than half as many iterations as Jacobi. The reason is visible in the last line: $\rho(T_G) = \rho(T_J)^2$, so one Gauss–Seidel step is worth two Jacobi steps. (This holds for all *consistently ordered* matrices, e.g. tridiagonal ones.)

7.3.1 Sufficient conditions for convergence

Computing $\rho(T)$ is usually as hard as solving the system. Simple sufficient conditions are therefore valuable.

Definition 7.4. A is *strictly diagonally dominant* (by rows) if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for every i.

Theorem 7.5. *If A is strictly diagonally dominant, then A is nonsingular, and both the Jacobi and Gauss–Seidel methods converge for every $\mathbf{x}^{(0)}$.*

Proof for Jacobi. $(T_J)_{ij} = -a_{ij}/a_{ii}$ for $j \neq i$ and 0 on the diagonal, so $\|T_J\|_\infty = \max_i \sum_{j \neq i} |a_{ij}|/|a_{ii}| < 1$. Hence $\rho(T_J) \leq \|T_J\|_\infty < 1$, and Theorem 7.1 applies. (Nonsingularity follows since $\rho(T_J) < 1$ implies $D^{-1}A = I - T_J$ is invertible.) □

Theorem 7.6. *If A is symmetric positive definite, the Gauss–Seidel method converges for every $\mathbf{x}^{(0)}$.*

△ Caution: convergence is not automatic

Jacobi and Gauss–Seidel may diverge, and one may converge while the other diverges. The matrix of Example 7.2 is not diagonally dominant (row 2: $4 < 3 + 1$ fails strictly), yet both methods converge because $\rho(T_J) = 0.79 < 1$. Reordering the equations can make a system diagonally dominant; always try this first.

Historical Note: Gauss, Jacobi and Seidel

In an 1823 letter to his student Christian Ludwig Gerling, **Gauss** described an iterative method for the normal equations of geodetic surveys, remarking that it could be done "half asleep" or while thinking of other things. **Carl Gustav Jacob Jacobi** published his method in 1845, also for least squares. **Philipp Ludwig von Seidel**, a student of Jacobi, published the Gauss–Seidel method in 1874. Gauss himself did not use it in the form now named after him, and Seidel recommended against it!

Exercises

7.3.1 Theory Perform two iterations of Jacobi and of Gauss–Seidel, by hand, for $10x_1 - x_2 + 2x_3 = 6$, $-x_1 + 11x_2 - x_3 + 3x_4 = 25$, $2x_1 - x_2 + 10x_3 - x_4 = -11$, $3x_2 - x_3 + 8x_4 = 15$ from $\mathbf{x}^{(0)} = \mathbf{0}$. (Solution $(1, 2, -1, 1)$.)

7.3.2 Theory Find T_J and T_G for $A = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ and compute their spectral radii.

7.3.3 Theory Show that for $A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$ the Jacobi method converges but Gauss–Seidel diverges. (Compute $\rho(T_J)$ and $\rho(T_G)$; Python may help.)

7.3.4 Theory Rearrange the equations $x_1 + 5x_2 = 6$, $7x_1 - x_2 = 6$ so that Jacobi converges, and explain why it diverges for the original ordering.

7.3.5 Python Modify Listing 7.1 to plot $\|\mathbf{x}^{(k)} - \mathbf{x}\|_\infty$ against k on a logarithmic scale for both methods, and measure the slopes. Compare with $\log_{10}\rho(T)$.

7.3.6 ★ Graduate Prove Theorem 7.5 for Gauss–Seidel. (Let λ be an eigenvalue of T_G with eigenvector $\mathbf{v}$, $\|\mathbf{v}\|_\infty = |v_i| = 1$; from $U\mathbf{v} = \lambda(D - L)\mathbf{v}$ derive $|\lambda| < 1$.)

7.4 Successive over-relaxation (SOR)

Gauss–Seidel moves each component from $x_i^{(k)}$ to the value $\tilde{x}_i$ given by (7.3). If the iterates approach the solution monotonically, it pays to move *further* in the same direction. With a *relaxation parameter* ω,

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\Big). \tag{7.4}$$

In matrix form $(D - \omega L)\mathbf{x}^{(k+1)} = [(1-\omega)D + \omega U]\mathbf{x}^{(k)} + \omega\mathbf{b}$, so

$$T_\omega = (D - \omega L)^{-1}\big[(1-\omega)D + \omega U\big], \qquad \mathbf{c}_\omega = \omega(D - \omega L)^{-1}\mathbf{b}.$$

$\omega = 1$ is Gauss–Seidel; $0 < \omega < 1$ is *under-relaxation*; $\omega > 1$ is *over-relaxation*.

Theorem 7.7 (Kahan; Ostrowski–Reich). *1. For every A with nonzero diagonal, $\rho(T_\omega) \geq |\omega - 1|$. Hence SOR can converge only if $0 < \omega < 2$.*

2. If A is symmetric positive definite, SOR converges for every $0 < \omega < 2$ and every $\mathbf{x}^{(0)}$.

3. (Young) If A is SPD and tridiagonal (more generally, consistently ordered), then $\rho(T_G) = \rho(T_J)^2$ and the optimal parameter is

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}}, \qquad \rho(T_{\omega_{\text{opt}}}) = \omega_{\text{opt}} - 1.$$

Example 7.8 (Example 7.2 with SOR). The matrix is SPD and tridiagonal with $\rho(T_J) = \sqrt{0.625} = 0.7906$, so $\omega_{\text{opt}} = 2/(1 + \sqrt{0.375}) \approx 1.24$ and $\rho(T_{\omega_{\text{opt}}}) \approx 0.24$, compared with 0.625 for Gauss–Seidel. With $\omega = 1.25$ the iteration is

$$\begin{aligned} x_1^{(k+1)} &= -0.25x_1^{(k)} - 0.9375x_2^{(k)} + 7.5, \\ x_2^{(k+1)} &= -0.9375x_1^{(k+1)} - 0.25x_2^{(k)} + 0.3125x_3^{(k)} + 9.375, \\ x_3^{(k+1)} &= 0.3125x_2^{(k+1)} - 0.25x_3^{(k)} - 7.5. \end{aligned}$$

Listing 7.2: SOR for several values of ω ch07/sor.py

```python
# Successive over-relaxation (SOR) and the choice of the relaxation parameter omega
import numpy as np
import matplotlib.pyplot as plt


def sor(A, b, omega, x0, tol=1e-10, max_iter=1000):
    """SOR: x_i <- (1 - omega) x_i + omega * (Gauss-Seidel value of x_i)."""
    n = len(b)
    x = np.array(x0, dtype=float)
    for k in range(1, max_iter + 1):
        x_old = x.copy()
        for i in range(n):
            gs = (b[i] - A[i, :i] @ x[:i] - A[i, i + 1:] @ x_old[i + 1:]) / A[i, i]
            x[i] = (1 - omega) * x_old[i] + omega * gs
        if np.linalg.norm(x - x_old, np.inf) < tol * np.linalg.norm(x, np.inf):
            return x, k
    return x, max_iter


A = np.array([[4, 3, 0], [3, 4, -1], [0, -1, 4]], dtype=float)
b = np.array([24, 30, -24], dtype=float)

D = np.diag(np.diag(A))
L = -np.tril(A, -1)
U = -np.triu(A, 1)


def rho_sor(omega):
    T = np.linalg.solve(D - omega * L, (1 - omega) * D + omega * U)
    return np.max(np.abs(np.linalg.eigvals(T)))


for omega in [0.8, 1.0, 1.1, 1.2, 1.24, 1.25, 1.3, 1.5, 1.8, 1.95]:
    x, k = sor(A, b, omega, np.ones(3))
    print(f"omega = {omega:4.2f}: {k:3d} iterations, rho(T_omega) = {rho_sor(omega):.4f}"
    ↪ )

rhoJ = np.max(np.abs(np.linalg.eigvals(np.linalg.solve(D, L + U))))
w_opt = 2 / (1 + np.sqrt(1 - rhoJ**2))
print(f"optimal omega = 2/(1+sqrt(1-rho_J^2)) = {w_opt:.6f}, rho = {rho_sor(w_opt):.6f}")

w = np.linspace(0.01, 1.99, 400)
plt.figure(figsize=(5.5, 3.3))
plt.plot(w, [rho_sor(t) for t in w])
plt.axvline(w_opt, ls="--", color="gray")
plt.xlabel(r"$\omega$")
plt.ylabel(r"$\rho(T_\omega)$")
plt.title("Spectral radius of the SOR iteration matrix")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch07_sor_rho.pdf")
```

Output

```
omega = 0.80:  63 iterations, rho(T_omega) = 0.7464
omega = 1.00:  43 iterations, rho(T_omega) = 0.6250
omega = 1.10:  34 iterations, rho(T_omega) = 0.5377
omega = 1.20:  23 iterations, rho(T_omega) = 0.4000
omega = 1.24:  18 iterations, rho(T_omega) = 0.2560
omega = 1.25:  19 iterations, rho(T_omega) = 0.2500
omega = 1.30:  21 iterations, rho(T_omega) = 0.3000
omega = 1.50:  36 iterations, rho(T_omega) = 0.5000
omega = 1.80: 109 iterations, rho(T_omega) = 0.8000
omega = 1.95: 469 iterations, rho(T_omega) = 0.9500
optimal omega = 2/(1+sqrt(1-rho_J^2)) = 1.240408, rho = 0.240408
```

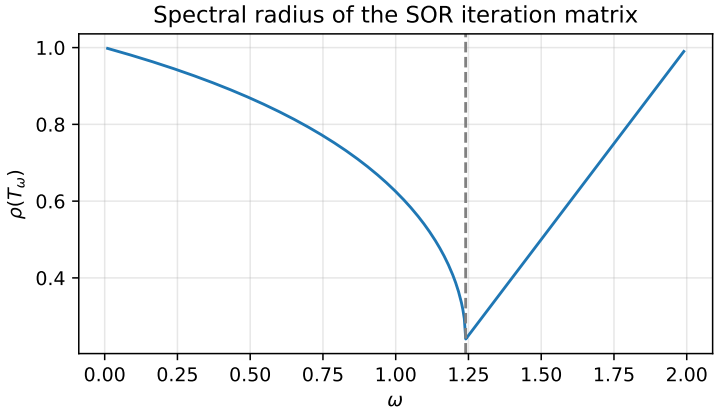


Figure 7.1: $\rho(T_\omega)$ for Example 7.2. The minimum is sharp, at $\omega_{\text{opt}} \approx 1.24$; to the right of it $\rho(T_\omega) = \omega - 1$.

Historical Note: David Young and the computer age

SOR was developed in 1950 by **David M. Young Jr.** in his Harvard PhD thesis, and independently by **Stanley Frankel**. Young's thesis gave the complete theory of Theorem 7.7(3) for the matrices arising from elliptic PDEs. On the computers of the 1950s and 60s, SOR made it possible for the first time to solve problems with thousands of unknowns, for example in the design of nuclear reactors, and it remained the method of choice until the 1970s.

Exercises

7.4.1 Theory Perform two SOR iterations with $\omega = 1.25$ for Example 7.2 from $\mathbf{x}^{(0)} = (1,1,1)^T$.

7.4.2 Theory Prove part (1) of Theorem 7.7. (Hint: $\det T_\omega = (1-\omega)^n$ is the product of the eigenvalues.)

7.4.3 Python For the tridiagonal matrix $\operatorname{tridiag}(-1,2,-1)$ of order $n = 50$, $\rho(T_J) = \cos(\pi/(n+1))$. Compute ω_{opt} and compare the number of iterations of Gauss–Seidel and optimal SOR for $\mathbf{b} = (1,\dots,1)^T$.

7.5 Gradient methods: steepest descent and conjugate gradients

When A is symmetric positive definite, solving $A\mathbf{x} = \mathbf{b}$ is equivalent to a minimisation problem.

Theorem 7.9. *Let A be SPD and $\phi(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$. Then $\nabla\phi(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = -\mathbf{r}$, and $\mathbf{x}^*$ solves $A\mathbf{x} = \mathbf{b}$ if and only if $\mathbf{x}^*$ minimises ϕ. Moreover $\phi(\mathbf{x}) - \phi(\mathbf{x}^*) = \tfrac{1}{2}(\mathbf{x} - \mathbf{x}^*)^T A(\mathbf{x} - \mathbf{x}^*) = \tfrac{1}{2}\|\mathbf{x} - \mathbf{x}^*\|_A^2$.*

The contours of ϕ are ellipsoids centred at the solution. A **line search** method chooses a search direction $\mathbf{v}^{(k)}$ and moves to the minimum of ϕ along it:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + t_k \mathbf{v}^{(k)}, \qquad t_k = \frac{\langle \mathbf{v}^{(k)}, \mathbf{r}^{(k)} \rangle}{\langle \mathbf{v}^{(k)}, A\mathbf{v}^{(k)} \rangle}, \qquad \mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}.$$

(Set $\frac{d}{dt}\phi(\mathbf{x} + t\mathbf{v}) = 0$ to obtain t_k.)

Steepest descent (gradient method). Choose $\mathbf{v}^{(k)} = \mathbf{r}^{(k)} = -\nabla\phi(\mathbf{x}^{(k)})$, the direction in which ϕ decreases fastest. It converges for every SPD matrix, but

$$\left\|\mathbf{x}^{(k)} - \mathbf{x}^*\right\|_A \le \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \left\|\mathbf{x}^{(0)} - \mathbf{x}^*\right\|_A, \qquad \kappa = \kappa_2(A),$$

which is very slow when κ is large: the iterates zig-zag across a long narrow valley.

Conjugate gradients (CG). Choose search directions that are *A-conjugate*: $\langle \mathbf{v}^{(i)}, A\mathbf{v}^{(j)} \rangle = 0$ for $i \ne j$. Then minimising along each direction never spoils the minimisation along previous ones, and the exact solution is reached in at most n steps. Remarkably, the new direction can be computed from the current residual and the previous direction alone:

Algorithm 4 Conjugate gradient method (Hestenes–Stiefel, 1952)

Require: SPD matrix A, $\mathbf{b}$, $\mathbf{x}^{(0)}$, tolerance ε

$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}$, $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$
for $k = 0, 1, 2, \dots$ **do**
  $\alpha_k = \dfrac{\mathbf{r}^{(k)T}\mathbf{r}^{(k)}}{\mathbf{p}^{(k)T} A\mathbf{p}^{(k)}}$ ▷ step length
  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}$, $\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)}$
  if $\left\|\mathbf{r}^{(k+1)}\right\|_2 < \varepsilon \|\mathbf{b}\|_2$ **then**
    return $\mathbf{x}^{(k+1)}$
  end if
  $\beta_k = \dfrac{\mathbf{r}^{(k+1)T}\mathbf{r}^{(k+1)}}{\mathbf{r}^{(k)T}\mathbf{r}^{(k)}}$, $\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}$ ▷ new conjugate direction
end for

Theorem 7.10 (properties of CG). *In exact arithmetic the residuals are mutually orthogonal, the directions are A-conjugate, CG terminates with the exact solution in at most n iterations, and*

$$\left\|\mathbf{x}^{(k)} - \mathbf{x}^*\right\|_A \le 2\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^k \left\|\mathbf{x}^{(0)} - \mathbf{x}^*\right\|_A.$$

Listing 7.3: Steepest descent and conjugate gradients ch07/gradient_cg.py

```
# Steepest descent and conjugate gradients for a symmetric positive definite system
import numpy as np
import matplotlib.pyplot as plt


def steepest_descent(A, b, x0, tol=1e-10, max_iter=10000):
```

```
    x = np.array(x0, dtype=float)
    path = [x.copy()]
    for k in range(1, max_iter + 1):
        r = b - A @ x                    # residual = -gradient of phi
        t = (r @ r) / (r @ (A @ r))      # exact line search
        x = x + t * r
        path.append(x.copy())
        if np.linalg.norm(r) < tol * np.linalg.norm(b):
            break
    return x, k, np.array(path)


def conjugate_gradient(A, b, x0, tol=1e-10, max_iter=None):
    """Hestenes-Stiefel conjugate gradient method."""
    x = np.array(x0, dtype=float)
    r = b - A @ x
    p = r.copy()
    rr = r @ r
    path = [x.copy()]
    max_iter = max_iter or 10 * len(b)
    for k in range(1, max_iter + 1):
        Ap = A @ p
        alpha = rr / (p @ Ap)            # step length
        x = x + alpha * p
        r = r - alpha * Ap
        path.append(x.copy())
        rr_new = r @ r
        if np.sqrt(rr_new) < tol * np.linalg.norm(b):
            break
        p = r + (rr_new / rr) * p        # new direction, A-conjugate to the old ones
        rr = rr_new
    return x, k, np.array(path)


A = np.array([[4.0, 1.0], [1.0, 3.0]])
b = np.array([1.0, 2.0])
x0 = np.array([2.0, 1.0])
xs, ks, ps = steepest_descent(A, b, x0)
xc, kc, pc = conjugate_gradient(A, b, x0)
print("exact solution    :", np.linalg.solve(A, b))
print(f"steepest descent  : {xs}  ({ks} iterations)")
print(f"conjugate gradient: {xc}  ({kc} iterations)")
lam = np.linalg.eigvalsh(A)
kappa = lam[-1] / lam[0]
print(f"kappa = {kappa:.4f},  SD rate (kappa-1)/(kappa+1) = {(kappa - 1) / (kappa + 1):.4f}
    ↪ ")

# picture: contours of phi(x) = x^T A x / 2 - b^T x and the two paths
X, Y = np.meshgrid(np.linspace(-0.6, 2.2, 200), np.linspace(-0.2, 1.4, 200))
P = 0.5 * (A[0, 0] * X**2 + 2 * A[0, 1] * X * Y + A[1, 1] * Y**2) - b[0] * X - b[1] * Y
plt.figure(figsize=(6, 3.6))
plt.contour(X, Y, P, levels=20, colors="lightgray")
plt.plot(ps[:12, 0], ps[:12, 1], "o-", ms=3, label="steepest descent")
plt.plot(pc[:, 0], pc[:, 1], "s-", ms=4, label="conjugate gradient")
plt.gca().set_aspect("equal")
plt.legend()
plt.title(r"Minimising $\phi(x)=\frac{1}{2} x^TAx-b^Tx$")
```

```
plt.tight_layout()
plt.savefig("ch07_sd_cg.pdf")
```

Output

```
exact solution    : [0.09090909 0.63636364]
steepest descent  : [0.09090909 0.63636364]  (13 iterations)
conjugate gradient: [0.09090909 0.63636364]  (2 iterations)
kappa = 1.9387,  SD rate (kappa-1)/(kappa+1) = 0.3194
```

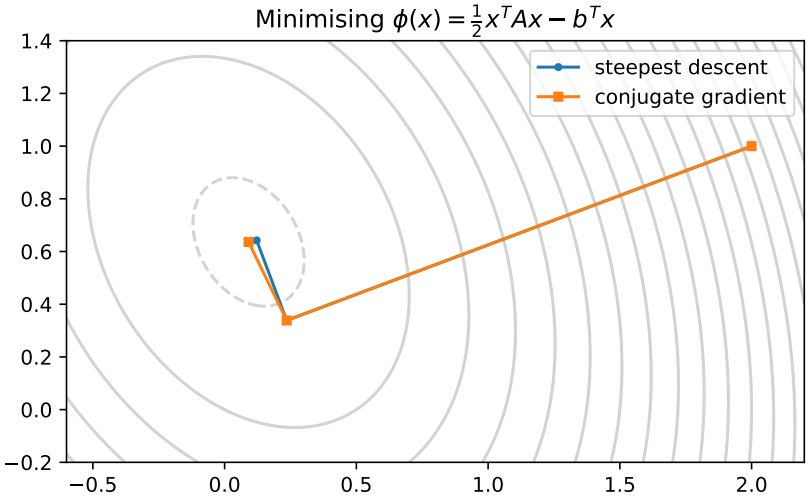


Figure 7.2: Steepest descent zig-zags towards the minimum; conjugate gradients reaches it in $n = 2$ steps.

Historical Note: Cauchy, Hestenes and Stiefel

The method of steepest descent was proposed by **Augustin-Louis Cauchy** in 1847 for solving systems of equations arising in astronomy. The conjugate gradient method was invented in 1952 by **Magnus Hestenes** (UCLA) and **Eduard Stiefel** (ETH Zürich), who discovered it independently and wrote a joint paper after meeting at the Institute for Numerical Analysis in Los Angeles. They viewed it as a *direct* method (exact after n steps), and it was largely forgotten because rounding errors destroy this property. It was revived in the 1970s by John Reid and others, who realised that as an *iterative* method it often gives excellent accuracy after far fewer than n steps. CG is included in lists of the top ten algorithms of the 20th century.

7.6 A large sparse example: the Poisson equation

The temperature $u(x, y)$ in a square plate heated uniformly satisfies the Poisson equation $-\left(u_{xx} + u_{yy}\right) = f$ with $u = 0$ on the boundary. On a grid with spacing $h = 1/(m+1)$ the five-point finite-difference approximation

$$4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = h^2 f_{i,j}$$

gives a linear system with $n = m^2$ unknowns, at most 5 nonzeros per row, SPD, with $\kappa_2(A) \approx \frac{4}{\pi^2 h^2}$. For this matrix $\rho(T_J) = \cos(\pi h)$ and $\omega_{\text{opt}} = 2/(1 + \sin \pi h)$.

Listing 7.4: Iterative methods for the 2D Poisson equation ($n = 400$) ch07/poisson_compare.py

```
# Comparing iterative methods on the 2D Poisson equation  -Laplace(u) = f  on the unit square
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt

m = 20                                  # interior grid points per direction
n = m * m                               # number of unknowns
h = 1.0 / (m + 1)
T = sp.diags([-1, 4, -1], [-1, 0, 1], shape=(m, m))
S = sp.diags([-1, -1], [-1, 1], shape=(m, m))
A = (sp.kron(sp.identity(m), T) + sp.kron(S, sp.identity(m))).tocsr()   # 5-point Laplacian * h^2
b = h**2 * np.ones(n)                   # f = 1
Ad = A.toarray()
tol, maxit = 1e-8, 5000
bnorm = np.linalg.norm(b)
print(f"n = {n} unknowns, nonzeros in A: {A.nnz} of {n * n}")


def run(update):
    x = np.zeros(n)
    res = []
    for k in range(maxit):
        x = update(x)
        res.append(np.linalg.norm(b - A @ x) / bnorm)
        if res[-1] < tol:
            break
    return res


d = Ad.diagonal()
jac = lambda x: x + (b - A @ x) / d


def gs_sweep(x, omega=1.0):
    x = x.copy()
    for i in range(n):
        row = A.indices[A.indptr[i]:A.indptr[i + 1]]
        val = A.data[A.indptr[i]:A.indptr[i + 1]]
        s = val @ x[row] - d[i] * x[i]
        x[i] = (1 - omega) * x[i] + omega * (b[i] - s) / d[i]
    return x


rhoJ = np.cos(np.pi * h)                # known spectral radius of Jacobi here
w_opt = 2 / (1 + np.sin(np.pi * h))
histories = {"Jacobi": run(jac),
             "Gauss-Seidel": run(gs_sweep),
             f"SOR (omega={w_opt:.3f})": run(lambda x: gs_sweep(x, w_opt))}

cg_res = []
record = lambda xk: cg_res.append(np.linalg.norm(b - A @ xk) / bnorm)
try:                                    # SciPy >= 1.12 calls the tolerance 'rtol'
```

```
    spla.cg(A, b, rtol=tol, maxiter=maxit, callback=record)
except TypeError:                       # older SciPy versions call it 'tol'
    spla.cg(A, b, tol=tol, maxiter=maxit, callback=record)
histories["Conjugate gradient"] = cg_res

print(f"rho(T_J) = cos(pi h) = {rhoJ:.5f},  rho(T_GS) = {rhoJ**2:.5f}")
for name, res in histories.items():
    print(f"{name:24s}: {len(res):5d} iterations")

plt.figure(figsize=(6.2, 3.8))
for name, res in histories.items():
    plt.semilogy(res, label=name)
plt.xlabel("iteration")
plt.ylabel("relative residual")
plt.xlim(0, 1700)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch07_poisson.pdf")
```

Output

```
n = 400 unknowns, nonzeros in A: 1920 of 160000
rho(T_J) = cos(pi h) = 0.98883,  rho(T_GS) = 0.97779
Jacobi                   :  1626 iterations
Gauss-Seidel             :   814 iterations
SOR (omega=1.741)        :    79 iterations
Conjugate gradient       :    36 iterations
```

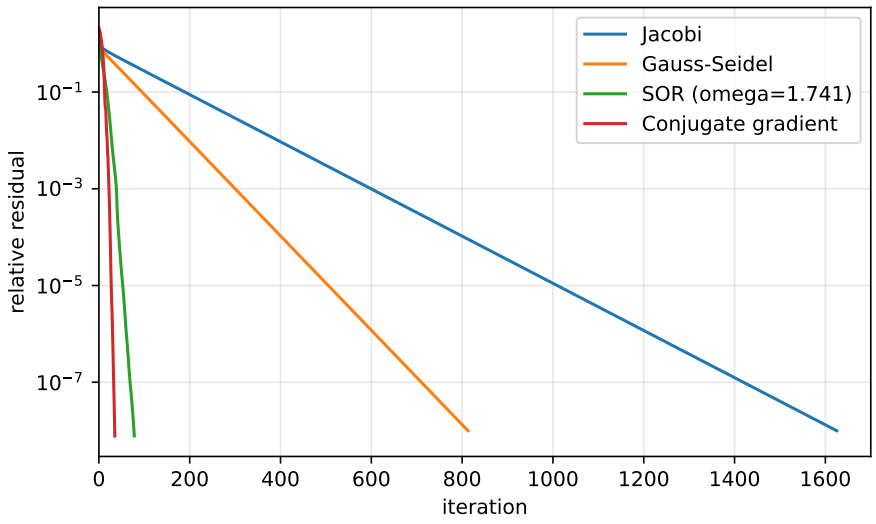


Figure 7.3: Convergence histories for the Poisson problem with $m = 20$.

The iteration counts behave like $O(h^{-2})$ for Jacobi and Gauss–Seidel, and like $O(h^{-1})$ for optimal SOR and CG. Halving h (four times as many unknowns) multiplies the Jacobi count by 4 but the CG count only by 2. Note that the sparse matrix has only 1920 nonzero entries out of 160 000.

Towards Graduate Studies: Krylov subspace methods and multigrid

CG is the first of the *Krylov subspace methods*: $\mathbf{x}^{(k)}$ is the best approximation from $\mathcal{K}_k = \operatorname{span}\{\mathbf{r}^{(0)}, A\mathbf{r}^{(0)}, \dots, A^{k-1}\mathbf{r}^{(0)}\}$ in the A-norm. For nonsymmetric matrices one uses GMRES (Saad & Schultz, 1986) or BiCGStab (van der Vorst, 1992). Their speed depends on $\kappa(A)$, so in practice one applies them to a *preconditioned* system $P^{-1}A\mathbf{x} = P^{-1}\mathbf{b}$ with $P \approx A$ cheap to invert (incomplete Cholesky, domain decomposition, …). *Multigrid* methods (Fedorenko 1964, Brandt 1977) use Jacobi or Gauss–Seidel only to smooth the error on a hierarchy of grids, and solve Poisson problems in $O(n)$ operations, which is optimal. These methods are the engines of computational fluid dynamics, weather prediction and structural engineering. See Saad, *Iterative Methods for Sparse Linear Systems* (free online), and Greenbaum [9].

Exercises

7.6.1 Theory Verify Theorem 7.9: compute $\nabla\phi$ and show $\phi(\mathbf{x}) - \phi(\mathbf{x}^*) = \tfrac{1}{2}\|\mathbf{x} - \mathbf{x}^*\|_A^2$.

7.6.2 Theory Derive the optimal step length t_k of a line search.

7.6.3 Theory Perform two CG iterations by hand for $A = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (1,2)^T$, $\mathbf{x}^{(0)} = \mathbf{0}$, and verify that $\mathbf{x}^{(2)}$ is the exact solution $(1/11, 7/11)^T$.

7.6.4 Python Solve the Poisson problem of Listing 7.4 for $m = 10, 20, 40$ and tabulate the iteration counts of all four methods. Confirm the $O(h^{-2})$ and $O(h^{-1})$ growth.

7.6.5 Python Solve a $10^6 \times 10^6$ tridiagonal SPD system with `scipy.sparse.linalg.cg`. How long does it take? Why would `np.linalg.solve` be impossible?

7.6.6 ★ Graduate Prove that in CG, $\mathbf{r}^{(k+1)} \perp \mathbf{r}^{(j)}$ and $\mathbf{p}^{(k+1)T} A \mathbf{p}^{(j)} = 0$ for all $j \le k$ (induction on k).

7.6.7 ★ Graduate **Project.** Implement *preconditioned* CG with the Jacobi preconditioner $P = D$ and test it on an SPD matrix with widely varying diagonal entries. How much does preconditioning help?

Chapter Summary

- Splitting $A = M - N$ gives $\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}$; convergence $\iff \rho(T) < 1$.
- Jacobi $T_J = D^{-1}(L+U)$; Gauss–Seidel $T_G = (D-L)^{-1}U$; SOR with $0 < \omega < 2$.
- Strict diagonal dominance $\Rightarrow$ Jacobi and GS converge; SPD $\Rightarrow$ GS and SOR converge.
- For SPD A, solving $A\mathbf{x} = \mathbf{b}$ = minimising $\tfrac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$; steepest descent rate $\frac{\kappa-1}{\kappa+1}$, CG rate $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$.
- For large sparse SPD systems, (preconditioned) CG is the method of choice.

Chapter 8

Norms, Condition Numbers and Error Estimates

Learning Objectives

After studying this chapter you should be able to:

- define vector norms and compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_\infty$;
- define induced matrix norms and compute $\|A\|_1$, $\|A\|_\infty$, $\|A\|_2$ and the spectral radius $\rho(A)$;
- define the condition number $\kappa(A)$ and use it to bound the error of a computed solution;
- explain why a small residual does not guarantee a small error;
- formulate stopping criteria for iterative methods in terms of norms.

To say that an approximate solution $\hat{\mathbf{x}} \in \mathbb{R}^n$ is "close" to the exact solution $\mathbf{x}$, or that an iteration $\mathbf{x}^{(k)}$ "converges", we need to measure the size of vectors and of matrices. This chapter introduces the tools, *norms*, and uses them to answer a basic question: *how accurate is the computed solution of* $A\mathbf{x} = \mathbf{b}$*?* These ideas are critical throughout numerical analysis, optimisation and machine learning. In Chapter 7 we used the maximum norm and the spectral radius with only a brief introduction; this chapter develops them properly, proves the results that were quoted there (in particular Theorem 8.7), and adds the condition number, which explains how accurately *any* method, direct or iterative, can solve a linear system.

8.1 Vector norms

Definition 8.1 (vector norm)**.** A *norm* on $\mathbb{R}^n$ is a function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ such that for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$:

1. $\|\mathbf{x}\| \geq 0$, and $\|\mathbf{x}\| = 0$ if and only if $\mathbf{x} = \mathbf{0}$; (positivity)
2. $\|\alpha\mathbf{x}\| = |\alpha| \, \|\mathbf{x}\|$; (homogeneity)
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$. (triangle inequality)

The *distance* between $\mathbf{x}$ and $\mathbf{y}$ is $\|\mathbf{x} - \mathbf{y}\|$.

Definition 8.2 (ℓ_p norms)**.** For $\mathbf{x} = (x_1, \dots, x_n)^T$,

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|, \qquad \|\mathbf{x}\|_2 = \Big(\sum_{i=1}^{n} x_i^2\Big)^{1/2}, \qquad \|\mathbf{x}\|_\infty = \max_{1\le i\le n} |x_i|,$$

and more generally $\|\mathbf{x}\|_p = \big(\sum |x_i|^p\big)^{1/p}$ for $1 \le p < \infty$. $\|\cdot\|_2$ is the Euclidean length; $\|\cdot\|_\infty$ is the *maximum* (or *Chebyshev*) norm.

Example 8.3. For $\mathbf{x} = (-1, 1, 2)^T$: $\|\mathbf{x}\|_1 = 4$, $\|\mathbf{x}\|_2 = \sqrt{6} \approx 2.4495$, $\|\mathbf{x}\|_\infty = 2$. If $\hat{\mathbf{x}} = (1.2001, 0.9991, 0.9250)^T$ approximates $\mathbf{x} = (1, 1, 1)^T$, then $\|\mathbf{x} - \hat{\mathbf{x}}\|_\infty = 0.2001$ and $\|\mathbf{x} - \hat{\mathbf{x}}\|_2 = \sqrt{0.2001^2 + 0.0009^2 + 0.075^2} \approx 0.2137$.

The triangle inequality for $\|\cdot\|_2$ follows from the **Cauchy–Schwarz inequality** $|\mathbf{x}^T\mathbf{y}| \le \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2$.

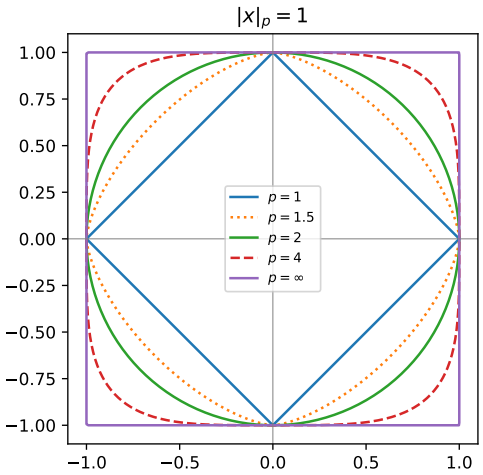


Figure 8.1: Unit circles $\{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\|_p = 1\}$. As p increases they grow from the diamond ($p = 1$) to the square ($p = \infty$).

Theorem 8.4 (equivalence of norms)**.** *All norms on $\mathbb{R}^n$ are equivalent: for any two norms there are constants $c, C > 0$ with $c\,\|\mathbf{x}\|_a \le \|\mathbf{x}\|_b \le C\,\|\mathbf{x}\|_a$ for all $\mathbf{x}$. In particular*

$$\|\mathbf{x}\|_\infty \le \|\mathbf{x}\|_2 \le \sqrt{n}\,\|\mathbf{x}\|_\infty, \qquad \|\mathbf{x}\|_2 \le \|\mathbf{x}\|_1 \le \sqrt{n}\,\|\mathbf{x}\|_2.$$

Consequently a sequence of vectors $\mathbf{x}^{(k)}$ converges to $\mathbf{x}$ in one norm if and only if it converges in every norm, if and only if each component converges: $\lim_k x_i^{(k)} = x_i$ for $i = 1, \dots, n$.

△ Caution: norms in infinite dimensions

The equivalence of norms fails in infinite-dimensional spaces such as $C[a, b]$. The functions $f_k(x) = x^k$ on $[0, 1]$ have $\|f_k\|_2 = (2k+1)^{-1/2} \to 0$ but $\|f_k\|_\infty = 1$ for all k. Which norm we use to measure the error of an approximation of a function really matters (Chapter 10).

Listing 8.1: Vector norms, matrix norms and the spectral radius ch08/norms.py

```
# Vector and matrix norms, and the spectral radius
import numpy as np

```

```
x = np.array([-1.0, 1.0, 2.0])
print("x =", x)
print("||x||_1   =", np.linalg.norm(x, 1))
print("||x||_2   =", np.linalg.norm(x), "= sqrt(6) =", np.sqrt(6))
print("||x||_inf =", np.linalg.norm(x, np.inf))

xhat = np.array([1.2001, 0.9991, 0.9250])
xtrue = np.ones(3)
print("\n||x - xhat||_2   =", np.linalg.norm(xtrue - xhat))
print("||x - xhat||_inf =", np.linalg.norm(xtrue - xhat, np.inf))

A = np.array([[1, 2, -1], [0, 3, -1], [5, -1, 1]], dtype=float)
print("\nA =\n", A)
print("||A||_1   (max column sum) =", np.abs(A).sum(axis=0).max(), "  numpy:", np.linalg.
    ↪ norm(A, 1))
print("||A||_inf (max row sum)    =", np.abs(A).sum(axis=1).max(), "  numpy:", np.linalg.
    ↪ norm(A, np.inf))
rho_AtA = np.max(np.abs(np.linalg.eigvals(A.T @ A)))
print("||A||_2 = sqrt(rho(A^T A)) =", np.sqrt(rho_AtA), "  numpy:", np.linalg.norm(A, 2))
print("||A||_F (Frobenius)        =", np.linalg.norm(A, "fro"))
print("eigenvalues of A:", np.round(np.linalg.eigvals(A), 6))
print("spectral radius rho(A)     =", np.max(np.abs(np.linalg.eigvals(A))))

# rho(A) <= ||A|| for every induced norm, but ||A|| can be much larger than rho(A)
B = np.array([[0.5, 10.0], [0.0, 0.5]])
print("\nB = [[0.5, 10], [0, 0.5]]:  rho(B) =", 0.5, "  ||B||_inf =", np.linalg.norm(B, np.
    ↪ inf))
for k in [1, 5, 10, 20, 40, 80]:
    print(f"   ||B^{k:<2d}||_inf = {np.linalg.norm(np.linalg.matrix_power(B, k), np.inf):.4
    ↪ e}")
```

Output

```
x = [-1.  1.  2.]
||x||_1   = 4.0
||x||_2   = 2.449489742783178 = sqrt(6) = 2.449489742783178
||x||_inf = 2.0

||x - xhat||_2   = 0.21369562466274308
||x - xhat||_inf = 0.20009999999999994

A =
 [[ 1.  2. -1.]
 [ 0.  3. -1.]
 [ 5. -1.  1.]]
||A||_1   (max column sum) = 6.0   numpy: 6.0
||A||_inf (max row sum)    = 7.0   numpy: 7.0
||A||_2 = sqrt(rho(A^T A)) = 5.282385477874271   numpy: 5.2823854778742705
||A||_F (Frobenius)        = 6.557438524302
eigenvalues of A: [1.+0.j       2.+1.732051j 2.-1.732051j]
spectral radius rho(A)     = 2.645751311064591

B = [[0.5, 10], [0, 0.5]]:  rho(B) = 0.5   ||B||_inf = 10.5
   ||B^1 ||_inf = 1.0500e+01
   ||B^5 ||_inf = 3.1562e+00
   ||B^10||_inf = 1.9629e-01
```

```
||B^20||_inf = 3.8242e-04
||B^40||_inf = 7.2851e-10
||B^80||_inf = 1.3243e-21
```

Exercises

8.1.1 Theory Compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_\infty$ for $\mathbf{x} = (3, -4, 0, \tfrac{3}{2})^T$ and $\mathbf{x} = (\sin k)_{k=1}^{10}$ (use Python for the second).

8.1.2 Theory Verify that $\|\cdot\|_1$ and $\|\cdot\|_\infty$ satisfy the three norm axioms.

8.1.3 Theory Prove the Cauchy–Schwarz inequality by considering $\|\mathbf{x} - t\mathbf{y}\|_2^2 \geq 0$ as a quadratic in t, and deduce the triangle inequality for $\|\cdot\|_2$.

8.1.4 Theory Prove the inequalities $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{n}\,\|\mathbf{x}\|_\infty$ and show that they are sharp (find vectors giving equality).

8.1.5 ★ Graduate Show that $\lim_{p\to\infty} \|\mathbf{x}\|_p = \|\mathbf{x}\|_\infty$.

8.2 Matrix norms and the spectral radius

Definition 8.5 (induced matrix norm)**.** Given a vector norm, the *induced* (or *natural*, or *operator*) matrix norm is

$$\|A\| = \max_{\mathbf{x}\neq\mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|} = \max_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|\,.$$

It is the largest factor by which A can stretch a vector.

Directly from the definition, induced norms satisfy $\|A\mathbf{x}\| \leq \|A\|\,\|\mathbf{x}\|$, $\|AB\| \leq \|A\|\,\|B\|$ and $\|I\| = 1$.

Theorem 8.6 (formulas for induced norms)**.** *For $A \in \mathbb{R}^{n\times n}$,*

$$\|A\|_1 = \max_j \sum_{i=1}^{n} |a_{ij}| \ \ \textit{(max column sum)}, \qquad \|A\|_\infty = \max_i \sum_{j=1}^{n} |a_{ij}| \ \ \textit{(max row sum)},$$

$$\|A\|_2 = \sqrt{\rho(A^TA)} = \sigma_{\max}(A) \ \ \textit{(the largest singular value)},$$

where the spectral radius *of a matrix B is $\rho(B) = \max\{|\lambda| : \lambda \text{ an eigenvalue of } B\}$. If A is symmetric, $\|A\|_2 = \rho(A)$.*

Proof for $\|\cdot\|_\infty$. For $\|\mathbf{x}\|_\infty = 1$, $|(A\mathbf{x})_i| = |\sum_j a_{ij}x_j| \leq \sum_j |a_{ij}|$, so $\|A\|_\infty \leq \max_i \sum_j |a_{ij}|$. If row p attains the maximum, choose $x_j = \operatorname{sign}(a_{pj})$; then $(A\mathbf{x})_p = \sum_j |a_{pj}|$, so equality holds. □

The *Frobenius norm* $\|A\|_F = \left(\sum_{i,j} a_{ij}^2\right)^{1/2}$ is a matrix norm but is not induced by any vector norm ($\|I\|_F = \sqrt{n}$).

Theorem 8.7. *For every induced matrix norm, $\rho(A) \leq \|A\|$. Moreover, the following are equivalent:*

1. *$\rho(A) < 1$;*
2. *$\lim_{k\to\infty} A^k = 0$ (the matrix is* convergent*);*
3. *$\lim_{k\to\infty} A^k\mathbf{x} = \mathbf{0}$ for every $\mathbf{x}$.*

Proof of the first claim. If $A\mathbf{x} = \lambda\mathbf{x}$ with $\|\mathbf{x}\| = 1$ then $|\lambda| = \|\lambda\mathbf{x}\| = \|A\mathbf{x}\| \le \|A\|$. □

⋆ Key Concept: the spectral radius controls powers

Whether $A^k \to 0$ is decided by the spectral radius, not by any particular norm. The matrix $B = \begin{pmatrix} 0.5 & 10 \\ 0 & 0.5 \end{pmatrix}$ has $\|B\|_\infty = 10.5$ but $\rho(B) = 0.5$, and the output above shows $\left\|B^k\right\|_\infty$ first growing and then decaying to zero. This is the key to the convergence of the iterative methods of Chapter 7: an iteration $\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}$ converges for every starting vector if and only if $\rho(T) < 1$.

Example 8.8. For $A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 3 & -1 \\ 5 & -1 & 1 \end{pmatrix}$: column sums $6, 6, 3$ give $\|A\|_1 = 6$; row sums $4, 4, 7$ give $\|A\|_\infty = 7$. The eigenvalues are 1 and $2 \pm i\sqrt{3}$, so $\rho(A) = \sqrt{7} \approx 2.646$, and indeed $\rho(A) \le \|A\|_2 = 5.282$.

Exercises

8.2.1 Theory Compute $\|A\|_1$, $\|A\|_\infty$, $\|A\|_F$ and $\|A\|_2$ for $A = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ and $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$.

8.2.2 Theory Prove the formula for $\|A\|_1$ in Theorem 8.6.

8.2.3 Theory Prove that $\|A\|_2^2 = \rho(A^TA)$. (Hint: $\|A\mathbf{x}\|_2^2 = \mathbf{x}^TA^TA\mathbf{x}$ and A^TA is symmetric positive semi-definite, so it has an orthonormal basis of eigenvectors.)

8.2.4 Theory Show that $\|A\|_2 \le \|A\|_F \le \sqrt{n}\,\|A\|_2$.

8.2.5 ★ Graduate Prove that for every $\epsilon > 0$ there is an induced norm with $\|A\| \le \rho(A) + \epsilon$. Use this to prove Theorem 8.7. (Use the Jordan or Schur form.)

8.3 The condition number of a matrix

How sensitive is the solution of $A\mathbf{x} = \mathbf{b}$ to changes in $\mathbf{b}$ and A?

Definition 8.9 (condition number)**.** For a nonsingular matrix A and an induced norm,

$$\kappa(A) = \operatorname{cond}(A) = \|A\| \left\|A^{-1}\right\|.$$

Always $\kappa(A) \ge \left\|AA^{-1}\right\| = 1$. For the 2-norm, $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$; for symmetric A, $\kappa_2(A) = |\lambda|_{\max}/|\lambda|_{\min}$.

Theorem 8.10 (perturbation of the right-hand side)**.** *Let $A\mathbf{x} = \mathbf{b}$ with $\mathbf{b} \neq \mathbf{0}$ and $A(\mathbf{x}+\delta\mathbf{x}) = \mathbf{b}+\delta\mathbf{b}$. Then*

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \le \kappa(A)\,\frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}.$$

Proof. $A\,\delta\mathbf{x} = \delta\mathbf{b}$ gives $\|\delta\mathbf{x}\| \le \left\|A^{-1}\right\|\|\delta\mathbf{b}\|$, and $\mathbf{b} = A\mathbf{x}$ gives $\|\mathbf{b}\| \le \|A\|\,\|\mathbf{x}\|$, i.e. $1/\|\mathbf{x}\| \le \|A\|/\|\mathbf{b}\|$. Multiply the two inequalities. □

Theorem 8.11 (residual and error)**.** *Let $\hat{\mathbf{x}}$ be any approximation of the solution of $A\mathbf{x} = \mathbf{b}$, with*

residual $\mathbf{r} = \mathbf{b} - A\hat{\mathbf{x}}$. *Then*

$$\frac{\|\mathbf{r}\|}{\kappa(A)\,\|\mathbf{b}\|} \;\le\; \frac{\|\mathbf{x}-\hat{\mathbf{x}}\|}{\|\mathbf{x}\|} \;\le\; \kappa(A)\frac{\|\mathbf{r}\|}{\|\mathbf{b}\|}.$$

Proof. $\mathbf{x} - \hat{\mathbf{x}} = A^{-1}\mathbf{r}$; apply Theorem 8.10 with $\delta\mathbf{b} = \mathbf{r}$ for the upper bound. The lower bound follows from $\|\mathbf{r}\| \le \|A\|\,\|\mathbf{x}-\hat{\mathbf{x}}\|$ and $\|\mathbf{x}\| \le \left\|A^{-1}\right\|\|\mathbf{b}\|$. □

Theorem 8.12 (perturbation of the matrix)**.** *If* $(A+\delta A)(\mathbf{x}+\delta\mathbf{x}) = \mathbf{b}$ *and* $\left\|A^{-1}\right\|\|\delta A\| < 1$, *then*

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \le \frac{\kappa(A)}{1-\kappa(A)\frac{\|\delta A\|}{\|A\|}}\left(\frac{\|\delta A\|}{\|A\|}\right).$$

⋆ Key Concept: what the condition number tells you

- A **small residual does not imply a small error** when $\kappa(A)$ is large.
- **Rule of thumb:** when solving $A\mathbf{x} = \mathbf{b}$ in floating-point arithmetic with unit roundoff $u \approx 10^{-16}$, a backward stable method gives a relative error of about $\kappa(A)\,u$. One loses about $\log_{10}\kappa(A)$ of the 16 significant decimal digits.
- $\kappa(A)$ measures the distance to singularity: $\min\{\|\delta A\|/\|A\| : A+\delta A \text{ singular}\} = 1/\kappa(A)$ (in the 2-norm). The determinant is *not* a measure of near-singularity: $\det(10^{-1}I_{100}) = 10^{-100}$ but $\kappa = 1$.

Example 8.13 (small residual, large error)**.** The system $\begin{pmatrix} 1 & 2 \\ 1.0001 & 2 \end{pmatrix}\mathbf{x} = \begin{pmatrix} 3 \\ 3.0001 \end{pmatrix}$ has solution $\mathbf{x} = (1,1)^T$. The very poor approximation $\hat{\mathbf{x}} = (3, -0.0001)^T$ has residual $\mathbf{r} = (0.0002, 0)^T$, which looks tiny. But $A^{-1} = \begin{pmatrix} -10000 & 10000 \\ 5000.5 & -5000 \end{pmatrix}$, so $\kappa_\infty(A) = 3.0001 \times 20000 = 60002$, and Theorem 8.11 allows a relative error of up to $60002 \times 0.0002/3.0001 \approx 4$.

Listing 8.2: Condition numbers and lost digits ch08/condition.py

```
# Condition numbers: small residual does not mean small error
import numpy as np

# A nearly singular 2x2 system (Burden & Faires)
A = np.array([[1.0, 2.0], [1.0001, 2.0]])
b = np.array([3.0, 3.0001])
x = np.linalg.solve(A, b)
xhat = np.array([3.0, -0.0001])
r = b - A @ xhat
print("exact solution x      =", x)
print("approximation xhat    =", xhat)
print("residual b - A xhat   =", r, "  ||r||_inf =", np.linalg.norm(r, np.inf))
print("error ||x - xhat||_inf=", np.linalg.norm(x - xhat, np.inf))
print("cond_inf(A)           =", np.linalg.cond(A, np.inf))
bound = np.linalg.cond(A, np.inf) * np.linalg.norm(r, np.inf) / np.linalg.norm(b, np.inf)
print(f"bound  K ||r||/||b||  = {bound:.4f} >= relative error "
      f"{np.linalg.norm(x - xhat, np.inf) / np.linalg.norm(x, np.inf):.4f}")

# Hilbert matrices: digits lost ~ log10(cond)
print("\n  n     cond_2(H_n)     rel. error of solve   correct digits")
for n in [2, 4, 6, 8, 10, 12, 14]:
    H = 1.0 / (np.arange(1, n + 1)[:, None] + np.arange(n)[None, :])
    xt = np.ones(n)
    xs = np.linalg.solve(H, H @ xt)
    rel = np.linalg.norm(xs - xt, np.inf) / np.linalg.norm(xt, np.inf)
    print(f"{n:3d}   {np.linalg.cond(H):12.4e}     {rel:12.4e}        {max(0, -np.log10(
    ↪ rel)):5.1f}")
```

Output

```
exact solution x      = [1. 1.]
approximation xhat    = [ 3.e+00 -1.e-04]
residual b - A xhat   = [2.0000000e-04 4.4408921e-16]   ||r||_inf = 0.00019999999999997797
error ||x - xhat||_inf= 1.9999999999980558
cond_inf(A)           = 60001.99999999003
bound  K ||r||/||b||  = 4.0000 >= relative error 2.0000

  n     cond_2(H_n)     rel. error of solve   correct digits
  2     1.9281e+01       6.6613e-16         15.2
  4     1.5514e+04       5.3735e-14         13.3
  6     1.4951e+07       6.6495e-11         10.2
  8     1.5258e+10       1.3089e-07          6.9
 10     1.6025e+13       5.2062e-05          4.3
 12     1.6426e+16       3.8249e-01          0.4
 14     3.2138e+17       2.2762e+01          0.0
```

The Hilbert matrices $H_n = \big(1/(i+j-1)\big)$ are notoriously ill-conditioned. The number of correct digits in the computed solution is almost exactly $16 - \log_{10}\kappa(H_n)$, as the rule of thumb predicts; for $n \geq 12$ no digit is correct.

8.3.1 Iterative refinement

If $\hat{\mathbf{x}}$ is a computed solution, compute the residual $\mathbf{r} = \mathbf{b} - A\hat{\mathbf{x}}$ (in higher precision), solve $A\mathbf{d} = \mathbf{r}$ with the *same* LU factors, and set $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + \mathbf{d}$. Repeat. The correction costs only $O(n^2)$ per step.

Refinement converges when $\kappa(A)u < 1$ (for the working precision u used in the factorisation), which the following experiment demonstrates:

Listing 8.3: Mixed-precision iterative refinement ch08/refinement.py

```python
# Iterative refinement: improve a computed solution using the residual
import numpy as np
import scipy.linalg as sla

n = 10
H = 1.0 / (np.arange(1, n + 1)[:, None] + np.arange(n)[None, :])
x_true = np.ones(n)
b = H @ x_true

# Solve in SINGLE precision (float32) to make the effect visible ...
H32, b32 = H.astype(np.float32), b.astype(np.float32)
lu = sla.lu_factor(H32)
x = sla.lu_solve(lu, b32).astype(np.float64)
print(f"step 0: error = {np.linalg.norm(x - x_true, np.inf):.3e}")
# ... and refine with residuals computed in DOUBLE precision
for k in range(1, 6):
    r = b - H @ x                       # residual in double precision
    d = sla.lu_solve(lu, r.astype(np.float32)).astype(np.float64)
    x = x + d
    print(f"step {k}: error = {np.linalg.norm(x - x_true, np.inf):.3e}")
print(f"cond(H_10) = {np.linalg.cond(H):.2e},  1/u_single = {1 / np.finfo(np.float32).eps
    ↪ :.2e}")

n = 6
H = 1.0 / (np.arange(1, n + 1)[:, None] + np.arange(n)[None, :])
x_true = np.ones(n)
b = H @ x_true
H32, b32 = H.astype(np.float32), b.astype(np.float32)
lu = sla.lu_factor(H32)
x = sla.lu_solve(lu, b32).astype(np.float64)
print(f"\nn = 6, cond = {np.linalg.cond(H):.2e}")
print(f"step 0: error = {np.linalg.norm(x - x_true, np.inf):.3e}")
for k in range(1, 6):
    r = b - H @ x
    x = x + sla.lu_solve(lu, r.astype(np.float32)).astype(np.float64)
    print(f"step {k}: error = {np.linalg.norm(x - x_true, np.inf):.3e}")
```

Output

```
step 0: error = 5.839e+00
step 1: error = 7.309e+00
step 2: error = 8.802e+00
step 3: error = 1.037e+01
step 4: error = 1.202e+01
step 5: error = 1.376e+01
cond(H_10) = 1.60e+13,  1/u_single = 8.39e+06

n = 6, cond = 1.50e+07
step 0: error = 1.631e-01
step 1: error = 8.592e-03
step 2: error = 4.657e-04
```

```
step 3: error = 2.524e-05
step 4: error = 1.367e-06
step 5: error = 7.384e-08
```

Historical Note: von Neumann, Turing and ill-conditioning

In 1947 **John von Neumann** and **Herman Goldstine** published a 79-page analysis of the rounding errors of Gaussian elimination for inverting matrices, generally regarded as the birth of modern numerical analysis. They were pessimistic about solving large systems. A year later **Alan Turing**, working on the ACE computer at the National Physical Laboratory, published "Rounding-off errors in matrix processes", in which he introduced the terms *condition number* and *ill-conditioned* and the LU factorisation, and gave a more optimistic analysis. The Hilbert matrix was introduced by **David Hilbert** in 1894 in a paper on approximation by polynomials, the very least-squares problem that leads to it (Chapter 10).

Towards Graduate Studies: condition numbers everywhere

The idea of a condition number applies to every problem: roots of polynomials, eigenvalues (Bauer–Fike theorem), least-squares problems (where $\kappa(A)^2$ appears for the normal equations), interpolation (the Lebesgue constant, Chapter 9), and ODE and PDE solvers. For large linear systems the condition number also controls the *speed* of iterative methods (the conjugate gradient method needs about $\sqrt{\kappa}$ iterations), which is why *preconditioning*, replacing $A\mathbf{x} = \mathbf{b}$ by an equivalent system with smaller κ, is a central research topic. Mixed-precision iterative refinement, in which the factorisation is done in fast low precision (16 or 32 bits) on GPUs, is currently a very active field (Carson & Higham, 2018).

Exercises

8.3.1 Theory Compute $\kappa_\infty(A)$ for $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $A = \begin{pmatrix} 1 & 1 \\ 1 & 1+\epsilon \end{pmatrix}$. What happens as $\epsilon \to 0$?

8.3.2 Theory Show that $\kappa(\alpha A) = \kappa(A)$ for $\alpha \neq 0$, that $\kappa_2(Q) = 1$ for orthogonal Q, and that $\kappa_2(A^TA) = \kappa_2(A)^2$.

8.3.3 Theory The system $\begin{pmatrix} 1.01 & 0.99 \\ 0.99 & 1.01 \end{pmatrix}\mathbf{x} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$ has solution $(1, 1)$. Change $\mathbf{b}$ to $(2.02, 1.98)$ and solve again. Compare the relative change in $\mathbf{x}$ with $\kappa(A)$ times the relative change in $\mathbf{b}$.

8.3.4 Python Compute κ_2 of the $n \times n$ Vandermonde matrix with nodes $0, 1/(n-1), \ldots, 1$ for $n = 5, 10, 15, 20$. What does this say about polynomial interpolation by solving a Vandermonde system (Chapter 9)?

8.3.5 Python Use `np.linalg.cond` to compute κ_2 of the tridiagonal matrix $\mathrm{tridiag}(-1, 2, -1)$ of order $n = 10, 100, 1000$ and verify that $\kappa_2 \approx \frac{4}{\pi^2}(n+1)^2$.

8.3.6 ★ Graduate Prove the matrix perturbation theorem. (Write $A + \delta A = A(I + A^{-1}\delta A)$ and use the Neumann series: if $\|B\| < 1$ then $I + B$ is invertible with $\|(I+B)^{-1}\| \le 1/(1 - \|B\|)$.)

8.4 Stopping criteria for iterative methods

An iterative method produces $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots$ and must decide when to stop. Common tests, for a tolerance ε, are:

1. **absolute change**: $\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\right\| < \varepsilon$;
2. **relative change**: $\dfrac{\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\right\|}{\left\|\mathbf{x}^{(k+1)}\right\|} < \varepsilon$;
3. **relative residual**: $\dfrac{\left\|\mathbf{b} - A\mathbf{x}^{(k)}\right\|}{\|\mathbf{b}\|} < \varepsilon$;
4. **iteration limit**: $k \geq N_{\max}$ (report failure).

Any norm may be used; $\|\cdot\|_\infty$ and $\|\cdot\|_2$ are the most common. By Theorem 8.11 the relative residual test (3) guarantees a relative error of at most $\kappa(A)\varepsilon$. For a linearly convergent iteration with contraction factor $\lambda < 1$, the error and the change are related by

$$\left\|\mathbf{x}^{(k+1)} - \mathbf{x}\right\| \leq \frac{\lambda}{1-\lambda}\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\right\|,$$

so a small change guarantees a small error only if λ is not close to 1.

Exercises

8.4.1 Theory Prove the last inequality, assuming $\left\|\mathbf{x}^{(k+1)} - \mathbf{x}\right\| \leq \lambda \left\|\mathbf{x}^{(k)} - \mathbf{x}\right\|$.

8.4.2 Theory An iteration has $\lambda = 0.999$ and the change in the last step was 10^{-8}. What can you say about the error?

Chapter Summary

- $\|\mathbf{x}\|_1 = \sum |x_i|$, $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$, $\|\mathbf{x}\|_\infty = \max |x_i|$; all norms on $\mathbb{R}^n$ are equivalent.
- $\|A\|_1 =$ max column sum, $\|A\|_\infty =$ max row sum, $\|A\|_2 = \sqrt{\rho(A^T A)}$; $\rho(A) \leq \|A\|$.
- $A^k \to 0 \iff \rho(A) < 1$.
- $\kappa(A) = \|A\|\left\|A^{-1}\right\|$; $\dfrac{\|\mathbf{x} - \hat{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \kappa(A)\dfrac{\|\mathbf{r}\|}{\|\mathbf{b}\|}$.
- About $\log_{10}\kappa(A)$ decimal digits are lost when solving $A\mathbf{x} = \mathbf{b}$.

Chapter 9

Polynomial Interpolation

Learning Objectives

After studying this chapter you should be able to:

- state the Weierstrass approximation theorem and the existence and uniqueness of the interpolating polynomial;
- construct interpolating polynomials by undetermined coefficients, in Lagrange form and in Newton form;
- build a divided-difference table and add a new data point without recomputing;
- state, prove and apply the interpolation error formula, and use the next-term rule;
- explain Runge's phenomenon and why Chebyshev nodes or splines are used in practice.

Given a table of values (x_i, y_i), for instance measurements, or values of a function that is expensive to evaluate, we often need values *between* the tabulated points. Polynomials are the natural tool: they are easy to evaluate, differentiate and integrate. Interpolation is also the foundation of the methods for numerical differentiation (Chapter 12) and integration (Chapter 13), and of methods for differential equations.

Historical Note: interpolation from astronomy tables to Runge

Interpolation is as old as astronomical tables: Babylonian astronomers interpolated linearly in their ephemerides, and the Chinese astronomer **Liu Zhuo** used quadratic interpolation around 600 AD. **Thomas Harriot** (1611) and **Henry Briggs** (1624, for his logarithm tables) used finite-difference formulas, which **Isaac Newton** developed into a general theory (*Principia*, 1687; *Methodus differentialis*, 1711). The "Lagrange" formula was published by Edward **Waring** in 1779, rediscovered by **Euler** in 1783 and published by **Joseph-Louis Lagrange** in 1795. In 1901 **Carl Runge** discovered that interpolation at equally spaced points can diverge, even for a very smooth function, the phenomenon now named after him.

9.1 Polynomial approximation

Definition 9.1. A polynomial of degree at most n is a function $P_n(x) = a_0 + a_1 x + \cdots + a_n x^n = \sum_{j=0}^{n} a_j x^j$. The set of all such polynomials is denoted $\mathbb{P}_n$; it is a vector space of dimension $n+1$.

Theorem 9.2 (Weierstrass approximation theorem, 1885)**.** *Let f be continuous on $[a,b]$. For every*

> *$\varepsilon > 0$ there is a polynomial P with*
>
> $$|f(x) - P(x)| < \varepsilon \quad \text{for all } x \in [a, b].$$

The theorem says polynomials *can* approximate any continuous function uniformly, but not *how* to find them. Taylor polynomials (Theorem 3.13) are excellent near the expansion point x_0 but their error grows rapidly away from it, and they require derivatives. Interpolation uses only function values spread over the interval.

Historical Note: Weierstrass and Bernstein

Karl Weierstrass proved Theorem 9.2 at the age of 70, in 1885. In 1912 **Sergei Bernstein** gave a beautiful constructive proof using the polynomials $B_n(f;x) = \sum_{k=0}^{n} f(k/n)\binom{n}{k}x^k(1-x)^{n-k}$, which arise from probability theory. Bernstein polynomials converge very slowly, but they became the basis of the Bézier curves (P. Bézier at Renault and P. de Casteljau at Citroën, around 1960) that describe every font you see on a computer screen.

9.2 The interpolation problem

Definition 9.3. Given $n+1$ distinct *nodes* $x_0, \dots, x_n$ and values $y_i = f(x_i)$, find $P_n \in \mathbb{P}_n$ with

$$P_n(x_i) = y_i, \qquad i = 0, 1, \dots, n.$$

P_n is the *interpolating polynomial*; we say P_n *interpolates* f at the nodes.

Theorem 9.4 (existence and uniqueness)**.** *If the nodes are distinct, there is exactly one polynomial $P_n \in \mathbb{P}_n$ interpolating the data.*

Proof. Uniqueness: if $P, Q \in \mathbb{P}_n$ both interpolate, then $P - Q \in \mathbb{P}_n$ has $n+1$ distinct zeros $x_0, \dots, x_n$. A nonzero polynomial of degree $\le n$ has at most n zeros, so $P - Q \equiv 0$. *Existence* follows from the Lagrange formula below (or: the linear map $\mathbb{P}_n \to \mathbb{R}^{n+1}$, $P \mapsto (P(x_0), \dots, P(x_n))$ is injective, hence surjective). □

9.2.1 Method of undetermined coefficients

Substitute the data into $P_n(x) = a_0 + a_1 x + \cdots + a_n x^n$ to obtain the linear system

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & & & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

The coefficient matrix V is the **Vandermonde matrix**; $\det V = \prod_{0 \le i < j \le n} (x_j - x_i) \neq 0$ for distinct nodes, which gives another proof of Theorem 9.4.

Example 9.5. Find the quadratic through $(0, 1.0000)$, $(0.5, 1.6487)$, $(1, 2.7183)$ (values of e^x). The system is $a_0 = 1$, $a_0 + 0.5a_1 + 0.25a_2 = 1.6487$, $a_0 + a_1 + a_2 = 2.7183$, so $0.5a_1 + 0.25a_2 = 0.6487$ and $a_1 + a_2 = 1.7183$, giving $a_2 = 0.8418$, $a_1 = 0.8765$:

$$P_2(x) = 1 + 0.8765x + 0.8418x^2.$$

Listing 9.1: Undetermined coefficients and the conditioning of Vandermonde matrices
ch09/vandermonde.py

```python
# Interpolation by the method of undetermined coefficients (a Vandermonde system)
import numpy as np

x = np.array([0.0, 0.5, 1.0])
y = np.array([1.0000, 1.6487, 2.7183])        # values of e^x
V = np.vander(x, increasing=True)              # columns 1, x, x^2
a = np.linalg.solve(V, y)
print("Vandermonde matrix:\n", V)
print("coefficients a0, a1, a2 =", a)
p = lambda t: a[0] + a[1] * t + a[2] * t**2
print(f"P2(0.25) = {p(0.25):.6f},   e^0.25 = {np.exp(0.25):.6f}")

# The Vandermonde matrix becomes extremely ill-conditioned as n grows
print("\n  n   cond(V) for equally spaced nodes in [0,1]")
for n in [5, 10, 15, 20, 25]:
    xs = np.linspace(0, 1, n)
    print(f"{n:3d}   {np.linalg.cond(np.vander(xs, increasing=True)):.3e}")
```

Output

```
Vandermonde matrix:
 [[1.   0.   0.  ]
 [1.   0.5  0.25]
 [1.   1.   1.  ]]
coefficients a0, a1, a2 = [1.     0.8765 0.8418]
P2(0.25) = 1.271737,   e^0.25 = 1.284025

  n   cond(V) for equally spaced nodes in [0,1]
  5   6.864e+02
 10   1.519e+07
 15   4.032e+11
 20   1.169e+16
 25   3.380e+18
```

△ Caution: do not solve Vandermonde systems for large n

The condition number of the Vandermonde matrix grows exponentially: for 20 equally spaced nodes it is about 10^{16}, so the computed coefficients may have no correct digits. The Lagrange and Newton forms avoid this system altogether.

9.3 Lagrange form

We look for $P_n(x) = y_0 L_0(x) + y_1 L_1(x) + \cdots + y_n L_n(x)$, where the *cardinal functions* $L_k \in \mathbb{P}_n$ satisfy

$$L_k(x_j) = \delta_{kj} = \begin{cases} 1, & j = k, \\ 0, & j \neq k. \end{cases}$$

Then indeed $P_n(x_j) = \sum_k y_k \delta_{kj} = y_j$. Since L_k vanishes at the n nodes x_j, $j \neq k$, it must be $L_k(x) = c \prod_{j \neq k}(x - x_j)$, and $L_k(x_k) = 1$ fixes c:

$$\boxed{L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}, \qquad P_n(x) = \sum_{k=0}^{n} y_k L_k(x).} \tag{9.1}$$

For three nodes:

$$L_0 = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)}, \quad L_1 = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)}, \quad L_2 = \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}.$$

Example 9.6. For the data of Example 9.5 ($x_0 = 0$, $x_1 = 0.5$, $x_2 = 1$):

$$L_0(x) = \frac{(x - 0.5)(x - 1)}{(0 - 0.5)(0 - 1)} = 2(x - 0.5)(x - 1), \qquad L_1(x) = \frac{x(x - 1)}{0.5(-0.5)} = -4x(x - 1),$$
$$L_2(x) = \frac{x(x - 0.5)}{1 \cdot 0.5} = 2x(x - 0.5),$$

and $P_2(x) = 1.0000 L_0(x) + 1.6487 L_1(x) + 2.7183 L_2(x)$. Expanding gives the same polynomial as in Example 9.5 (uniqueness!). At $x = 0.25$: $L_0 = 0.375$, $L_1 = 0.75$, $L_2 = -0.125$, so $P_2(0.25) = 0.375 + 1.2365 - 0.3398 = 1.2717$, while $e^{0.25} = 1.2840$.

Listing 9.2: Lagrange interpolation ch09/lagrange.py

```python
# Lagrange interpolation
import numpy as np
import matplotlib.pyplot as plt


def lagrange_basis(xnodes, k, t):
    """The k-th Lagrange cardinal polynomial L_k evaluated at t."""
    L = np.ones_like(t, dtype=float)
    for j, xj in enumerate(xnodes):
        if j != k:
            L *= (t - xj) / (xnodes[k] - xj)
    return L


def lagrange_interp(xnodes, ynodes, t):
    """P_n(t) = sum_k y_k L_k(t)."""
    t = np.asarray(t, dtype=float)
    return sum(yk * lagrange_basis(xnodes, k, t) for k, yk in enumerate(ynodes))


x = np.array([0.0, 0.5, 1.0])
y = np.array([1.0000, 1.6487, 2.7183])
for t in [0.25, 0.75]:
    Ls = [float(lagrange_basis(x, k, np.array(t))) for k in range(3)]
    print(f"t = {t}: L0 = {Ls[0]:.4f}, L1 = {Ls[1]:.4f}, L2 = {Ls[2]:.4f}, "
          f"P2 = {float(lagrange_interp(x, y, t)):.6f}, e^t = {np.exp(t):.6f}")

# Plot the cardinal functions for 5 equally spaced nodes
xn = np.linspace(0, 1, 5)
t = np.linspace(0, 1, 300)
plt.figure(figsize=(6, 3.4))
for k in range(5):
    plt.plot(t, lagrange_basis(xn, k, t), label=f"$L_{k}$")
plt.plot(xn, np.zeros(5), "ko", ms=4)
plt.plot(xn, np.ones(5), "k+", ms=8)
plt.legend(ncol=5, fontsize=8, loc="lower center")
plt.title("Lagrange cardinal functions: $L_k(x_j)=\\delta_{kj}$")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch09_cardinal.pdf")
```

Output

```
t = 0.25: L0 = 0.3750, L1 = 0.7500, L2 = -0.1250, P2 = 1.271737, e^t = 1.284025
t = 0.75: L0 = -0.1250, L1 = 0.7500, L2 = 0.3750, P2 = 2.130888, e^t = 2.117000
```

★ Key Concept: Lagrange form: pros and cons

The Lagrange form is explicit and elegant, which makes it ideal for *theory* (deriving quadrature and differentiation formulas). But evaluating it naively costs $O(n^2)$ per point, and adding a new node requires recomputing every L_k. The Newton form fixes the second problem; the *barycentric* form (Section 9.6) fixes the first.

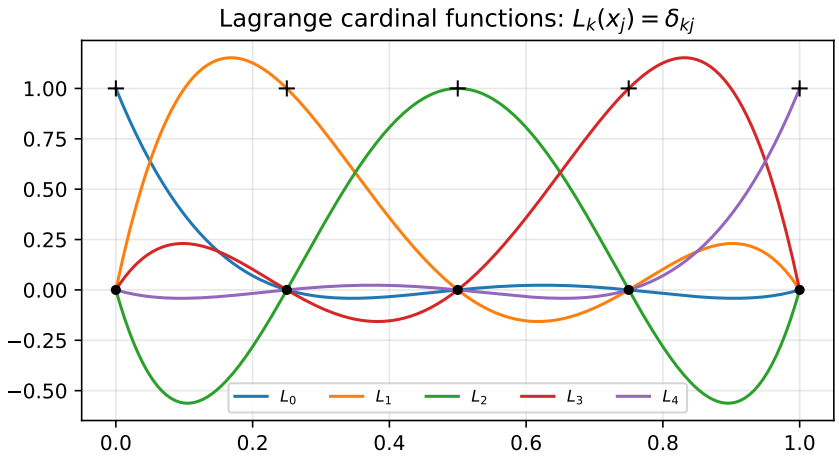


Figure 9.1: The five Lagrange cardinal functions for equally spaced nodes on $[0, 1]$.

Exercises

9.3.1 Theory Find the Lagrange interpolating polynomial for $(1, 2)$, $(2, 3)$, $(4, 1)$ and evaluate it at $x = 3$.

9.3.2 Theory Show that $\sum_{k=0}^{n} L_k(x) = 1$ for all x. (Hint: interpolate $f \equiv 1$.) More generally, $\sum_k x_k^m L_k(x) = x^m$ for $m \le n$.

9.3.3 Theory Use the Lagrange form to derive the linear interpolation formula $P_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$.

9.3.4 Python Tabulate $\sin x$ at $x = 0, \pi/6, \pi/4, \pi/3, \pi/2$ and interpolate. Plot the error $\sin x - P_4(x)$ on $[0, \pi/2]$.

9.3.5 Python **Ghana's population.** The census populations of Ghana (in millions) were 6.73 (1960), 8.56 (1970), 12.30 (1984), 18.91 (2000), 24.66 (2010), 30.83 (2021). Interpolate by a polynomial of degree 5 and estimate the population in 1990 and in 2030. Plot the polynomial on $[1950, 2035]$. Do you trust the estimates? (Compare with the least-squares model of Chapter 10.)

9.4 Newton's divided-difference form

Write the interpolating polynomial as

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1}). \tag{9.2}$$

The advantage: $P_n = P_{n-1} + a_n(x - x_0)\cdots(x - x_{n-1})$, so adding a node adds one term and does not change the earlier coefficients. Putting $x = x_0$ gives $a_0 = f(x_0)$. Putting $x = x_1$ gives $f(x_1) = a_0 + a_1(x_1 - x_0)$, so

$$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0} =: f[x_0, x_1],$$

the *first divided difference*. Putting $x = x_2$ and simplifying (see the exercises),

$$a_2 = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} =: f[x_0, x_1, x_2].$$

Definition 9.7 (divided differences)**.** The zeroth divided difference is $f[x_i] = f(x_i)$, and recursively

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Theorem 9.8 (Newton's interpolation formula)**.** *The interpolating polynomial is*

$$P_n(x) = f[x_0] + \sum_{k=1}^{n} f[x_0, x_1, \dots, x_k]\,(x - x_0)(x - x_1)\cdots(x - x_{k-1}).$$

Divided differences are symmetric functions of their arguments (the order of the nodes does not matter).

The coefficients are computed in a **divided-difference table**, each entry being the difference of its two neighbours on the left divided by the difference of the corresponding extreme x-values:

x_i	$f[x_i]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$\mathbf{f[x_0]}$			
		$\mathbf{f[x_0, x_1]}$		
x_1	$f[x_1]$		$\mathbf{f[x_0, x_1, x_2]}$	
		$f[x_1, x_2]$		$\mathbf{f[x_0, \dots, x_3]}$
x_2	$f[x_2]$		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3]$		
x_3	$f[x_3]$			

The Newton coefficients are the entries on the top diagonal (in bold).

Example 9.9. Construct the divided-difference table for

i	0	1	2	3	4
x_i	0.0	0.5	0.8	1.0	1.4
y_i	1.0000	1.6487	2.2255	2.7183	4.0552

and approximate $f(0.7)$. For instance $f[x_0, x_1] = (1.6487 - 1)/0.5 = 1.2974$, $f[x_1, x_2] = (2.2255 - 1.6487)/0.3 = 1.92267$, and $f[x_0, x_1, x_2] = (1.92267 - 1.2974)/0.8 = 0.78158$.

Listing 9.3: Divided differences, Newton form and error estimates ch09/divided_diff.py

```python
# Newton's divided differences and interpolation error
import numpy as np


def divided_differences(x, y):
    """Return the full divided-difference table F, with F[i, j] = f[x_{i-j}, ..., x_i].
    The Newton coefficients are the diagonal entries F[j, j]."""
    n = len(x)
    F = np.zeros((n, n))
    F[:, 0] = y
    for j in range(1, n):
        for i in range(j, n):
            F[i, j] = (F[i, j - 1] - F[i - 1, j - 1]) / (x[i] - x[i - j])
    return F


def newton_eval(x, coef, t):
    """Evaluate the Newton form by nested multiplication (Horner-like)."""
    p = coef[-1]
    for k in range(len(coef) - 2, -1, -1):
        p = p * (t - x[k]) + coef[k]
    return p


x = np.array([0.0, 0.5, 0.8, 1.0, 1.4])
y = np.array([1.00, 1.6487, 2.2255, 2.7183, 4.0552])     # e^x rounded
F = divided_differences(x, y)
print("Divided-difference table:")
print("  x_i     f[x_i]      1st         2nd         3rd         4th")
for i in range(len(x)):
    row = "  ".join(f"{F[i, j]:9.5f}" for j in range(i + 1))
    print(f"{x[i]:4.1f}  {row}")
coef = np.diag(F)
print("Newton coefficients:", np.round(coef, 5))
for n in range(1, 5):
    print(f"P_{n}(0.7) = {newton_eval(x[:n + 1], coef[:n + 1], 0.7):.6f}")
print(f"e^0.7    = {np.exp(0.7):.6f}")

# f(0.7) by quadratics on two different sets of nodes, with error estimates
print()
for nodes, extra in (([0.0, 0.5, 0.8], 1.0), ([0.5, 0.8, 1.0], 1.4)):
    xs = np.array(nodes)
    ys = np.exp(xs).round(4)                          # the tabulated values
    c = np.diag(divided_differences(xs, ys))
    p = newton_eval(xs, c, 0.7)
    w = np.prod(0.7 - xs)                             # (0.7-x0)(0.7-x1)(0.7-x2)
    bound = abs(w) * np.exp(xs.max()) / 6             # max |f^(3)| / 3!  with f^(3) = e^x
    # next-term rule: the term that one more node (extra) would add
    x4 = np.append(xs, extra)
    nxt = np.diag(divided_differences(x4, np.exp(x4).round(4)))[-1] * w
    print(f"nodes {nodes}: P2(0.7) = {p:.6f}, true error = {np.exp(0.7) - p:+.2e},")
    print(f"      error bound = {bound:.2e}, next-term estimate = {nxt:+.2e}")
```

Output

```
Divided-difference table:
  x_i     f[x_i]      1st        2nd        3rd        4th
 0.0    1.00000
 0.5    1.64870    1.29740
 0.8    2.22550    1.92267    0.78158
 1.0    2.71830    2.46400    1.08267    0.30108
 1.4    4.05520    3.34225    1.46375    0.42343    0.08739
Newton coefficients: [1.      1.2974  0.78158 0.30108 0.08739]
P_1(0.7) = 1.908180
P_2(0.7) = 2.017602
P_3(0.7) = 2.013386
P_4(0.7) = 2.013754
e^0.7    = 2.013753

nodes [0.0, 0.5, 0.8]: P2(0.7) = 2.017602, true error = -3.85e-03,
      error bound = 5.19e-03, next-term estimate = -4.22e-03
nodes [0.5, 0.8, 1.0]: P2(0.7) = 2.011580, true error = +2.17e-03,
      error bound = 2.72e-03, next-term estimate = +2.54e-03
```

The cubic polynomial is $P_3(x) = 1 + 1.2974x + 0.78158x(x-0.5) + 0.30108x(x-0.5)(x-0.8)$, and the sequence $P_1(0.7), P_2(0.7), P_3(0.7), P_4(0.7)$ approaches $e^{0.7} = 2.013753$. (The data are e^x rounded to 4 decimals.)

⋆ Key Concept: evaluating the Newton form

Like Horner's rule, the Newton form is evaluated by nested multiplication: $P_3(x) = a_0 + (x-x_0)\big(a_1 + (x-x_1)\big(a_2 + (x-x_2)a_3\big)\big)$, which costs $O(n)$ per point.

Exercises

9.4.1 Theory Complete the derivation of $a_2 = f[x_0, x_1, x_2]$ from $P_n(x_2) = f(x_2)$.

9.4.2 Theory Construct the divided-difference table for $(-1, 2)$, $(0, 1)$, $(1, 2)$, $(2, -7)$ and write down the cubic interpolant. Then add the point $(3, 10)$ and find the quartic with one extra column.

9.4.3 Theory Show that $f[x_0, x_1] = f[x_1, x_0]$ and $f[x_0, x_1, x_2]$ is symmetric in its three arguments.

9.4.4 Theory For equally spaced nodes $x_i = x_0 + ih$ show that $f[x_0, \dots, x_k] = \dfrac{\Delta^k f_0}{k!\, h^k}$, where $\Delta f_i = f_{i+1} - f_i$ is the forward difference. Deduce *Newton's forward-difference formula*.

9.4.5 Python Write a function that returns the Newton coefficients using only a one-dimensional array (overwrite in place from the bottom up), and test it on Example 9.9.

9.5 The error of polynomial interpolation

Theorem 9.10 (interpolation error). *Let $f \in C^{n+1}[a,b]$ and let P_n interpolate f at distinct nodes $x_0, \dots, x_n \in [a,b]$. Then for each $x \in [a,b]$ there is $\xi_x \in (a,b)$ such that*

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!}\,(x-x_0)(x-x_1)\cdots(x-x_n). \tag{9.3}$$

Proof. If x is a node both sides vanish. Otherwise fix x, let $w(t)=\prod_{i=0}^{n}(t-x_i)$ and define

$$g(t)=f(t)-P_n(t)-\frac{f(x)-P_n(x)}{w(x)}\,w(t).$$

Then g vanishes at the $n+2$ distinct points $x_0,\dots,x_n,x$. By Rolle's theorem g' has at least $n+1$ zeros, g'' at least $n,\dots,$ and $g^{(n+1)}$ at least one zero ξ_x. Since $P_n^{(n+1)}\equiv 0$ and $w^{(n+1)}\equiv (n+1)!$,

$$0=g^{(n+1)}(\xi_x)=f^{(n+1)}(\xi_x)-\frac{f(x)-P_n(x)}{w(x)}(n+1)!,$$

which is (9.3). □

Corollary 9.11. $f[x_0,x_1,\dots,x_n]=\dfrac{f^{(n)}(\xi)}{n!}$ *for some ξ between the smallest and largest node.*

★ Key Concept: the two factors of the error

The error (9.3) is a product of

- $f^{(n+1)}(\xi)/(n+1)!$, which depends on the *smoothness of f*, and
- $w(x)=\prod(x-x_i)$, which depends only on the *position of the nodes*.

The error is small near the nodes and grows quickly outside $[x_0,x_n]$: **extrapolation is dangerous**. For a given x, choose the nodes closest to x.

The next-term rule. Usually $f^{(n+1)}$ is unknown. By Corollary 9.11, $f^{(n+1)}(\xi)/(n+1)!\approx f[x_0,\dots,x_n,x_{n+1}]$ for one extra node, so

$$f(x)-P_n(x)\approx f[x_0,\dots,x_{n+1}]\,(x-x_0)\cdots(x-x_n),$$

which is exactly the next term of the Newton form. This is how the error is estimated in practice.

Example 9.12. Approximate $f(0.7)$ from the table of Example 9.9 by quadratics through (i) $0.0,0.5,0.8$ and (ii) $0.5,0.8,1.0$. Since $f=e^x$, $f'''(\xi)\le e^{0.8}$ in case (i), so $|E(0.7)|\le \frac{e^{0.8}}{6}|0.7(0.2)(-0.1)|=5.2\times10^{-3}$. The output of Listing 9.3 shows the true errors (-3.9×10^{-3} and 2.2×10^{-3}), the bounds and the next-term estimates. The second set of nodes, which surrounds 0.7 more closely, is better.

Exercises

9.5.1 Theory Bound the error of linear interpolation of $f(x)=\sin x$ between x_0 and $x_1=x_0+h$, and show $|E|\le h^2/8$. What h is needed for a table of $\sin x$ with linear interpolation accurate to 10^{-6}?

9.5.2 Theory Show that $\max_{x\in[x_0,x_1]}|(x-x_0)(x-x_1)|=h^2/4$ and derive the bound $\frac{h^2}{8}\max|f''|$ for linear interpolation.

9.5.3 Theory Let P_3 interpolate $f(x)=\ln x$ at $1,2,3,4$. Bound $|f(2.5)-P_3(2.5)|$ and compare with the actual error.

9.5.4 ★ Graduate Prove Corollary 9.11, and use it to define $f[x_0,x_0]=f'(x_0)$ (confluent divided differences, the basis of Hermite interpolation).

9.6 Runge's phenomenon and Chebyshev nodes

Does $P_n \to f$ as $n \to \infty$? For equally spaced nodes the answer can be *no*. Runge's example $f(x) = 1/(1+25x^2)$ on $[-1,1]$ is infinitely differentiable, yet the interpolants at equally spaced nodes oscillate wildly near the end points, and the error grows without bound.

Listing 9.4: Runge's phenomenon ch09/runge.py

```python
# Runge's phenomenon: equally spaced versus Chebyshev nodes
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import BarycentricInterpolator

f = lambda x: 1 / (1 + 25 * x**2)
t = np.linspace(-1, 1, 2001)

print("  n   max error (equispaced)   max error (Chebyshev)")
for n in [4, 8, 12, 16, 20, 40]:
    xe = np.linspace(-1, 1, n + 1)
    xc = np.cos((2 * np.arange(n + 1) + 1) * np.pi / (2 * n + 2))   # Chebyshev nodes
    pe = BarycentricInterpolator(xe, f(xe))
    pc = BarycentricInterpolator(xc, f(xc))
    print(f"{n:3d}   {np.max(np.abs(f(t) - pe(t))):18.4e}   {np.max(np.abs(f(t) - pc(t)))
    ↪ :18.4e}")

n = 12
xe = np.linspace(-1, 1, n + 1)
xc = np.cos((2 * np.arange(n + 1) + 1) * np.pi / (2 * n + 2))
fig, ax = plt.subplots(1, 2, figsize=(8.5, 3.3), sharey=True)
for a, xn, title in [(ax[0], xe, "13 equally spaced nodes"), (ax[1], xc, "13 Chebyshev
    ↪ nodes")]:
    a.plot(t, f(t), "k", lw=1, label="$f(x)=1/(1+25x^2)$")
    a.plot(t, BarycentricInterpolator(xn, f(xn))(t), "C3", label="$P_{12}$")
    a.plot(xn, f(xn), "o", ms=4)
    a.set_ylim(-1.5, 2.2)
    a.set_title(title, fontsize=10)
    a.grid(alpha=0.3)
ax[0].legend(fontsize=8, loc="upper center")
plt.tight_layout()
plt.savefig("ch09_runge.pdf")
```

Output

```
  n   max error (equispaced)   max error (Chebyshev)
  4           4.3836e-01           4.0202e-01
  8           1.0452e+00           1.7083e-01
 12           3.6633e+00           6.9216e-02
 16           1.4394e+01           3.2613e-02
 20           5.9822e+01           1.5333e-02
 40           1.0464e+05           2.8941e-04
```

The remedy is to cluster the nodes near the end points. The factor $\max|w(x)|$ in (9.3) is minimised by the **Chebyshev nodes**.

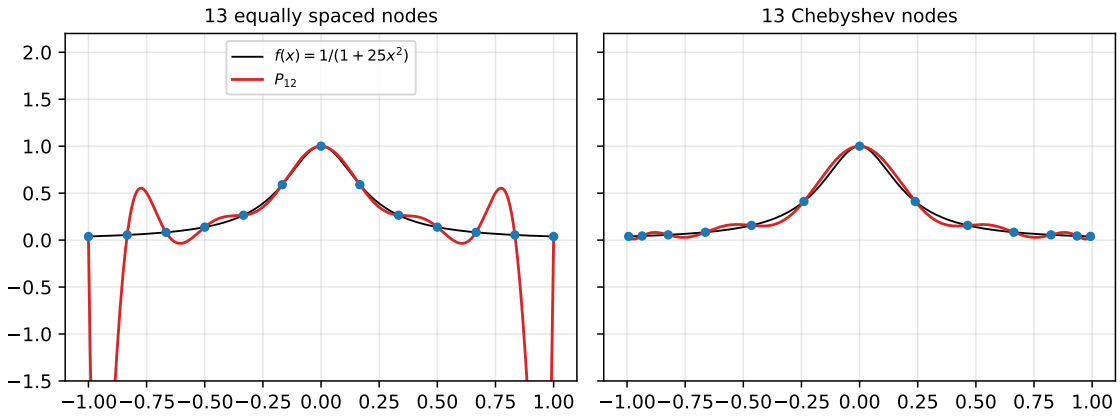


Figure 9.2: Interpolating Runge's function with 13 nodes.

Theorem 9.13 (Chebyshev nodes). *Among all choices of $n+1$ nodes in $[-1,1]$, $\max_{x\in[-1,1]}|(x-x_0)\cdots(x-x_n)|$ is minimised by the zeros of the Chebyshev polynomial $T_{n+1}(x)=\cos((n+1)\arccos x)$,*

$$x_k = \cos\frac{(2k+1)\pi}{2(n+1)}, \qquad k = 0,1,\dots,n,$$

and then the maximum equals 2^{-n}. On $[a,b]$ use $\frac{a+b}{2}+\frac{b-a}{2}x_k$.

For Chebyshev nodes the interpolants of every function that is (Lipschitz) continuous on $[a,b]$ converge, and for analytic functions they converge exponentially fast, as the table shows.

Towards Graduate Studies: barycentric interpolation and Chebfun

The *barycentric formula* $P_n(x)=\dfrac{\sum_k \frac{w_k}{x-x_k}y_k}{\sum_k \frac{w_k}{x-x_k}}$ with $w_k = 1/\prod_{j\neq k}(x_k-x_j)$ evaluates the interpolant in $O(n)$ operations and is numerically stable (Higham, 2004). `scipy.interpolate.BarycentricInterpolator` uses it. With Chebyshev points, polynomial interpolation of degree thousands is perfectly practical: the MATLAB system *Chebfun* (Trefethen and collaborators, 2004–) represents functions by such interpolants and computes with them to machine precision. Read Trefethen, *Approximation Theory and Approximation Practice* [19], a wonderful graduate-level book written as executable code. The *Lebesgue constant* $\Lambda_n=\max_x\sum_k|L_k(x)|$ is the condition number of interpolation: it grows like $2^{n+1}/(en\ln n)$ for equispaced nodes but only like $\frac{2}{\pi}\log n$ for Chebyshev nodes.

9.7 Piecewise interpolation and splines

Another way to avoid high-degree oscillations is to use *low-degree polynomials on subintervals*.

- **Piecewise linear** interpolation ("connect the dots", `np.interp`) has error $\le \frac{h^2}{8}\max|f''|$.
- A **cubic spline** S is a cubic on each $[x_i,x_{i+1}]$ with S, S' and S'' continuous at the nodes. The $4n$ coefficients are fixed by $2n$ interpolation conditions, $2(n-1)$ smoothness conditions and two end conditions, e.g. $S''(x_0)=S''(x_n)=0$ (*natural* spline). The resulting tridiagonal system is solved in $O(n)$ operations (Listing 6.4). The error is $O(h^4)$.

Listing 9.5: Piecewise linear and cubic spline interpolation of Runge's function `ch09/splines.py`

```python
# Piecewise interpolation: linear and cubic splines for the Runge function
import numpy as np
from scipy.interpolate import CubicSpline

f = lambda x: 1 / (1 + 25 * x**2)
t = np.linspace(-1, 1, 4001)
print("  n    piecewise linear     natural cubic spline   not-a-knot spline")
for n in [8, 16, 32, 64, 128]:
    x = np.linspace(-1, 1, n + 1)
    lin = np.interp(t, x, f(x))
    nat = CubicSpline(x, f(x), bc_type="natural")(t)
    nak = CubicSpline(x, f(x))(t)                # default end condition
    print(f"{n:4d}   {np.max(np.abs(lin - f(t))):14.3e}   {np.max(np.abs(nat - f(t))):18.3e
    ↪ }"
          f"   {np.max(np.abs(nak - f(t))):16.3e}")
```

Output

```
  n    piecewise linear     natural cubic spline   not-a-knot spline
  8        6.390e-02            5.607e-02          5.615e-02
 16        5.355e-02            3.745e-03          3.745e-03
 32        2.070e-02            6.555e-04          6.555e-04
 64        5.850e-03            4.032e-05          4.032e-05
128        1.509e-03            2.522e-06          2.380e-06
```

Doubling n reduces the linear error by about 4 ($O(h^2)$) and the spline error by about 16 ($O(h^4)$).

Historical Note: splines

Before computers, draughtsmen drew smooth curves through points (for ship hulls and aircraft) with a thin flexible strip of wood called a *spline*, held in place by lead weights ("ducks"). The bent strip minimises its bending energy $\int S''(x)^2 dx$, and so does the natural cubic spline. The mathematical theory was created by **Isaac Schoenberg** in 1946, and splines are now used everywhere in computer graphics, CAD and statistics.

Exercises

9.7.1 Python Reproduce the Runge table with $f(x) = |x|$ and with $f(x) = e^x$. What do you observe?

9.7.2 Python Compute the Lebesgue constant Λ_n for equispaced and Chebyshev nodes, $n = 5, 10, 20$, by evaluating $\sum_k |L_k(x)|$ on a fine grid.

9.7.3 Theory Show that $T_{n+1}(x) = \cos((n+1)\arccos x)$ is a polynomial of degree $n+1$ with leading coefficient 2^n, using the recurrence $T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x)$. Find its zeros.

9.7.4 Theory *Hermite interpolation.* Find the cubic H with $H(0) = f(0)$, $H'(0) = f'(0)$, $H(1) = f(1)$, $H'(1) = f'(1)$ for $f(x) = e^x$, using a divided-difference table with repeated nodes $0, 0, 1, 1$.

9.7.5 Python Fit a natural cubic spline to the Ghana census data above and compare its estimates for 1990 and 2030 with the degree-5 polynomial.

9.7.6 ★ Graduate Prove that the natural cubic spline minimises $\int_a^b g''(x)^2\,dx$ among all C^2

functions g interpolating the data (Holladay's theorem, 1957).

Chapter Summary

- A unique $P_n \in \mathbb{P}_n$ interpolates $n+1$ distinct points.
- Lagrange: $P_n = \sum y_k L_k$, $L_k = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}$. Newton: coefficients from the divided-difference table.
- Error: $f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod (x - x_i)$; next-term rule for estimates.
- Equispaced high-degree interpolation can diverge (Runge); use Chebyshev nodes or splines.
- Python: `scipy.interpolate` (`BarycentricInterpolator`, `CubicSpline`), `np.interp`.

Chapter 10

Least-Squares Approximation and Orthogonal Polynomials

Learning Objectives

After studying this chapter you should be able to:

- formulate a data-fitting problem as an overdetermined linear system $A\mathbf{a} \approx \mathbf{y}$;
- derive the normal equations $A^{T}A\mathbf{a} = A^{T}\mathbf{y}$ by calculus and by orthogonality;
- fit straight lines, polynomials, and exponential and power-law models to data;
- explain why QR is preferred to the normal equations;
- compute continuous least-squares approximations, and explain why orthogonal polynomials make this easy;
- work with Legendre and Chebyshev polynomials and their three-term recurrences.

Interpolation forces the approximating function through every data point. This is inappropriate when the data contain measurement errors, or when there are many more data points than parameters in the model: a degree-99 polynomial through 100 noisy points is useless. Instead we seek a simple function that fits the data *as well as possible* in the sense of minimising the sum of the squares of the errors.

Historical Note: Legendre, Gauss and the lost planet

On 1 January 1801 the Italian astronomer **Giuseppe Piazzi** discovered the dwarf planet Ceres, but after 41 days it disappeared behind the Sun. Astronomers had only a few observations covering a tiny arc of its orbit. The 24-year-old **Carl Friedrich Gauss** computed its orbit, using a method of least squares he said he had used since 1795, and Ceres was rediscovered on 31 December 1801 almost exactly where he predicted, which made him famous. The method was first *published*, however, by **Adrien-Marie Legendre** in 1805 (“Méthode des moindres carrés”, in an appendix on comet orbits), and a bitter priority dispute followed. Gauss’s 1809 book justified the method probabilistically, assuming normally distributed errors; this is why the normal distribution is also called the Gaussian distribution. The *Gauss–Markov theorem* (1821) shows that least squares gives the best linear unbiased estimates, which is why it is the foundation of regression in statistics.

10.1 Discrete least squares

Example 10.1. Fit a straight line $y = a_0 + a_1 x$ to the points $(1,-1), (2,3), (3,7), (4,10)$. Requiring the line to pass through every point gives four equations in two unknowns:

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} = \begin{pmatrix} -1 \\ 3 \\ 7 \\ 10 \end{pmatrix}, \qquad A\mathbf{a} = \mathbf{y}.$$

This system is *overdetermined* (more equations than unknowns) and *inconsistent*: the points are not collinear, so there is no exact solution.

Let $r_i = y_i - (a_0 + a_1 x_i)$ be the *residual* (vertical deviation) of the i-th point. The method of least squares chooses a_0, a_1 to minimise

$$E(a_0, a_1) = \sum_{i=1}^{m} r_i^2 = \sum_{i=1}^{m} \big(y_i - a_0 - a_1 x_i\big)^2 = \|\mathbf{y} - A\mathbf{a}\|_2^2 .$$

Why squares? $\sum |r_i|$ (the ℓ_1 fit) and $\max |r_i|$ (the minimax or Chebyshev fit) are also sensible, but lead to non-differentiable problems. Squares give a smooth function whose minimum is found by solving a *linear* system, and they have the statistical justification mentioned above.

10.1.1 The normal equations

In general, let $A \in \mathbb{R}^{m \times n}$ with $m > n$ (the *design matrix*), and minimise $E(\mathbf{a}) = \|\mathbf{y} - A\mathbf{a}\|_2^2 = \mathbf{r}^T\mathbf{r}$ with $\mathbf{r} = \mathbf{y} - A\mathbf{a}$. Since $r_i = y_i - \sum_j a_{ij} a_j$, we have $\partial r_i / \partial a_k = -a_{ik}$, and

$$\frac{\partial E}{\partial a_k} = \sum_{i=1}^{m} 2 r_i \frac{\partial r_i}{\partial a_k} = -2 \sum_{i=1}^{m} a_{ik} r_i = -2\,(A^T \mathbf{r})_k .$$

Setting all partial derivatives to zero gives $A^T\mathbf{r} = \mathbf{0}$, i.e. $A^T(\mathbf{y} - A\mathbf{a}) = \mathbf{0}$:

$$\boxed{A^T A\,\mathbf{a} = A^T \mathbf{y}} \qquad \text{(the normal equations).} \tag{10.1}$$

Theorem 10.2. *If the columns of A are linearly independent, then A^TA is symmetric positive definite and the least-squares problem has the unique solution $\hat{\mathbf{a}} = (A^TA)^{-1}A^T\mathbf{y}$.*

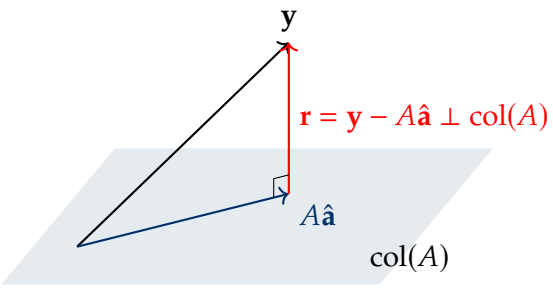
Proof. $\mathbf{x}^T A^T A \mathbf{x} = \|A\mathbf{x}\|_2^2 > 0$ for $\mathbf{x} \neq \mathbf{0}$, since $A\mathbf{x} \neq \mathbf{0}$ when the columns are independent. For any $\mathbf{a}$, writing $\mathbf{a} = \hat{\mathbf{a}} + \mathbf{d}$,

$$\|\mathbf{y} - A\mathbf{a}\|_2^2 = \|(\mathbf{y} - A\hat{\mathbf{a}}) - A\mathbf{d}\|_2^2 = \|\mathbf{y} - A\hat{\mathbf{a}}\|_2^2 - 2\mathbf{d}^T \underbrace{A^T(\mathbf{y} - A\hat{\mathbf{a}})}_{=\mathbf{0}} + \|A\mathbf{d}\|_2^2 \geq \|\mathbf{y} - A\hat{\mathbf{a}}\|_2^2 ,$$

with equality only if $A\mathbf{d} = \mathbf{0}$, i.e. $\mathbf{d} = \mathbf{0}$. □

⋆ Key Concept: the geometry of least squares

The normal equations say $A^T\mathbf{r} = \mathbf{0}$: **the residual is orthogonal to every column of** A. The vector $A\hat{\mathbf{a}}$ is the *orthogonal projection* of $\mathbf{y}$ onto the column space of A, the point of that subspace closest to $\mathbf{y}$. Every least-squares problem, discrete or continuous, is a projection problem.



Straight line. For $y = a_0 + a_1 x$ the normal equations are

$$\begin{pmatrix} m & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \end{pmatrix}.$$

In Example 10.1: $m = 4$, $\sum x_i = 10$, $\sum x_i^2 = 30$, $\sum y_i = 19$, $\sum x_i y_i = 66$, so $4a_0 + 10a_1 = 19$, $10a_0 + 30a_1 = 66$, giving $a_0 = -4.5$, $a_1 = 3.7$: the best line is $y = -4.5 + 3.7x$.

Listing 10.1: The least-squares line ch10/linear_fit.py

```
# Least-squares straight line: normal equations, lstsq and polyfit
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([-1.0, 3.0, 7.0, 10.0])

# design matrix: columns 1 and x
A = np.column_stack([np.ones_like(x), x])
print("A^T A =\n", A.T @ A, "\nA^T y =", A.T @ y)
a_normal = np.linalg.solve(A.T @ A, A.T @ y)          # normal equations
a_lstsq, res, rank, sv = np.linalg.lstsq(A, y, rcond=None)   # QR/SVD based
a_poly = np.polyfit(x, y, 1)                           # highest power first!
print("normal equations: a0, a1 =", a_normal)
print("np.linalg.lstsq : a0, a1 =", a_lstsq)
print("np.polyfit      : a1, a0 =", a_poly)
r = y - A @ a_normal
print("residuals r =", r, "  sum of squares E =", r @ r)
print("A^T r =", A.T @ r, " (residual is orthogonal to the columns of A)")

plt.figure(figsize=(5, 3.4))
plt.plot(x, y, "o", label="data")
t = np.linspace(0.5, 4.5, 10)
plt.plot(t, a_normal[0] + a_normal[1] * t, label=f"y = {a_normal[0]:.1f} + {a_normal[1]:.1f
    ↪ }x")
for xi, yi, fi in zip(x, y, A @ a_normal):
    plt.plot([xi, xi], [yi, fi], "r:", lw=1)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch10_linefit.pdf")
```

Output

```
A^T A =
 [[ 4. 10.]
 [10. 30.]]
```

```
A^T y = [19. 66.]
normal equations: a0, a1 = [-4.5  3.7]
np.linalg.lstsq : a0, a1 = [-4.5  3.7]
np.polyfit      : a1, a0 = [ 3.7 -4.5]
residuals r = [-0.2  0.1  0.4 -0.3]   sum of squares E = 0.3000000000000004
A^T r = [1.33226763e-15 4.44089210e-16]  (residual is orthogonal to the columns of A)
```

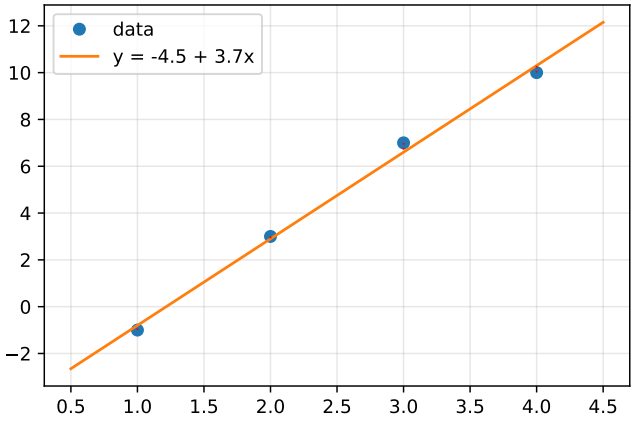


Figure 10.1: The least-squares line minimises the sum of squared vertical distances (dotted).

Polynomials. To fit $P_n(x) = a_0 + a_1 x + \cdots + a_n x^n$ to $m > n+1$ points, use the $m \times (n+1)$ design matrix with columns $1, x_i, x_i^2, \ldots, x_i^n$ (a rectangular Vandermonde matrix). The normal equations are $\sum_{j=0}^{n} a_j \left(\sum_i x_i^{j+k}\right) = \sum_i y_i x_i^k$, $k = 0, \ldots, n$. In Python use `np.polyfit(x, y, n)` (which returns the coefficients with the highest power first).

10.1.2 Normal equations versus QR

△ Caution: the normal equations square the condition number

$\kappa_2(A^TA) = \kappa_2(A)^2$. If $\kappa(A) = 10^8$, forming A^TA in double precision loses all 16 digits. With the QR factorisation $A = QR$ (Section 6.5), $\|\mathbf{y} - A\mathbf{a}\|_2 = \|Q^T\mathbf{y} - R\mathbf{a}\|_2$ (plus a constant), and the solution is obtained from the triangular system $R\hat{\mathbf{a}} = Q^T\mathbf{y}$, which involves only $\kappa(A)$. `np.linalg.lstsq` uses the (even more robust) singular value decomposition.

Listing 10.2: Normal equations versus QR for polynomial fitting `ch10/normal_vs_qr.py`

```
# Normal equations square the condition number; QR does not
import numpy as np

m = 50
x = np.linspace(0, 1, m)
print(" deg   cond(A)     cond(A^T A)   error normal eq.   error QR")
for n in [3, 6, 9, 12, 15]:
    A = np.vander(x, n + 1, increasing=True)
    c_true = np.ones(n + 1)                    # data generated by a known polynomial
    y = A @ c_true
    c_ne = np.linalg.solve(A.T @ A, A.T @ y)   # normal equations
    Q, R = np.linalg.qr(A)                     # reduced QR factorisation
```

```
    c_qr = np.linalg.solve(R, Q.T @ y)
    print(f"{n:3d}  {np.linalg.cond(A):10.2e}  {np.linalg.cond(A.T @ A):12.2e}"
          f"  {np.max(np.abs(c_ne - c_true)):14.2e}  {np.max(np.abs(c_qr - c_true)):10.2e}"
    )
```

Output

```
 deg    cond(A)      cond(A^T A)    error normal eq.    error QR
   3    1.18e+02       1.39e+04          1.33e-12     4.66e-15
   6    1.98e+04       3.90e+08          4.90e-09     2.39e-12
   9    3.56e+06       1.27e+13          5.37e-04     2.63e-10
  12    6.79e+08       3.67e+18          8.07e+02     2.44e-08
  15    1.38e+11       5.17e+18          2.59e+01     2.08e-06
```

Exercises

10.1.1 Theory Find the least-squares line for $(0,1)$, $(1,3)$, $(2,4)$, $(3,4)$ and compute E.

10.1.2 Theory Find the least-squares quadratic for $(-1,2)$, $(0,0)$, $(1,1)$, $(2,5)$.

10.1.3 Theory Show that the least-squares line passes through the centroid $(\bar{x},\bar{y})$ of the data, and that $a_1 = \dfrac{\sum (x_i-\bar{x})(y_i-\bar{y})}{\sum (x_i-\bar{x})^2}$.

10.1.4 Theory Show that fitting a constant $y = c$ by least squares gives the mean $c = \bar{y}$, while minimising $\sum |y_i - c|$ gives the median.

10.1.5 Python The following are the average monthly maximum temperatures (°C) at a weather station in Cape Coast (illustrative data): Jan 30.9, Feb 31.2, Mar 31.3, Apr 31.0, May 30.1, Jun 28.6, Jul 27.4, Aug 27.0, Sep 27.6, Oct 28.8, Nov 30.2, Dec 30.6. Fit $T(t) = c_0 + c_1\cos(2\pi t/12) + c_2\sin(2\pi t/12)$ by least squares. (The model is *linear in the parameters* even though it is not a polynomial.)

10.1.6 ★ Graduate Prove that $\kappa_2(A^TA) = \kappa_2(A)^2$ using the singular value decomposition.

10.2 Nonlinear models and linearisation

Many laws of nature are not polynomial: exponential growth and decay, power laws. Some can be made linear by a change of variables.

Model	transformation	linear model
$y = ae^{bx}$	$Y = \ln y$	$Y = \ln a + bx$
$y = ax^{b}$	$Y = \ln y,\ X = \ln x$	$Y = \ln a + bX$
$y = \dfrac{ax}{b+x}$ (saturation)	$Y = 1/y,\ X = 1/x$	$Y = \dfrac{1}{a} + \dfrac{b}{a}X$

Example 10.3 (demand for pens)**.** The number of pens sold at various prices is

Price (GH¢) x_i	6.5	8.3	11.2	14.3
Number sold y_i	33	21	13	9

Fit $y = ae^{bx}$. Fitting $\ln y_i$ by a straight line gives $\ln a = 4.4745$, $b = -0.16364$, i.e. $y = 87.75e^{-0.1636x}$.

△ Caution: linearisation changes the problem

Minimising $\sum(\ln y_i - \ln a - b x_i)^2$ is *not* the same as minimising $\sum(y_i - a e^{b x_i})^2$: the logarithm gives more weight to small values of y. The linearised fit is an excellent starting guess for a genuinely nonlinear least-squares solver such as `scipy.optimize.curve_fit`, which uses the Gauss–Newton/Levenberg–Marquardt method (Chapter 5).

Example 10.4 (population of Ghana)**.** The census populations of Ghana (millions) were 6.727 (1960), 8.559 (1970), 12.296 (1984), 18.912 (2000), 24.659 (2010) and 30.832 (2021). Fitting $P(t) = P_0 e^{bt}$ (t in years since 1960) gives a growth rate of about 2.5% per year and a doubling time of about 27 years.

Listing 10.3: Exponential fits: linearisation and nonlinear least squares ch10/exp_fit.py

```python
# Fitting exponential models: linearisation versus nonlinear least squares
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# 1. Demand for pens: price (GHS) and number sold
x = np.array([6.5, 8.3, 11.2, 14.3])
y = np.array([33.0, 21.0, 13.0, 9.0])
b, c = np.polyfit(x, np.log(y), 1)            # ln y = ln a + b x
a = np.exp(c)
print(f"linearised fit : y = {a:.4f} exp({b:.5f} x)")
(a2, b2), _ = curve_fit(lambda t, a, b: a * np.exp(b * t), x, y, p0=(a, b))
print(f"nonlinear fit  : y = {a2:.4f} exp({b2:.5f} x)")
for name, (aa, bb) in [("linearised", (a, b)), ("nonlinear", (a2, b2))]:
    print(f"   {name:10s}: sum of squared errors = {np.sum((y - aa * np.exp(bb * x))**2)
    ↪ :.4f}")

# 2. Population of Ghana (census, millions)
year = np.array([1960, 1970, 1984, 2000, 2010, 2021])
pop = np.array([6.727, 8.559, 12.296, 18.912, 24.659, 30.832])
t = year - 1960
b, c = np.polyfit(t, np.log(pop), 1)
P0 = np.exp(c)
print(f"\nGhana: P(t) = {P0:.3f} exp({b:.5f} t),  t = years since 1960")
print(f"growth rate = {100 * b:.2f}% per year, doubling time = {np.log(2) / b:.1f} years"
    ↪ )
for yr in [1990, 2030]:
    print(f"model estimate for {yr}: {P0 * np.exp(b * (yr - 1960)):.2f} million")

tt = np.linspace(0, 75, 200)
fig, ax = plt.subplots(1, 2, figsize=(8.5, 3.2))
ax[0].plot(year, pop, "o", label="census")
ax[0].plot(1960 + tt, P0 * np.exp(b * tt), label="exponential fit")
ax[0].set_xlabel("year")
ax[0].set_ylabel("millions")
ax[0].legend()
ax[1].semilogy(year, pop, "o")
ax[1].semilogy(1960 + tt, P0 * np.exp(b * tt))
ax[1].set_xlabel("year")
ax[1].set_title("log scale: a straight line", fontsize=10)
for a_ in ax:
    a_.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch10_ghana.pdf")
```

Output

```
linearised fit : y = 87.7491 exp(-0.16364 x)
nonlinear fit  : y = 107.8296 exp(-0.18710 x)
   linearised: sum of squared errors = 11.1596
   nonlinear : sum of squared errors = 6.9445

Ghana: P(t) = 6.711 exp(0.02548 t),  t = years since 1960
growth rate = 2.55% per year, doubling time = 27.2 years
model estimate for 1990: 14.41 million
model estimate for 2030: 39.94 million
```

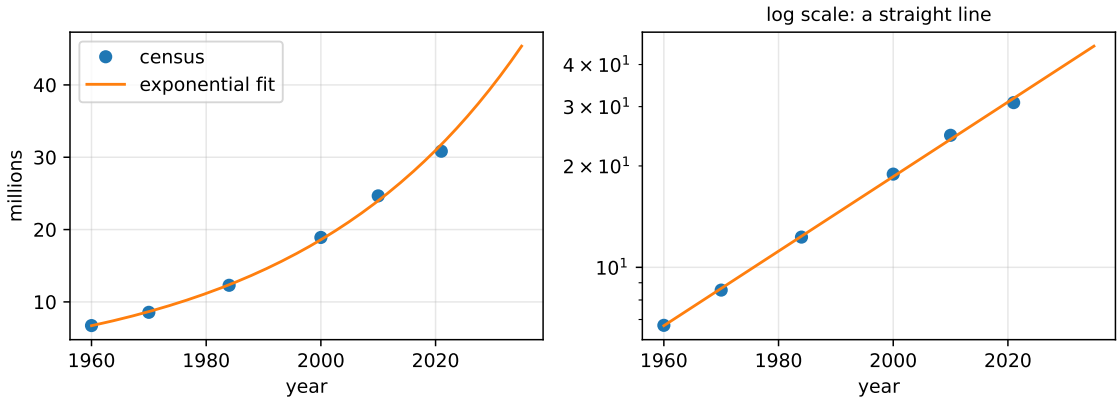


Figure 10.2: Exponential growth model for the population of Ghana.

Exercises

10.2.1 Theory Fit $y = ax^b$ to $(1, 0.5)$, $(2, 2)$, $(3, 4.5)$, $(4, 8)$ by linearisation. (The data lie exactly on $y = x^2/2$.)

10.2.2 Python Compare the Ghana population predictions of the exponential model with the degree-5 interpolating polynomial of Chapter 9 for 1950, 1990 and 2030. Which model would you use, and why?

10.2.3 Python The *logistic model* $P(t) = K/(1 + ce^{-rt})$ has three parameters. Fit it to the Ghana data with `curve_fit`, using the exponential fit to choose starting values. What carrying capacity K does it predict? Is this reliable?

10.2.4 Python Kepler's third law says $T = aR^b$ with $b = 3/2$, where R is the mean distance of a planet from the Sun and T its period. Using R (in AU) $= 0.387, 0.723, 1, 1.524, 5.203, 9.537$ and T (in years) $= 0.241, 0.615, 1, 1.881, 11.86, 29.46$, estimate b by a log–log least-squares fit.

10.3 Continuous least squares

Now let f be a continuous function on $[a, b]$. We seek $P_n(x) = \sum_{j=0}^{n} a_j x^j$ minimising

$$E(a_0, \dots, a_n) = \|f - P_n\|_2^2 = \int_a^b \Big(f(x) - \sum_{j=0}^{n} a_j x^j \Big)^2 dx.$$

Setting $\partial E/\partial a_k = -2\int_a^b \big(f(x) - \sum_j a_j x^j\big)x^k dx = 0$ gives the normal equations

$$\sum_{j=0}^{n} a_j \int_a^b x^{j+k}\,dx = \int_a^b x^k f(x)\,dx, \qquad k = 0, 1, \ldots, n.$$

On $[0,1]$, $\int_0^1 x^{j+k}dx = \frac{1}{j+k+1}$, so the matrix is the **Hilbert matrix** $H_{ij} = 1/(i+j-1)$.

Example 10.5. Find the least-squares quadratic approximation of e^x on $[0,1]$. Since $\int_0^1 e^x dx = e-1$, $\int_0^1 xe^x dx = 1$ and $\int_0^1 x^2 e^x dx = e-2$,

$$\begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{pmatrix}\begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} e-1 \\ 1 \\ e-2 \end{pmatrix} \Longrightarrow\ P_2(x) = 1.0130 + 0.8511x + 0.8392x^2.$$

Listing 10.4: Continuous least squares in the monomial and Legendre bases
ch10/continuous_ls.py

```
# Continuous least squares: best quadratic approximation of e^x on [0,1]
import numpy as np
from scipy.integrate import quad
from numpy.polynomial import legendre as leg

# (a) monomial basis 1, x, x^2 -> normal equations with the Hilbert matrix
H = np.array([[1 / (i + j + 1) for j in range(3)] for i in range(3)])
rhs = np.array([quad(lambda x: x**k * np.exp(x), 0, 1)[0] for k in range(3)])
a = np.linalg.solve(H, rhs)
print("Hilbert matrix H_3 =\n", H)
print("right-hand side  =", rhs, " ( = [e-1, 1, e-2] )")
print("P2(x) = %.6f + %.6f x + %.6f x^2" % tuple(a))
xs = np.linspace(0, 1, 1001)
P = a[0] + a[1] * xs + a[2] * xs**2
err_ls = np.exp(xs) - P
print(f"max |e^x - P2|          = {np.max(np.abs(err_ls)):.3e}")
print(f"L2 error ||e^x - P2||_2 = {np.sqrt(quad(lambda x: (np.exp(x) - np.polyval(a
    ↪ [::-1], x))**2, 0, 1)[0]):.3e}")
T = 1 + xs + xs**2 / 2
print(f"Taylor 1 + x + x^2/2: max error = {np.max(np.abs(np.exp(xs) - T)):.3e}")

# (b) Legendre basis on [-1,1]: the normal equations are DIAGONAL
print("\ne^x on [-1,1] in Legendre polynomials: c_k = (2k+1)/2 * <f, P_k>")
for n in [2, 4, 6, 8]:
    c = [(2 * k + 1) / 2 * quad(lambda x: np.exp(x) * leg.legval(x, [0] * k + [1]), -1,
    ↪ 1)[0]
         for k in range(n + 1)]
    t = np.linspace(-1, 1, 2001)
    print(f"degree {n}: max error = {np.max(np.abs(np.exp(t) - leg.legval(t, c))):.3e}")
print("first coefficients:", np.round(c[:4], 6))
```

Output

```
Hilbert matrix H_3 =
 [[1.         0.5        0.33333333]
 [0.5        0.33333333 0.25      ]
 [0.33333333 0.25       0.2       ]]
right-hand side  = [1.71828183 1.         0.71828183]  ( = [e-1, 1, e-2] )
P2(x) = 1.012991 + 0.851125 x + 0.839184 x^2
max |e^x - P2|          = 1.498e-02
L2 error ||e^x - P2||_2 = 5.276e-03
Taylor 1 + x + x^2/2: max error = 2.183e-01

e^x on [-1,1] in Legendre polynomials: c_k = (2k+1)/2 * <f, P_k>
degree 2: max error = 8.163e-02
degree 4: max error = 1.207e-03
degree 6: max error = 8.158e-06
degree 8: max error = 3.136e-08
first coefficients: [1.175201 1.103638 0.357814 0.070456]
```

The maximum error 0.015 is spread over the whole interval, whereas the Taylor polynomial $1 + x + x^2/2$ of the same degree is excellent near 0 but has error 0.22 at $x = 1$.

△ Caution: the monomial basis is a bad basis

The Hilbert matrix is catastrophically ill-conditioned ($\kappa_2(H_{10}) \approx 1.6 \times 10^{13}$, Listing 8.2). The functions $1, x, x^2, \dots$ look increasingly alike on $[0,1]$, so the normal equations are nearly singular. Moreover, raising the degree from n to $n+1$ changes all the coefficients. Both problems disappear with an *orthogonal* basis.

10.4 Orthogonal polynomials

Definition 10.6 (inner product and orthogonality)**.** Let $w(x) > 0$ be a *weight function* on (a,b). For functions f, g define the inner product and norm

$$\langle f, g \rangle = \int_a^b w(x) f(x) g(x)\, dx, \qquad \|f\| = \sqrt{\langle f, f \rangle}.$$

Functions $\phi_0, \phi_1, \dots, \phi_n$ are *orthogonal* if $\langle \phi_j, \phi_k \rangle = 0$ for $j \neq k$, and *orthonormal* if in addition $\|\phi_k\| = 1$.

Theorem 10.7 (best approximation in an orthogonal basis)**.** *If $\phi_0, \dots, \phi_n$ are orthogonal (with ϕ_k of exact degree k), the polynomial $P = \sum_{k=0}^{n} c_k \phi_k$ minimising $\|f - P\|$ over $\mathbb{P}_n$ has*

$$c_k = \frac{\langle f, \phi_k \rangle}{\langle \phi_k, \phi_k \rangle}, \qquad k = 0, 1, \dots, n.$$

Proof. The normal equations are $\sum_j c_j \langle \phi_j, \phi_k \rangle = \langle f, \phi_k \rangle$. By orthogonality the matrix is *diagonal*, so each equation involves only c_k. □

⋆ Key Concept: why orthogonal polynomials are so useful

In an orthogonal basis the normal equations are diagonal: no linear system, no ill-conditioning, and increasing the degree from n to $n+1$ **only adds one new coefficient** c_{n+1}, leaving $c_0, \dots, c_n$ unchanged. This is exactly the idea of Fourier series, with polynomials instead of sines and cosines.

Theorem 10.8 (Gram–Schmidt and the three-term recurrence). *The monic orthogonal polynomials for a weight w on (a,b) are generated by $\phi_0(x)=1$, $\phi_1(x)=x-B_1$ and*

$$\phi_k(x) = (x - B_k)\phi_{k-1}(x) - C_k\phi_{k-2}(x), \qquad k \ge 2,$$

where $B_k = \dfrac{\langle x\phi_{k-1}, \phi_{k-1}\rangle}{\langle \phi_{k-1}, \phi_{k-1}\rangle}$ *and* $C_k = \dfrac{\langle x\phi_{k-1}, \phi_{k-2}\rangle}{\langle \phi_{k-2}, \phi_{k-2}\rangle}$. *Moreover ϕ_n has n distinct real zeros, all in (a,b).*

The zeros of orthogonal polynomials are the nodes of Gaussian quadrature (Section 13.4).

10.4.1 Legendre polynomials

For $w(x)=1$ on $[-1,1]$ the orthogonal polynomials are the **Legendre polynomials**

$$P_0 = 1, \quad P_1 = x, \quad P_2 = \tfrac{1}{2}(3x^2-1), \quad P_3 = \tfrac{1}{2}(5x^3-3x), \quad P_4 = \tfrac{1}{8}(35x^4-30x^2+3), \ \dots$$

normalised so that $P_n(1)=1$. They satisfy $(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x)$, Rodrigues' formula $P_n(x) = \frac{1}{2^n n!}\frac{d^n}{dx^n}(x^2-1)^n$, and $\langle P_n, P_n\rangle = \frac{2}{2n+1}$. On an interval $[a,b]$ use the substitution $x = \frac{2t-a-b}{b-a}$.

10.4.2 Chebyshev polynomials

For $w(x) = (1-x^2)^{-1/2}$ on $(-1,1)$ the orthogonal polynomials are the **Chebyshev polynomials**

$$T_n(x) = \cos(n \arccos x): \quad T_0 = 1,\ T_1 = x,\ T_2 = 2x^2-1,\ T_3 = 4x^3-3x, \quad T_{n+1} = 2xT_n - T_{n-1}.$$

With $x = \cos\theta$, orthogonality of T_n is just orthogonality of $\cos n\theta$ on $[0,\pi]$. T_n oscillates between -1 and $+1$ exactly $n+1$ times on $[-1,1]$, and $2^{1-n}T_n$ is the monic polynomial of degree n with the smallest maximum norm on $[-1,1]$ (Theorem 9.13). Chebyshev expansions are therefore nearly optimal in the maximum norm; this is the basis of `numpy.polynomial.chebyshev` and of Chebfun.

Listing 10.5: Legendre and Chebyshev polynomials: recurrences and orthogonality
`ch10/orthopoly.py`

```python
# Legendre and Chebyshev polynomials: recurrences, orthogonality and plots
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad


def legendre(n, x):
    """P_n(x) by the three-term recurrence (k+1) P_{k+1} = (2k+1) x P_k - k P_{k-1}."""
    p0, p1 = np.ones_like(x), x
    if n == 0:
        return p0
    for k in range(1, n):
        p0, p1 = p1, ((2 * k + 1) * x * p1 - k * p0) / (k + 1)
    return p1
```

```python


def chebyshev(n, x):
    """T_n(x) by the recurrence T_{k+1} = 2x T_k - T_{k-1}."""
    t0, t1 = np.ones_like(x), x
    if n == 0:
        return t0
    for _ in range(1, n):
        t0, t1 = t1, 2 * x * t1 - t0
    return t1


print("Gram matrix <P_i, P_j> on [-1,1] (should be diagonal with 2/(2i+1)):")
G = np.array([[quad(lambda x: legendre(i, x) * legendre(j, x), -1, 1)[0]
               for j in range(4)] for i in range(4)])
G[np.abs(G) < 1e-12] = 0.0                                  # hide rounding noise
print(np.round(G, 10))
print("Chebyshev Gram matrix with weight 1/sqrt(1-x^2):")
Gc = np.array([[quad(lambda th: chebyshev(i, np.cos(th)) * chebyshev(j, np.cos(th)), 0, np.
    ↪ pi)[0]
                for j in range(4)] for i in range(4)])     # substitution x = cos(theta)
Gc[np.abs(Gc) < 1e-12] = 0.0
print(np.round(Gc, 10))

x = np.linspace(-1, 1, 400)
fig, ax = plt.subplots(1, 2, figsize=(8.5, 3.2), sharey=True)
for n in range(6):
    ax[0].plot(x, legendre(n, x), label=f"$P_{n}$")
    ax[1].plot(x, chebyshev(n, x), label=f"$T_{n}$")
ax[0].set_title("Legendre polynomials", fontsize=10)
ax[1].set_title("Chebyshev polynomials", fontsize=10)
for a in ax:
    a.grid(alpha=0.3)
    a.legend(fontsize=7, ncol=3, loc="lower right")
plt.tight_layout()
plt.savefig("ch10_orthopoly.pdf")
```

Output

```
Gram matrix <P_i, P_j> on [-1,1] (should be diagonal with 2/(2i+1)):
[[2.         0.         0.         0.        ]
 [0.         0.66666667 0.         0.        ]
 [0.         0.         0.4        0.        ]
 [0.         0.         0.         0.28571429]]
Chebyshev Gram matrix with weight 1/sqrt(1-x^2):
[[3.14159265 0.         0.         0.        ]
 [0.         1.57079633 0.         0.        ]
 [0.         0.         1.57079633 0.        ]
 [0.         0.         0.         1.57079633]]
```

Historical Note: Legendre and Chebyshev

Legendre introduced his polynomials in 1782–85 while computing the gravitational attraction of ellipsoidal planets: they appear in the expansion $\frac{1}{|\mathbf{r}-\mathbf{r}'|} = \sum_{n} \frac{r'^n}{r^{n+1}} P_n(\cos\gamma)$ of the Newtonian potential.

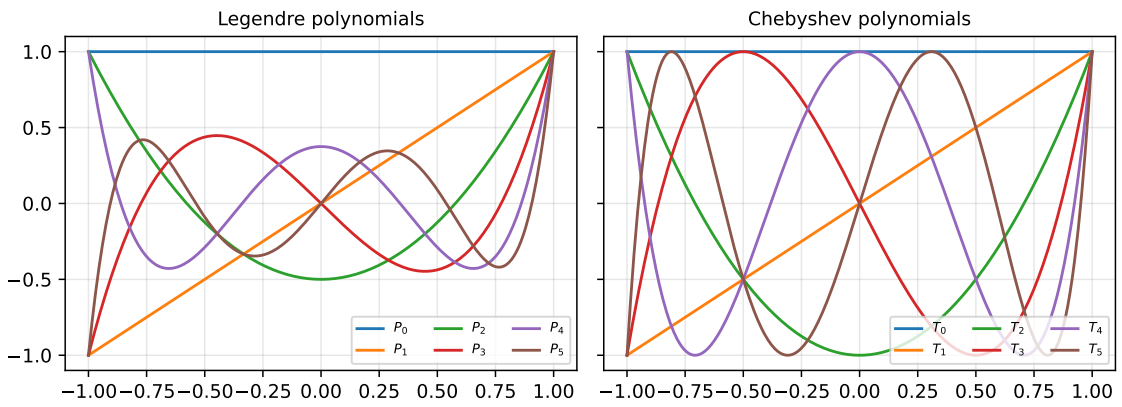


Figure 10.3: The first six Legendre and Chebyshev polynomials.

Pafnuty Chebyshev (St. Petersburg, 1821–1894) was led to his polynomials in 1854 by a very practical question: how to design the linkages of steam engines (Watt's parallel motion) so that a point moves as nearly as possible along a straight line, i.e. to minimise a *maximum* deviation. He founded the theory of best uniform approximation and the St. Petersburg school of mathematics (Markov, Lyapunov).

Towards Graduate Studies: Hilbert spaces, regression and machine learning

The theory of this chapter is the finite-dimensional shadow of the *projection theorem* in Hilbert spaces: in any inner product space the best approximation from a closed subspace is the orthogonal projection, and orthonormal expansions $f = \sum \langle f, \phi_k \rangle \, \phi_k$ (generalised Fourier series) converge in norm. This is the foundation of Fourier analysis, quantum mechanics, spectral methods for PDEs and the finite element method (Galerkin's method is a least-squares-type projection). In statistics and machine learning, least squares is linear regression; adding a penalty, $\min \|\mathbf{y} - A\mathbf{a}\|^2 + \lambda \|\mathbf{a}\|^2$, gives *ridge regression* (Tikhonov regularisation), and $\lambda \|\mathbf{a}\|_1$ gives the LASSO. See Trefethen [19] and Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning*.

Exercises

10.4.1 Theory Find the least-squares linear approximation of $f(x) = x^2$ on $[0, 1]$ (a) using the monomial basis, (b) using the shifted Legendre polynomials $1, 2x - 1$.

10.4.2 Theory Verify directly that P_0, P_1, P_2, P_3 are orthogonal on $[-1, 1]$ and that $\langle P_2, P_2 \rangle = 2/5$.

10.4.3 Theory Use Gram–Schmidt (Theorem 10.8) with $w = 1$ on $[0, 1]$ to construct ϕ_0, ϕ_1, ϕ_2, and use them to find the least-squares quadratic for e^x on $[0, 1]$. Compare with Example 10.5.

10.4.4 Theory Find the Legendre coefficients c_0, c_1, c_2 of e^x on $[-1, 1]$ and check against the output above ($c_0 = \sinh 1 = 1.175201$).

10.4.5 Theory Prove that $T_n(x) = \cos(n \arccos x)$ satisfies $T_{n+1} = 2xT_n - T_{n-1}$ (use $\cos(n+1)\theta + \cos(n-1)\theta = 2\cos\theta\cos n\theta$) and that $\int_{-1}^{1} \frac{T_m T_n}{\sqrt{1-x^2}} dx = 0$ for $m \neq n$.

10.4.6 Python Compute the least-squares approximations of $f(x) = |x|$ on $[-1, 1]$ in Legendre polynomials of degree $2, 4, \ldots, 20$ and plot $\|f - P_n\|_2$ and $\|f - P_n\|_\infty$ against n on a log–log scale. Estimate the rates.

10.4.7 ★ Graduate Prove that ϕ_n in Theorem 10.8 has n distinct zeros in (a, b). (Suppose it changes sign at only $k < n$ points $z_1, \dots, z_k$ and consider $\langle \phi_n, \prod (x - z_i) \rangle$.)

Chapter Summary

- Least squares minimises $\|\mathbf{y} - A\mathbf{a}\|_2^2$; normal equations $A^T A\mathbf{a} = A^T\mathbf{y}$; residual $\perp$ col(A).
- Prefer QR (`np.linalg.lstsq`, `np.polyfit`) to the normal equations: $\kappa(A^T A) = \kappa(A)^2$.
- Exponential/power models can be linearised with logarithms (weighted differently!).
- Continuous LS in monomials gives the ill-conditioned Hilbert matrix; orthogonal polynomials give diagonal normal equations with $c_k = \langle f, \phi_k \rangle / \langle \phi_k, \phi_k \rangle$.
- Legendre: $w = 1$; Chebyshev: $w = (1 - x^2)^{-1/2}$; both satisfy three-term recurrences.

Chapter 11

Approximation by Rational Functions

Learning Objectives

After studying this chapter you should be able to:

- explain the limitations of polynomial approximation and the advantages of rational functions;
- derive and solve the linear system for the Padé approximant $r_{n,m} = p_n/q_m$ of a function;
- compare Taylor and Padé approximations numerically;
- describe Chebyshev rational approximation and the idea of best (minimax) rational approximation.

Polynomials are smooth, have no poles, and grow without bound as $|x| \to \infty$. They therefore approximate poorly functions that have poles or near-singularities ($\tan x$ near $\pi/2$, $1/(1+25x^2)$), that tend to a finite limit (e^{-x}, $\arctan x$), or that change rapidly. A **rational function**

$$r(x) = \frac{p(x)}{q(x)} = \frac{p_0 + p_1 x + \cdots + p_n x^n}{q_0 + q_1 x + \cdots + q_m x^m}$$

can do all of these things. We normalise $q_0 = 1$, so r has $N+1$ free parameters, where $N = n + m$. Rational approximations are used inside computers and calculators to evaluate e^x, $\ln x$, $\tan x$, $\arctan x$ and many special functions.

11.1 Padé approximation

The Taylor polynomial of degree N is the polynomial that agrees with f and its first N derivatives at $x = 0$. The **Padé approximant** is the rational function that does the same.

Definition 11.1 (Padé approximant)**.** Let f have the Maclaurin series $f(x) = \sum_{i \ge 0} a_i x^i$. The Padé approximant $r_{n,m} = p/q$ with $\deg p \le n$, $\deg q \le m$, $q_0 = 1$, is defined by

$$f^{(k)}(0) = r^{(k)}(0), \qquad k = 0, 1, \dots, N = n + m,$$

equivalently $f(x)q(x) - p(x) = O(x^{N+1})$.

Deriving the equations. Since $q(0) = 1 \neq 0$, $f - r = \dfrac{fq - p}{q}$, and we need the coefficients of $x^0, x^1, \ldots, x^N$ in

$$f(x)q(x) - p(x) = \Big(\sum_{i=0}^{\infty} a_i x^i\Big)\Big(\sum_{j=0}^{m} q_j x^j\Big) - \sum_{k=0}^{n} p_k x^k$$

to vanish. The coefficient of x^k gives

$$\sum_{j=0}^{\min(k,m)} a_{k-j}\, q_j = p_k, \qquad k = 0, 1, \ldots, N, \tag{11.1}$$

with the convention $p_k = 0$ for $k > n$. The last m equations ($k = n+1, \ldots, N$) are a linear system for $q_1, \ldots, q_m$; the first $n+1$ equations then give $p_0, \ldots, p_n$ directly.

Example 11.2 (Padé approximant of e^{-x} with $n = 3$, $m = 2$)**.** The Maclaurin coefficients are $a_0 = 1$, $a_1 = -1$, $a_2 = \frac{1}{2}$, $a_3 = -\frac{1}{6}$, $a_4 = \frac{1}{24}$, $a_5 = -\frac{1}{120}$. Equations (11.1) for $k = 4, 5$ (where $p_4 = p_5 = 0$):

$$a_4 + a_3 q_1 + a_2 q_2 = 0: \ \ \tfrac{1}{24} - \tfrac{1}{6} q_1 + \tfrac{1}{2} q_2 = 0, \qquad a_5 + a_4 q_1 + a_3 q_2 = 0: \ \ -\tfrac{1}{120} + \tfrac{1}{24} q_1 - \tfrac{1}{6} q_2 = 0.$$

The solution is $q_1 = \frac{2}{5}$, $q_2 = \frac{1}{20}$. Then $p_0 = a_0 = 1$, $p_1 = a_1 + a_0 q_1 = -\frac{3}{5}$, $p_2 = a_2 + a_1 q_1 + a_0 q_2 = \frac{3}{20}$, $p_3 = a_3 + a_2 q_1 + a_1 q_2 = -\frac{1}{60}$. Hence

$$r_{3,2}(x) = \frac{1 - \frac{3}{5}x + \frac{3}{20}x^2 - \frac{1}{60}x^3}{1 + \frac{2}{5}x + \frac{1}{20}x^2}.$$

Listing 11.1: Padé approximation of e^{-x} in exact rational arithmetic and with SciPy ch11/pade.py

```
# Pade approximation of e^{-x}: r(x) = p(x)/q(x) with deg p = n, deg q = m
import math
from fractions import Fraction
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import pade


def pade_coefficients(a, n, m):
    """Pade approximant from Maclaurin coefficients a[0..n+m] (exact fractions).

    Solves  sum_{j=0}^{k} a_{k-j} q_j = p_k  (k = 0..n+m), with q_0 = 1,
    p_k = 0 for k > n and q_j = 0 for j > m."""
    N = n + m
    # equations k = n+1..N determine q_1..q_m
    M = [[a[k - j] if k - j >= 0 else Fraction(0) for j in range(1, m + 1)]
         for k in range(n + 1, N + 1)]
    rhs = [-a[k] for k in range(n + 1, N + 1)]
    # Gaussian elimination in exact rational arithmetic
    for c in range(m):
        piv = next(r for r in range(c, m) if M[r][c] != 0)
        M[c], M[piv] = M[piv], M[c]
        rhs[c], rhs[piv] = rhs[piv], rhs[c]
        for r in range(m):
            if r != c and M[r][c] != 0:
                f = M[r][c] / M[c][c]
                M[r] = [x - f * y for x, y in zip(M[r], M[c])]
                rhs[r] -= f * rhs[c]
    q = [Fraction(1)] + [rhs[i] / M[i][i] for i in range(m)]
    p = [sum(a[k - j] * q[j] for j in range(0, min(k, m) + 1)) for k in range(n + 1)]
    return p, q


a = [Fraction((-1) ** k, math.factorial(k)) for k in range(6)]     # e^{-x}
p, q = pade_coefficients(a, 3, 2)
print("p:", [str(c) for c in p])
print("q:", [str(c) for c in q])

# the same with SciPy (floating point); note: pade returns numpy poly1d objects
P, Q = pade([float(c) for c in a], 2)          # 2 = degree of the denominator
print("scipy p:", P.coeffs[::-1])
print("scipy q:", Q.coeffs[::-1] / Q.coeffs[-1])

r = lambda x: np.polyval([float(c) for c in p[::-1]], x) / np.polyval([float(c) for c in
    ↪ q[::-1]], x)
taylor5 = lambda x: sum(float(c) * x**k for k, c in enumerate(a))
print("\n  x      e^-x        Taylor P5    error       Pade r32     error")
for x in [0.2, 0.4, 0.6, 0.8, 1.0]:
    e = math.exp(-x)
    print(f"{x:.1f}  {e:.8f}  {taylor5(x):.8f}  {abs(e - taylor5(x)):.2e}  {r(x):.8f}  {
    ↪ abs(e - r(x)):.2e}")

xs = np.linspace(0, 3, 400)
plt.figure(figsize=(6, 3.4))
plt.semilogy(xs, np.abs(np.exp(-xs) - taylor5(xs)) + 1e-17, label="Taylor, degree 5")
plt.semilogy(xs, np.abs(np.exp(-xs) - r(xs)) + 1e-17, label="Pade [3/2]")
plt.xlabel("x")
plt.ylabel("absolute error")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch11_pade.pdf")
```

Output

```
p: ['1', '-3/5', '3/20', '-1/60']
q: ['1', '2/5', '1/20']
scipy p: [ 1.         -0.6         0.15       -0.01666667]
scipy q: [1.   0.4  0.05]

  x      e^-x        Taylor P5    error       Pade r32     error
0.2  0.81873075  0.81873067  8.64e-08  0.81873075  7.55e-09
0.4  0.67032005  0.67031467  5.38e-06  0.67031963  4.11e-07
0.6  0.54881164  0.54875200  5.96e-05  0.54880763  4.00e-06
0.8  0.44932896  0.44900267  3.26e-04  0.44930966  1.93e-05
1.0  0.36787944  0.36666667  1.21e-03  0.36781609  6.33e-05
```

Both approximations use the same six Taylor coefficients, but the Padé approximant is 10 to 20 times more accurate. Python's `fractions.Fraction` lets us reproduce the exact hand computation.

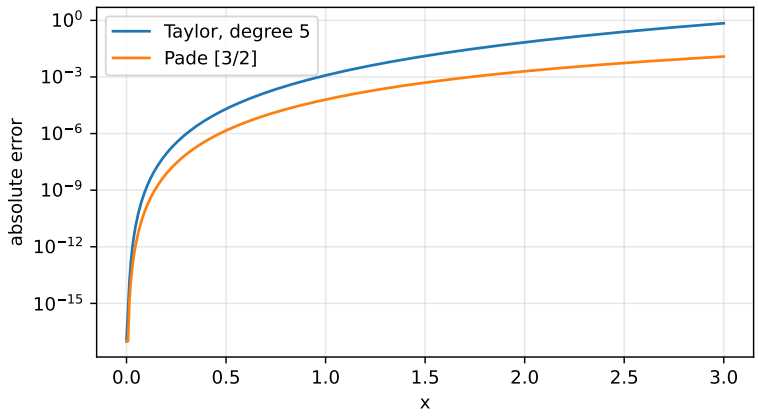


Figure 11.1: Errors of the Taylor polynomial of degree 5 and of the Padé approximant $r_{3,2}$ for e^{-x} on $[0, 3]$.

⋆ Key Concept: why rational approximations work well

A polynomial can only grow or oscillate; a rational function can also *level off* and *blow up*. The Padé approximant of $\tan x$ has poles very close to $\pm\pi/2$, so it follows $\tan x$ all the way up to the singularity, where the Taylor polynomial fails completely:

Listing 11.2: Padé approximation reproduces the pole of $\tan x$ ch11/pade_tan.py

```
# Rational functions can reproduce poles: tan(x) near pi/2
import numpy as np
from scipy.interpolate import pade

# Maclaurin coefficients of tan x = x + x^3/3 + 2x^5/15 + 17x^7/315 + 62x^9/2835
a = [0, 1, 0, 1 / 3, 0, 2 / 15, 0, 17 / 315, 0, 62 / 2835]
p, q = pade(a, 4)                       # numerator degree 5, denominator degree 4
taylor = np.poly1d(a[::-1])
print("denominator roots (poles of r):", np.round(np.sort(q.roots.real), 6))
```

```python
print("pi/2 =", np.pi / 2)
print("\n   x        tan x          Taylor(9)       Pade[5/4]")
for x in [0.5, 1.0, 1.3, 1.5, 1.55]:
    print(f"{x:5.2f}  {np.tan(x):14.8f}  {taylor(x):14.8f}  {p(x) / q(x):14.8f}")
```

Output

```
denominator roots (poles of r): [-5.052976 -1.570808  1.570808  5.052976]
pi/2 = 1.5707963267948966

   x        tan x          Taylor(9)       Pade[5/4]
 0.50      0.54630249      0.54629767      0.54630249
 1.00      1.55740772      1.54250441      1.55740741
 1.30      3.60210245      3.09794843      3.60208073
 1.50     14.10141995      5.40033482     14.10000000
 1.55     48.07848248      6.27351693     48.05526984
```

△ Caution: things that can go wrong

The linear system for $q_1, \dots, q_m$ may be singular (then the Padé approximant of that type does not exist), and the denominator q may vanish inside the interval of interest, producing a spurious pole. Always check the zeros of q. The Padé approximant is a *local* approximation (like Taylor), so its error grows away from $x = 0$.

Historical Note: Padé, Frobenius and continued fractions

Rational approximations of this kind appear in the work of Lagrange and in **Georg Frobenius** (1881), but the systematic theory, including the two-dimensional *Padé table* of all approximants $r_{n,m}$, was the subject of the 1892 doctoral thesis of **Henri Padé**, a student of Charles Hermite in Paris. Padé approximants are closely connected with continued fractions, which Euler used to study e, and with **Hermite**'s 1873 proof that e is transcendental. Today they are used in the algorithm for the matrix exponential `scipy.linalg.expm` (the "scaling and squaring" method uses a Padé approximant of degree [13/13]), in control theory to approximate time delays, and in theoretical physics to sum divergent series.

Exercises

11.1.1 Theory Find the Padé approximants $r_{1,1}$ and $r_{2,2}$ of e^x and compare them with the Taylor polynomials of degrees 2 and 4 at $x = 0.5$ and $x = 1$.

11.1.2 Theory Show that $r_{n,0}$ is the Taylor polynomial of degree n, and that $r_{0,1}$ of e^x is $1/(1-x)$.

11.1.3 Theory Find $r_{2,2}$ for $f(x) = \ln(1+x)/x$ and use it to approximate $\ln 2$.

11.1.4 Theory Find $r_{1,1}$ for $\arctan x$ (the Maclaurin series is $x - x^3/3 + \dots$) and compare with $\arctan 1 = \pi/4$.

11.1.5 Python Use `scipy.interpolate.pade` to compute $r_{m,m}$ of e^x for $m = 1, \dots, 6$ and plot the maximum error on $[-1, 1]$ against m on a semilog scale. Compare with Taylor polynomials of degree $2m$.

11.1.6 ★ Graduate Show that the diagonal Padé approximants of e^x satisfy $r_{m,m}(-x) = 1/r_{m,m}(x)$, and that $|r_{m,m}(z)| < 1$ for $\operatorname{Re} z < 0$. (This makes them *A-stable* when used as time-stepping methods for stiff ODEs: $r_{1,1}$ is the trapezoidal rule.)

11.2 Chebyshev rational approximation

Padé approximants concentrate their accuracy at $x = 0$. For a good approximation on a whole interval $[-1, 1]$ we replace the powers x^k by Chebyshev polynomials $T_k(x)$ (Section 10.4), whose "equal ripple" behaviour spreads the error evenly. Write

$$f(x) = \sum_{k=0}^{\infty} a_k T_k(x), \qquad r_T(x) = \frac{\sum_{k=0}^{n} p_k T_k(x)}{\sum_{k=0}^{m} q_k T_k(x)}, \quad q_0 = 1,$$

and require that the Chebyshev coefficients of $T_0, \dots, T_N$ in $f(x)q(x) - p(x)$ vanish. Products of Chebyshev polynomials are computed with the identity

$$T_i(x)T_j(x) = \tfrac{1}{2}\big(T_{i+j}(x) + T_{|i-j|}(x)\big),$$

which follows from $\cos i\theta \cos j\theta = \frac{1}{2}[\cos(i+j)\theta + \cos(i-j)\theta]$. The result is again a linear system of $N + 1$ equations in the $N + 1$ unknowns $p_0, \dots, p_n, q_1, \dots, q_m$.

Listing 11.3: Chebyshev rational approximation of e^{-x} on $[-1, 1]$ ch11/cheb_rational.py

```
# Chebyshev rational approximation of e^{-x} on [-1, 1], compared with Pade
import math
import numpy as np
from numpy.polynomial import chebyshev as C

f = lambda x: np.exp(-x)
n, m = 3, 2
N = n + m
a = C.chebinterpolate(f, 30)            # (essentially exact) Chebyshev coefficients of f

# Unknowns: p_0..p_n and q_1..q_m (q_0 = 1). Require the Chebyshev coefficients
# of  f*q - p  to vanish for T_0, ..., T_N.
M = np.zeros((N + 1, n + 1 + m))
for j in range(n + 1):
    M[j, j] = -1.0                      # -p_j contributes to T_j
for j in range(1, m + 1):
    col = C.chebmul(a, [0] * j + [1])   # coefficients of f(x) * T_j(x)
    M[:, n + j] = col[:N + 1]           # f * T_j
rhs = -a[:N + 1]                        # from the q_0 = 1 term: f * T_0
sol = np.linalg.solve(M, rhs)
p, q = sol[:n + 1], np.concatenate([[1.0], sol[n + 1:]])
print("numerator   (Chebyshev coefficients):", np.round(p, 6))
print("denominator (Chebyshev coefficients):", np.round(q, 6))

x = np.linspace(-1, 1, 4001)
rT = C.chebval(x, p) / C.chebval(x, q)
# Pade [3/2] of e^{-x} about 0 (from pade.py)
pade = (1 - 3 * x / 5 + 3 * x**2 / 20 - x**3 / 60) / (1 + 2 * x / 5 + x**2 / 20)
taylor5 = sum((-x) ** k / math.factorial(k) for k in range(6))
cheb5 = C.chebval(x, a[:6])             # truncated Chebyshev series of degree 5
for name, g in [("Taylor degree 5", taylor5), ("Chebyshev series deg 5", cheb5),
                ("Pade [3/2]", pade), ("Chebyshev rational [3/2]", rT)]:
    print(f"{name:26s}: max error on [-1,1] = {np.max(np.abs(f(x) - g)):.3e}")
```

Output

```
numerator   (Chebyshev coefficients): [ 1.050031 -0.600434  0.073831 -0.004068]
denominator (Chebyshev coefficients): [1.       0.387972 0.023804]
Taylor degree 5           : max error on [-1,1] = 1.615e-03
Chebyshev series deg 5    : max error on [-1,1] = 4.839e-05
Pade [3/2]                : max error on [-1,1] = 3.331e-04
Chebyshev rational [3/2]  : max error on [-1,1] = 8.197e-06
```

With exactly the same number of parameters, the Chebyshev rational approximation is about 40 times more accurate than the Padé approximant on $[-1, 1]$, and 200 times more accurate than the Taylor polynomial.

11.2.1 Best approximation

The ultimate goal is the *best* (minimax) approximation, which minimises $\max_{x\in[a,b]} |f(x) - r(x)|$. Chebyshev proved that the best approximation is characterised by *equioscillation*: the error attains its maximum absolute value, with alternating signs, at least $N + 2$ times (for polynomials: $n + 2$ times). The **Remez algorithm** (Evgeny Remez, 1934) computes best approximations iteratively by forcing equioscillation. The Chebyshev rational approximation above is usually very close to the best one, and is much cheaper to compute.

Towards Graduate Studies: rational approximation today

Rational functions can approximate functions with singularities spectacularly well: for $|x|$ on $[-1, 1]$, the best polynomial of degree n has error $\approx 0.28/n$, but the best rational function of type (n, n) has error $\approx 8e^{-\pi\sqrt{n}}$ (Newman 1964; Stahl 1993). This root-exponential convergence underlies modern algorithms such as the *AAA algorithm* (Nakatsukasa, Sète & Trefethen, 2018), which builds rational approximations in barycentric form from data; *lightning* solvers for PDEs in domains with corners; rational Krylov methods in model order reduction; and the best rational approximations of e^{-x} used in exponential integrators. See Trefethen [19], Chapters 23–28.

Exercises

11.2.1 Theory Verify the product formula $T_i T_j = \frac{1}{2}(T_{i+j} + T_{|i-j|})$ for $i = 2$, $j = 3$ by direct multiplication.

11.2.2 Theory Find the Chebyshev coefficients a_0, a_1, a_2 of $f(x) = x^2$ and of $f(x) = x^3$. (Express x^k in terms of $T_0, \ldots, T_k$.)

11.2.3 Python Modify Listing 11.3 to compute the Chebyshev rational approximation of type $(2, 2)$ of $\arctan x$ on $[-1, 1]$ and of $1/(1 + 25x^2)$ (Runge's function). Compare with polynomial interpolation at Chebyshev nodes with the same number of parameters.

11.2.4 Python Plot the error $e^{-x} - r_T(x)$ of the Chebyshev rational approximation on $[-1, 1]$. Count the number of equioscillation points. How close is it to being a best approximation?

11.2.5 ★ Graduate Show that the best polynomial approximation of degree $n - 1$ to x^n on $[-1, 1]$ is $x^n - 2^{1-n}T_n(x)$, using the equioscillation theorem.

Chapter Summary

- Rational functions p_n/q_m can model poles and horizontal asymptotes; they have $N+1 = n+m+1$ free parameters.
- Padé: match the Maclaurin series to order N; solve $\sum_j a_{k-j} q_j = p_k$, $k = 0, \ldots, N$.
- Padé is usually much more accurate than the Taylor polynomial with the same number of coefficients.
- Chebyshev rational approximation spreads the error evenly over $[-1, 1]$; the best (minimax) approximation equioscillates and is computed by the Remez algorithm.

Chapter 12

Numerical Differentiation

Learning Objectives

After studying this chapter you should be able to:

- derive forward, backward, central and higher-order difference formulas from Taylor series and from interpolation;
- determine the order of accuracy of a formula, theoretically and experimentally;
- explain the conflict between truncation and rounding errors and choose a good step size h;
- apply Richardson extrapolation;
- explain why differentiating noisy data is ill-conditioned.

Derivatives are needed everywhere: velocities and accelerations from position data, rates of reaction, the Jacobian in Newton's method (Chapter 5), and above all the discretisation of differential equations, which we met in the bungee jumper (Section 1.7) and Poisson (Section 7.6) problems. When f is only known at discrete points, or is given by a complicated program, we approximate its derivatives from function values.

Historical Note: from Newton to Richardson's forecast factory

The derivative was defined by Newton and Leibniz as the limit of a difference quotient, so numerical differentiation is as old as calculus; Newton, Stirling and Gregory derived many of the finite-difference formulas still in use. In 1911 the English mathematician and meteorologist **Lewis Fry Richardson** used finite differences to compute stresses in a masonry dam, and introduced the idea of combining results with different step sizes, which he called the "deferred approach to the limit" (1927), now *Richardson extrapolation*. During the First World War, while serving as an ambulance driver in France, he spent weeks computing a six-hour weather forecast by hand with finite differences. The forecast was badly wrong (the reason was later found to be unbalanced initial data), but his 1922 book *Weather Prediction by Numerical Process* imagined a "forecast factory" of 64 000 human computers. His dream came true in 1950, when Charney, Fjørtoft and von Neumann made the first computer weather forecast on the ENIAC.

12.1 Finite-difference formulas

Let $h > 0$ be a small step. By Taylor's theorem,

$$f(x+h) = f(x) + hf'(x) + \tfrac{h^2}{2}f''(x) + \tfrac{h^3}{6}f'''(x) + \tfrac{h^4}{24}f^{(4)}(x) + \cdots, \tag{12.1}$$

$$f(x-h) = f(x) - hf'(x) + \tfrac{h^2}{2}f''(x) - \tfrac{h^3}{6}f'''(x) + \tfrac{h^4}{24}f^{(4)}(x) - \cdots. \tag{12.2}$$

Forward and backward differences. From (12.1),

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2} f''(\xi), \qquad f'(x) = \frac{f(x) - f(x-h)}{h} + \frac{h}{2} f''(\eta).$$

Both are *first-order* accurate: the truncation error is $O(h)$.

Central difference. Subtracting (12.2) from (12.1), the even-order terms cancel:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6} f'''(\xi). \tag{12.3}$$

This is *second-order* accurate, at the same cost of two function evaluations.

Second derivative. Adding (12.1) and (12.2), the odd-order terms cancel:

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} - \frac{h^2}{12} f^{(4)}(\xi). \tag{12.4}$$

Five-point formula. Combining central differences with steps h and $2h$ to cancel the h^2 term gives the fourth-order formula

$$f'(x) = \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h} + \frac{h^4}{30} f^{(5)}(\xi).$$

Derivation by interpolation. Differentiating the Lagrange interpolating polynomial through $(x_j, f(x_j))$ gives $f'(x) \approx \sum_j f(x_j) L_j'(x)$, and the error follows from (9.3). This is how formulas for unequally spaced points and one-sided formulas are derived, e.g. the three-point end-point formula

$$f'(x_0) = \frac{-3f(x_0) + 4f(x_0+h) - f(x_0+2h)}{2h} + \frac{h^2}{3} f'''(\xi).$$

> **Example 12.1.** Let $f(x) = xe^x$, so $f'(2) = 3e^2 = 22.167168$ and $f''(2) = 4e^2 = 29.556224$. With $h = 0.1$: forward difference $(f(2.1) - f(2))/0.1 = (17.148957 - 14.778112)/0.1 = 23.708$ (error 1.54); central difference $(f(2.1) - f(1.9))/0.2 = (17.148957 - 12.703199)/0.2 = 22.228790$ (error 0.0616).

Listing 12.1: Orders of accuracy of difference formulas ch12/fd_formulas.py

```python
# Finite-difference formulas and their orders of accuracy
import numpy as np

f = lambda x: x * np.exp(x)
df = lambda x: (x + 1) * np.exp(x)          # exact f'
d2f = lambda x: (x + 2) * np.exp(x)         # exact f''
x0 = 2.0

forward = lambda h: (f(x0 + h) - f(x0)) / h
backward = lambda h: (f(x0) - f(x0 - h)) / h
central = lambda h: (f(x0 + h) - f(x0 - h)) / (2 * h)
five_point = lambda h: (f(x0 - 2 * h) - 8 * f(x0 - h) + 8 * f(x0 + h) - f(x0 + 2 * h)) /
    ↪ (12 * h)
second = lambda h: (f(x0 + h) - 2 * f(x0) + f(x0 - h)) / h**2

print(f"f(x) = x e^x at x0 = 2:  f'(2) = {df(x0):.10f},  f''(2) = {d2f(x0):.10f}\n")
print("    h       forward     central     5-point    2nd deriv   (errors)")
prev = None
for h in [0.2, 0.1, 0.05, 0.025, 0.0125]:
    errs = [abs(forward(h) - df(x0)), abs(central(h) - df(x0)),
            abs(five_point(h) - df(x0)), abs(second(h) - d2f(x0))]
    print(f"{h:7.4f}  " + "  ".join(f"{e:10.3e}" for e in errs))
    if prev is not None:
        print("  ratio  " + "  ".join(f"{p / e:10.2f}" for p, e in zip(prev, errs)))
    prev = errs
print("\nexpected ratios when h is halved: 2 (O(h)), 4 (O(h^2)), 16 (O(h^4)), 4 (O(h^2))"
    ↪ )
```

Output

```
f(x) = x e^x at x0 = 2:  f'(2) = 22.1671682968,  f''(2) = 29.5562243957

    h       forward     central     5-point    2nd deriv   (errors)
 0.2000   3.217e+00   2.470e-01   2.776e-03   1.480e-01
 0.1000   1.541e+00   6.162e-02   1.727e-04   3.696e-02
  ratio        2.09        4.01       16.07        4.01
 0.0500   7.545e-01   1.540e-02   1.078e-05   9.237e-03
  ratio        2.04        4.00       16.02        4.00
 0.0250   3.733e-01   3.849e-03   6.735e-07   2.309e-03
  ratio        2.02        4.00       16.00        4.00
 0.0125   1.857e-01   9.621e-04   4.209e-08   5.773e-04
  ratio        2.01        4.00       16.00        4.00

expected ratios when h is halved: 2 (O(h)), 4 (O(h^2)), 16 (O(h^4)), 4 (O(h^2))
```

Each time h is halved the errors fall by factors of 2, 4, 16 and 4, confirming the orders 1, 2, 4 and 2.

★ Key Concept: checking the order of a method

If $E(h) \approx Ch^p$ then $E(h)/E(h/2) \approx 2^p$, i.e. $p \approx \log_2\big(E(h)/E(h/2)\big)$. On a log–log plot of error against h, a method of order p appears as a straight line of slope p. This test should be applied to *every* numerical method you implement: it is the most effective way of finding

bugs.

Exercises

12.1.1 Theory Derive the backward difference formula and its error term.

12.1.2 Theory Derive the three-point end-point formula above from the Lagrange polynomial through x_0, x_0+h, x_0+2h.

12.1.3 Theory Derive the five-point formula by applying Richardson extrapolation (Section 12.3) to (12.3).

12.1.4 Theory Use the most accurate formula possible to estimate $f'(x)$ at each point of the table

x	1.1	1.2	1.3	1.4
$f(x)$	9.025013	11.02318	13.46374	16.44465

(the values are e^{2x}; compute the errors).

12.1.5 Python Verify the orders of the formulas in Listing 12.1 for $f(x)=\sin x$ at $x=\pi/4$ by producing a log–log plot of the errors against h and measuring slopes.

12.1.6 Theory Derive a second-order approximation for the mixed derivative $f_{xy}(x,y)$ using the four points $(x\pm h, y\pm h)$.

12.2 Rounding error and the choice of h

The truncation error suggests taking h as small as possible. But the difference $f(x+h)-f(x)$ subtracts nearly equal numbers: *catastrophic cancellation* (Section 3.3). If f is computed with relative error at most ε, each value carries an absolute error of about $\varepsilon|f|$, and the total error of the forward difference is roughly

$$E(h) \approx \underbrace{\frac{h}{2}|f''(x)|}_{\text{truncation}} + \underbrace{\frac{2\varepsilon|f(x)|}{h}}_{\text{rounding}}.$$

The first term decreases and the second increases as $h\to 0$. Minimising over h gives

$$h_{\text{opt}} = 2\sqrt{\frac{\varepsilon|f|}{|f''|}} \sim \sqrt{\varepsilon} \approx 10^{-8}, \qquad E(h_{\text{opt}}) \sim \sqrt{\varepsilon} \approx 10^{-8}.$$

For the central difference, $E(h)\approx \frac{h^2}{6}|f'''| + \frac{\varepsilon|f|}{h}$, giving $h_{\text{opt}}\sim \varepsilon^{1/3}\approx 10^{-5}$ and $E\sim\varepsilon^{2/3}\approx 10^{-11}$.

Listing 12.2: Total error of difference quotients as a function of h ch12/roundoff_h.py

```
# Truncation error versus rounding error: the best step size h
import numpy as np
import matplotlib.pyplot as plt

f, df, x0 = np.sin, np.cos, 1.0
hs = 10.0 ** np.arange(-1, -16.5, -0.25)
err_fwd = [abs((f(x0 + h) - f(x0)) / h - df(x0)) for h in hs]
err_cen = [abs((f(x0 + h) - f(x0 - h)) / (2 * h) - df(x0)) for h in hs]
```

```

i, j = int(np.argmin(err_fwd)), int(np.argmin(err_cen))
eps = np.finfo(float).eps
print(f"forward : best h = {hs[i]:.1e}, error = {err_fwd[i]:.1e}   (theory: h ~ sqrt(eps) = {np.sqrt(eps):.1e})")
print(f"central : best h = {hs[j]:.1e}, error = {err_cen[j]:.1e}   (theory: h ~ eps^(1/3) = {eps ** (1 / 3):.1e})")
for h in [1e-2, 1e-5, 1e-8, 1e-11, 1e-14]:
    print(f"h = {h:.0e}: forward error {abs((f(x0 + h) - f(x0)) / h - df(x0)):.2e}, "
          f"central error {abs((f(x0 + h) - f(x0 - h)) / (2 * h) - df(x0)):.2e}")

plt.figure(figsize=(6, 3.8))
plt.loglog(hs, err_fwd, "o-", ms=3, label="forward difference")
plt.loglog(hs, err_cen, "s-", ms=3, label="central difference")
plt.loglog(hs, 0.5 * hs * abs(np.sin(x0)), "k:", lw=1, label="truncation ~ h")
plt.loglog(hs, eps / hs, "k--", lw=1, label="rounding ~ eps/h")
plt.ylim(1e-12, 1)
plt.xlabel("h")
plt.ylabel("error in f'(1)")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch12_roundoff.pdf")
```

Output

```
forward : best h = 5.6e-09, error = 2.5e-09   (theory: h ~ sqrt(eps) = 1.5e-08)
central : best h = 5.6e-06, error = 3.8e-13   (theory: h ~ eps^(1/3) = 6.1e-06)
h = 1e-02: forward error 4.22e-03, central error 9.00e-06
h = 1e-05: forward error 4.21e-06, central error 1.11e-11
h = 1e-08: forward error 2.97e-09, central error 2.58e-09
h = 1e-11: forward error 1.17e-06, central error 1.17e-06
h = 1e-14: forward error 3.71e-03, central error 3.71e-03
```

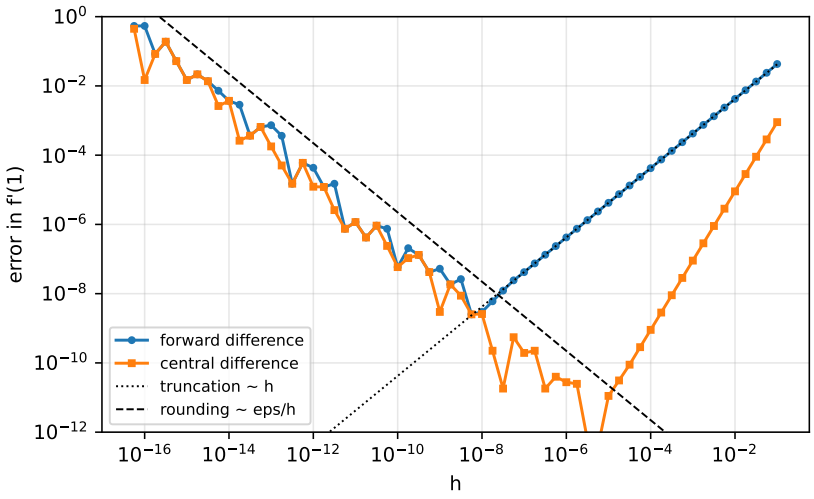


Figure 12.1: The "V-shaped" error curve of numerical differentiation for $f = \sin$ at $x = 1$. To the right of the minimum the truncation error dominates (slopes 1 and 2); to the left, rounding error dominates (slope -1).

△ Caution: numerical differentiation is ill-conditioned

Unlike integration, which averages out errors, differentiation *amplifies* them: an error δ in the data produces an error of order δ/h in the derivative. With double precision the best attainable accuracy for f' is about 8 digits (forward) or 11 digits (central), never 16. For measured data with noise level $\delta \gg u$ the situation is much worse, and the data should first be smoothed, for example by a least-squares fit (Chapter 10), and the fit differentiated.

Listing 12.3: Differentiating noisy velocity measurements ch12/noisy_data.py

```python
# Differentiating noisy data amplifies the noise
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(407)                # fixed seed: reproducible output
t = np.linspace(0, 10, 101)                     # time (s), h = 0.1
g, m, cd = 9.81, 68.1, 0.25
v_true = np.sqrt(g * m / cd) * np.tanh(np.sqrt(g * cd / m) * t)
a_true = g - cd / m * v_true**2                 # exact acceleration dv/dt
v_meas = v_true + rng.normal(0, 0.05, t.size)   # measured velocity, noise 5 cm/s

h = t[1] - t[0]
a_fd = np.gradient(v_meas, h)                   # central differences (one-sided at the
    ↪ ends)
coef = np.polyfit(t, v_meas, 6)                 # smooth first by least squares ...
a_ls = np.polyval(np.polyder(coef), t)          # ... then differentiate the fit
print(f"noise in v             : {0.05:.3f} m/s")
print(f"max error, central diff: {np.max(np.abs(a_fd - a_true)):.3f} m/s^2")
print(f"max error, LS fit      : {np.max(np.abs(a_ls - a_true)):.3f} m/s^2")
print(f"expected noise amplification ~ noise/h = {0.05 / h:.2f}")

fig, ax = plt.subplots(1, 2, figsize=(8.5, 3.2))
ax[0].plot(t, v_meas, ".", ms=3, label="measured v")
ax[0].plot(t, v_true, "k", lw=1, label="true v")
ax[0].set_xlabel("t (s)")
ax[0].legend(fontsize=8)
ax[1].plot(t, a_fd, ".", ms=3, label="central differences")
ax[1].plot(t, a_ls, "C2", label="derivative of LS fit")
ax[1].plot(t, a_true, "k", lw=1, label="true dv/dt")
ax[1].set_xlabel("t (s)")
ax[1].legend(fontsize=8)
for a in ax:
    a.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch12_noisy.pdf")
```

Output

```
noise in v             : 0.050 m/s
max error, central diff: 0.866 m/s^2
max error, LS fit      : 0.142 m/s^2
expected noise amplification ~ noise/h = 0.50
```

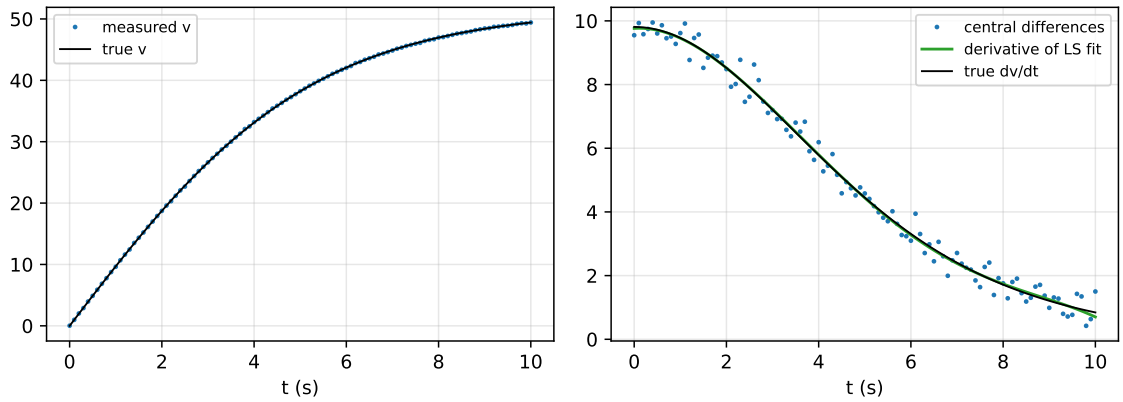


Figure 12.2: A noise level of 5 cm/s, invisible in the velocity data (left), dominates the finite-difference acceleration (right).

Exercises

12.2.1 Theory Derive h_{opt} for the central difference and for the second-derivative formula (12.4). What accuracy can be expected for f'' in double precision?

12.2.2 Python Repeat Listing 12.2 in single precision (`np.float32`). Where is the minimum now?

12.2.3 Python Redo Listing 12.3 with noise levels 0.005, 0.05 and 0.5, and with $h = 0.1$ and $h = 0.01$. Explain the results.

12.3 Richardson extrapolation

Suppose a quantity M is approximated by $N(h)$ with an error expansion in powers of h:

$$M = N(h) + K_1 h^p + K_2 h^q + \cdots, \qquad p < q.$$

Writing the same with $h/2$ and eliminating K_1 gives an approximation of higher order:

$$M = \underbrace{N(h/2) + \frac{N(h/2) - N(h)}{2^p - 1}}_{N_2(h)} + O(h^q). \tag{12.5}$$

For the central difference, $p = 2$ and the error contains only even powers of h, so repeated extrapolation removes $h^2, h^4, h^6, \ldots$ one at a time:

$$R_{i,j} = R_{i,j-1} + \frac{R_{i,j-1} - R_{i-1,j-1}}{4^j - 1}, \qquad R_{i,0} = N\left(h/2^i\right).$$

Example 12.2. For $f(x) = xe^x$ at $x = 2$ with $h = 0.4$ the central differences are $N(0.4) = 23.16346$ and $N(0.2) = 22.41416$, and

$$N_2 = 22.41416 + \frac{22.41416 - 23.16346}{3} = 22.16439,$$

reducing the error from 0.25 to 0.0028 at no extra cost.

Listing 12.4: The Richardson extrapolation table ch12/richardson.py

```python
# Richardson extrapolation of the central difference
import numpy as np


def richardson_table(D, h, levels):
    """R[i][0] = D(h / 2^i);  R[i][j] = R[i][j-1] + (R[i][j-1] - R[i-1][j-1]) / (4^j - 1).

    Valid when the error of D has an expansion in EVEN powers of h (central differences). """
    R = np.zeros((levels, levels))
    for i in range(levels):
        R[i, 0] = D(h / 2**i)
        for j in range(1, i + 1):
            R[i, j] = R[i, j - 1] + (R[i, j - 1] - R[i - 1, j - 1]) / (4**j - 1)
    return R


f = lambda x: x * np.exp(x)
x0, exact = 2.0, 3 * np.exp(2.0)
central = lambda h: (f(x0 + h) - f(x0 - h)) / (2 * h)
R = richardson_table(central, 0.4, 5)
print("Richardson table for f'(2), f(x) = x e^x, starting with h = 0.4")
for i in range(5):
    print("  ".join(f"{R[i, j]:.10f}" for j in range(i + 1)))
print(f"exact f'(2) = {exact:.10f}")
print("errors on the diagonal:", "  ".join(f"{abs(R[i, i] - exact):.1e}" for i in range(5)))
```

Output

```
Richardson table for f'(2), f(x) = x e^x, starting with h = 0.4
23.1634642931
22.4141606570  22.1643927783
22.2287868803  22.1669956214  22.1671691443
22.1825648578  22.1671575170  22.1671683100  22.1671682968
22.1710169319  22.1671676232  22.1671682970  22.1671682968  22.1671682968
exact f'(2) = 22.1671682968
errors on the diagonal: 1.0e+00 2.8e-03 8.5e-07 3.6e-11 8.2e-14
```

Five central differences, the best of which has error 4×10^{-3}, are combined into an approximation with error 10^{-13}.

⋆ Key Concept: extrapolation

Richardson extrapolation is a general principle, not a formula for derivatives. Whenever the error of a method has a known asymptotic expansion in h, results for several values of h can be combined to cancel the leading error terms. We use it again for integration (Romberg integration, Section 13.3); it is also used in ODE solvers (the Bulirsch–Stoer method).

Towards Graduate Studies: automatic differentiation

Two techniques avoid the rounding-error dilemma altogether. The *complex-step* method (Lyness & Moler, 1967; Squire & Trapp, 1998) uses $f'(x) \approx \operatorname{Im} f(x+ih)/h$ for a real-analytic f; there is no subtraction, so $h = 10^{-20}$ may be used and full machine accuracy is obtained. *Automatic differentiation* (AD) goes further: it applies the chain rule to each elementary operation of the program that computes f, and gives derivatives exact to rounding error. Reverse-mode AD ("backpropagation") computes the gradient of a function of millions of variables at the cost of a few function evaluations. It is the engine of deep learning (PyTorch, JAX, TensorFlow) and of modern optimisation and sensitivity analysis. See Griewank & Walther, *Evaluating Derivatives* (SIAM, 2008).

Listing 12.5: The complex-step derivative ch12/complex_step.py

```python
# The complex-step derivative: f'(x) ~ Im f(x + i h) / h, with no cancellation
import numpy as np

f = lambda x: np.exp(x) / np.sqrt(np.sin(x) ** 3 + np.cos(x) ** 3)   # Squire & Trapp's test function
x0 = 1.5
# reference value from a very accurate complex step
exact = np.imag(f(x0 + 1e-100j)) / 1e-100
print(f"f'(1.5) = {exact:.15f}")
print("     h      central difference error   complex step error")
for h in [1e-2, 1e-4, 1e-6, 1e-8, 1e-10, 1e-12, 1e-20]:
    cd = (f(x0 + h) - f(x0 - h)) / (2 * h)
    cs = np.imag(f(x0 + 1j * h)) / h
    print(f"{h:8.0e}   {abs(cd - exact):20.2e}   {abs(cs - exact):18.2e}")
```

Output

```
f'(1.5) = 4.053427893898621
     h      central difference error   complex step error
   1e-02                5.36e-04             5.36e-04
   1e-04                5.36e-08             5.36e-08
   1e-06                5.63e-10             5.36e-12
   1e-08                3.39e-08             8.88e-16
   1e-10                8.10e-07             8.88e-16
   1e-12                2.18e-04             8.88e-16
   1e-20                4.05e+00             0.00e+00
```

Exercises

12.3.1 Theory Apply one step of Richardson extrapolation to the *forward* difference ($p = 1$) and show that the result is the three-point end-point formula.

12.3.2 Theory Given $N(0.2) = 2.2362$, $N(0.1) = 2.2100$ and $N(0.05) = 2.2028$, where N has an error expansion in even powers of h, build the Richardson table and estimate M.

12.3.3 Python Use Richardson extrapolation on the second-derivative formula (12.4) for $f(x) = \cos x$ at $x = 0.5$.

12.3.4 ★ Graduate Prove that the complex-step formula has error $O(h^2)$ by expanding $f(x+ih)$ in a Taylor series, and explain why there is no cancellation.

12.3.5 ★ Graduate Implement forward-mode automatic differentiation with *dual numbers* $a + b\varepsilon$, $\varepsilon^2 = 0$, as a Python class with overloaded `__add__`, `__mul__`, `__truediv__`, and functions `sin`, `exp`. Use it to compute $f'(1.5)$ for the function in Listing 12.5.

Chapter Summary

- Forward/backward differences: $O(h)$; central difference and f'' formula: $O(h^2)$; five-point: $O(h^4)$.
- Total error = truncation ($\downarrow$ as $h \to 0$) + rounding ($\uparrow$ as $h \to 0$); $h_{\text{opt}} \approx \sqrt{u}$ (forward), $u^{1/3}$ (central).
- Differentiation amplifies noise: smooth measured data before differentiating.
- Richardson: $N_2(h) = N(h/2) + \frac{N(h/2) - N(h)}{2^p - 1}$ raises the order at almost no cost.

Chapter 13

Numerical Integration and Adaptive Quadrature

Learning Objectives

After studying this chapter you should be able to:

- derive the trapezoidal and Simpson rules by integrating interpolating polynomials, with their error terms;
- define and determine the degree of precision of a quadrature rule;
- apply composite rules and choose the number of subintervals to meet an error tolerance;
- use Romberg integration and Gauss–Legendre quadrature;
- explain and implement adaptive quadrature, and use `scipy.integrate.quad`.

Most definite integrals cannot be evaluated in closed form: $\int_0^1 e^{-x^2}\,dx$, $\int_0^{\pi} \sqrt{1+\cos^2 x}\,dx$ (the length of a sine arch), or integrals of functions known only from measurements or from a computer program. A **quadrature rule** approximates an integral by a weighted sum of function values:

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n} w_i f(x_i),$$

with *nodes* x_i and *weights* w_i. Different choices of nodes and weights give the methods of this chapter.

Historical Note: from Archimedes to Simpson

Archimedes (c. 250 BC) computed the area of a parabolic segment by exhausting it with triangles, the first "quadrature" (the word originally meant constructing a square of equal area). In 1615 **Johannes Kepler**, annoyed by the way wine merchants measured the volume of barrels, published the *Keplersche Fassregel* (barrel rule), which is Simpson's rule. Bonaventura **Cavalieri** (1639) and James **Gregory** (1668) also knew it, and **Thomas Simpson**, a self-taught weaver's son who became a professor at the Royal Military Academy, published it in 1743. **Roger Cotes** derived the general formulas for equally spaced nodes; they were published in 1722, six years after his death at the age of 33. Newton said of him: "If he had lived, we might have known something."

13.1 Newton–Cotes formulas

Replace f by its interpolating polynomial P_n at equally spaced nodes and integrate:

$$\int_a^b f(x)\,dx \approx \int_a^b P_n(x)\,dx = \sum_{i=0}^{n} f(x_i)\underbrace{\int_a^b L_i(x)\,dx}_{w_i}.$$

Trapezoidal rule ($n=1$). Integrating the line through $(a,f(a))$ and $(b,f(b))$ and the error term $\frac{f''(\xi(x))}{2}(x-a)(x-b)$ of (9.3) (using the weighted mean value theorem, since $(x-a)(x-b)\le 0$):

$$\int_a^b f(x)\,dx = \frac{h}{2}\big[f(a)+f(b)\big] - \frac{h^3}{12}f''(\xi), \qquad h=b-a. \tag{13.1}$$

Simpson's rule ($n=2$). With the midpoint $m=(a+b)/2$ and $h=(b-a)/2$:

$$\int_a^b f(x)\,dx = \frac{h}{3}\big[f(a)+4f(m)+f(b)\big] - \frac{h^5}{90}f^{(4)}(\xi). \tag{13.2}$$

Remarkably, although Simpson's rule is based on a quadratic, its error involves $f^{(4)}$: it is exact for cubics as well.

> **Definition 13.1** (degree of precision)**.** The *degree of precision* of a quadrature rule is the largest integer d such that the rule is exact for all polynomials of degree $\le d$, i.e. for $1, x, \dots, x^d$.

Rule	formula (h = node spacing)	error	degree
Midpoint	$2h\,f(x_1)$, on $[x_0,x_2]$	$+\frac{h^3}{3}f''(\xi)$	1
Trapezoidal	$\frac{h}{2}(f_0+f_1)$	$-\frac{h^3}{12}f''(\xi)$	1
Simpson	$\frac{h}{3}(f_0+4f_1+f_2)$	$-\frac{h^5}{90}f^{(4)}(\xi)$	3
Simpson $\frac{3}{8}$	$\frac{3h}{8}(f_0+3f_1+3f_2+f_3)$	$-\frac{3h^5}{80}f^{(4)}(\xi)$	3
Boole	$\frac{2h}{45}(7f_0+32f_1+12f_2+32f_3+7f_4)$	$-\frac{8h^7}{945}f^{(6)}(\xi)$	5

Listing 13.1: Newton–Cotes rules and their degree of precision ch13/newton_cotes.py

```python
# Basic Newton-Cotes rules and their degree of precision
import numpy as np


def trapezoid(f, a, b):
    return (b - a) / 2 * (f(a) + f(b))


def simpson(f, a, b):
    m = (a + b) / 2
    return (b - a) / 6 * (f(a) + 4 * f(m) + f(b))


def simpson38(f, a, b):
    h = (b - a) / 3
    return 3 * h / 8 * (f(a) + 3 * f(a + h) + 3 * f(a + 2 * h) + f(b))
```

```python


def midpoint(f, a, b):
    return (b - a) * f((a + b) / 2)


rules = {"midpoint": midpoint, "trapezoid": trapezoid, "Simpson": simpson, "Simpson 3/8": simpson38}

print("Integral of e^x over [0, 1] = e - 1 =", np.e - 1)
for name, Q in rules.items():
    val = Q(np.exp, 0.0, 1.0)
    print(f"  {name:12s}: {val:.8f}   error = {abs(val - (np.e - 1)):.2e}")

print("\nDegree of precision: is x^k integrated exactly on [0, 1]?")
print("  k   " + "".join(f"{name:>13s}" for name in rules))
for k in range(6):
    exact = 1 / (k + 1)
    row = ["yes" if abs(Q(lambda x: x**k, 0.0, 1.0) - exact) < 1e-14 else "no" for Q in rules.values()]
    print(f"  {k}   " + "".join(f"{r:>13s}" for r in row))
```

Output

```
Integral of e^x over [0, 1] = e - 1 = 1.718281828459045
  midpoint    : 1.64872127   error = 6.96e-02
  trapezoid   : 1.85914091   error = 1.41e-01
  Simpson     : 1.71886115   error = 5.79e-04
  Simpson 3/8 : 1.71854015   error = 2.58e-04

Degree of precision: is x^k integrated exactly on [0, 1]?
  k       midpoint    trapezoid      Simpson  Simpson 3/8
  0            yes          yes          yes          yes
  1            yes          yes          yes          yes
  2             no           no          yes          yes
  3             no           no          yes          yes
  4             no           no           no           no
  5             no           no           no           no
```

Example 13.2. Using Simpson's rule on $[0,2]$: $\int_0^2 x^4 dx \approx \frac{1}{3}[0+4\cdot 1+16] = 6.667$ (exact 6.4), while $\int_0^2 x^3 dx \approx \frac{1}{3}[0+4+8] = 4$, which is exact, as the degree of precision 3 predicts.

Exercises

13.1.1 Theory Derive the weights of Simpson's rule by the *method of undetermined coefficients*: require $\int_{-h}^{h} p(x)dx = w_0 p(-h) + w_1 p(0) + w_2 p(h)$ for $p = 1, x, x^2$, and then show it also holds for x^3 but not x^4.

13.1.2 Theory Approximate $\int_0^{0.5} \frac{2}{x-4} dx$ by the trapezoidal and Simpson rules, and compare with the exact value $2\ln(3.5/4)$. Verify that the actual errors are within the error bounds.

13.1.3 Theory Find the degree of precision of $\int_{-1}^{1} f(x)dx \approx f(-\frac{1}{\sqrt{3}}) + f(\frac{1}{\sqrt{3}})$.

13.1.4 Theory Derive the error term of the trapezoidal rule (13.1) from the interpolation error

formula.

13.2 Composite rules

High-order Newton–Cotes formulas are not used (they suffer from Runge's phenomenon, and for $n \geq 8$ some weights are negative). Instead, divide $[a,b]$ into n subintervals of width $h = (b-a)/n$ with nodes $x_j = a + jh$, and apply a low-order rule on each.

Theorem 13.3 (composite trapezoidal and Simpson rules)**.** *Let $f_j = f(x_j)$. Then for some $\mu \in (a,b)$,*

$$\int_a^b f\,dx = \frac{h}{2}\Big[f_0 + 2\sum_{j=1}^{n-1} f_j + f_n\Big] - \frac{(b-a)h^2}{12} f''(\mu),$$

$$\int_a^b f\,dx = \frac{h}{3}\Big[f_0 + 4\sum_{j\ odd} f_j + 2\sum_{\substack{j\ even\\ 0<j<n}} f_j + f_n\Big] - \frac{(b-a)h^4}{180} f^{(4)}(\mu) \quad (n\ even).$$

Proof for the trapezoidal rule. Summing (13.1) over the n subintervals gives the error $-\frac{h^3}{12}\sum_j f''(\xi_j)$. By the intermediate value theorem $\frac{1}{n}\sum_j f''(\xi_j) = f''(\mu)$ for some μ, and $nh^3 = (b-a)h^2$. □

⋆ Key Concept: order of composite rules

One subinterval: the trapezoidal error is $O(h^3)$ (local). $n = (b-a)/h$ subintervals: the error is $O(h^2)$ (global). Composite trapezoid is second order, composite Simpson fourth order: halving h reduces the errors by factors 4 and 16.

Example 13.4. Determine n so that the composite rules approximate $\int_0^\pi \sin x\,dx = 2$ with error less than 2×10^{-5}.
Trapezoid: $|f''| \leq 1$, so we need $\frac{\pi h^2}{12} = \frac{\pi^3}{12n^2} < 2\times 10^{-5}$, i.e. $n \geq 360$. *Simpson:* $|f^{(4)}| \leq 1$, so we need $\frac{\pi h^4}{180} = \frac{\pi^5}{180n^4} < 2\times 10^{-5}$, i.e. $n \geq 18$. Simpson needs 20 times fewer function evaluations.

Listing 13.2: Composite trapezoidal and Simpson rules ch13/composite.py

```python
# Composite trapezoidal and Simpson rules: convergence rates
import numpy as np
import matplotlib.pyplot as plt


def composite_trapezoid(f, a, b, n):
    """n subintervals of width h = (b-a)/n."""
    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n
    return h * (0.5 * y[0] + y[1:-1].sum() + 0.5 * y[-1])


def composite_simpson(f, a, b, n):
    """n subintervals (n must be even)."""
    if n % 2:
        raise ValueError("n must be even for Simpson's rule")
    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n
    return h / 3 * (y[0] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum() + y[-1])


exact = 2.0                                     # integral of sin over [0, pi]
print("  n     trapezoid error  ratio   Simpson error   ratio")
ns = [2, 4, 8, 16, 32, 64, 128]
et, es = [], []
for n in ns:
    et.append(abs(composite_trapezoid(np.sin, 0, np.pi, n) - exact))
    es.append(abs(composite_simpson(np.sin, 0, np.pi, n) - exact))
    rt = f"{et[-2] / et[-1]:6.2f}" if len(et) > 1 else "      "
    rs = f"{es[-2] / es[-1]:6.2f}" if len(es) > 1 else "      "
    print(f"{n:4d}   {et[-1]:14.3e}  {rt}   {es[-1]:13.3e}  {rs}")

# A periodic integrand: the trapezoidal rule converges exponentially fast!
f = lambda x: np.exp(np.cos(x))
exact_p = 7.954926521012845                     # = 2 pi I_0(1)
print("\nperiodic integrand e^{cos x} on [0, 2 pi]:")
for n in [2, 4, 8, 12, 16]:
    print(f"  n = {n:2d}: trapezoid error = {abs(composite_trapezoid(f, 0, 2 * np.pi, n)
    ↪ - exact_p):.2e}")

hs = [np.pi / n for n in ns]
plt.figure(figsize=(5.5, 3.6))
plt.loglog(hs, et, "o-", label="composite trapezoid")
plt.loglog(hs, es, "s-", label="composite Simpson")
plt.loglog(hs, 0.2 * np.array(hs) ** 2, "k:", label="slope 2")
plt.loglog(hs, 0.01 * np.array(hs) ** 4, "k--", label="slope 4")
plt.xlabel("h")
plt.ylabel("error")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch13_composite.pdf")
```

Output

```
  n     trapezoid error  ratio   Simpson error   ratio
  2        4.292e-01               9.440e-02
  4        1.039e-01    4.13       4.560e-03   20.70
  8        2.577e-02    4.03       2.692e-04   16.94
 16        6.430e-03    4.01       1.659e-05   16.22
 32        1.607e-03    4.00       1.033e-06   16.06
 64        4.016e-04    4.00       6.453e-08   16.01
128        1.004e-04    4.00       4.032e-09   16.00

periodic integrand e^{cos x} on [0, 2 pi]:
  n =  2: trapezoid error = 1.74e+00
  n =  4: trapezoid error = 3.44e-02
  n =  8: trapezoid error = 1.25e-06
  n = 12: trapezoid error = 6.53e-12
  n = 16: trapezoid error = 8.88e-16
```

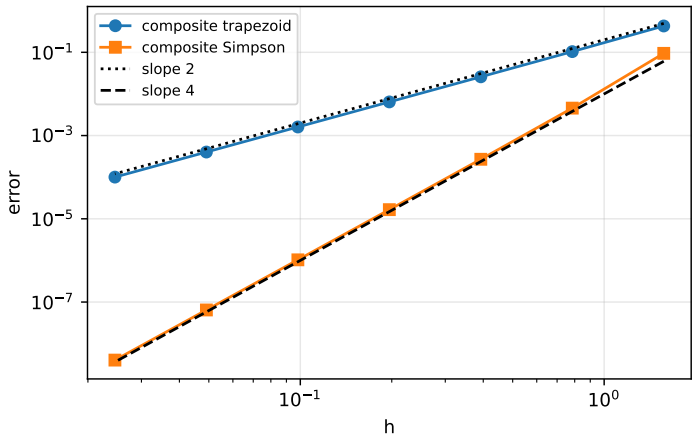


Figure 13.1: Errors of the composite rules for $\int_0^\pi \sin x\,dx$: slopes 2 and 4 on a log–log plot.

⋆ Key Concept: numerical integration is stable

Unlike differentiation, integration is *well conditioned*: the weights of the composite rules are positive and sum to $b - a$, so errors of size ε in the function values produce an error of at most $(b - a)\varepsilon$ in the integral, independent of h. We can take h as small as we like.

△ Caution: periodic functions: a pleasant surprise

The last lines of the output show the trapezoidal rule converging *exponentially* for the periodic function $e^{\cos x}$ on a full period: 16 points give full machine accuracy. The error expansion of the trapezoidal rule (the Euler–Maclaurin formula, Section 13.3) involves only the derivatives at the end points, and for periodic functions these terms all cancel. For periodic integrands, never use Simpson's rule: the trapezoidal rule is far better.

Exercises

13.2.1 Theory Use the composite trapezoidal rule with $n = 4$ and composite Simpson with $n = 4$ to approximate $\int_1^2 x \ln x \, dx$ and compare with the exact value $2\ln 2 - \frac{3}{4}$.

13.2.2 Theory How large must n be for the composite Simpson rule to approximate $\int_0^2 e^{2x} \sin 3x \, dx$ with error below 10^{-4}?

13.2.3 Python **Normal distribution.** The probability that a standard normal variable lies in $[-1.96, 1.96]$ is $\frac{2}{\sqrt{2\pi}} \int_0^{1.96} e^{-t^2/2} dt$. Compute it with composite Simpson to 8 digits and compare with `scipy.stats.norm.cdf`.

13.2.4 Python The length of one arch of $y = \sin x$ is $\int_0^{\pi} \sqrt{1 + \cos^2 x}\, dx$. Compute it to 10 digits with the composite trapezoidal rule. Explain why it converges so quickly.

13.2.5 Python A car's speed (km/h) was recorded every 6 minutes: 0, 42, 67, 80, 88, 90, 86, 71, 50, 22, 0. Estimate the distance travelled in the hour with the trapezoidal and Simpson rules. (This is integration of *data*.)

13.3 Romberg integration

Theorem 13.5 (Euler–Maclaurin formula). *If $f \in C^{2k+2}[a,b]$, the composite trapezoidal approximation $T(h)$ satisfies*

$$T(h) = \int_a^b f\,dx + c_1 h^2 + c_2 h^4 + \cdots + c_k h^{2k} + O(h^{2k+2}),$$

where $c_j = \frac{B_{2j}}{(2j)!}\big(f^{(2j-1)}(b) - f^{(2j-1)}(a)\big)$ (B_{2j} are the Bernoulli numbers) do not depend on h.

Since the error expansion contains only even powers of h, we can apply Richardson extrapolation (Section 12.3) repeatedly:

$$R_{k,0} = T\!\left(\frac{b-a}{2^k}\right), \qquad R_{k,j} = R_{k,j-1} + \frac{R_{k,j-1} - R_{k-1,j-1}}{4^j - 1}.$$

The column $R_{k,1}$ is exactly the composite Simpson rule; $R_{k,2}$ is composite Boole's rule. Each new row reuses all previous function values (only the new midpoints are evaluated).

Listing 13.3: Romberg integration ch13/romberg.py

```
# Romberg integration = composite trapezoid + Richardson extrapolation
import numpy as np


def romberg(f, a, b, levels):
    """R[k, 0] = trapezoid with 2^k subintervals, reusing old function values;
    R[k, j] = R[k, j-1] + (R[k, j-1] - R[k-1, j-1]) / (4^j - 1)."""
    R = np.zeros((levels, levels))
    h = b - a
    R[0, 0] = h / 2 * (f(a) + f(b))
    for k in range(1, levels):
        h /= 2
        new_points = a + h * np.arange(1, 2**k, 2)           # only the NEW midpoints
        R[k, 0] = R[k - 1, 0] / 2 + h * np.sum(f(new_points))
```

```
        for j in range(1, k + 1):
            R[k, j] = R[k, j - 1] + (R[k, j - 1] - R[k - 1, j - 1]) / (4**j - 1)
    return R


R = romberg(np.sin, 0, np.pi, 6)
print("Romberg table for the integral of sin x over [0, pi] (exact value 2):")
for k in range(6):
    print("  ".join(f"{R[k, j]:.10f}" for j in range(k + 1)))
print("error of R[5,5] =", abs(R[5, 5] - 2), " using", 2**5 + 1, "function values")
```

Output

```
Romberg table for the integral of sin x over [0, pi] (exact value 2):
0.0000000000
1.5707963268  2.0943951024
1.8961188979  2.0045597550  1.9985707318
1.9742316019  2.0002691699  1.9999831309  2.0000055500
1.9935703438  2.0000165910  1.9999997525  2.0000000163  1.9999999946
1.9983933610  2.0000010334  1.9999999962  2.0000000001  2.0000000000  2.0000000000
error of R[5,5] = 1.3216094885137863e-12  using 33 function values
```

With 33 function values Romberg integration achieves an error of 10^{-12}, whereas the composite trapezoidal rule with the same points has error 1.6×10^{-3}.

Historical Note: Euler, Maclaurin and Romberg

The Euler–Maclaurin formula was found independently by **Leonhard Euler** (1736) and **Colin Maclaurin** (1742), who used it in the other direction, to evaluate slowly convergent sums by integrals (Euler used it to compute $\sum 1/k^2 = \pi^2/6$ to 20 decimal places). The German astrophysicist **Werner Romberg**, who fled the Nazis and worked in the Soviet Union and Norway, published his method in 1955 in a four-page paper in Trondheim.

Exercises

13.3.1 Theory Compute $R_{2,2}$ by hand for $\int_0^1 x^2 e^{-x}\,dx$.

13.3.2 Theory Show that $R_{k,1} = \frac{4T(h/2) - T(h)}{3}$ equals the composite Simpson rule with step $h/2$.

13.3.3 Python Apply Romberg integration to $\int_0^1 \sqrt{x}\,dx$. Why does extrapolation help so little? (Consider the Euler–Maclaurin hypotheses.)

13.4 Gaussian quadrature

Newton–Cotes rules fix the nodes and choose the weights: $n+1$ free parameters. If we choose *both* the n nodes and the n weights, we have $2n$ parameters and can hope to integrate all polynomials of degree $2n-1$ exactly.

Example 13.6 (two-point Gauss rule). Find c_1, c_2, x_1, x_2 such that $\int_{-1}^{1} f(x)dx = c_1 f(x_1) + c_2 f(x_2)$ for $f = 1, x, x^2, x^3$:

$$c_1 + c_2 = 2, \quad c_1 x_1 + c_2 x_2 = 0, \quad c_1 x_1^2 + c_2 x_2^2 = \tfrac{2}{3}, \quad c_1 x_1^3 + c_2 x_2^3 = 0.$$

The solution is $c_1 = c_2 = 1$, $x_{1,2} = \mp\frac{1}{\sqrt{3}}$:

$$\int_{-1}^{1} f(x)\,dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right),$$

exact for all cubics, with only two function evaluations.

Theorem 13.7 (Gauss–Legendre quadrature). *Let $x_1, \dots, x_n$ be the zeros of the Legendre polynomial P_n (Section 10.4), and $w_i = \int_{-1}^{1} L_i(x)dx$. Then $\int_{-1}^{1} p(x)dx = \sum_{i=1}^{n} w_i p(x_i)$ for every polynomial of degree $\le 2n-1$. No n-point rule has a higher degree of precision. All weights are positive.*

Proof. Let $p \in \mathbb{P}_{2n-1}$. Divide by P_n: $p = qP_n + r$ with $q, r \in \mathbb{P}_{n-1}$. Since P_n is orthogonal to all polynomials of lower degree, $\int_{-1}^{1} qP_n\,dx = 0$, so $\int p = \int r$. On the other hand $p(x_i) = r(x_i)$ because $P_n(x_i) = 0$, and the rule is exact for $r \in \mathbb{P}_{n-1}$ (it is interpolatory). Hence $\sum w_i p(x_i) = \sum w_i r(x_i) = \int r = \int p$. □

An integral over $[a, b]$ is transformed to $[-1, 1]$ by $x = \frac{b-a}{2}t + \frac{a+b}{2}$:

$$\int_a^b f(x)\,dx = \frac{b-a}{2}\int_{-1}^{1} f\left(\frac{b-a}{2}t + \frac{a+b}{2}\right)dt \approx \frac{b-a}{2}\sum_{i=1}^{n} w_i f\left(\frac{b-a}{2}t_i + \frac{a+b}{2}\right).$$

Listing 13.4: Gauss–Legendre quadrature ch13/gauss.py

```python
# Gauss-Legendre quadrature
import numpy as np
from math import erf, sqrt, pi


def gauss_legendre(f, a, b, n):
    """n-point Gauss-Legendre rule on [a, b] (exact for polynomials of degree 2n-1)."""
    x, w = np.polynomial.legendre.leggauss(n)          # nodes/weights on [-1, 1]
    t = (b - a) / 2 * x + (a + b) / 2                  # change of variable
    return (b - a) / 2 * np.sum(w * f(t))


for n in [1, 2, 3, 4]:
    x, w = np.polynomial.legendre.leggauss(n)
    print(f"n = {n}: nodes = {np.round(x, 10)}, weights = {np.round(w, 10)}")

f = lambda x: np.exp(-x**2)
exact = sqrt(pi) / 2 * (erf(1.5) - erf(1.0))
print(f"\nintegral of exp(-x^2) over [1, 1.5] = {exact:.10f}")
for n in [2, 3, 4, 5]:
    g = gauss_legendre(f, 1.0, 1.5, n)
    print(f"  Gauss, n = {n}: {g:.10f}   error = {abs(g - exact):.2e}")
x = np.linspace(1.0, 1.5, 5)            # Simpson with 5 points, for comparison
s = 0.125 / 3 * (f(x[0]) + 4 * f(x[1]) + 2 * f(x[2]) + 4 * f(x[3]) + f(x[4]))
print(f"  Simpson, 5 points: {s:.10f}   error = {abs(s - exact):.2e}")

print("\nDegree of precision of the 3-point rule on [-1, 1]:")
for k in range(8):
    exact_k = (1 - (-1) ** (k + 1)) / (k + 1)
    val = gauss_legendre(lambda t: t**k, -1, 1, 3)
    print(f"  x^{k}: rule = {val:+.12f}, exact = {exact_k:+.12f}")
```

Output

```
n = 1: nodes = [0.], weights = [2.]
n = 2: nodes = [-0.57735027  0.57735027], weights = [1. 1.]
n = 3: nodes = [-0.77459667  0.          0.77459667], weights = [0.55555556 0.88888889
    ↪ 0.55555556]
n = 4: nodes = [-0.86113631 -0.33998104  0.33998104  0.86113631], weights = [0.34785485
    ↪ 0.65214515 0.65214515 0.34785485]

integral of exp(-x^2) over [1, 1.5] = 0.1093642608
  Gauss, n = 2: 0.1094002612   error = 3.60e-05
  Gauss, n = 3: 0.1093641960   error = 6.48e-08
  Gauss, n = 4: 0.1093642605   error = 2.91e-10
  Gauss, n = 5: 0.1093642608   error = 2.13e-12
  Simpson, 5 points: 0.1093609550   error = 3.31e-06

Degree of precision of the 3-point rule on [-1, 1]:
  x^0: rule = +2.000000000000, exact = +2.000000000000
  x^1: rule = +0.000000000000, exact = +0.000000000000
  x^2: rule = +0.666666666667, exact = +0.666666666667
  x^3: rule = +0.000000000000, exact = +0.000000000000
  x^4: rule = +0.400000000000, exact = +0.400000000000
  x^5: rule = +0.000000000000, exact = +0.000000000000
  x^6: rule = +0.240000000000, exact = +0.285714285714
  x^7: rule = +0.000000000000, exact = +0.000000000000
```

Gauss with 3 points is 50 times more accurate than Simpson with 5 points, and Gauss with 5 points is correct to 12 digits. The 3-point rule is exact up to x^5 and fails for x^6, confirming the degree of precision $2n - 1 = 5$.

Historical Note: Gauss, Jacobi, Golub and Welsch

Gauss discovered his quadrature rules in 1814 using continued fractions, and computed the nodes and weights up to $n = 7$ to 16 digits by hand. **Jacobi** (1826) gave the proof via orthogonal polynomials used above. For a long time nodes and weights were taken from tables; in 1969 Gene **Golub** and John **Welsch** showed that they are the eigenvalues (and, from the eigenvectors, the weights) of a symmetric tridiagonal matrix built from the three-term recurrence, the method used by `np.polynomial.legendre.leggauss`. Gauss rules with other weight functions (Gauss–Chebyshev, Gauss–Laguerre on $[0, \infty)$, Gauss–Hermite on $(-\infty, \infty)$) integrate $\int w(x) f(x)\,dx$ with the corresponding orthogonal polynomials.

Exercises

13.4.1 Theory Use the 2- and 3-point Gauss rules to approximate $\int_1^{1.5} x^2 \ln x \, dx$ and $\int_0^1 x^2 e^{-x} dx$.

13.4.2 Theory Verify that the 3-point Gauss nodes $0, \pm\sqrt{3/5}$ are the zeros of $P_3(x) = \frac{1}{2}(5x^3 - 3x)$, and that the weights are $\frac{8}{9}, \frac{5}{9}, \frac{5}{9}$.

13.4.3 Theory Show that no two-point rule $c_1 f(x_1) + c_2 f(x_2)$ can have degree of precision 4, which proves the optimality claim of Theorem 13.7 for $n = 2$. (Hint: apply the rule to $(x - x_1)^2 (x - x_2)^2$.)

13.4.4 Python Compute the Gauss nodes and weights for $n = 5$ as the eigenvalues of the Jacobi matrix with off-diagonal entries $\beta_k = k/\sqrt{4k^2 - 1}$ (Golub–Welsch) and compare with `leggauss(5)`.

13.5 Adaptive quadrature

Composite rules use equally spaced points. But most integrands are smooth in some regions and vary rapidly in others; uniform spacing wastes evaluations where f is smooth. **Adaptive quadrature** places points automatically where they are needed.

Error estimation. Let $S(a,b)$ denote Simpson's rule on $[a,b]$ and $m = (a+b)/2$. With $h = (b-a)/2$,

$$I = S(a,b) - \frac{h^5}{90} f^{(4)}(\xi), \qquad I = S(a,m) + S(m,b) - \frac{1}{16}\,\frac{h^5}{90} f^{(4)}(\tilde{\xi}).$$

Assuming $f^{(4)}(\xi) \approx f^{(4)}(\tilde{\xi})$ and subtracting, the error of the more accurate value is estimated as

$$\bigl|I - S(a,m) - S(m,b)\bigr| \approx \frac{1}{15}\bigl|S(a,m) + S(m,b) - S(a,b)\bigr|. \tag{13.3}$$

This estimate costs nothing extra: it uses values already computed.

Algorithm 5 Adaptive Simpson quadrature $\mathrm{AS}(f,a,b,\varepsilon)$

1: $m \leftarrow (a+b)/2$; compute $S_1 = S(a,b)$ and $S_2 = S(a,m) + S(m,b)$
2: **if** $|S_2 - S_1| < 15\varepsilon$ **then**
3: **return** $S_2 + (S_2 - S_1)/15$ ▷ accept, with Richardson correction
4: **else**
5: **return** $\mathrm{AS}(f,a,m,\varepsilon/2) + \mathrm{AS}(f,m,b,\varepsilon/2)$ ▷ subdivide, split the tolerance
6: **end if**

Example 13.8. Compute $\int_1^3 \frac{100}{x^2} \sin\frac{10}{x}\,dx$. The substitution $t = 10/x$ gives the exact value $10\bigl(\cos\frac{10}{3} - \cos 10\bigr) = -1.4260247563\ldots$, which we use to measure the errors. The integrand oscillates rapidly near $x = 1$ and is smooth near $x = 3$.

Listing 13.5: Adaptive Simpson quadrature ch13/adaptive.py

```python
# Adaptive Simpson quadrature
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

evaluations = []                            # record every x at which f is evaluated


def f(x):
    evaluations.append(x)
    return 100 / x**2 * np.sin(10 / x)


def simpson(fa, fm, fb, a, b):
    return (b - a) / 6 * (fa + 4 * fm + fb)


def adaptive_simpson(f, a, b, tol):
    """Integrate f over [a, b] with estimated absolute error below tol."""
    fa, fm, fb = f(a), f((a + b) / 2), f(b)
    whole = simpson(fa, fm, fb, a, b)
    return _asr(f, a, b, fa, fm, fb, whole, tol, depth=0)


def _asr(f, a, b, fa, fm, fb, whole, tol, depth):
    m = (a + b) / 2
    lm, rm = (a + m) / 2, (m + b) / 2
    flm, frm = f(lm), f(rm)                 # only two new evaluations per call
    left = simpson(fa, flm, fm, a, m)
    right = simpson(fm, frm, fb, m, b)
    err = (left + right - whole) / 15       # error estimate of left + right
    if abs(err) < tol or depth > 50:
        return left + right + err           # accept (with Richardson correction)
    return (_asr(f, a, m, fa, flm, fm, left, tol / 2, depth + 1) +
            _asr(f, m, b, fm, frm, fb, right, tol / 2, depth + 1))


exact = 10 * (np.cos(10 / 3) - np.cos(10))   # exact value, by the substitution t = 10/x
for tol in [1e-3, 1e-6, 1e-9]:
    evaluations.clear()
    val = adaptive_simpson(f, 1.0, 3.0, tol)
    print(f"tol = {tol:.0e}: integral = {val:.12f}, error = {abs(val - exact):.1e}, "
          f"{len(evaluations)} function evaluations")

ref, est = quad(lambda x: 100 / x**2 * np.sin(10 / x), 1, 3, epsabs=1e-13, limit=200)
print(f"scipy.integrate.quad: {ref:.13f}, error = {abs(ref - exact):.1e} (its own
    ↪ estimate {est:.1e})")
print(f"exact value         : {exact:.13f}")

# how many points would composite Simpson need for the same accuracy?
for n in [64, 128, 256, 512]:
    x = np.linspace(1, 3, n + 1)
    y = 100 / x**2 * np.sin(10 / x)
    s = (2 / n) / 3 * (y[0] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum() + y[-1])
    print(f"composite Simpson, {n + 1:4d} points: error = {abs(s - exact):.1e}")

evaluations.clear()
adaptive_simpson(f, 1.0, 3.0, 1e-4)
xs = np.linspace(1, 3, 1000)
plt.figure(figsize=(6.5, 3.4))
plt.plot(xs, 100 / xs**2 * np.sin(10 / xs), lw=1)
pts = np.array(sorted(set(evaluations)))
```

Output

```
tol = 1e-03: integral = -1.426015318981, error = 9.4e-06, 53 function evaluations
tol = 1e-06: integral = -1.426024684569, error = 7.2e-08, 293 function evaluations
tol = 1e-09: integral = -1.426024756346, error = 4.1e-14, 1465 function evaluations
scipy.integrate.quad: -1.4260247563463, error = 3.8e-15 (its own estimate 1.7e-08)
exact value         : -1.4260247563463
composite Simpson,   65 points: error = 6.2e-04
composite Simpson,  129 points: error = 3.9e-05
composite Simpson,  257 points: error = 2.4e-06
composite Simpson,  513 points: error = 1.5e-07
```

With 53 function evaluations the adaptive method is about 60 times more accurate than composite Simpson with 65 equally spaced points.

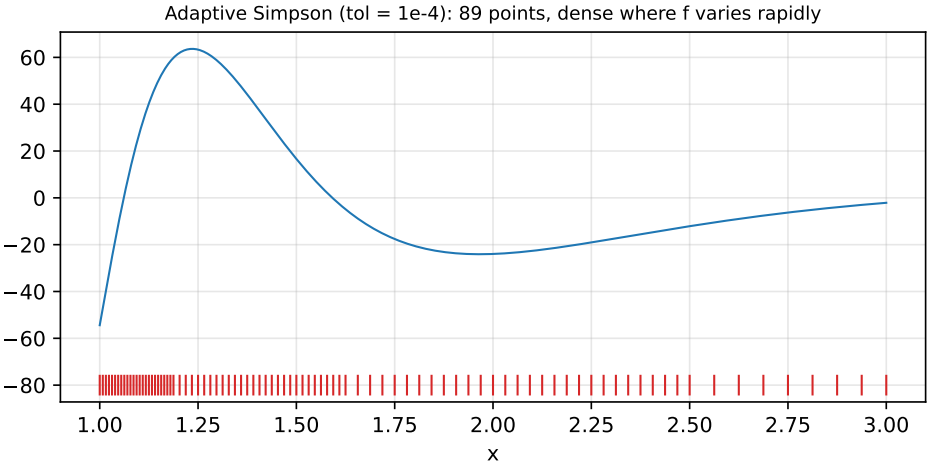


Figure 13.2: The points chosen by adaptive Simpson quadrature cluster where the integrand oscillates.

△ Caution: no black box is infallible

Any quadrature routine samples f at finitely many points and can be fooled: a narrow spike between sample points is simply not seen, and the error estimate (13.3) assumes $f^{(4)}$ is nearly constant. Always check the returned error estimate and the warnings of `scipy.integrate.quad`, and exploit what you know about f (singularities, oscillations, periodicity) by splitting the interval (`points=`) or by a change of variables.

Historical Note: adaptive quadrature and QUADPACK

Adaptive Simpson quadrature was published in 1962 by G. F. **Kuncir** and W. M. **McKeeman** as short ALGOL programs in the *Communications of the ACM*. In 1983 R. Piessens, E. de Doncker-Kapenga, C. Überhuber and D. Kahaner published **QUADPACK**, a Fortran library based on adaptive Gauss–Kronrod rules (Kronrod, 1964), which add $n+1$ nodes to an n-point Gauss rule to obtain an error estimate. QUADPACK is still used today: `scipy.integrate.quad` calls it. W. Gander and W. Gautschi's paper "Adaptive quadrature, revisited" (2000) is a good modern account.

Towards Graduate Studies: integration in high dimensions

In d dimensions a product rule with n points per direction needs n^d function evaluations: for $d = 100$ and $n = 2$ that is 10^{30}, the *curse of dimensionality*. *Monte Carlo* integration (Metropolis & Ulam, 1949) estimates an integral as an average of f at random points with error $O(N^{-1/2})$ *independent of* d; quasi-Monte Carlo (Sobol, Halton sequences) and sparse grids (Smolyak, 1963) do better for smooth integrands. These methods are essential in finance (option pricing), statistical physics, Bayesian statistics (Markov chain Monte Carlo) and uncertainty quantification. For one-dimensional integrals, the analysis of quadrature leads to deep results in approximation theory, e.g. Trefethen's "Is Gauss quadrature better than Clenshaw–Curtis?" (*SIAM Review*, 2008).

Exercises

13.5.1 Theory Apply one level of the adaptive Simpson test (13.3) by hand to $\int_0^1 e^{2x}dx$ with $\varepsilon = 10^{-3}$. Is the interval accepted?

13.5.2 Python Modify Listing 13.5 to return the list of accepted subintervals, and plot their lengths against their midpoints.

13.5.3 Python Use `scipy.integrate.quad` to compute (a) $\int_0^1 \frac{\ln x}{\sqrt{x}}dx$ (exact -4), (b) $\int_0^\infty e^{-x^2}dx$ (exact $\sqrt{\pi}/2$), (c) $\int_0^{100} \sin^2(x^2)dx$. Report the error estimates and any warnings, and compare with the exact values.

13.5.4 Python Implement adaptive quadrature based on the 2-point and 3-point Gauss rules (use their difference as the error estimate), and compare the number of function evaluations with adaptive Simpson on the example above.

13.5.5 Python **Project: Monte Carlo.** Estimate $\int_{[0,1]^d} \prod_{i=1}^d \frac{\pi}{2}\sin(\pi x_i)\,d\mathbf{x} = 1$ for $d = 1, 2, 5, 10$ with $N = 10^2, \dots, 10^6$ random points, and verify the $N^{-1/2}$ convergence.

13.5.6 ★ Graduate Prove that the composite trapezoidal rule with n points integrates e^{ikx} over $[0, 2\pi]$ exactly for $|k| < n$, and deduce the exponential convergence for periodic analytic functions observed in Listing 13.2.

Chapter Summary

- Newton–Cotes: integrate the interpolating polynomial. Trapezoid $O(h^3)$ locally, degree 1; Simpson $O(h^5)$ locally, degree 3.
- Composite trapezoid: $O(h^2)$; composite Simpson: $O(h^4)$. Integration is well conditioned.
- Romberg = trapezoid + Richardson extrapolation (Euler–Maclaurin expansion in h^2).
- Gauss–Legendre: n nodes at the zeros of P_n, degree of precision $2n - 1$, positive weights.
- Adaptive quadrature estimates the error locally, $\frac{1}{15}|S_2 - S_1|$, and refines only where needed. In practice: `scipy.integrate.quad`.

Chapter A

Python Quick Reference

Installing Python

- **Anaconda** (`anaconda.com`) installs Python with NumPy, SciPy, Matplotlib, Jupyter and the Spyder editor.

- **Google Colab** (`colab.research.google.com`) runs Jupyter notebooks in a browser; nothing to install.

- With a plain Python installation: `pip install numpy scipy matplotlib`.

All programs in these notes are in the folder `code/`. The script `run_all.py` runs every one of them and regenerates the outputs and figures used in this document.

Python, NumPy and MATLAB/Octave

Task	Python / NumPy	MATLAB / Octave
import libraries	`import numpy as np`	(built in)
row vector	`np.array([1, 2, 3])`	`[1 2 3]`
column vector	`np.array([[1], [2], [3]])`	`[1; 2; 3]`
matrix	`np.array([[1, 2], [3, 4]])`	`[1 2; 3 4]`
equally spaced points	`np.linspace(0, 1, 11)`	`linspace(0, 1, 11)`
range with step	`np.arange(0, 1.01, 0.1)`	`0:0.1:1`
zeros, ones, identity	`np.zeros((m, n))`, `np.ones(n)`, `np.eye(n)`	`zeros(m,n)`, `ones(n,1)`, `eye(n)`
first element	`x[0]`	`x(1)`
last element	`x[-1]`	`x(end)`
row i, column j	`A[i-1, :]`, `A[:, j-1]`	`A(i,:)`, `A(:,j)`
transpose	`A.T`	`A'`
matrix product	`A @ B`	`A * B`
element-wise product	`A * B`	`A .* B`
element-wise power	`A ** 2`	`A .^ 2`
solve $A\mathbf{x} = \mathbf{b}$	`np.linalg.solve(A, b)`	`A \ b`
inverse, determinant	`np.linalg.inv(A)`, `np.linalg.det(A)`	`inv(A)`, `det(A)`
eigenvalues	`np.linalg.eig(A)`	`eig(A)`
norms, condition number	`np.linalg.norm(x, p)`, `np.linalg.cond(A)`	`norm(x, p)`, `cond(A)`
LU, Cholesky, QR	`scipy.linalg.lu(A)`, `np.linalg.cholesky(A)`, `np.linalg.qr(A)`	`lu(A)`, `chol(A)'`, `qr(A)`
polynomial fit/evaluate	`np.polyfit(x, y, n)`, `np.polyval(p, t)`	`polyfit`, `polyval`
polynomial roots	`np.roots(p)`	`roots(p)`
root of f in $[a, b]$	`scipy.optimize.brentq(f, a, b)`	`fzero(f, [a b])`
nonlinear system	`scipy.optimize.fsolve(F, x0)`	`fsolve(F, x0)`
integral	`scipy.integrate.quad(f, a, b)`	`integral(f, a, b)`
interpolation	`np.interp`, `scipy.interpolate.CubicSpline`	`interp1`, `spline`
anonymous function	`f = lambda x: x**2 - 2`	`f = @(x) x.^2 - 2`
plot	`plt.plot(x, y); plt.show()`	`plot(x, y)`
print formatted	`print(f"{x:10.6f}")`	`fprintf('%10.6f\n', x)`

A template for an iterative method

```python
def my_method(f, x0, tol=1e-10, max_iter=100):
    """Describe what the function computes, its arguments and its return values."""
    x = x0
    for k in range(1, max_iter + 1):
        x_new = x - f(x) / 2.0                # replace by the update of your method
        if abs(x_new - x) <= tol * max(1.0, abs(x_new)):
            return x_new, k                   # converged: return result and iterations
        x = x_new
    raise RuntimeError(f"no convergence after {max_iter} iterations")
```

Common errors

- `IndentationError`: Python uses indentation to define blocks; use four spaces consistently.
- `IndexError: index 3 is out of bounds`: indices run from 0 to $n-1$.
- `ValueError: shapes (3,) and (4,) not aligned`: check dimensions with `A.shape`.
- Integer arrays: `np.array([1, 2, 3])` has integer type, so assigning `0.5` to an entry stores `0`. Use `np.array([1, 2, 3], dtype=float)` for numerical work.
- Copies: `B = A` does *not* copy an array (both names refer to the same data). Use `B = A.copy()`.
- `2^3` is not 2^3 (it is the bitwise exclusive or, = 1); write `2**3`.

Chapter B

Answers and Hints to Selected Exercises

Exercise numbers refer to the exercise lists at the end of each section (e.g. 4.2.3 is Exercise 3 of Section 4.2). All numerical answers were computed in Python.

Chapter 2

2.1.1 (a) 53; (b) 511; (c) 48879; (d) 0.4375; (e) $5\frac{7}{9} = 5.777\ldots$

2.1.2 $2026 = (11111101010)_2 = (3752)_8 = (7EA)_{16}$.

2.1.3 $0.3 = (0.0\overline{1001})_2$ (infinite); $0.75 = (0.11)_2$ (finite); $\frac{1}{3} = (0.\overline{01})_2$ (infinite).

2.2.1 (a) $101000_2 = 40$; (b) $1110_2 = 14$; (c) $1000001_2 = 65$; (d) $1001_2 = 9$.

2.2.2 $(E6)_{16} = 230$.

2.3.1 $-37 = (11011011)_2$.

Chapter 3

3.1.1 $\mathbb{F}(10, 2, -1, 1)$ has 270 positive numbers (541 in total); the largest is 9.9, the smallest positive is 0.01.

3.1.5 $n = 2^{53} = 9\,007\,199\,254\,740\,992$: $2^{53} + 1$ is not a double and rounds (ties to even) to 2^{53}.

3.2.1 Relative errors: (a) 4.0×10^{-4}; (b) 8.5×10^{-8}; (c) 1.0×10^{-4}; (d) 1.5×10^{-4}. With the definition of significant digits in Section 3.2: (a) 4, (b) 7, (c) 4, (d) 4.

3.3.1 (a) $\dfrac{1}{\sqrt{x+1} + \sqrt{x}}$; (b) $\ln(1 + 1/x)$, computed with `np.log1p(1/x)`; (c) $\tan(x/2)$; (d) `np.expm1` ↪ `(x)`; (e) $2\cos\left(a + \frac{x}{2}\right)\sin\frac{x}{2}$.

3.4.1 $\kappa = |x|$ for e^x; $1/|\ln x|$ for $\ln x$ (ill conditioned near $x = 1$); $|x \cot x|$ for $\sin x$ (ill conditioned near $k\pi$, $k \neq 0$); n for x^n; $|x/(1-x)|$ for $1/(1-x)$ (ill conditioned near $x = 1$).

Chapter 4

4.2.1 $f(1) = -5$, $f(2) = 14$. $p_1 = 1.5$ ($f = 2.375$), $p_2 = 1.25$ ($f = -1.7969$), $p_3 = 1.375$ ($f = 0.1621$). The root is 1.3652300134.

4.2.2 $n \geq \log_2 10^8 = 26.6$, so 27 iterations.

4.2.5 With Algorithm 1: (a) 2; (b) 0 (the first midpoint is a root); (c) 0.

4.4.1 The fixed point is $(3-\sqrt{13})/2 = -0.302776$. With $k = \frac{2}{3}$ and $\max(x_0 - a, b - x_0) = 1$: $(2/3)^n \le 10^{-5}$ gives $n \ge 29$.

4.4.2 At $p = 1.36523$: $|g_a'(p)| \approx 15.5$ (diverges), $|g_b'(p)| \approx 0.51$ (linear), $|g_c'(p)| \approx 0.127$ (fast linear), $g_d'(p) = 0$ (Newton: quadratic).

4.4.5 $g'(\sqrt{2}) = 1 - 2\sqrt{2}\,c$; convergence for $0 < c < 1/\sqrt{2}$; quadratic for $c = 1/(2\sqrt{2})$.

4.5.4 $x_{n+1} = x_n - 3x_n = -2x_n$: the iterates double in size and alternate in sign. $f'(0)$ does not exist.

4.5.5 $x = 4.4934094579$ (the first positive root after $x = 0$).

4.5.6 $i = 0.021211$ per month (2.12%); nominal annual rate 25.45%, effective annual rate 28.64%.

4.6.1 $x_2 = 0.7363841$, $x_3 = 0.7390581$, $x_4 = 0.7390851$.

Chapter 5

5.2.1 $\mathbf{x}^{(1)} = (2, 0.5)$, $\mathbf{x}^{(2)} = (1.93333, 0.51667)$. The exact solution is $(1.93185, 0.51764)$.

5.2.2 $\mathbf{x}^{(1)} = (0.8, 0.88)$.

Chapter 6

6.2.1 $\mathbf{x} = (1, 2, -1)$.

6.3.1 $L = \begin{pmatrix} 1&0&0\\ 2&1&0\\ 4&3&1 \end{pmatrix}$, $U = \begin{pmatrix} 2&1&1\\ 0&1&1\\ 0&0&2 \end{pmatrix}$, $\det A = 4$; $\mathbf{x} = (0.5, 0.5, -0.5)$ and $\mathbf{x} = (-0.5, 2.5, -1.5)$.

6.4.2 $L = \begin{pmatrix} 2&0&0\\ 1&2&0\\ 1&1&2 \end{pmatrix}$; $\mathbf{x} = (0.3125, -0.375, 0.75)$; $D = \operatorname{diag}(4, 4, 4)$, and the unit lower triangular factor is $\frac{1}{2}L$.

6.5.1 $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1, 0)^T$, $\mathbf{q}_2 = \frac{1}{\sqrt{6}}(1, -1, 2)^T$, $R = \begin{pmatrix} \sqrt{2} & 1/\sqrt{2} \\ 0 & \sqrt{3/2} \end{pmatrix}$.

6.5.2 $Q^T\mathbf{b} = (5, 0, -1)^T$ and $\mathbf{x} = (2.9, -0.1, -0.25)$.

Chapter 7

7.3.1 Jacobi: $\mathbf{x}^{(1)} = (0.6, 2.2727, -1.1, 1.875)$, $\mathbf{x}^{(2)} = (1.0473, 1.7159, -0.8052, 0.8852)$. Gauss–Seidel: $\mathbf{x}^{(1)} = (0.6, 2.3273, -0.9873, 0.8789)$, $\mathbf{x}^{(2)} = (1.0302, 2.0369, -1.0145, 0.9843)$.

7.3.2 $T_J = \begin{pmatrix} 0 & 1/2 \\ 1/2 & 0 \end{pmatrix}$, $\rho = \frac{1}{2}$; $T_G = \begin{pmatrix} 0 & 1/2 \\ 0 & 1/4 \end{pmatrix}$, $\rho = \frac{1}{4}$.

7.3.3 $\rho(T_J) = 0$ ($T_J^3 = 0$, so Jacobi gives the exact solution in 3 steps) but $\rho(T_G) = 2$.

7.6.3 $\alpha_0 = \frac{1}{4}$, $\mathbf{x}^{(1)} = (0.25, 0.5)$; $\alpha_1 = \frac{4}{11}$, $\mathbf{x}^{(2)} = (\frac{1}{11}, \frac{7}{11})$.

Chapter 8

8.1.1 $\|\mathbf{x}\|_1 = 8.5$, $\|\mathbf{x}\|_2 = \sqrt{27.25} = 5.2202$, $\|\mathbf{x}\|_\infty = 4$.

8.2.1 First matrix: $\|A\|_1 = \|A\|_\infty = \|A\|_2 = 3$, $\|A\|_F = \sqrt{10}$. Second matrix: $\|A\|_1 = \|A\|_\infty = 2$, $\|A\|_F = \sqrt{3}$, $\|A\|_2 = \frac{1+\sqrt{5}}{2}$.

8.3.1 $\kappa_\infty = 21$; $\kappa_\infty = (2+\epsilon)^2/\epsilon \to \infty$ as $\epsilon \to 0$.

8.3.3 The new solution is $(2, 0)$: relative change 1 (100%) in $\|\cdot\|_\infty$, although $\mathbf{b}$ changed by 1%. Here $\kappa_\infty(A) = 2 \times 50 = 100$, so the bound of Theorem 8.10 is attained.

Chapter 9

9.3.1 $P_2(x) = -\frac{2}{3}x^2 + 3x - \frac{1}{3}$; $P_2(3) = \frac{8}{3}$.

9.4.2 Newton coefficients $2, -1, 1, -2$: $P_3(x) = 2 - (x+1) + (x+1)x - 2(x+1)x(x-1)$. Adding $(3, 10)$ gives the new coefficient 2: $P_4 = P_3 + 2(x+1)x(x-1)(x-2)$.

9.5.1 $|E| \le \frac{h^2}{8} \max|\sin''| \le \frac{h^2}{8}$; $h \le \sqrt{8 \times 10^{-6}} \approx 2.8 \times 10^{-3}$.

9.5.3 $P_3(2.5) = 0.92122$, $\ln 2.5 = 0.91629$, error 4.9×10^{-3}. The bound $\frac{6}{4!} \times 0.5625 = 0.14$ is pessimistic because $|f^{(4)}(\xi)| = 6/\xi^4$ is much smaller than 6 for most ξ.

Chapter 10

10.1.1 $y = 1.5 + x$, $E = 1$.

10.1.2 $y = 1.5x^2 - 0.5x$, which passes through all four points ($E = 0$).

10.2.1 $a = 0.5$, $b = 2$ (exactly).

10.4.1 $P_1(x) = x - \frac{1}{6}$ in both cases; with Legendre polynomials, $c_0 = \frac{1}{3}$, $c_1 = \frac{1}{2}$ in $\frac{1}{3} + \frac{1}{2}(2x-1)$.

Chapter 11

11.1.1 $r_{1,1}(x) = \dfrac{1 + x/2}{1 - x/2}$, $r_{2,2}(x) = \dfrac{1 + x/2 + x^2/12}{1 - x/2 + x^2/12}$.

Chapter 12

12.1.4 Using the three-point end-point formulas at $x = 1.1$ and 1.4 and central differences at 1.2 and 1.3: 17.76971, 22.19364, 27.10735, 32.51085; exact values 18.05003, 22.04635, 26.92748, 32.88929.

12.3.2 First column of extrapolations: 2.201267, 2.200400; then 2.200342.

Chapter 13

13.1.2 Trapezoidal -0.267857, Simpson -0.267063, exact -0.267063.

13.2.1 Trapezoid 0.639900, Simpson 0.636310, exact 0.636294.

13.2.2 $\max_{[0,2]}|f^{(4)}| \approx 4475$, and the error bound requires $n \ge 54$ (even).

13.2.3 0.9500042.

13.2.4 3.8201977890.

13.2.5 Trapezoidal 59.6 km; Simpson 60.1 km.

13.4.1 $\int_1^{1.5} x^2 \ln x\, dx$: 0.1922687 ($n = 2$), 0.1922594 ($n = 3$), exact 0.1922594. $\int_0^1 x^2 e^{-x} dx$: 0.1594104 ($n = 2$), 0.1605954 ($n = 3$), exact 0.1606028.

Bibliography

[1] K. E. Atkinson, *An Introduction to Numerical Analysis*, 2nd ed., Wiley, 1989.

[2] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed., Brooks/Cole, Cengage Learning, 2011.

[3] S. C. Chapra, *Applied Numerical Methods with MATLAB for Engineers and Scientists*, 3rd ed., McGraw-Hill, 2012.

[4] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed., McGraw-Hill, 2015.

[5] D. G. Duffy, *Advanced Engineering Mathematics with MATLAB*, 4th ed., CRC Press, 2017.

[6] C.-E. Fröberg, *Introduction to Numerical Analysis*, 2nd ed., Addison-Wesley, 1969.

[7] C. F. Gerald and P. O. Wheatley, *Applied Numerical Analysis*, 7th ed., Addison-Wesley, 2004.

[8] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.

[9] A. Greenbaum and T. P. Chartier, *Numerical Methods: Design, Analysis, and Computer Implementation of Algorithms*, Princeton University Press, 2012.

[10] E. Hairer, S. P. Nørsett and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer, 1993.

[11] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.

[12] J. Kiusalaas, *Numerical Methods in Engineering with Python 3*, Cambridge University Press, 2013.

[13] E. Kreyszig, *Advanced Engineering Mathematics*, 10th ed., Wiley, 2011 (Part E: Numeric Analysis).

[14] S. E. Moore, *Lecture Notes in Numerical Analysis*, University of Cape Coast, 2023 (online).

[15] A. Quarteroni, R. Sacco and F. Saleri, *Numerical Mathematics*, 2nd ed., Springer, 2007.

[16] T. Sauer, *Numerical Analysis*, 3rd ed., Pearson, 2018.

[17] E. Süli and D. F. Mayers, *An Introduction to Numerical Analysis*, Cambridge University Press, 2003.

[18] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997.

[19] L. N. Trefethen, *Approximation Theory and Approximation Practice*, extended ed., SIAM, 2019.

[20] J. L. Chabert (ed.), *A History of Algorithms: From the Pebble to the Microchip*, Springer, 1999.

[21] H. H. Goldstine, *A History of Numerical Analysis from the 16th through the 19th Century*, Springer, 1977.

Suggestions for further reading. For the level of this course: Burden & Faires [2], Sauer [16], and Chapra [3] for applications. For Python: Kiusalaas [12]. For a first graduate course: Süli & Mayers [17], Quarteroni, Sacco & Saleri [15], Trefethen & Bau [18] and Higham [11]. For the history of the subject: Goldstine [21] and Chabert [20].